

Rust for C Programmers

Learn Rust Fast by Building on What You Already Know

Written for dwang, with Devin as guide

2026

Contents

Preface	10
What this book is	10
The one big promise (and the one hard idea)	11
The biggest shifts from C (your mental map)	11
A few things C programmers must <i>unlearn</i>	12
How the book is organized	12
Conventions	12
 Chapter 1 — Why Rust for a C Programmer	 13
Where Rust comes from	13
The Rust philosophy (and how it differs from C's)	13
What Rust removes (the relief)	14
What Rust adds (the new tools)	14
Your first program: “Hello, World”	14
A slightly bigger taste	15
The one idea that makes Rust different: the borrow checker	16
How a Rust program is built and run	16
Is Rust fast? (an honest answer)	18
When <i>not</i> to choose Rust	18
Key takeaways	18
Watch out (gotchas for C programmers)	19
Interview questions	19
Try it	19
 Chapter 2 — Installing Rust and the Cargo Toolchain	 20
The big picture: one tool instead of five	20
Installing Rust with rustup	20
A command-by-command tour of cargo	22
The daily workflow cheat-box	25
The anatomy of Cargo.toml	25
Calling rustc directly (rare)	25
Where everything lives on disk	26
How cargo build drives rustc	26

Editions in one paragraph	27
Key takeaways	27
Watch out (gotchas for C programmers)	28
Interview questions	28
Try it	29
Chapter 3 — Program Structure: Crates, Modules, and Visibility	29
No headers, no <code>#include</code> , no forward declarations	29
Crates and packages	29
Modules with <code>mod</code>	30
Paths: naming items across the tree	32
<code>use</code> : bringing names into scope	33
Visibility: private by default	34
The prelude: what you get for free	36
Using an external crate	36
Key takeaways	36
Watch out (gotchas for C programmers)	37
Interview questions	37
Try it	38
Chapter 4 — Variables, Types, and Constants	38
Variables with <code>let</code>	38
Immutable by default	38
Shadowing vs mutation	39
Type inference and annotations	40
Scalar types: integers	40
Scalar types: floating point, bool, char	41
Compound types: tuples	42
Compound types: arrays	43
No implicit conversions	44
Constants and statics	45
C type to Rust type	46
Key takeaways	47
Watch out (gotchas for C programmers)	47
Interview questions	47
Try it	48
Chapter 5 — Control Flow	48
<code>if</code> and <code>else</code>	48
Loops	50
<code>match</code> : the multi-way branch	52
<code>if let</code> , <code>let . . . else</code> , and <code>while let</code>	53
How the pieces fit	55
Key takeaways	55
Watch out (gotchas for C programmers)	55
Interview questions	55
Try it	56

Chapter 6 — Functions and Closures	56
Functions: the shape	56
Statements vs expressions	57
Returning a value: no <code>return</code> needed	58
The unit type <code>()</code> and diverging functions	59
No overloading, no default arguments	59
Function pointers	60
Closures: functions that capture their environment	61
How a closure captures: by reference, by mut reference, by value	62
The three closure traits: <code>Fn</code> , <code>FnMut</code> , <code>FnOnce</code>	65
Taking closures as parameters	65
Returning closures	66
Key takeaways	67
Watch out (gotchas for C programmers)	67
Interview questions	68
Try it	68
Chapter 7 — Ownership and Moves	68
The problem ownership solves	69
The three rules of ownership	69
Drop: automatic, deterministic cleanup (RAII)	69
Move semantics	71
Copy types: bit-for-bit duplication	72
Clone: an explicit, possibly expensive copy	73
Ownership and functions	74
Putting it together: the lifecycle of an owned value	76
Key takeaways	77
Watch out (gotchas for C programmers)	77
Interview questions	77
Try it	78
Chapter 8 — Borrowing and References	78
References: <code>&T</code> and <code>&mut T</code>	79
Borrowing in functions	79
The borrow rules	80
The borrow checker in action	82
Dangling references are impossible	83
References vs C pointers	84
A preview: <code>&self</code> and <code>&mut self</code>	84
Key takeaways	85
Watch out (gotchas for C programmers)	85
Interview questions	85
Try it	86
Chapter 9 — Lifetimes	86
What a lifetime is	86
Why functions need lifetimes	87
Lifetimes on structs	88

Lifetime elision: why most code needs no lifetimes	89
The <code>'static</code> lifetime	90
The classic lifetime errors and their fixes	90
Picture: overlapping vs disjoint lifetimes	91
Key takeaways	92
Watch out (gotchas for C programmers)	92
Interview questions	93
Try it	93
Chapter 10 — Slices and Strings	93
Slices: a borrowed view into a sequence	94
<code>Vec<T></code> : the owned array a slice borrows from	96
Strings: <code>String</code> and <code>&str</code>	96
UTF-8 reality (the big difference from C)	97
Conversions between <code>String</code> and <code>&str</code>	99
API guidance: borrow <code>&str</code> and <code>&[T]</code> , not <code>&String</code> and <code>&Vec<T></code>	100
Key takeaways	101
Watch out (gotchas for C programmers)	101
Interview questions	102
Try it	102
Chapter 11 — Structs and Methods	102
Structs: grouping related data	102
Building and updating structs	104
Mutability is per-binding, not per-field	104
Memory layout: like C, but fields may be reordered	105
Methods with <code>impl</code>	106
Deriving common traits	110
C struct + functions vs Rust struct + methods	111
Key takeaways	111
Watch out (gotchas for C programmers)	112
Interview questions	112
Try it	113
Chapter 12 — Enums and Pattern Matching	113
What a C <code>enum</code> is, and why Rust's is more	113
Defining an enum with data	114
How an enum is stored: tag plus payload	115
<code>match</code> : the heart of pattern matching	115
More pattern tools	119
<code>Option<T></code> : Rust has no null	120
<code>Result<T, E></code> : success or error, as a value	121
<code>if let</code> , <code>let . . . else</code> , and <code>while let</code>	121
Methods on enums with <code>impl</code>	122
Make illegal states unrepresentable	123
Matching can move the value	124
Key takeaways	125
Watch out (gotchas for C programmers)	125

Interview questions	125
Try it	126
Chapter 13 — Error Handling	126
Two kinds of failure	126
<code>Result<T, E></code> is just an enum	126
The <code>?</code> operator: clean propagation	127
Handling a <code>Result</code> with combinators	130
Defining your own error type	131
<code>panic!:</code> for unrecoverable errors	132
C <code>errno</code> vs Rust <code>Result</code> , side by side	135
Key takeaways	135
Watch out (gotchas for C programmers)	135
Interview questions	136
Try it	136
Chapter 14 — Generics	136
The problem: writing the same code twice	137
Generic functions	137
Generic structs	138
Generic enums	139
Generic methods	140
Trait bounds in depth	140
Monomorphization: generics with zero runtime cost	142
Const generics	144
The turbofish: helping inference	144
Static dispatch (a look ahead)	145
Key takeaways	145
Watch out (gotchas for C programmers)	145
Interview questions	146
Try it	146
Chapter 15 — Traits	147
What a trait is	147
Implementing a trait	147
Using traits to write generic code	149
Static vs dynamic dispatch	150
The orphan rule (coherence)	153
The standard-library traits you must know	153
Associated types	155
Supertraits	156
Blanket implementations	156
How traits map to C, summarized	157
Key takeaways	157
Watch out (gotchas for C programmers)	158
Interview questions	158
Try it	159

Chapter 16 — Collections and Iterators	159
The problem these solve	159
Vec<T>: the growable array	159
String: a UTF-8 Vec<u8> (recap)	161
HashMap<K, V>: the key-value store	161
Iterators: walking data without index loops	163
A complete example: word frequencies	168
Key takeaways	168
Watch out (gotchas for C programmers)	169
Interview questions	169
Try it	169
Chapter 17 — Smart Pointers	170
What is a smart pointer?	170
Box<T>: a single owner on the heap	170
Deref, DerefMut, and deref coercion	172
Drop: automatic cleanup (recap)	172
Rc<T>: shared ownership by reference counting	173
RefCell<T>: interior mutability	174
Rc<RefCell<T>>: shared mutable data	175
Weak<T>: breaking reference cycles	176
Picking the right pointer	177
Key takeaways	177
Watch out (gotchas for C programmers)	177
Interview questions	178
Try it	178
Chapter 18 — Memory Without a Garbage Collector	178
The big picture: same speed as C, no GC, no free	178
Stack vs heap (a quick recap)	179
Moves: transferring ownership, not copying data	180
How freeing happens: Drop and scope	180
The allocator: who actually calls malloc	182
Can you leak memory in safe Rust?	183
Performance: how Rust stays in C's class	183
Key takeaways	184
Watch out (gotchas for C programmers)	185
Interview questions	185
Try it	185
Chapter 19 — Threads and Fearless Concurrency	186
Starting a thread	186
move closures: giving data to a thread	187
Fearless concurrency: data races caught at compile time	187
Send and Sync: the thread-safety markers	187
Sharing ownership: Arc	189
Shared mutation: Mutex and RwLock	189
Scoped threads: borrowing local data	191

What Rust does <i>not</i> prevent	191
Key takeaways	192
Watch out (gotchas for C programmers)	192
Interview questions	192
Try it	193
Chapter 20 — Channels and Shared State	193
Channels: <code>std::sync::mpsc</code>	193
A producer/consumer example	196
Channels vs shared state	199
Shared state: <code>Arc<Mutex<T>></code>	199
Faster channels in the ecosystem	201
Key takeaways	202
Watch out (gotchas for C programmers)	202
Interview questions	202
Try it	203
Chapter 21 — Async/Await	203
The problem: thousands of connections	203
<code>async fn</code> returns a future	204
Futures need a runtime	205
Concurrency within a task	206
<code>async</code> and <code>Send</code>	208
When <code>async</code> , when threads	209
Key takeaways	210
Watch out (gotchas for C programmers)	210
Interview questions	210
Try it	211
Chapter 22 — Cargo, Crates, and Workspaces	211
The problem C never solved	211
<code>Cargo.toml</code> in depth	212
<code>Cargo.lock</code> : reproducible builds	213
Version requirements and semver	214
The crates.io ecosystem	215
Cargo features	215
Workspaces: many crates, one repository	216
Publishing and build scripts (briefly)	219
Key takeaways	219
Watch out (gotchas for C programmers)	220
Interview questions	220
Try it	220
Chapter 23 — Testing	221
Testing is built in, not bolted on	221
Unit tests	221
Running tests with <code>cargo test</code>	223
Integration tests	224

Documentation tests	224
What cargo test runs, all at once	225
Organizing tests and fixtures	225
Ecosystem crates for testing	227
Key takeaways	227
Watch out (gotchas for C programmers)	228
Interview questions	228
Try it	228
Chapter 24 — Tooling	229
One ecosystem, not a pile of separate tools	229
rustfmt: one canonical style	229
clippy: the linter you should always run	230
rust-analyzer: the editor language server	231
cargo doc: documentation from comments	232
Debugging	233
Miri: finding undefined behavior in unsafe code	233
More analysis and inspection tools	234
Profiling	235
Key takeaways	236
Watch out (gotchas for C programmers)	236
Interview questions	236
Try it	237
Chapter 25 — Unsafe Rust and FFI	237
Why this chapter matters to a C programmer	237
What unsafe actually allows: the five superpowers	237
Raw pointers: C pointers, back again	238
Calling C from Rust	239
Calling Rust from C	241
bindgen and the *-sys convention	242
Wrapping unsafe in a safe abstraction	243
The big picture: Rust and C coexisting	244
Key takeaways	245
Watch out (gotchas for C programmers)	245
Interview questions	245
Try it	246
Chapter 26 — Macros	246
What a macro is, and why a language needs one	246
The C preprocessor, and its traps	247
Declarative macros with macro_rules!	248
The macros you already use (and why they must be macros)	251
Procedural macros (an overview — you will mostly <i>use</i> them)	251
C macros vs Rust macros, side by side	253
Seeing the expansion: cargo expand	254
Key takeaways	254
Watch out (gotchas for C programmers)	255

Interview questions	255
Try it	256
Chapter 27 — Idioms and Style	256
Make illegal states unrepresentable	256
The newtype pattern	257
Conversions: <code>From</code> , <code>Into</code> , <code>TryFrom</code> , <code>AsRef</code>	258
<code>Default</code> , struct-update syntax, and the builder pattern	260
Function signatures: borrow on the way in, own on the way out	261
Prefer iterators, combinators, and <code>?</code>	262
Error style: <code>thiserror</code> for libraries, <code>anyhow</code> for applications	262
Naming conventions	263
<code>Display</code> and <code>Debug</code>	264
Keep the API small, run clippy, follow the guidelines	264
Translating C idioms to Rust idioms	265
Key takeaways	266
Watch out (gotchas for C programmers)	266
Interview questions	266
Try it	267
Chapter 28 — Gotchas for C Programmers (the checklist)	267
Variables and types	267
Ownership, moves, and borrows	270
Strings and characters	272
Collections and indexing	274
Null, errors, and return values	275
Expressions and control flow	276
Macros and functions	277
Structs and layout	278
Smart pointers and shared state	279
Concurrency	280
Key takeaways	281
Interview questions	281
Chapter 29 — Interview Questions and Answers	282
Basics	282
Ownership and borrowing	283
Lifetimes	284
Enums and pattern matching	284
Error handling	285
Traits and generics	285
Smart pointers	286
Concurrency	287
Memory and runtime	287
Unsafe and FFI	288
Cargo and tooling	289
Predict / explain (small puzzles)	289
Design questions	291

Chapter 30 — Capstone Projects	292
Project 1 — <code>linecount</code> : a <code>wc</code> -like command-line tool	293
Project 2 — <code>parallel_count</code> : a multithreaded word-frequency counter	301
Project 3 — <code>Stack<T></code> and an RPN evaluator: a generic library with tests	307
Key takeaways	314
Watch out (gotchas for C programmers)	314
Interview questions	315
Try it	315
Appendix A — C to Rust Cheat Sheet	315
Types	316
Declarations	316
Memory management	317
Strings	318
Pointers and references	319
Control flow	319
Error handling	320
Concurrency	320
libc headers → Rust	321
Idioms (the Rust way)	322
cargo command cheat sheet	322
Common one-liners	323
Appendix B — Glossary	323
References and Further Reading	328
Official documentation	328
Books	329
Practice	329
Tools	329
Community	329
Especially for C programmers	330

Preface

What this book is

This is a textbook for one specific reader: an **experienced C programmer** who needs to learn **Rust** quickly and well. It does not assume you know any other language. It does assume you are comfortable with the things C teaches deeply: pointers, memory, the stack and the heap, structs, `malloc/free`, and how source files become a running program.

That C background is the perfect starting point. Rust was built for exactly the jobs C is used for — operating systems, browsers, embedded devices, network services, command-line tools — and it makes the same core promise as C: **no garbage collector, no hidden runtime, and full control over memory and speed**. The difference is that Rust’s compiler *checks your memory and threading for you*, and refuses to build the program until it can prove there are no dangling pointers, no use-after-free, no double-free, and no data races.

The writing is deliberately simple. Sentences are short. Every new word is explained the first time it appears. The goal is that the **ideas** are the only hard part — never the language the book is written in.

The one big promise (and the one hard idea)

Most of Rust will feel familiar. Integers, structs, functions, control flow, the stack and heap, and compiling to a fast native binary are all here and behave much like C.

There is really just **one big new idea** to learn, and the whole language grows out of it: **ownership**. In C, *you* keep track of who allocated a buffer and who must free it, and the bugs that follow (use-after-free, double-free, leaks, data races) are some of the worst in our industry. In Rust, the compiler tracks this for you using a few simple rules. Learning those rules is the real work of this book. Once they click, Rust stops feeling like a fight and starts feeling like a very strict, very helpful colleague.

Mental model. Rust is “C where the compiler is your `malloc/free` accountant and your thread-safety reviewer.” You still control memory; you just write down the ownership rules the compiler then enforces.

The biggest shifts from C (your mental map)

Keep these in mind. The whole book is an expansion of this list.

1. **Ownership replaces `malloc/free`.** Every value has exactly one *owner*. When the owner goes out of scope, the value is freed automatically (this is RAII — the same idea as a C++ destructor). You never call `free`, and you can never free twice. (Chapter 7)
2. **Borrowing has rules the compiler enforces.** You can lend out a value by reference. The rule: **either many shared `&` readers, or exactly one `&mut` writer — never both at once.** This single rule removes data races and dangling pointers. (Chapter 8)
3. **No NULL, no uninitialized memory.** There is no null pointer. “Maybe a value, maybe nothing” is expressed with the `Option<T>` type, and the compiler forces you to handle the “nothing” case. Every variable must be initialized before use. (Chapter 12)
4. **Errors are values, not exceptions.** A function that can fail returns `Result<T, E>`, and the `?` operator makes checking it short and clean. There are no exceptions and no `errno`. (Chapter 13)
5. **Values move by default.** Assigning or passing a value *moves* its ownership; the old variable can no longer be used. This is how the compiler guarantees a single owner. Cheap “plain data” types are `Copy` instead and behave like C. (Chapter 7)
6. **Immutable by default.** `let x = 5;` is read-only. You must write `let mut x` to allow changes. This is the reverse of C, and it is a big part of why Rust code is easy to reason about. (Chapter 4)
7. **Traits instead of inheritance; generics are zero-cost.** Shared behavior is described by *traits* (like a type-checked interface), and generics are compiled into specialized machine code (monomorphization), so abstraction costs nothing at runtime. (Chapters 14–15)
8. **`unsafe` is a small, explicit escape hatch.** Raw pointers, pointer arithmetic, and C interop live inside `unsafe` blocks. The other 99% of your code is checked. This is how Rust talks to

C and the hardware. (Chapter 25)

If those ideas sink in, everything else is detail.

A few things C programmers must *unlearn*

- **You do not free memory.** Do not look for the **free**. The owner’s scope ending *is* the free. (Chapter 7)
- **No pointer arithmetic in normal code.** To walk memory you use slices and iterators, which are bounds-checked. Raw pointer math exists only in **unsafe**.
- **No implicit numeric conversions.** You must write `x as i64` (or use `From/Into`). The compiler will not quietly mix `i32` and `i64`. (Chapter 4)
- **Variables are immutable until you say otherwise.** Reach for `mut` only when you truly need to change a value.
- **Strings are UTF-8, not NUL-terminated char arrays.** A `String` knows its length, is not null-terminated, and indexing by integer is deliberately not allowed. (Chapter 10)

How the book is organized

The chapters build on each other, but you do not have to read them all before writing code. The **Start Here** part (Chapters 1–6) plus the **Ownership** part (Chapters 7–10) is the real core. The `README.md` lays out a **fast-track** reading order; follow it, then read the rest as your work demands.

Two reference sections are meant to stay open on your desk:

- **Appendix A — the C → Rust cheat sheet:** a translation table for nearly every construct, plus the commands you’ll type all day.
- **Chapter 28 — gotchas:** the borrow-checker traps that catch C programmers, in one checklist.

Conventions

You will see short **callout boxes**. Each one always means the same thing:

Mental model. A quick analogy to build intuition.

C vs Rust. A direct comparison with C.

Watch out. A common trap or surprise. Read these carefully.

Rule of thumb. Advice you can apply right away.

Deep dive. An optional, deeper aside. Safe to skip on the fast track.

Try it. A tiny exercise.

Code blocks are labeled by language — **rust**, **c**, or **sh** (a shell command you type in a terminal). Some Rust examples **intentionally do not compile**, to show a mistake the compiler catches; those begin with a `// COMPILER ERROR:` comment. Most chapters end with **Key takeaways**, a **Watch out** checklist, and a few **Interview questions** with answers, so the book doubles as interview preparation.

Let’s begin with *why* Rust exists, and what it feels like coming from C.

Chapter 1 — Why Rust for a C Programmer

What you’ll learn. Why Rust was created, how its philosophy compares to C’s, exactly what Rust removes and adds relative to C, and how to read and run your first Rust program. You will also see the single idea — the *borrow checker* — that makes Rust feel different from every language you have used.

Where Rust comes from

Rust began as a personal project of Graydon Hoare around 2006 and was then sponsored by Mozilla, which needed a language to build a safe, parallel browser engine (Servo) without the constant memory-safety bugs that plague large C and C++ codebases. Rust 1.0 shipped in 2015.

The problem it set out to solve is the one every C programmer knows in their bones: **memory bugs**. Use-after-free, double-free, buffer overruns, dangling pointers, and data races are responsible for a huge share of security vulnerabilities. Study after study from Microsoft, Google, and others found that roughly **70% of serious security bugs** in large C/C++ codebases are memory-safety bugs.

Rust’s bet is that a compiler can prevent all of those *at compile time*, with **no garbage collector and no runtime cost**. That bet paid off: Rust is now used in the Linux kernel, Android, Windows, Firefox, AWS (Firecracker), Cloudflare, Discord, Dropbox, and countless command-line tools, and it has topped Stack Overflow’s “most admired language” survey for years.

Mental model. Rust is “C with a compiler that refuses to let you write a memory bug.” You keep C’s speed and control; you give up the freedom to shoot yourself in the foot.

The Rust philosophy (and how it differs from C’s)

C’s philosophy is “trust the programmer.” The language is small and gives you sharp tools, including ones with undefined behavior. The programmer is responsible for every byte.

Rust’s philosophy is “**safety, speed, and concurrency — pick all three.**” It achieves this with two ideas:

- **Ownership and borrowing:** a small set of rules, checked by the compiler, that track who is responsible for each value and who may read or write it. This is the heart of the language (Chapters 7–9).
- **Zero-cost abstractions:** high-level features (iterators, generics, traits) compile down to the same machine code you would have written by hand in C. You do not pay at runtime for the convenience.

Topic	C	Rust
Memory management	Manual <code>malloc/free</code>	Automatic via ownership (no GC, no <code>free</code>)
Memory safety	Your responsibility	Guaranteed by the compiler (in safe code)
Null pointers	NULL everywhere	No null; <code>Option<T></code> instead
Error handling	Return codes / <code>errno</code>	<code>Result<T, E></code> values + ?

Topic	C	Rust
Concurrency safety	Your responsibility	Checked at compile time (Send/Sync)
Mutability	Mutable by default	Immutable by default
Pointer arithmetic	Yes	Only inside unsafe
Generics	Macros / <code>void *</code>	Real generics, monomorphized (zero-cost)
Build system	You write Makefiles	Built in (cargo)
Undefined behavior	Pervasive	Only possible inside unsafe

Watch out. Rust’s compiler is strict and will reject code that a C compiler would happily build. This feels slow at first. The trade is that once it compiles, an enormous class of bugs is simply gone. Rust programmers call this feeling “if it compiles, it works” — an exaggeration, but a telling one.

What Rust removes (the relief)

Coming from C, you get to stop worrying about a long list of dangers:

- **Manual memory management.** No `malloc/free`, no “who owns this pointer?”, no use-after-free, no double-free, no leak hunts with Valgrind. The compiler frees memory for you, deterministically, with no garbage collector. (Chapter 7)
- **Dangling pointers and buffer overruns.** References can never outlive what they point to, and slices are bounds-checked. (Chapters 8–10)
- **Null pointer dereferences.** There is no null. (Chapter 12)
- **Data races.** The compiler will not let two threads share mutable data without synchronization. (Chapter 19)
- **Most undefined behavior.** Integer sizes are fixed, signed overflow is defined (it panics in debug builds), and uninitialized memory cannot be read.

What Rust adds (the new tools)

In exchange, Rust gives you features C does not have:

- **The ownership system and borrow checker** — the compile-time memory and thread safety described above. (Chapters 7–9)
- **`Option<T>` and `Result<T, E>`** — types that make “no value” and “an error” explicit and impossible to ignore. (Chapters 12–13)
- **Enums and pattern matching** — powerful tagged unions with exhaustive `match`. (Chapter 12)
- **Traits and generics** — type-checked, zero-cost abstraction. (Chapters 14–15)
- **Cargo** — one tool that builds, tests, formats, lints, fetches dependencies, and generates docs. (Chapter 2)
- **Fearless concurrency** — threads, channels, and `async`, all checked for safety. (Chapters 19–21)

Your first program: “Hello, World”

Here it is in C and in Rust, side by side.

```

/* hello.c - compile: cc hello.c -o hello && ./hello */
#include <stdio.h>

int main(void) {
    printf("Hello, World\n");
    return 0;
}

// hello.rs - run: rustc hello.rs && ./hello (or: cargo run)
fn main() {
    println!("Hello, World");
}

```

Line by line, with the C version as our reference:

- `fn main()` — `fn` declares a function; `main` is the entry point, like in C. Note that `main` does **not** return `int` here; to exit with a status code you return a `Result` or call `std::process::exit(code)`.
- `println!("Hello, World")` — prints a line and adds the newline for you. The `!` means **`println!` is a macro**, not a function. Macros run at compile time and can do things functions cannot (here, checking the format string). You will meet many **`name!` macros**: `println!`, `vec!`, `format!`, `panic!`. (Chapter 26)
- There is no `#include`. Common items come from the *prelude*, which is imported automatically; everything else you bring in with `use` (Chapter 3).

Watch out. The `!` on `println!` is not a typo and not “not”. It marks a macro call. Forgetting it (`println(...)`) is a common first-day error.

A slightly bigger taste

This program shows several Rust ideas at once: a growable vector, an iterator pipeline, an explicit numeric conversion, and an `Option` handled with `match`. Read the comments; we cover each idea in detail later.

```

fn main() {
    let nums = vec![3, 1, 4, 1, 5, 9]; // a Vec<i32>: a heap array that knows its length
    let total: i32 = nums.iter().sum(); // iterators are lazy and zero-cost
    let count = nums.len();
    let avg = total as f64 / count as f64; // `as` is an explicit conversion

    println!("sum={total} count={count} avg={avg:.2}");

    // `max()` returns Option<E>i32>: Some(&value), or None if the Vec were empty.
    match nums.iter().max() {
        Some(m) => println!("max={m}"),
        None => println!("the list was empty"),
    }
}

```

Things to notice compared to C:

- `let nums = vec![...]` declares **and** initializes a vector. No `malloc`, no size, no `free`. When `nums` goes out of scope at the end of `main`, its heap memory is released automatically.
- `nums.iter().sum()` is an **iterator pipeline**. It compiles to a tight loop with no extra allocation — a zero-cost abstraction.
- `total as f64` is an **explicit conversion**. Rust will not divide an `i32` by an `i32` and silently give you a float, nor mix `i32` and `f64`.
- `max()` cannot return null. It returns `Option`, and the compiler forces you to handle both the `Some` and `None` cases. A whole class of crashes disappears.
- `println!("{total}")` puts the variable right in the string — like a safer, type-checked `printf`, with no format-specifier mismatches.

The one idea that makes Rust different: the borrow checker

Here is the example that surprises every newcomer. In C, this compiles and is a classic bug:

```
char *s1 = strdup("hello");
char *s2 = s1;      /* both point to the same buffer */
free(s1);
free(s2);           /* double free - undefined behavior, maybe a crash or exploit */
```

The equivalent Rust does **not** compile — and that is the point:

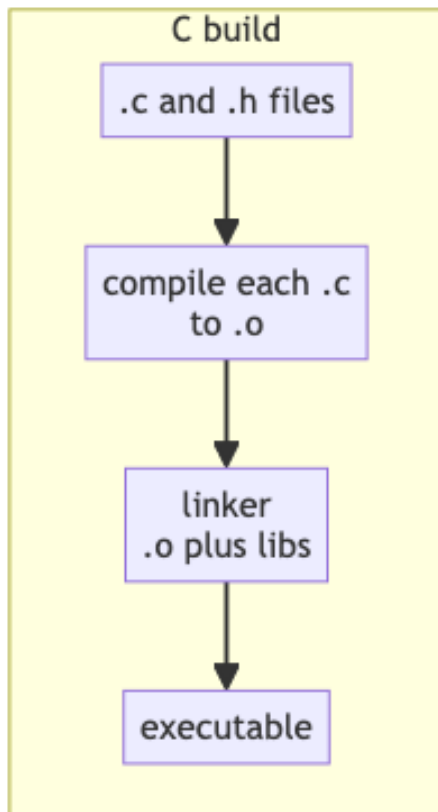
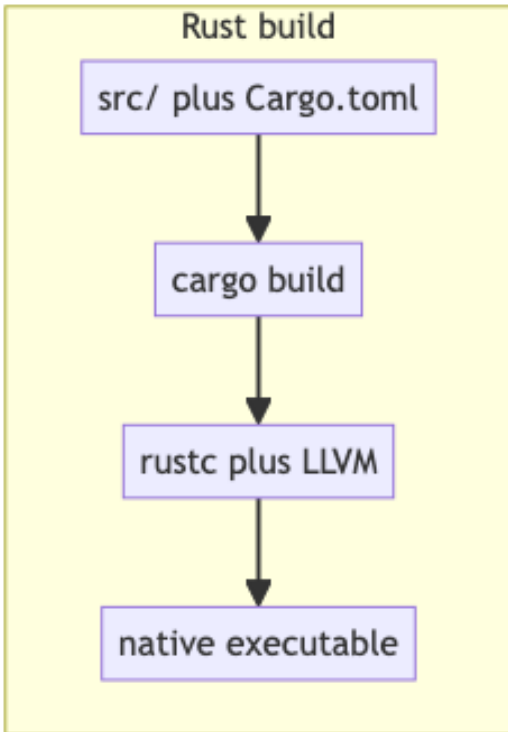
```
// COMPILE ERROR: borrow of moved value: `s1`
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;      // ownership MOVES from s1 to s2; s1 is now invalid
    println!("{s1}"); // error[E0382]: borrow of moved value: `s1`
}
```

When you write `let s2 = s1`, ownership of the string **moves** to `s2`. Rust now considers `s1` empty, so using it is a compile error. Because there is exactly one owner, the memory is freed exactly once, automatically, when that owner's scope ends. Double-free and use-after-free are impossible. This rule — *ownership* — is the subject of Chapter 7, and almost everything else in Rust follows from it.

Mental model. Think of ownership like a single physical key to a room. You can hand the key to someone else (a move), but then *you* no longer have it. Only the key-holder may lock up (free) the room, and there is only ever one key.

How a Rust program is built and run

Like C, Rust compiles ahead of time to native machine code (through the LLVM backend, the same backend `clang` uses). But you rarely call the compiler (`rustc`) directly. You use **Cargo**, Rust's build tool and package manager, which reads a `Cargo.toml` file and handles compiling, dependencies, and linking.



Practical consequences:

- **One command.** `cargo run` builds and runs; `cargo build --release` produces an optimized binary in `target/release/`. No Makefile to write.
- **Native, fast binaries.** The output is machine code with no interpreter and no garbage collector. Startup is instant.
- **Slower compiles than C.** All that checking and optimization takes time; Rust build times are a real, often-mentioned trade-off. `cargo check` (type-check without producing a binary) is much faster and what you run while editing.
- **Easy cross-compilation and dependencies.** `rustup target add ...` and Cargo fetch and build everything for you.

Is Rust fast? (an honest answer)

Yes. Rust is in the same performance class as C and C++. Its abstractions are “zero-cost”: an iterator chain or a generic function compiles to the same instructions you would write by hand. There is no garbage collector, so there are no GC pauses, and memory use is predictable.

The trade-offs versus C are not at runtime; they are at *development* time:

- **The learning curve.** Ownership and borrowing take real effort to internalize — that is what this book is for.
- **Compile times.** Larger than C, because the compiler does much more work.
- **Some patterns are harder.** Data structures with lots of shared mutable pointers (graphs, doubly linked lists) fight the borrow checker and need extra tools (Chapter 17) or `unsafe`.

Rule of thumb. If you would reach for C or C++ for performance and control — a systems tool, a network service, an embedded device, a CLI, a game engine component — Rust gives you that performance with memory and thread safety guaranteed. That is its sweet spot.

When *not* to choose Rust

Be honest about the edges. Rust is a weaker fit when you need:

- **Throwaway scripts or quick prototypes**, where a garbage-collected language (Python, Go) lets you move faster and you do not care about the last bit of performance.
- **To extend a large existing C/C++ codebase in place** — though Rust interoperates with C through FFI (Chapter 25) and can be introduced module by module.
- **A team with no time to climb the learning curve** right now. Rust rewards the investment, but the investment is real.
- **Very dynamic, reflection-heavy designs** that lean on a runtime; Rust prefers to resolve things at compile time.

For systems programming, infrastructure, performance-critical services, embedded work, and robust command-line tools, Rust is an outstanding choice — and a natural next language for a C programmer who is tired of memory bugs.

Key takeaways

- Rust was created to eliminate memory-safety bugs (about 70% of serious security bugs in C/C++) **without** a garbage collector and **without** runtime cost.

- It keeps C’s strengths — compiled, native, fast, low-level control — and removes the dangers: manual **free**, dangling pointers, buffer overruns, null, and data races.
- The one big new idea is **ownership**, enforced by the **borrow checker**: every value has a single owner, and moving it invalidates the old name.
- New tools you gain: **Option/Result**, enums and pattern matching, traits and generics, Cargo, and fearless concurrency.
- **cargo run** builds and runs; abstractions are zero-cost; the trade-offs are the learning curve and compile times, not runtime speed.

Watch out (gotchas for C programmers)

- **println! is a macro** — the **!** is required.
- **main does not return int**. Use a **Result** return or **std::process::exit**.
- **No implicit numeric conversions** — write **x as f64**.
- **Variables are immutable by default** — you will need **let mut** sooner than you think.
- The compiler fighting you is normal and expected at first; the error messages are unusually good — read them.

Interview questions

Q: What problem was Rust designed to solve, and how does it differ from a garbage-collected language? A: Rust was designed to give memory and thread safety with the performance and control of C/C++. Unlike garbage-collected languages, it has no GC and no runtime overhead; safety is enforced at compile time by the ownership and borrowing rules, and memory is freed deterministically when its owner goes out of scope.

Q: What is “zero-cost abstraction”? A: The principle that high-level features (iterators, generics, traits) compile down to machine code as efficient as hand-written low-level code, so you do not pay a runtime penalty for using them. Generics, for example, are monomorphized into specialized code rather than dispatched dynamically.

Q: In Rust, why does using a variable after assigning it to another variable sometimes fail to compile? A: Because assignment can *move* ownership. For a non-Copy type like **String**, **let b = a**; moves ownership to **b**, leaving **a** invalid. Using **a** afterward is a compile error. This guarantees a single owner, so the value is freed exactly once.

Q: Name two whole classes of bugs that safe Rust prevents at compile time that C does not. A: Examples: use-after-free and double-free (prevented by ownership), dangling references (prevented by the borrow checker and lifetimes), buffer overruns (slices are bounds-checked), null-pointer dereferences (no null; **Option** instead), and data races (the **Send/Sync** rules).

Try it

1. Install Rust if you have not (Chapter 2 covers it) and save the “Hello, World” program. Run it with **rustc hello.rs && ./hello**.
2. Type the “bigger taste” program into **src/main.rs** of a **cargo new** project and run **cargo run**. Then make the **nums** vector empty (**vec! []**), and annotate it as **let nums: Vec<i32> = vec! []**; and watch the **None** branch print.
3. Type the borrow-checker example and read the full compiler error. Notice how it names the move and even suggests **.clone()**. That is the borrow checker talking to you — get used to

its voice.

Chapter 2 — Installing Rust and the Cargo Toolchain

What you'll learn. How to install Rust with `rustup`, what each tool in the toolchain does, and a command-by-command tour of `cargo` — the one program that replaces your C compiler, `make`, package manager, formatter, and linter all at once.

The big picture: one tool instead of five

In C, building a real project means stitching together separate programs: a compiler (`gcc` or `clang`), a build system (`make`, CMake, Autotools), a way to get libraries (your distro's package manager, or copying source into the tree), a formatter (`clang-format`), and a linter (`clang-tidy`, `cppcheck`). Each has its own config file and its own learning curve.

Rust ships these as one coherent toolchain, and you drive almost all of it through a single command called `cargo`.

Job	C (typical)	Rust
Compile	<code>gcc</code> / <code>clang</code>	<code>rustc</code> (called for you by cargo)
Build / incremental builds	<code>make</code> , CMake	<code>cargo build</code>
Get a dependency	distro package, vendored source	<code>cargo add</code> (from crates.io)
Pin dependency versions	by hand / lockfiles you write	<code>Cargo.lock</code> (automatic)
Format code	<code>clang-format</code>	<code>cargo fmt</code>
Lint	<code>clang-tidy</code> , <code>cppcheck</code>	<code>cargo clippy</code>
Run tests	a framework you bolt on	<code>cargo test</code> (built in)
Build docs	Doxygen	<code>cargo doc</code>
Install the toolchain itself	distro package	<code>rustup</code>

Mental model. `rustup` manages *which Rust* you have (versions and targets). `cargo` manages *your project* (building, testing, dependencies). `rustc` is the actual compiler underneath, which you rarely call by hand.

Installing Rust with rustup

`rustup` is the **toolchain manager**. It installs the compiler and tools, keeps them updated, and lets you switch between Rust versions and cross-compilation targets. On Linux and macOS, the official one-line installer is:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

This downloads a small script and runs it. It asks one question (press Enter for the default), then installs everything into your home directory — no `sudo`, no system-wide changes. On Windows, download and run `rustup-init.exe` from <https://rustup.rs> instead.

Watch out. Piping a script from the internet into `sh` is exactly the kind of thing a careful C programmer is right to pause over. This is the method the Rust project officially documents. If you prefer, open the URL in a browser and read the script first, or install `rustup` from your OS package manager.

When it finishes, the installer tells you to add `~/.cargo/bin` to your `PATH`. It usually edits your shell profile for you. To use Rust in the **current** shell without opening a new one:

```
source "$HOME/.cargo/env"
```

What just got installed

A default install gives you four programs, all living under `~/.cargo/bin`:

- **rustc** — the compiler. Turns Rust source into a native binary, the same job `gcc` does for C. Built on the LLVM backend (the same backend `clang` uses).
- **cargo** — the build tool and package manager. This is the command you type all day.
- **clippy** — the linter (run as `cargo clippy`). Catches likely mistakes and suggests more idiomatic code.
- **rustfmt** — the code formatter (run as `cargo fmt`). One official style, so formatting is never an argument.

You also get `rust-std` (the standard library, precompiled for your platform) and local copies of the documentation.

Verify the install

```
rustc --version    # e.g. rustc 1.96.0 (ac68faa20 2026-05-25)
cargo --version    # e.g. cargo 1.96.0 (30a34c682 2026-05-25)
```

If both print a version, you are ready.

Keeping Rust up to date

Rust ships a new stable release every six weeks. Updating is one command:

```
rustup update      # update all installed toolchains to the latest
rustup self update # update rustup itself
```

C vs Rust. In C, upgrading your compiler is an OS-level event (a new distro package, maybe a new `gcc` you build yourself). In Rust, `rustup update` swaps in the new compiler in seconds, per user, and you can keep several versions side by side.

Components: adding clippy, rustfmt, and more

If a component is missing, or you used a minimal install profile, add it with `rustup component add`:

```
rustup component add clippy rustfmt
rustup component add rust-src          # std source, useful for IDE tooling
rustup component add rust-analyzer    # the language server (Chapter 24 - Tooling)
```

Targets: cross-compiling

A **target** is a platform you compile *for* (a CPU + OS combination, written as a “target triple” like `aarch64-unknown-linux-gnu`). In C this means installing a whole cross-toolchain. In Rust you add the target’s standard library and Cargo does the rest:

```
rustup target add aarch64-unknown-linux-gnu      # add a target
rustup target list                               # see all available targets
cargo build --release --target aarch64-unknown-linux-gnu
```

You still need a linker for that platform, but the Rust side is one command.

Toolchains: stable, beta, nightly

Rust has three release channels:

- **stable** — what you use for almost everything. New stable every six weeks.
- **beta** — the next stable, for testing your code early.
- **nightly** — built every night; required for a few unstable, experimental features.

```
rustup toolchain install nightly      # install the nightly toolchain
rustup default stable                # set the default toolchain
rustup override set nightly          # use nightly in THIS directory only
cargo +nightly build                  # one-off: build with nightly
```

Rule of thumb. Stay on **stable** unless a specific tool or feature demands nightly. Most production Rust is written on stable.

A command-by-command tour of cargo

These are the commands you will actually use. Each entry is the one-line purpose plus a real example.

cargo new / cargo init — start a project

`cargo new` creates a new project directory; `cargo init` turns the *current* directory into one.

```
cargo new hello      # make ./hello, a binary project (src/main.rs)
cargo new mylib --lib # make a library project (src/lib.rs) instead
cargo init            # turn an existing directory into a project
```

`cargo new hello` produces:

```
hello/
Cargo.toml      # project manifest (metadata + dependencies)
.gitignore      # ignores /target by default
src/
  main.rs       # your code; a "Hello, world!" is generated for you
```

C vs Rust. There is no equivalent to “scaffold a buildable C project with a Makefile and a `.gitignore`” in standard C. `cargo new` gives you a project that builds and runs immediately.

cargo build — compile the project

```
cargo build      # debug build -> target/debug/hello
cargo build --release # optimized build -> target/release/hello
```

A **debug** build compiles fast and keeps debug info and runtime checks (for example, integer-overflow checks panic in debug). A **release** build turns on optimizations (`-O`-style) and is much faster at

runtime but slower to compile. Everything goes into the `target/` directory; you never clean up `.o` files by hand.

C vs Rust. Debug vs release is `-O0 -g` vs `-O2` in C, but you do not manage the flags or output paths yourself — `cargo build` and `cargo build --release` handle it, and the two builds live in separate folders so they never clash.

cargo run — build and run in one step

```
cargo run                # build (if needed) then run the binary
cargo run --release      # same, optimized
cargo run -- --input file.txt # pass args AFTER -- to YOUR program
```

Cargo rebuilds only if something changed, then runs the result. Anything after `--` is passed to your program, not to cargo.

cargo check — type-check without building a binary

```
cargo check          # analyze the code; report errors; produce NO executable
```

`cargo check` runs the full front end of the compiler — parsing, type-checking, and **borrow-checking** — but stops before code generation and linking, the expensive back-end stages. That is why it is much faster than `cargo build`. It is what you run in a tight edit loop to ask “does this compile?” without waiting for an actual binary.

C vs Rust. The closest C analog is `gcc -fsyntax-only`, which parses but does not generate code. `cargo check` goes further: it also runs the borrow checker, so it catches Rust’s ownership errors too. Run `cargo check` constantly while editing; save `cargo build` for when you need to actually run the program.

cargo test — run tests

```
cargo test          # compile and run all tests
cargo test parse    # run only tests whose name contains "parse"
```

Test support is built into the language and the tool: functions marked `#[test]`, integration tests in `tests/`, and even code examples inside doc comments all run with this one command. Full details are in Chapter 23 — Testing.

cargo fmt — format code (rustfmt)

```
cargo fmt          # reformat the whole project to the standard style
cargo fmt --check   # report unformatted files without changing them (CI)
```

There is one official style, so code looks the same everywhere. Run it before you commit. See Chapter 24 — Tooling.

cargo clippy — lint the code

```
cargo clippy        # extra warnings beyond the compiler's
cargo clippy -- -D warnings # treat warnings as errors (good for CI)
```

Clippy is the linter. It flags likely bugs, slow patterns, and un-idiomatic code, often with a suggested fix. Think `clang-tidy`, but tuned for Rust and very beginner-friendly. See Chapter 24 — Tooling.

cargo add — add a dependency from crates.io

```
cargo add serde                                # add the latest serde to Cargo.toml
cargo add serde --features derive              # enable an optional feature
cargo add rand@0.8                             # pin a version requirement
```

A **crate** is a Rust library (or binary) package. **crates.io** is the central public registry, like a language-wide package repository. `cargo add` edits your `Cargo.toml` and the next build downloads and compiles the crate.

C vs Rust. In C, adding a library means finding it, installing the right `-dev` package system-wide (or vendoring the source), then editing your Makefile for include paths and `-l` flags. `cargo add serde` does the whole thing, and the dependency is recorded *in the project*, not in your OS.

cargo doc — build API documentation

```
cargo doc --open                               # build docs for your crate + its deps, open in a browser
cargo doc --no-deps                           # only your crate
```

Cargo generates browsable HTML from your code and your doc comments — like Doxygen, but built in and used by the whole ecosystem. The same tooling powers <https://docs.rs>.

cargo bench — run benchmarks

```
cargo bench                                   # run functions marked as benchmarks
```

Runs performance benchmarks. The built-in harness needs nightly; on stable, the common choice is the `criterion` crate. See Chapter 24 — Tooling.

cargo update — update dependencies within your version rules

```
cargo update                                # bump dependencies to newer compatible versions
cargo update -p serde                       # update just one
```

This refreshes `Cargo.lock` to newer versions that still satisfy the requirements in `Cargo.toml` (more on this in Chapter 22 — Cargo, Crates, and Workspaces).

cargo clean — delete build artifacts

```
cargo clean                                # remove the entire target/ directory
```

Like `make clean`, but it simply deletes `target/`. The next build is a full rebuild.

cargo install — install a binary crate as a command

```
cargo install ripgrep                        # build and install the `rg` command into ~/.cargo/bin
cargo install --path .                      # install the binary from the current project
```

This compiles a program from crates.io and drops the executable in `~/.cargo/bin` so you can run it as a normal command. It is how many Rust command-line tools are distributed.

cargo tree — show the dependency graph

```
cargo tree           # print dependencies as a tree
cargo tree -i regex  # show what pulls in the `regex` crate (inverted)
```

Useful for understanding *why* some transitive crate is in your build.

The daily workflow cheat-box

These are the six commands you type all day. Keep them in muscle memory.

```
cargo check          # fast: does it compile? (run this most often)
cargo run            # build and run during development
cargo test           # run the tests
cargo fmt            # format before committing
cargo clippy         # lint for bugs and style
cargo build --release # the optimized binary, when you ship
```

The anatomy of Cargo.toml

Cargo.toml is the project **manifest**: the file Cargo reads to know your project’s name, edition, and dependencies. It is written in TOML, a simple key-value format. A fresh **cargo new** gives you:

```
[package]
name = "hello"
version = "0.1.0"
edition = "2024"
```

[dependencies]

- [package] — metadata about this project. **name** is the crate name, **version** follows semantic versioning, and **edition** selects the language edition (more below).
- [dependencies] — the crates you depend on. **cargo add** fills this in for you, for example **serde = "1"**.

Alongside it, Cargo maintains **Cargo.lock**, a file that records the *exact* version of every dependency (and transitive dependency) that was resolved. You edit **Cargo.toml** (your requirements); Cargo writes **Cargo.lock** (the exact resolution). Commit **Cargo.lock** for applications so builds are reproducible. The full story — version requirements, features, workspaces — is Chapter 22 — Cargo, Crates, and Workspaces.

C vs Rust. Cargo.toml is your Makefile *and* your dependency list in one small, declarative file. Cargo.lock is the reproducible “exactly these versions” record that, in C, you would have to build yourself with pinned submodules or a vendor directory.

Calling rustc directly (rare)

You almost never invoke the compiler by hand, but you can, and it is handy for a single throwaway file with no project around it:

```
rustc --edition 2024 main.rs    # compile main.rs -> ./main
./main
```

Without `--edition`, `rustc` defaults to the 2015 edition, so pass it explicitly. For anything bigger than one file, use Cargo — it manages dependencies, editions, profiles, and output paths for you.

Where everything lives on disk

Three locations matter. None require root; everything is per user or per project.

```
~/.rustup/                                # managed by rustup
  toolchains/                             #   the actual rustc, cargo, std for each version
    stable-.../
    nightly-.../
  settings.toml                           #   default toolchain, overrides

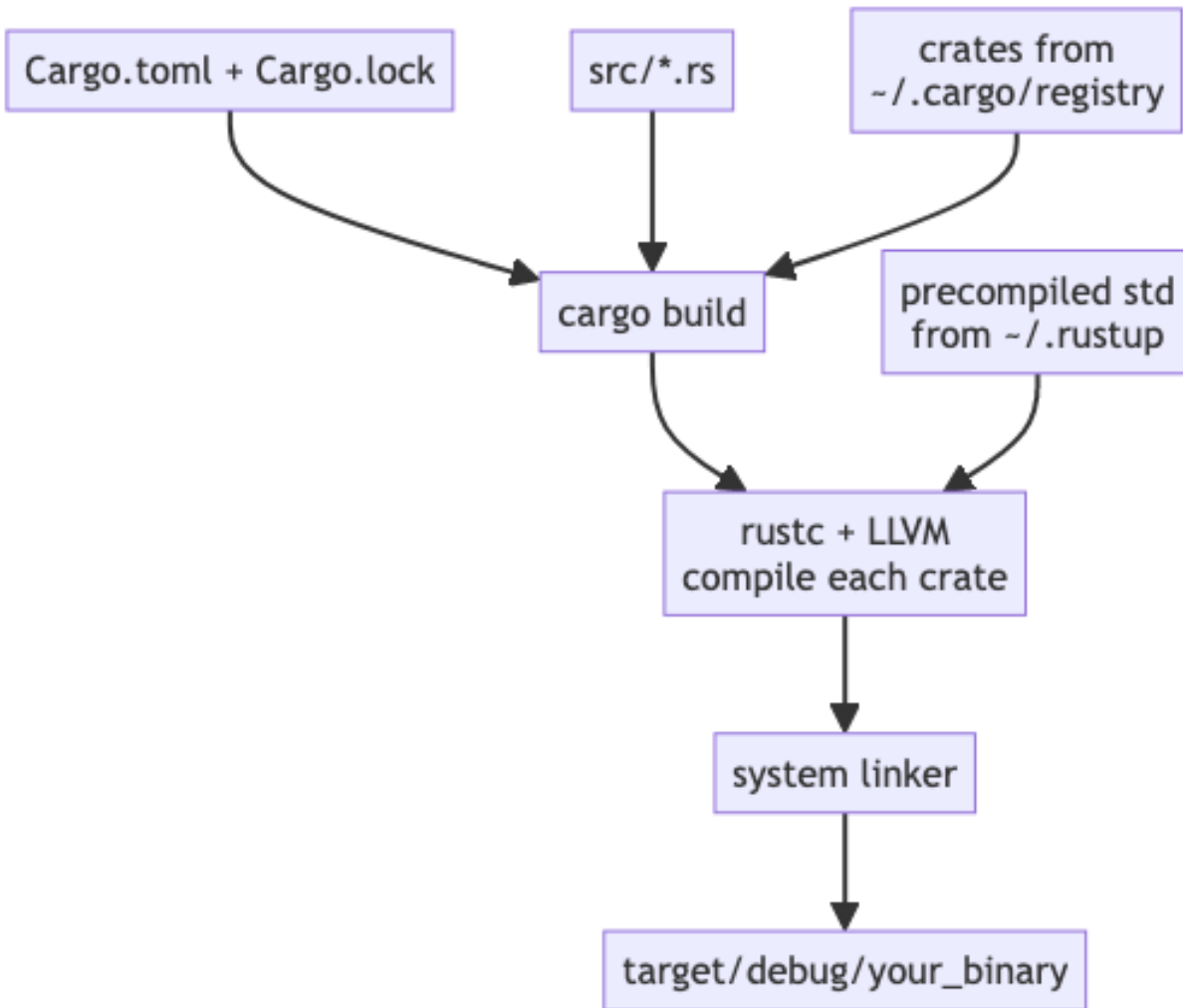
~/.cargo/                                 # managed by cargo
  bin/                                    #   rustc, cargo, clippy, rustfmt, + cargo install'd tools
                                           #   ** put this on your PATH **
  registry/                              #   downloaded + cached crates from crates.io
  config.toml                             #   global cargo settings

my_project/
  Cargo.toml                             # the manifest you edit
  Cargo.lock                             # exact resolved versions (generated)
  src/                                    # your source code
  target/                                # ALL build output (debug/, release/); safe to delete
```

- `~/.rustup` holds the toolchains (compilers and standard libraries).
- `~/.cargo` holds the downloaded-crate cache (`registry/`) and installed binaries (`bin/`). Make sure `~/.cargo/bin` is on your `PATH`.
- `target/` is per project. It is your `*.o` and final binaries; add it to `.gitignore` (the generated `.gitignore` already does). Deleting it just forces a rebuild.

How cargo build drives rustc

`cargo` is the conductor; `rustc` does the compiling; the system linker produces the final binary. The standard library is already compiled and comes from the toolchain.



The key point: you talk to **cargo**, and cargo decides what needs recompiling, fetches dependencies, calls **rustc** with the right flags for each crate, and runs the linker. This is the work a hand-written Makefile does in C — here it is automatic and incremental.

Editions in one paragraph

An **edition** is a named snapshot of the language’s syntax and idioms: 2015, 2018, 2021, and 2024. Editions let Rust make small breaking changes (like turning a word into a keyword) without breaking old code: each crate declares its edition in **Cargo.toml**, and crates of different editions link together fine. This book targets **edition 2024**. Think of it like a C standard level (**-std=c11** vs **-std=c17**), except that mixing editions across libraries is fully supported and normal.

Key takeaways

- **rustup** installs and updates the toolchain; **cargo** builds your project; **rustc** is the compiler underneath that you rarely call directly.
- Install with the one-line `curl ... | sh`; verify with `rustc --version` and `cargo --version`; update with `rustup update`.

- A default install includes `rustc`, `cargo`, `clippy`, and `rustfmt`. Add pieces with `rustup component add` and cross-compile targets with `rustup target add`.
- One tool, `cargo`, replaces C’s separate compiler, `make`, package manager, formatter, and linter. The daily six: `check`, `run`, `test`, `fmt`, `clippy`, `build --release`.
- `cargo check` is faster than `cargo build` because it skips code generation and linking; run it constantly while editing.
- Files live in `~/.rustup` (toolchains), `~/.cargo` (crate cache + installed binaries; put `~/.cargo/bin` on `PATH`), and per-project `target/`.
- `Cargo.toml` declares your requirements; `Cargo.lock` pins exact versions for reproducible builds. This book uses **edition 2024**.

Watch out (gotchas for C programmers)

- **Add `~/.cargo/bin` to your `PATH`.** If `cargo` is “not found” after install, this is almost always why. Run `source "$HOME/.cargo/env"` or open a new shell.
- **Bare `rustc` defaults to edition 2015.** Pass `--edition 2024` when you compile a single file by hand.
- **`cargo run -- args`:** arguments for *your* program go after `--`. Without it, `cargo` tries to interpret them.
- **Debug builds are slow at runtime.** They are not optimized and include overflow checks. Always benchmark and ship with `--release`.
- **Do not commit `target/`.** It is large and regenerable. The generated `.gitignore` already excludes it.
- **`cargo check` does not produce a binary.** When you need to actually run the program, use `cargo run` or `cargo build`.

Interview questions

Q: What is the difference between `rustup`, `cargo`, and `rustc`? A: `rustup` is the toolchain manager — it installs, updates, and switches between Rust versions and cross-compilation targets. `cargo` is the build tool and package manager you use day to day to build, test, and manage dependencies. `rustc` is the compiler itself; `cargo` calls it for you, so you rarely run it directly.

Q: Why is `cargo check` faster than `cargo build`? A: `cargo check` runs the compiler’s front end — parsing, type-checking, and borrow-checking — and then stops. It skips the expensive back-end stages: code generation (LLVM optimization) and linking. It reports the same errors but produces no executable, so it is much faster for a tight edit-and-check loop.

Q: What is the difference between `Cargo.toml` and `Cargo.lock`? A: `Cargo.toml` is the manifest you write: it lists your dependency *requirements* (for example, `serde = "1"`) plus package metadata and the edition. `Cargo.lock` is generated by `cargo` and records the *exact* resolved version of every direct and transitive dependency, so builds are reproducible. You edit the first; `cargo` maintains the second.

Q: How does Rust’s toolchain replace the separate tools a C project needs? A: One install via `rustup` and one command, `cargo`, cover the whole workflow: `cargo build` replaces `make`/`CMake`, `cargo add` plus `crates.io` replaces the system package manager and vendored libraries, `cargo fmt` replaces `clang-format`, `cargo clippy` replaces `clang-tidy`, `cargo test` provides the test harness, and `cargo doc` replaces `Doxygen` — all driven through the same project manifest.

Q: What is a Rust “edition,” and how is it like or unlike a C standard? A: An edition (2015, 2018, 2021, 2024) is a named snapshot of the language’s syntax and idioms, declared per crate in `Cargo.toml`. Like a C standard (`-std=c17`) it selects a language level. Unlike C standards, editions can mix freely across linked crates — a 2024 crate can depend on a 2018 crate — so the ecosystem can adopt new editions gradually without breaking old code.

Try it

1. Run `cargo new playground` `&& cd playground`, then `cargo run`. Note the `Compiling/Finished/Running` lines and the new `target/` directory.
2. Edit `src/main.rs` and run `cargo check`, then `cargo build`. Time both (`time cargo check` vs `time cargo build --release`) and compare.
3. Run `cargo add rand`, look at how `Cargo.toml` and `Cargo.lock` changed, then run `cargo tree` to see what `rand` pulled in.

Chapter 3 — Program Structure: Crates, Modules, and Visibility

What you’ll learn. How a Rust program is organized without headers: what a crate and a package are, how `mod` builds a tree of modules, how paths and `use` name things, and why everything is private by default — the opposite of C.

No headers, no `#include`, no forward declarations

In C, a program is a set of **translation units**: each `.c` file is compiled on its own, sees only what it has `#included`, and the linker joins the results. You manage visibility by hand with `extern` (share a symbol) and `static` (hide one), and you write **header files** to declare what one file exposes to another. You also order things carefully or write forward declarations, because the compiler reads top to bottom.

Rust throws all of that out. The compiler reads a whole **crate** at once, so:

- There are **no header files**. You declare a function or struct in exactly one place.
- There is **no `#include`**. You bring names into scope with `use`.
- There are **no forward declarations** and **order does not matter**. A function can call another defined later in the file; a module can use a type defined below it.

C vs Rust. A C header is a promise repeated in two places (the `.h` and the `.c`), which can drift out of sync. Rust has a single source of truth per item, and the compiler’s whole-crate view means it already knows every declaration.

Crates and packages

Two words you must keep straight:

- A **crate** is one unit of compilation: either a **binary** (an executable with a `main`) or a **library** (code other crates use). The compiler processes one crate as a whole. This is the closest thing Rust has to C’s “the program the linker produces,” except a library crate is one unit too, not a pile of `.o` files.
- A **package** is what `cargo new` creates: a directory with a `Cargo.toml` and one or more crates. A package has **at most one library crate** and **any number of binary crates**.

Each crate has a **root** file — the file the compiler starts from:

File	Crate kind	Role
<code>src/main.rs</code>	binary	the default binary crate root; has <code>fn main()</code>
<code>src/lib.rs</code>	library	the library crate root
<code>src/bin/foo.rs</code>	binary	an extra binary crate named <code>foo</code>

A common, idiomatic layout is a library crate (`src/lib.rs`) with all the logic plus a thin binary (`src/main.rs`) that calls into it. That way the logic is testable and reusable, and the binary is just a front door.

Mental model. A crate root is like the single `.c` file you would pass to the compiler if that file could pull in every other file of the program itself. From the root, `mod` statements pull in the rest of the tree.

Modules with `mod`

A **module** is a named container for items (functions, structs, enums, other modules). Modules give you a namespace and a visibility boundary *inside* a crate. They are not separate compilation units — they are organization within one crate.

C vs Rust. In C, the file *is* the unit of scope and you split code into files. In Rust, the **module** is the unit of scope, and modules do not have to match files one-to-one — though it is common and tidy to make them match.

Inline modules

The simplest module is written inline with a body:

```
mod math {
    pub fn square(x: i32) -> i32 {
        x * x
    }

    pub mod trig {
        pub fn deg_to_rad(d: f64) -> f64 {
            d * std::f64::consts::PI / 180.0
        }
    }
}

fn main() {
    let s = math::square(5);
    let r = math::trig::deg_to_rad(180.0);
    println!("{s} {r:.4}");
}
```

Modules can nest, forming a tree. Note the `pub` keywords — without them the items would be private to the module and `main` could not reach them. More on that below.

File-based modules

For real code you put modules in their own files. Writing `mod foo;` (with a semicolon, no body) tells the compiler to load the module's body from another file. The compiler looks for **either**:

- `foo.rs` (next to the current file), or
- `foo/mod.rs` (the older style; still valid).

So this crate root:

```
// src/main.rs
mod geometry;           // load src/geometry.rs
mod io_utils;           // load src/io_utils.rs

fn main() {
    let a = geometry::area_of_circle(2.0);
    io_utils::log(&format!("area = {a:.3}"));
}
```

pairs with:

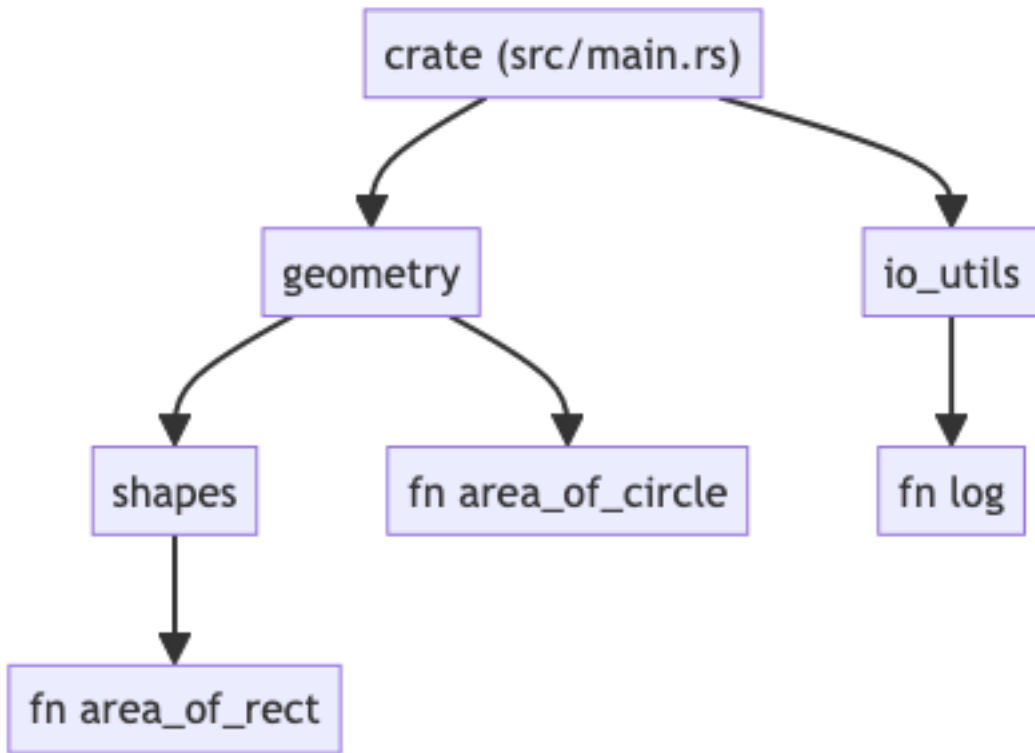
```
// src/geometry.rs
pub fn area_of_circle(r: f64) -> f64 {
    std::f64::consts::PI * r * r
}
```

Submodules nest as folders. A module `geometry` with a child `shapes` lives in either `src/geometry/shapes.rs` (with `mod shapes;` written inside `src/geometry.rs`) or the older `src/geometry/mod.rs` form.

C vs Rust. `mod foo;` looks a little like `#include "foo.h"`, but it is not textual inclusion. It declares that module `foo` exists and tells the compiler where its source is. The contents are parsed as their own module, not pasted in.

The module tree

Every crate has an implicit root module called `crate`, and `mod` declarations build a tree under it. For the files above, the tree is:



And here is how files on disk map to the module path each one provides:

src/main.rs	->	crate	(the binary crate root)
src/geometry.rs	->	crate::geometry	
src/geometry/shapes.rs	->	crate::geometry::shapes	
src/io_utils.rs	->	crate::io_utils	

Paths: naming items across the tree

To use an item you name it by its **path**, with `::` between segments (like `.` in a filesystem path, or `::` in C++). Paths can be absolute or relative:

- **Absolute** paths start from the crate root with `crate::`, for example `crate::geometry::area_of_circle`.
- **Relative** paths start from the current module, optionally using:
 - `self::` — this module (explicit relative start).
 - `super::` — the parent module (like `..` in a filesystem).

```

mod network {
    pub fn connect() {
        println!("connecting");
    }

    pub mod client {
        pub fn start() {
            super::connect();           // call parent's connect()
            crate::network::connect();  // same call, absolute path
            self::helper();              // a sibling in this module
        }
    }
}
  
```



```

    }

    fn helper() {}
}

fn main() {
    network::client::start();
}

```

Rule of thumb. Use **absolute** paths (`crate::...`) for items from elsewhere in the crate, and `super::`/`self::` for things close by. Absolute paths survive moving code around better.

To name items from an **external** crate (one you added with `cargo add`), start the path with the crate’s name instead of `crate`, for example `rand::random()`. The standard library is the crate `std`, as in `std::collections::HashMap`.

use: bringing names into scope

Writing full paths everywhere is noisy. The `use` keyword creates a shortcut in the current scope, so you can write the last segment alone. This is Rust’s answer to repeating long names — not a textual include.

```

use std::collections::HashMap;

fn main() {
    let mut counts: HashMap<String, i32> = HashMap::new(); // no std::collections::
    counts.insert("a".to_string(), 1);
    println!("{counts:?}");
}

```

Aliases with as

When two names would clash, or a name is long, rename it on import:

```

use std::io::Result as IoResult;
use std::fmt::Result as FmtResult;

```

Grouped and nested use

Combine several imports from the same root to keep the top of a file tidy:

```

use std::collections::{HashMap, HashSet, BTreeMap};
use std::io::{self, Read, Write}; // `self` imports std::io itself too

```

Glob imports

`use foo::*` brings in *everything* public from `foo`. Use it sparingly — it hides where names come from — but it is common in two places: a module’s tests (`use super::*;`) and crates that publish a `prelude` module.

```
use std::collections::*;    // pulls in HashMap, HashSet, ... (use with care)
```

Re-exporting with `pub use`

A plain `use` is private: it is a shortcut for the current module only. `pub use` both imports a name *and* re-exports it, so callers of your module can reach it through your path. This lets you build a clean public API that hides your internal layout.

```
mod internal {
    pub mod deep {
        pub fn important() {}
    }
}

pub use internal::deep::important;    // now callers can use crate::important

fn main() {
    important();
}
```

C vs Rust. `use` is closest to nothing in C — there is no “import a name” statement; you just rely on the header you included. `pub use` is how Rust builds a flat, friendly public API on top of a nested private structure, something C libraries fake by carefully choosing what goes in the public header.

Visibility: private by default

Here is the biggest shift from C, and it is the reverse of what you are used to.

C vs Rust. In C, a top-level symbol is **global by default**; you write `static` to *hide* it from other translation units. In Rust, every item is **private by default**; you write `pub` to *expose* it. Rust is “deny by default”; C is “allow by default.”

Private means “visible only within the current module and its descendants.” To make an item usable from outside its module, mark it `pub`.

```
mod bank {
    pub struct Account {
        pub owner: String,    // public field
        balance: i64,         // PRIVATE field, even though the struct is pub
    }

    impl Account {
        pub fn new(owner: &str) -> Account {
            Account { owner: owner.to_string(), balance: 0 }
        }

        pub fn deposit(&mut self, amount: i64) {
            self.balance += amount;    // ok: same module
        }
    }
}
```

```

        pub fn balance(&self) -> i64 {
            self.balance
        }
    }

fn main() {
    let mut a = bank::Account::new("Ada");
    a.deposit(100);
    println!("{}", a.owner, a.balance()); // owner + balance() are pub
}

```

This **does not** compile, because struct fields are private by default even when the struct is `pub`:

```

// COMPILE ERROR: field `balance` of struct `Account` is private
mod bank {
    pub struct Account {
        pub owner: String,
        balance: i64,
    }
}

fn main() {
    let a = bank::Account { owner: "Ada".into(), balance: 100 }; // error[E0451]
    println!("{}", a.owner);
}

```

Watch out. Making a `struct pub` does **not** make its fields `pub`. Each field needs its own `pub`. This is deliberate: it lets a type keep invariants (like “balance is never set directly”) the same way a C module hides a struct behind an opaque pointer and accessor functions.

Finer-grained visibility

Between fully private and fully public, Rust offers restricted `pub`:

Marker	Visible to	Closest C idea
(none)	this module and its child modules	a static symbol / file-local
<code>pub</code>	everyone, including other crates	a non- static symbol in a header
<code>pub(crate)</code>	anywhere in <i>this</i> crate, but not outside	a symbol shared across <code>.c</code> files but not in the public header
<code>pub(super)</code>	the parent module	a symbol shared with one related file
<code>pub(in path)</code>	a specific ancestor module	(no direct C analog)

`pub(crate)` is the workhorse: “share this throughout my own crate, but do not make it part of my public API.”

```
mod engine {
    pub(crate) fn internal_tick() {}    // any module in this crate may call it
    pub fn run() {}                    // part of the crate's public API
    fn secret() {}                     // only this module + children
}
```

Mental model. Think of three rings: the item's own module (private), the whole crate (`pub(crate)`), and the outside world (`pub`). You open exactly the ring you need, no wider.

The prelude: what you get for free

Even though there is no `#include`, common types and traits are available in every file without any `use`. That set is the **prelude** — a small list the compiler imports automatically into every module. It is why you can write `Vec`, `String`, `Option`, `Result`, `println!`, `Box`, and `Clone` with no import.

In edition 2024 the prelude also includes `Future` and `IntoFuture` (used by `async`). Anything *not* in the prelude — like `HashMap` — you bring in with `use`.

C vs Rust. The prelude is a bit like a header that is always implicitly included, except it is curated by the language and intentionally tiny, so it does not bloat compile times the way a giant always-included header would in C.

Using an external crate

After you run `cargo add rand` (Chapter 2 — Installing Rust and the Cargo Toolchain), the crate is available by name. In modern editions you do **not** write `extern crate`; you just `use` it or name it by path.

```
// after: cargo add rand    (this needs the rand dependency to compile)
use rand::RngExt;

fn main() {
    let mut rng = rand::rng();
    let roll: u8 = rng.random_range(1..=6);
    println!("you rolled {roll}");
}
```

Watch out. Older tutorials show `extern crate rand`; at the top of the file. That was required in edition 2015 and is unnecessary (and un-idiomatic) in editions 2018 and later. Just add the dependency and `use` it.

Key takeaways

- A **crate** is one compilation unit (a binary or a library); a **package** has a `Cargo.toml` and one or more crates. `src/main.rs` is a binary root, `src/lib.rs` a library root.
- The compiler sees the **whole crate at once**: no headers, no `#include`, no forward declarations, and order does not matter.
- `mod` builds a tree of modules. `mod foo { ... }` is inline; `mod foo`; loads `foo.rs` or `foo/mod.rs`.

- Name items by **path** with `:: crate::` (absolute), `self::`/`super::` (relative). `use` makes a shortcut; `as` aliases; `{}` groups; `pub use` re-exports.
- **Everything is private by default** — the opposite of C. `pub` exposes an item; `pub(crate)`/`pub(super)` restrict it. Struct fields are private even in a `pub` struct.
- The **prelude** auto-imports common items; external crates are used by name with no `extern crate` in modern editions.

Watch out (gotchas for C programmers)

- **Default is private, not global.** This is the reverse of C’s non-`static` symbols. Forgetting `pub` is the most common “why can’t I call this?” error.
- **A `pub` struct does not make fields `pub`.** Each field needs its own `pub`.
- **`mod foo`; is not `#include`.** It declares a module and points at its file; it does not paste text. There are no header/source pairs to keep in sync.
- **`use` does not compile or copy code.** It only creates a name shortcut in the current scope; the item is compiled once wherever it is defined.
- **No `extern crate` needed.** After `cargo add`, just `use` the crate. The old `extern crate` line is legacy.
- **Module `path` is not the file name you `cargo run`.** The crate root is `main.rs/lib.rs`; a file’s module path comes from the `mod` chain that loads it, not from being passed on a command line.

Interview questions

Q: What is the difference between a crate and a package? A: A crate is a single compilation unit — either a binary (with `main`) or a library — that the compiler processes as a whole. A package is a Cargo project: a directory with a `Cargo.toml` that contains one or more crates, with at most one library crate and any number of binary crates.

Q: Rust has no headers. How does one part of a program see another? A: The compiler reads the entire crate at once, so every declaration is already known regardless of order. You organize code into modules with `mod`, refer to items by path (`crate::`, `super::`, `self::`), and use `use` to create shortcuts. There are no headers, no `#include`, and no forward declarations.

Q: How does Rust’s default visibility differ from C’s, and how do you control it? A: In C a top-level symbol is global by default and you write `static` to hide it. In Rust every item is private by default and you write `pub` to expose it. Finer control comes from `pub(crate)` (visible throughout the crate but not outside) and `pub(super)` (visible to the parent module).

Q: If a struct is declared `pub`, can other modules construct it directly? A: Not necessarily. Marking the struct `pub` exposes the type, but its fields are still private unless each is also marked `pub`. If any field is private, outside code cannot build the struct with a struct literal and must use a public constructor function instead — this is how a type protects its invariants.

Q: What does `pub use` do and why is it useful? A: `pub use` imports a name and simultaneously re-exports it from the current module. It lets you present a clean, flat public API while keeping a deeply nested private module structure inside — callers reach the item through your chosen path instead of your internal one.

Try it

1. In a `cargo new` project, add `mod math { pub fn double(x: i32) -> i32 { x * 2 } }` and call `math::double(21)` from `main`. Remove the `pub` and read the error.
2. Move the module into `src/math.rs`, replace the inline block with `mod math;`, and confirm it still builds.
3. Add a `pub struct Point { x: i32, y: i32 }` in a module and try to build one from `main` with a struct literal. Watch the private-field error, then add `pub` to the fields to fix it.

Chapter 4 — Variables, Types, and Constants

What you'll learn. How `let` bindings work and why variables are immutable by default, the difference between shadowing and mutation, Rust's exact-sized scalar and compound types, why there are no implicit conversions, and how `const` and `static` differ.

Variables with `let`

In C you declare a variable by writing its type and name: `int x = 5;`. In Rust you write `let`, and the type is usually inferred:

```
fn main() {
    let x = 5;           // type inferred as i32
    let y: i32 = 5;      // type written explicitly
    println!("{x} {y}");
}
```

The keyword `let` introduces a **binding**: it ties a name to a value. The compiler figures out the type from the value on the right, so you rarely write the type. When you want to be explicit, or when inference cannot decide, you add `: Type` after the name, as in `let y: i32 = 5`.

C vs Rust. C: `int x = 5;` (type first). Rust: `let x = 5;` (type inferred, or written after the name as `let x: i32 = 5;`). The name comes before the type in Rust, the reverse of C.

Immutable by default

Here is the first big surprise for a C programmer. In C, every variable is mutable: you can assign to it again whenever you like. In Rust, a `let` binding is **immutable** by default — once set, you cannot change it.

```
// COMPILER ERROR: cannot assign twice to immutable variable `x`
fn main() {
    let x = 5;
    x = 6;           // error[E0384]: cannot assign twice to immutable variable `x`
    println!("{x}");
}
```

To make a variable changeable, add the keyword `mut` (short for “mutable”):

```
fn main() {
    let mut x = 5;
```

```

x = 6;           // ok: x is mutable
x += 1;
println!("{x}"); // prints 7
}

```

This is the opposite default from C. In C you write `const` to *forbid* change; in Rust you write `mut` to *allow* it. The default is the safe one.

C vs Rust. C is mutable by default; you add `const` to lock a value. Rust is immutable by default; you add `mut` to unlock it. The defaults are reversed.

Rule of thumb. Start without `mut`. Add it only when the compiler tells you that you need it. Code with fewer `mut` bindings is easier to reason about, because you know those values never change.

Shadowing vs mutation

Rust lets you declare a new binding with the **same name** as an old one. This is called **shadowing**. The new binding hides the old one from that point on:

```

fn main() {
    let x = 5;
    let x = x + 1; // a NEW binding, also named x, value 6
    let x = x * 2; // another NEW binding, value 12
    println!("{x}"); // prints 12
}

```

Shadowing is not mutation. Each `let` creates a brand-new variable that happens to reuse the name. The old value is still there underneath; it is just hidden. Because it is a new variable, it can even have a **different type**:

```

fn main() {
    let spaces = "  "; // spaces is &str (text)
    let spaces = spaces.len(); // spaces is now usize (a number)
    println!("{spaces}"); // prints 3
}

```

Compare that with `mut`, which keeps the *same* variable and so cannot change its type:

```

// COMPILE ERROR: mismatched types
fn main() {
    let mut spaces = "  ";
    spaces = spaces.len(); // error[E0308]: expected `&str`, found `usize`
}

```

Mental model. Mutation changes the value inside one box. Shadowing throws the old box away and puts a new box (possibly a different shape) under the same label. C has no shadowing in the same scope — a second declaration of the same name is an error.

Watch out. Shadowing is easy to confuse with mutation. `let x = ...` again is a new variable; `x = ...` (no `let`) is assignment and needs `mut`. Reach for shadowing when you transform a value into a new form (often a new type); reach for `mut` when one value genuinely changes over time.

Type inference and annotations

Rust uses **type inference**: the compiler reads how a value is created and used, then assigns a type. This is more powerful than C's, because it can look at later uses, not just the initializer. Sometimes that means you must help it:

```
fn main() {  
    let guess: u32 = "42".parse().expect("not a number"); // annotation needed  
    println!("{guess}");  
}
```

Here `parse` can produce many number types, so the compiler cannot guess. The `: u32` tells it which one. Without the annotation you get a compile error asking you to specify the type.

Scalar types: integers

Rust's integer types state their size and signedness **in the name**. There is no guessing about how many bits an `int` has, the way there is in C.

Length	Signed	Unsigned
8-bit	<code>i8</code>	<code>u8</code>
16-bit	<code>i16</code>	<code>u16</code>
32-bit	<code>i32</code>	<code>u32</code>
64-bit	<code>i64</code>	<code>u64</code>
128-bit	<code>i128</code>	<code>u128</code>
pointer-sized	<code>isize</code>	<code>usize</code>

```
i8   : [#]           1 byte  
i16  : [##]          2 bytes  
i32  : [####]         4 bytes  <-- default  
i64  : [#####]       8 bytes  
i128 : [#####]       16 bytes  
isize/usize: pointer-sized (8 bytes on a 64-bit machine)
```

The `i` means signed (it can be negative), the `u` means unsigned (zero or positive). The number is the width in bits. The two odd ones are `isize` and `usize`: they are **pointer-sized**, meaning they match the address width of the machine (4 bytes on a 32-bit target, 8 bytes on a 64-bit target). This is like `intptr_t/uintptr_t` and `ptrdiff_t/size_t` in C.

`usize` is special: Rust uses it for **all indexing and lengths**. The length of an array or a `Vec`, and any value you use to index into one, is a `usize`. This is the same reason C uses `size_t` for `sizeof` and array sizes.

Watch out. If you do not annotate an integer and nothing forces a type, Rust defaults to `i32`. In C the default integer type is also `int`, usually 32-bit, so this will feel familiar — but in Rust it is a precise, fixed-width `i32`, not an “at least 16 bits” `int`.

Integer literals

You can write numbers in several bases and add a `_` as a digit separator to make big numbers readable. You can also glue the type onto the end as a **suffix**:


```
fn main() {
    let a = 1_000_000;    // decimal, underscores ignored by the compiler
    let b = 0xff;         // hex
    let c = 0o77;         // octal
    let d = 0b1010_0001;  // binary
    let e = b'A';         // a byte literal: the u8 value 65
    let f = 1_000u64;     // suffix: this literal is a u64
    println!("{a} {b} {c} {d} {e} {f}");
}
```

The suffix (`1_000u64`) is a quick way to pick a type without a separate annotation. It is handy when a literal alone would be ambiguous.

Integer overflow

In C, signed integer overflow is **undefined behavior**, and unsigned overflow wraps silently. Rust is stricter and more predictable:

- In a **debug** build, an overflowing `+`, `-`, or `*` **panics** (the program stops with an error message) instead of silently giving a wrong answer.
- In a **release** build (`--release`), it **wraps around** using two's complement, like C's unsigned arithmetic — fast, but you opted into it.

Relying on either default is usually a mistake. When you actually want a specific overflow behavior, say so with a method:

```
fn main() {
    let big: u8 = 250;

    println!("{}", big.wrapping_add(10));    // 4 (wraps around 256)
    println!("{:?}", big.checked_add(10));    // None (would overflow)
    println!("{:?}", big.checked_add(3));     // Some(253)
    println!("{}", big.saturating_add(10));   // 255 (clamps at the max)
    let (v, overflowed) = big.overflowing_add(10);
    println!("{v} {overflowed}");            // 4 true
}
```

- `wrapping_*` — wrap around on overflow (the C unsigned behavior).
- `checked_*` — return `Option: Some(result)` or `None` if it overflowed.
- `saturating_*` — stop at the type's minimum or maximum instead of wrapping.
- `overflowing_*` — return the wrapped result plus a `bool` flag.

C vs Rust. C: signed overflow is undefined behavior, a real source of bugs. Rust: overflow panics in debug builds so you find it, and you choose explicit behavior with `wrapping_*`, `checked_*`, or `saturating_*` when you need it.

Scalar types: floating point, bool, char

Floating point

Rust has exactly two floating-point types: `f32` (single precision) and `f64` (double precision). The default is `f64`. They follow the IEEE 754 standard, the same as C's `float` and `double`.

```
fn main() {
    let x = 2.0;           // f64 by default
    let y: f32 = 3.0;      // f32
    println!("{x} {y}");
}
```

Booleans

`bool` has two values, `true` and `false`, and is one byte. The key difference from C: a `bool` is **not** a number. You cannot use an integer where a `bool` is expected, and you cannot do arithmetic on a `bool`.

```
// COMPILE ERROR: mismatched types, expected `bool`, found integer
fn main() {
    let flag = 1;
    if flag {           // error[E0308]: expected `bool`, found integer
        println!("yes");
    }
}
```

In C, `if (n)` and `if (ptr)` are common: any nonzero value is “true.” Rust has no such rule. A condition must be a real `bool`. We cover this fully in Chapter 5 — Control Flow.

Characters

A Rust `char` is a **Unicode scalar value**: it is **4 bytes** and can hold any single Unicode character, not just ASCII. This is very different from a C `char`, which is one byte and is really just a small integer.

```
fn main() {
    let letter = 'A';      // char, 4 bytes
    let pi = ' ';          // also one char
    let crab = ' ';        // a single char, even though it is multi-byte as text
    println!("{letter} {pi} {crab}");
}
```

Write a `char` with single quotes (`'A'`) and text with double quotes (`"A"`). A `char` is one character; a string is a sequence of bytes encoded as UTF-8. If you want C’s “one byte” meaning, use `u8` (and the byte literal `b'A'`).

C vs Rust. C `char` is one byte and doubles as a tiny integer. Rust `char` is a 4-byte Unicode scalar value and is **not** an integer. For a raw byte, use `u8`. We cover text and UTF-8 in Chapter 10 — Slices and Strings.

Watch out. Single quotes and double quotes mean different types in Rust. `'a'` is a `char`; `"a"` is a string slice (`&str`). They are not interchangeable, unlike the loose `char`/string handling habits from C.

Compound types: tuples

A **tuple** groups a fixed number of values, which may have different types, into one value. It is like an anonymous, lightweight struct.

```
fn main() {
    let point: (i32, f64, char) = (1, 2.5, 'z');

    // Destructuring: unpack the tuple into separate names.
    let (a, b, c) = point;
    println!("{a} {b} {c}");

    // Or access fields by index with a dot.
    println!("{}", point.0, point.1, point.2);
}
```

You read a tuple field with `.0`, `.1`, `.2`, and so on. **Destructuring** lets you unpack all the fields at once into separate names. The size and the type of each position are fixed when you write the tuple.

The unit type

The empty tuple `()` is called the **unit type**. It has exactly one value, also written `()`, and carries no information. It is what a function returns when it returns “nothing” — the rough equivalent of C’s `void`. You will see it in Chapter 6 — Functions and Closures.

Compound types: arrays

An **array** in Rust is a fixed-length sequence of values that all have the same type. Its type is written `[T; N]`: element type `T`, length `N`. Crucially, the **length is part of the type**. A `[i32; 3]` and a `[i32; 4]` are different, incompatible types.

```
fn main() {
    let nums: [i32; 3] = [10, 20, 30];
    let zeros = [0u8; 16];           // sixteen bytes, all 0 (like memset to 0)

    println!("{}", nums[0]);         // indexing with []
    println!("{}", nums.len());      // 3
    println!("{}", zeros.len());     // 16
}
```

Rust arrays are **stack-allocated** and **bounds-checked**. If you index past the end, the program panics instead of reading stray memory:

```
use std::hint::black_box;

// This compiles, but PANICS at runtime: index out of bounds.
fn main() {
    let nums = [10, 20, 30];
    // `black_box` hides the value from the compiler so it cannot reject this
    // at compile time; in real code the index usually comes from input.
    let i = black_box(5);
    println!("{}", nums[i]); // thread panicked: index out of bounds: len is 3
}
```

Watch out. If you index with a *constant* the compiler already knows is out of range (e.g. `nums[5]` written literally), Rust rejects it at **compile time** instead — an even nicer outcome. Indexes computed at runtime are the ones checked at runtime.

```
let nums = [10, 20, 30];    type [i32; 3]
```

```
index:    0      1      2
          +-----+-----+-----+
stack:    | 10  | 20  | 30  |
          +-----+-----+-----+
          i32   i32   i32      <- 3 * 4 = 12 bytes, on the stack
```

```
nums[5] -> PANIC: index out of bounds (checked at runtime)
```

C vs Rust. A C array decays to a pointer and carries no length, so out-of-bounds access is silent and dangerous. A Rust array knows its length (it is in the type), stays on the stack, and is bounds-checked at runtime.

Watch out. Because the length is part of the type, a function that takes `[i32; 3]` will not accept `[i32; 4]`. For a view into a sequence of any length, you use a **slice** (`&[T]`), and for a growable, heap-allocated array you use `Vec<T>`. Both are covered in Chapter 10 — Slices and Strings.

No implicit conversions

C quietly converts between numeric types all the time: assign an `int` to a `long`, mix `int` and `double` in an expression, pass a `char` where an `int` is expected. This is convenient and a frequent source of bugs.

Rust does **none** of this automatically. Different numeric types never mix:

```
// COMPILER ERROR: mismatched types
fn main() {
    let a: i32 = 5;
    let b: i64 = 10;
    let c = a + b;           // error[E0308]: expected `i32`, found `i64`
    println!("{c}");
}
```

To convert, you ask for it explicitly with the `as` keyword:

```
fn main() {
    let a: i32 = 5;
    let b: i64 = 10;
    let c = a as i64 + b;    // convert a to i64, then add
    println!("{c}");

    let big: i32 = 300;
    let small = big as u8;   // truncates: 300 mod 256 = 44
    println!("{small}");    // prints 44
}
```

The `as` cast is like a C cast: it can **truncate** or change the value. Casting `300i32` to `u8` keeps only the low 8 bits, giving 44. `as` never panics and never reports an error, so use it knowing it can silently change the value, just like in C.

For safe, lossless conversions, idiomatic Rust prefers the `From` and `Into` traits, such as `i64::from(a)`, which only compile when the conversion cannot lose data. We cover those in Chapter 27 — Idioms and Style.

Rule of thumb. Use `as` for deliberate, possibly-lossy casts (and for casting to and from raw pointers later). Prefer `From/Into` when you want the compiler to guarantee the conversion is lossless.

Constants and statics

Rust has two ways to declare a fixed, named value: `const` and `static`. Both replace C's `#define` constants and `const/static` globals, but they are not the same as each other.

`const`

A `const` is a compile-time constant. It must have a type, its value must be computable at compile time, and the compiler **inlines** it wherever it is used — there is no single storage location, much like a C `#define` but type-checked.

```
const MAX_USERS: u32 = 100_000;
const SECONDS_PER_DAY: u32 = 60 * 60 * 24;    // computed at compile time

fn main() {
    println!("{MAX_USERS} {SECONDS_PER_DAY}");
}
```

Names use `SCREAMING_SNAKE_CASE` by convention. A `const` can be declared in any scope, including inside a function or a module, and it has no fixed address.

C vs Rust. A Rust `const` is closest to a C `#define` of a literal or to an `enum` constant, but it is fully typed and scoped. It is *not* a variable: it has no address and never changes.

`static`

A `static` is a value with a **single, fixed memory address** that lives for the entire run of the program. This is like a C global variable. Because it has one real location, you can take a reference to it, and it is not inlined.

```
static GREETING: &str = "hello";

fn main() {
    println!("{GREETING}");
}
```

An immutable `static` is safe. A **mutable** global is exactly the kind of shared mutable state that causes data races in C, so Rust makes it hard on purpose. The old `static mut` form is now strongly

discouraged — in edition 2024 the compiler even refuses to let you take a reference to one. The safe, modern way to have a mutable global counter is an **atomic**:

```
use std::sync::atomic::{AtomicU32, Ordering};

static COUNTER: AtomicU32 = AtomicU32::new(0);

fn main() {
    COUNTER.fetch_add(1, Ordering::Relaxed); // safe: no `unsafe`, no data race
    println!("{}", COUNTER.load(Ordering::Relaxed));
}
```

Rule of thumb. Reach for `const` almost always. Use `static` only when you truly need a single fixed address (for example, a large lookup table). For a mutable global, use an `Atomic` or a `Mutex` (Chapter 20 — Channels and Shared State), not `static mut`.

Feature	<code>const</code>	<code>static</code>
Has a fixed address	No (inlined)	Yes (one location)
Can be mutable	No	Only <code>static mut</code> , and that is <code>unsafe</code>
Lives for the whole program	Effectively yes (inlined)	Yes (' <code>static</code> ')
Type required	Yes	Yes
Closest C idea	typed <code>#define</code>	global variable

C type to Rust type

C type	Rust type	Note
<code>int8_t</code> / signed char	<code>i8</code>	exact width
<code>uint8_t</code> / unsigned char	<code>u8</code>	also Rust's "raw byte"
<code>int16_t</code>	<code>i16</code>	
<code>uint16_t</code>	<code>u16</code>	
<code>int</code> / <code>int32_t</code>	<code>i32</code>	Rust's default integer
<code>unsigned</code> / <code>uint32_t</code>	<code>u32</code>	
<code>long long</code> / <code>int64_t</code>	<code>i64</code>	
<code>uint64_t</code>	<code>u64</code>	
<code>size_t</code>	<code>usize</code>	indexing and lengths
<code>ptrdiff_t</code> / <code>intptr_t</code>	<code>isize</code>	pointer-sized signed
<code>float</code>	<code>f32</code>	IEEE 754 single
<code>double</code>	<code>f64</code>	IEEE 754 double, the default
<code>_Bool</code> / <code>bool</code>	<code>bool</code>	not a number in Rust
<code>char</code> (one byte)	<code>u8</code>	a byte, not text
a Unicode character	<code>char</code>	4 bytes, Unicode scalar value
<code>void</code> (no value)	<code>()</code>	the unit type
<code>T arr[N]</code>	<code>[T; N]</code>	length is part of the type
<code>struct { ... }</code> (anonymous group)	<code>(A, B, C)</code>	tuple

Key takeaways

- `let` binds a name to a value. Bindings are **immutable by default**; add `mut` to allow change. This is the reverse of C.
- **Shadowing** (`let x = ... again`) makes a new variable with the same name and may change its type; **mutation** (`x = ... with mut`) changes one variable's value and keeps its type.
- Integers state their size and signedness: `i8..i128`, `u8..u128`, plus pointer-sized `isize/usize`. The default integer is `i32`; `usize` is used for indexing and lengths.
- Overflow **panics in debug builds** and wraps in release; use `wrapping_*`, `checked_*`, or `saturating_*` to choose behavior explicitly.
- `bool` is not a number, and `char` is a **4-byte Unicode scalar value**, not a C-style byte (use `u8` for a byte).
- Tuples group mixed types; arrays `[T; N]` are stack-allocated, bounds-checked, and the **length is part of the type**. `()` is the unit type (like `void`).
- There are **no implicit numeric conversions**: cast with `as` (may truncate) or use `From/Into` for lossless conversions.
- `const` is an inlined, typed compile-time constant; `static` has a single fixed address; `static mut` is **unsafe**.

Watch out (gotchas for C programmers)

- **Immutable by default**. You will need `let mut` more often than you expect.
- **Shadowing is not mutation**. A second `let` makes a new variable; assignment without `let` needs `mut`.
- **char is 4 bytes**, not one. Use `u8` for a raw byte. Single quotes are a `char`; double quotes are a string.
- **Use `usize` for indexing and lengths**. Mixing index types causes compile errors; `as usize` is common when you have, say, an `i32` index.
- **Overflow panics in debug builds**. Do not rely on silent wraparound; choose `wrapping_*/checked_*/saturating_*` when you mean it.
- **No implicit conversions**. `i32 + i64` will not compile. Write `as` or use `From/Into`.
- **Array length is part of the type**. `[i32; 3]` and `[i32; 4]` are different types; use a slice `&[T]` or `Vec<T>` for variable length (Chapter 10).

Interview questions

Q: What is the difference between shadowing and mutation in Rust? A: Mutation (`x = ...`, requires `let mut`) changes the value stored in one variable and cannot change its type. Shadowing (`let x = ... again`) creates a brand-new variable that reuses the name; the old one is hidden, and the new one may have a different type. Shadowing works even on immutable bindings.

Q: How big is a Rust `char`, and how does it differ from a C `char`? A: A Rust `char` is 4 bytes and holds a Unicode scalar value, so it can represent any single Unicode character. A C `char` is one byte and is really a small integer. For a single raw byte in Rust you use `u8`, not `char`.

Q: What happens on integer overflow in Rust? A: In a debug build, an overflowing arithmetic operation panics, so the bug is caught. In a release build it wraps around using two's complement. To control the behavior explicitly you use `wrapping_*` (wrap), `checked_*` (returns `Option`), `saturating_*` (clamp to min/max), or `overflowing_*` (result plus a flag).

Q: Why does Rust require `as` for numeric conversions when C does them implicitly? A: Rust has no implicit numeric conversions, to prevent the silent value changes and bugs that implicit conversions cause in C. You write `as` for a deliberate, possibly-lossy cast, or use the `From/Into` traits for conversions the compiler can prove are lossless.

Q: When would you use `const` versus `static`? A: Use `const` for a typed compile-time constant; it has no fixed address and is inlined where used. Use `static` when you need a single value at one fixed memory address that lives for the whole program (like a C global). A mutable `static` `mut` requires `unsafe`, so prefer safe shared-state types like `Mutex` or `atomics`.

Try it

1. Write `let x = 5`; then `x = 6`; and read the error. Add `mut` to fix it.
2. Shadow a value to change its type: `let s = "hello"`; `let s = s.len()`; and print it. Then try the same with `mut` and watch it fail.
3. Set `let n: u8 = 255`; and print `n + 1` in a debug build (`cargo run`) to see the panic, then replace it with `n.wrapping_add(1)` and `n.checked_add(1)`.

Chapter 5 — Control Flow

What you'll learn. How `if`, `loop`, `while`, and `for` work in Rust, why most of them are **expressions** that produce a value, why there is no C-style `for(;;)`, and a first look at `match`, `if let`, and `while let`.

`if` and `else`

An `if` in Rust looks almost like C, with two differences you must remember: the condition needs **no parentheses**, and the braces are **required** even for a single statement.

```
fn main() {
    let n = 7;

    if n % 2 == 0 {
        println!("even");
    } else if n % 3 == 0 {
        println!("divisible by 3");
    } else {
        println!("something else");
    }
}
```

The condition must be a `bool`

This is the rule that bites C programmers most. In C, the condition of an `if` is any value, and “nonzero means true.” So `if (n)` and `if (ptr)` are everyday code. In Rust, the condition **must be a `bool`** — a real `true` or `false`. There is no “truthiness.”

```
// COMPILE ERROR: mismatched types, expected `bool`, found integer
fn main() {
    let n = 5;
```



```

    if n {                                     // error[E0308]: expected `bool`, found integer
        println!("nonzero");
    }
}

```

Write the comparison you actually mean:

```

fn main() {
    let n = 5;
    if n != 0 {
        println!("nonzero");
    }
}

```

C vs Rust. C: `if (n)` and `if (ptr)` rely on “nonzero is true” and “non-null is true.” Rust: the condition is always a `bool`, so you write `if n != 0` and `if ptr.is_some()`. No implicit conversion to truth.

if is an expression

In C, `if` is a statement; to choose a value you use the ternary operator `cond ? a : b`. Rust has **no ternary operator**, because it does not need one: `if` is an **expression** that produces a value. You can assign its result directly.

```

fn main() {
    let c = true;
    let x = if c { 1 } else { 2 }; // replaces int x = c ? 1 : 2;
    println!("{x}");
}

```

The value of an `if` expression is the value of the block that runs. Note there is no semicolon after the 1 or 2: the last expression in a block, written without a semicolon, is the block’s value. (We cover expression-vs-statement in detail in Chapter 6 — Functions and Closures.)

Because both branches feed the same variable, they must have the **same type**. And if you use the value, you must have an **else**:

```

// COMPILE ERROR: `if` may be missing an `else` clause
fn main() {
    let c = true;
    let x = if c { 1 }; // error[E0317]: expected `i32`, found `()`
    println!("{x}");
}

```

Without **else**, the `if` would produce nothing `()` when the condition is false, which cannot become an `i32`. C’s ternary always has both sides, and Rust’s value-producing `if` enforces the same completeness.

Mental model. Think of `if { } else { }` as Rust’s ternary that grew braces and can hold many statements. The last expression in each branch is the value it hands back.

Loops

Rust has three loop keywords: `loop`, `while`, and `for`. There is **no C-style** `for(init; cond; step)` loop at all.

`loop`: the infinite loop

`loop` repeats forever until you `break` out of it. It is clearer than C's `while (1)` or `for (;;)` .

```
fn main() {
    let mut count = 0;
    loop {
        count += 1;
        if count == 5 {
            break;
        }
    }
    println!("{count}"); // prints 5
}
```

Uniquely, `loop` can **return a value**: put the value after `break`, and the whole loop expression evaluates to it. This is handy for “retry until it works” code.

```
fn main() {
    let mut n = 1;
    let result = loop {
        n *= 2;
        if n > 100 {
            break n; // the loop expression becomes this value
        }
    };
    println!("{result}"); // prints 128
}
```

C vs Rust. C: `while (1) { ... }` with a `break`. Rust: `loop { ... }`, and a `break` value can hand a result back out — something C's loops cannot do.

`while`: loop with a condition

`while` works like C's, minus the parentheses, and the condition is again a real `bool`.

```
fn main() {
    let mut n = 3;
    while n != 0 {
        println!("{n}");
        n -= 1;
    }
    println!("liftoff");
}
```

for: the iterator loop

In C, `for` is a counting loop you build by hand, and indexing past the end is a classic bug. In Rust, `for x in iterable` walks over the items of something that can be iterated — a range, an array, or any **iterator**. This is the loop you will use almost all the time.

```
fn main() {
    let nums = [10, 20, 30];

    for n in nums {           // walk the array's elements directly
        println!("{n}");
    }
}
```

To count, you loop over a **range**. There are two forms:

- `0..n` is **exclusive**: it yields `0`, `1`, ..., `n - 1` (stops before `n`).
- `0..=n` is **inclusive**: it yields `0`, `1`, ..., `n` (includes `n`).

```
fn main() {
    for i in 0..5 {           // 0 1 2 3 4   (like for (i = 0; i < 5; i++))
        print!("{i} ");
    }
    println!();

    for i in 1..=5 {          // 1 2 3 4 5   (like for (i = 1; i <= 5; i++))
        print!("{i} ");
    }
    println!();
}
```

Because the loop drives the iterator, there is no index variable to get wrong and no chance of running off the end. When you do need both the index and the value, ask the iterator with `.enumerate()`:

```
fn main() {
    let names = ["Ada", "Linus", "Grace"];
    for (i, name) in names.iter().enumerate() {
        println!("{i}: {name}");
    }
}
```

C vs Rust. C: `for (int i = 0; i < n; i++)` with manual bounds. Rust: `for i in 0..n`. The iterator handles the counting and bounds, so off-by-one and out-of-bounds bugs largely disappear. There is no C-style three-part `for`.

Watch out. `..` excludes the end; `..=` includes it. `0..n` runs `n` times ending at `n - 1`; `0..=n` runs `n + 1` times ending at `n`. Mixing these up is the Rust version of an off-by-one error.

Loop labels

To **break** or **continue** an **outer** loop from inside a nested loop, give the loop a **label**: a name starting with a single quote, like `'outer:..`. C has no labeled loops (only `goto`), so this replaces the `goto` cleanup; pattern for breaking out of nesting.

```
fn main() {
    'outer: for i in 0..5 {
        for j in 0..5 {
            if i + j == 4 {
                println!("stopping at i={i} j={j}");
                break 'outer;      // break the OUTER loop, not just the inner
            }
        }
    }
}
```

`continue 'outer;` works the same way: it skips to the next iteration of the labeled loop.

match: the multi-way branch

`match` compares a value against a series of **patterns** and runs the first arm that fits. It is like C's `switch`, but far more powerful and much safer. This is a first look; the full treatment, including enums, comes in Chapter 12 — Enums and Pattern Matching.

```
fn main() {
    let n = 3;
    match n {
        1 => println!("one"),
        2 => println!("two"),
        3 => println!("three"),
        _ => println!("something else"),    // _ is the catch-all, like default
    }
}
```

Each line is an **arm**: a pattern, then `=>`, then the code to run. Two things are different from C's `switch`:

- **No fall-through.** Each arm runs on its own and then the `match` ends. There is no `break` and no accidental fall into the next case. (In C you must write `break;` after every case or risk a bug.)
- **It must be exhaustive.** The arms must cover every possible value. If they do not, the program will not compile. The `_` wildcard matches anything left over, like `default` in C.

```
// COMPILER ERROR: non-exhaustive patterns: `i32::MIN..=0_i32` and others not covered
fn main() {
    let n = 3;
    match n {
        1 => println!("one"),
        2 => println!("two"),
        // no arm for every other i32 -> error[E0004]
```

```

    }
}

```

Richer patterns

match arms can do much more than match a single value:

```

fn main() {
    let n = 7;
    match n {
        0 => println!("zero"),
        1 | 2 | 3 => println!("small"),           // multiple patterns with |
        4..=9 => println!("medium"),              // an inclusive range pattern
        x if x % 2 == 0 => println!("big even {x}"), // a guard: extra `if` test
        x => println!("big odd {x}"),             // bind the value to a name `x`
    }
}

```

- 1 | 2 | 3 — match any of several patterns.
- 4..=9 — match a range of values.
- x if x % 2 == 0 — a **guard**: an extra if condition the arm must also pass.
- x — a **binding**: capture the matched value in a new variable x.

match is an expression

Like if, match produces a value, so you can assign its result. Every arm must yield the same type. This is the idiomatic way to map an input to an output:

```

fn main() {
    let code = 2;
    let label = match code {
        0 => "off",
        1 => "low",
        2 => "high",
        _ => "unknown",
    };
    println!("{label}");
}

```

C vs Rust. C switch falls through unless you **break**, only matches integer constants, and **default** is optional. Rust **match** never falls through, matches rich patterns (ranges, alternatives, guards, bindings), must be exhaustive, and is an expression that returns a value.

if let, let ... else, and while let

A full match is overkill when you only care about **one** pattern. Rust offers shorter forms. These shine with **Option** and **Result** (Chapter 12 — Enums and Pattern Matching), but here is the shape.

if let

if let runs a block only when a value matches one pattern, and binds the inner value at the same time:

```
fn main() {
    let maybe = Some(5);

    if let Some(x) = maybe {
        println!("got {x}");      // runs only when `maybe` is Some
    } else {
        println!("nothing");
    }
}
```

This is shorter than a two-arm **match** when you only act on one case.

let ... else

let ... else binds a pattern, but if it does **not** match, the **else** block must leave the current scope (with **return**, **break**, **continue**, or a **panic**). After it, the bound value is available without extra nesting:

```
fn parse_first(words: &[&str]) -> i32 {
    let Some(first) = words.first() else {
        return -1;          // bail out if the slice is empty
    };
    // `first` is usable here, no nesting needed
    first.parse().unwrap_or(0)
}

fn main() {
    println!("{}", parse_first(&["42", "x"])); // 42
    println!("{}", parse_first(&[]));          // -1
}
```

This keeps the “happy path” un-indented, similar to an early **return** after a guard check in C.

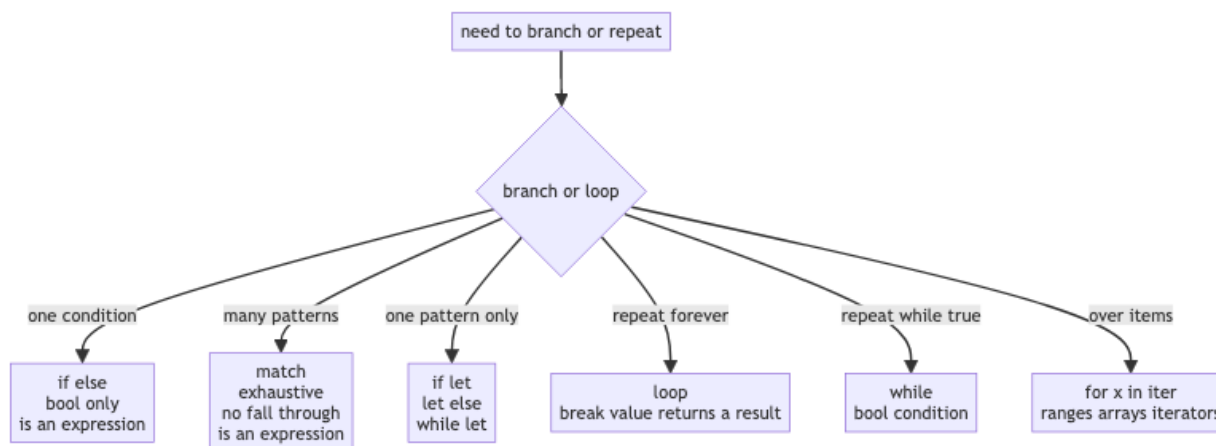
while let

while let keeps looping as long as a pattern matches. It is a clean way to drain a value, such as popping from a stack until it is empty:

```
fn main() {
    let mut stack = vec![1, 2, 3];

    while let Some(top) = stack.pop() { // pop() returns Some until empty, then None
        println!("{}", top);           // prints 3, 2, 1
    }
}
```

How the pieces fit



Key takeaways

- `if/else` take a `bool` condition (no truthiness), need no parentheses, and require braces. `if` is an **expression**: `let x = if c { 1 } else { 2 };`. There is no ternary operator.
- `loop` is the infinite loop; `break value` makes the `loop` produce a value.
- `while` loops on a `bool` condition.
- `for x in iter` is the everyday loop: over ranges (`0..n` exclusive, `0..=n` inclusive), arrays, and iterators. There is **no C-style three-part for**.
- Loop labels (`'outer:'`) let `break/continue` target an outer loop.
- `match` is an exhaustive, expression-valued multi-way branch with **no fall-through**; arms support `_`, `|`, ranges, bindings, and `if` guards.
- `if let`, `let ... else`, and `while let` are shorthands for handling a single pattern without a full match.

Watch out (gotchas for C programmers)

- **The condition must be a `bool`.** `if (n)` and `if (ptr)` do not work; write `if n != 0` or `if ptr.is_some()`.
- **`if`, `match`, and `loop` are expressions** that produce values. Use that instead of the missing ternary operator.
- **No C-style `for`.** Use `for i in 0..n` or iterate items directly; there is no `for (i = 0; i < n; i++)`.
- **`..` vs `..=`.** `0..n` stops at `n - 1`; `0..=n` includes `n`. This is the common off-by-one trap.
- **`match` does not fall through and must be exhaustive.** No `break` is needed between arms; you must cover every case (often with `_`).
- **Braces are mandatory** on every `if/loop` body, even a single line. There is no brace-less one-liner.

Interview questions

Q: Why does `if n { ... }` fail to compile in Rust when `n` is an integer? A: Because an `if` condition must be a `bool`. Rust has no “truthiness”: an integer or pointer is not implicitly treated as true or false. You write an explicit comparison such as `if n != 0`.

Q: Rust has no ternary operator. What replaces `cond ? a : b`? A: An `if` expression: `let x = if cond { a } else { b };`. Because `if` is an expression that produces a value, it serves the role of the ternary. Both branches must have the same type, and an `else` is required when the value is used.

Q: How does Rust's `match` differ from C's `switch`? A: `match` never falls through (no `break` needed), must be exhaustive (cover every case or use `_`), can match rich patterns (ranges, alternatives with `|`, bindings, and `if` guards) rather than only integer constants, and is an expression that returns a value.

Q: What is special about `loop` compared to `while` and `for`? A: `loop` repeats forever until a `break`, and a `break` value; makes the entire `loop` expression evaluate to that value. `while` repeats while a `bool` condition holds, and `for` iterates over items of a range, array, or iterator.

Q: When would you use `if let` or `while let` instead of `match`? A: When you only care about a single pattern. `if let Some(x) = opt { ... }` runs a block and binds `x` only when the value matches, and `while let` loops while a pattern keeps matching (for example, popping from a collection until it is empty). They avoid the boilerplate of a full multi-arm `match`.

Try it

1. Rewrite `int max = a > b ? a : b;` as a Rust `let max = if ... { ... } else { ... };`.
2. Print `1..=10` with a `for` loop, then change `..=` to `..` and note that the last number disappears.
3. Write a `match` over an `i32` with arms for `0, 1 | 2, 3..=9`, an `x if x < 0` guard, and a final `x` binding. Remove the last arm and read the non-exhaustive error.

Chapter 6 — Functions and Closures

What you'll learn. How Rust functions are written and how “the last expression is the return value” works, why there is no function overloading or default arguments, what diverging functions and the unit type are, and how closures capture their environment — the thing C cannot do without a manual `void *` context.

Functions: the shape

A Rust function looks much like a C one, but the syntax is rearranged. The keyword is `fn`, the parameter types come *after* each name, and the return type comes after an arrow `->`.

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    let s = add(2, 3);
    println!("{s}");
}
```

The same idea in C:


```
int add(int a, int b) {
    return a + b;
}
```

Part	C	Rust
Keyword	(none)	<code>fn</code>
Parameter type	<code>int a</code>	<code>a: i32</code>
Return type	before the name	after <code>-></code>
No return value	<code>void f(void)</code>	<code>fn f()</code> (returns <code>()</code>)
Forward declaration	needed in a header	never needed

C vs Rust. Order does not matter in Rust. A function may call another that is defined later in the same file, and there are no headers or forward declarations (see Chapter 3 — Program Structure: Crates, Modules, and Visibility). The compiler reads the whole crate before checking calls.

Parameters are **always** typed. Rust never infers a parameter’s type from the call site, the way template-like guessing might. This keeps function signatures a clear, fixed contract — exactly like a C prototype.

Statements vs expressions

This is the single idea that surprises C programmers most, so read it slowly.

Rust splits code into two kinds of thing:

- An **expression** produces a value. `2 + 3`, `add(1, 2)`, and `x` are expressions.
- A **statement** performs an action and produces **no** useful value. `let x = 5;` is a statement.

In C, `a + b` is “an expression statement” and the value is just thrown away. In Rust the distinction is load-bearing: **the value of a block `{ ... }` is the value of its last expression, if that last line has no semicolon.**

```
fn main() {
    let y = {
        let a = 2;
        let b = 3;
        a + b           // no semicolon: this is the block's value
    };
    println!("{y}");    // prints 5
}
```

So the block evaluates to 5, and `y` becomes 5. The `let a = 2;` lines are statements; the final `a + b` is the block’s result.

Mental model. A `{ ... }` block in Rust is like a tiny function: it runs its statements, then “returns” its last expression. C’s blocks have no value; Rust’s blocks do.

The trailing semicolon changes the meaning

A semicolon turns an expression into a statement. The statement’s value is `()`, the **unit type** (more on that below). This is the most common beginner bug.

```
fn square(x: i32) -> i32 {
    x * x                // last expression, no semicolon: this is the return value
}

// COMPILER ERROR: mismatched types - expected i32, found ()
fn square_broken(x: i32) -> i32 {
    x * x;               // the semicolon makes this a statement; the block returns ()
}

// error[E0308]: expected `i32`, found `()``
```

The first version returns `i32`. The second returns `()` because the semicolon discarded the value, and the compiler complains that `()` is not the `i32` you promised.

Watch out. A stray semicolon on the last line is the classic “why won’t this compile?” error. If a function should return a value, its final expression must have **no** semicolon.

Returning a value: no return needed

Because the last expression *is* the return value, you usually do not write `return` at all. It is there for **early exit**.

```
fn classify(n: i32) -> &'static str {
    if n < 0 {
        return "negative";    // early exit
    }
    if n == 0 {
        return "zero";
    }
    "positive"                // the final value, no `return`, no semicolon
}

fn main() {
    println!("{}", classify(-4));
    println!("{}", classify(0));
    println!("{}", classify(7));
}
```

Rule of thumb. Use the bare last expression for the normal result. Use `return` only to bail out early. Idiomatic Rust rarely writes `return` on the last line.

Because `if` is itself an expression (see Chapter 5 — Control Flow), the function above can also be written without any `return`:

```
fn classify(n: i32) -> &'static str {
    if n < 0 {
        "negative"
    } else if n == 0 {
        "zero"
    } else {
        "positive"
    }
}
```

```
}
```

The unit type () and diverging functions

A function with no `-> ...` returns the **unit type**, written `()`. Unit is a value that carries no information — the closest C idea is `void`, but `()` is a real value you can store and pass, not the absence of a type.

```
fn log(msg: &str) {           // returns ()
    println!("[log] {msg}");
}
```

Some functions never return at all. Their return type is `!`, the **never type**, and they are called **diverging functions**. They panic, loop forever, or exit the process.

```
fn fail(msg: &str) -> ! {
    panic!("fatal: {msg}");    // never returns
}

fn main() {
    let n: i32 = "42".parse().unwrap_or_else(|_| fail("not a number"));
    println!("{n}");
}
```

`std::process::exit(1)` and an infinite loop `{}` also have type `!`. The `!` type is useful because it fits anywhere: a branch that calls `fail(...)` can sit next to a branch that produces an `i32`, because `!` coerces to any type.

C vs Rust. C marks such functions with `_Noreturn` (or `__attribute__((noreturn))`) as a hint. Rust's `!` is a real type the compiler understands and uses for flow analysis.

No overloading, no default arguments

Two features C++ has and C does not — and Rust deliberately does not have either.

- **No function overloading.** You cannot define two functions with the same name and different parameter types. Each name means one function.
- **No default argument values.** You cannot write `fn f(x: i32 = 0)`.

This keeps every call unambiguous: the name plus the arguments always picks exactly one function, with no hidden resolution rules. Here is how Rust programmers get the same results.

Idiomatic alternative 1 — distinct names. Just give each variant a clear name. The standard library does this everywhere (`new`, `with_capacity`, `from_utf8`).

```
struct Buffer {
    bytes: Vec<u8>,
}

impl Buffer {
    fn new() -> Buffer {
        Buffer { bytes: Vec::new() }
    }
}
```

```

    fn with_capacity(n: usize) -> Buffer {
        Buffer { bytes: Vec::with_capacity(n) }
    }
}

```

Idiomatic alternative 2 — Option parameters. An argument that is sometimes “use the default” becomes an `Option<T>`. The caller passes `None` to mean “default” (see Chapter 12 — Enums and Pattern Matching for `Option`).

```

fn greet(name: &str, greeting: Option<&str>) {
    let g = greeting.unwrap_or("Hello");    // default when None
    println!("{g}, {name}");
}

fn main() {
    greet("Ada", None);                      // Hello, Ada
    greet("Ada", Some("Welcome"));           // Welcome, Ada
}

```

Idiomatic alternative 3 — the builder pattern. When a type has many optional settings, give it a builder with one method per setting. This replaces both overloading and default arguments cleanly (see Chapter 27 — Idioms and Style).

C vs Rust. In C you also have no overloading or defaults, so you already use distinct names (`strncpy` vs `strcpy`) and pass extra flags by hand. Rust’s `Option` and builders are the tidy, type-checked version of that habit.

Function pointers

Rust has function pointers, just like C. The type is written `fn(ArgTypes) -> Ret` — note the lowercase `fn` in a *type* position. You can store one in a variable, pass it to a function, or return it.

```

fn double(x: i32) -> i32 {
    x * 2
}

fn triple(x: i32) -> i32 {
    x * 3
}

fn apply(f: fn(i32) -> i32, v: i32) -> i32 {
    f(v)
}

fn main() {
    let g: fn(i32) -> i32 = double;    // store a function pointer
    println!("{}", g(10));             // 20
    println!("{}", apply(triple, 10)); // 30
}

```

The C version is the same idea with C’s harder-to-read syntax:

```

int double_it(int x) { return x * 2; }

int apply(int (*f)(int), int v) {
    return f(v);
}

int main(void) {
    int (*g)(int) = double_it;    /* a function pointer */
    return apply(double_it, 10);
}

```

Concept	C	Rust
Function pointer type	<code>int (*)(int)</code>	<code>fn(i32) -> i32</code>
Carries captured state	no	no (use a closure for that)
Can be null	yes	no (use <code>Option<fn(...)></code>)

Watch out. A plain `fn` pointer holds **no** captured data — it is just an address, exactly like in C. To carry state alongside the code, you need a *closure*, which is next.

Closures: functions that capture their environment

A **closure** is an anonymous function that can **capture** variables from the scope where it is written. C has nothing like this. In C, when you pass a callback you also pass a `void *` “context” pointer by hand, and the callback casts it back. A Rust closure bundles that context for you, type-checked.

The syntax uses vertical bars `| |` around the parameters:

```

fn main() {
    let add_one = |x| x + 1;           // a closure; types are inferred
    println!("{}", add_one(41));      // 42

    let factor = 3;
    let scale = |x| x * factor;        // captures `factor` from the environment
    println!("{}", scale(10));        // 30
}

```

Notice `scale` uses `factor`, a variable from `main`. The closure “closes over” `factor` — it captures it. The C equivalent must thread that value through a context pointer:

```

typedef int (*cb)(int x, void *ctx);

int scale(int x, void *ctx) {
    int factor = *(int *)ctx;
    return x * factor;
}

int main(void) {
    int factor = 3;
    cb f = scale;
}

```

```

    return f(10, &factor);    /* you carry the context by hand */
}

```

Mental model. A closure is a small struct the compiler writes for you: it holds the captured variables as fields, plus the code to run. That hidden struct is the `void *` context you would have built by hand in C — but type-safe and automatic.

Closure type inference

Closure parameter and return types are usually inferred from how the closure is used, so you rarely write them. You *may* annotate them, and the body needs braces if it has more than one line:

```

fn main() {
    let typed = |x: i32| -> i32 { x * x };
    println!("{}", typed(5));    // 25
}

```

Each closure has its own unique, unnameable type generated by the compiler. That is why function parameters that take closures use *trait bounds* (below), not a single concrete type.

How a closure captures: by reference, by mut reference, by value

A closure captures each variable in the **least demanding** way that still lets the body work. There are three modes:

1. **By shared reference** (`&T`) — the closure only reads the variable.
2. **By mutable reference** (`&mut T`) — the closure modifies the variable.
3. **By value** (**moves it in**) — forced with the `move` keyword, or chosen when the body needs ownership.

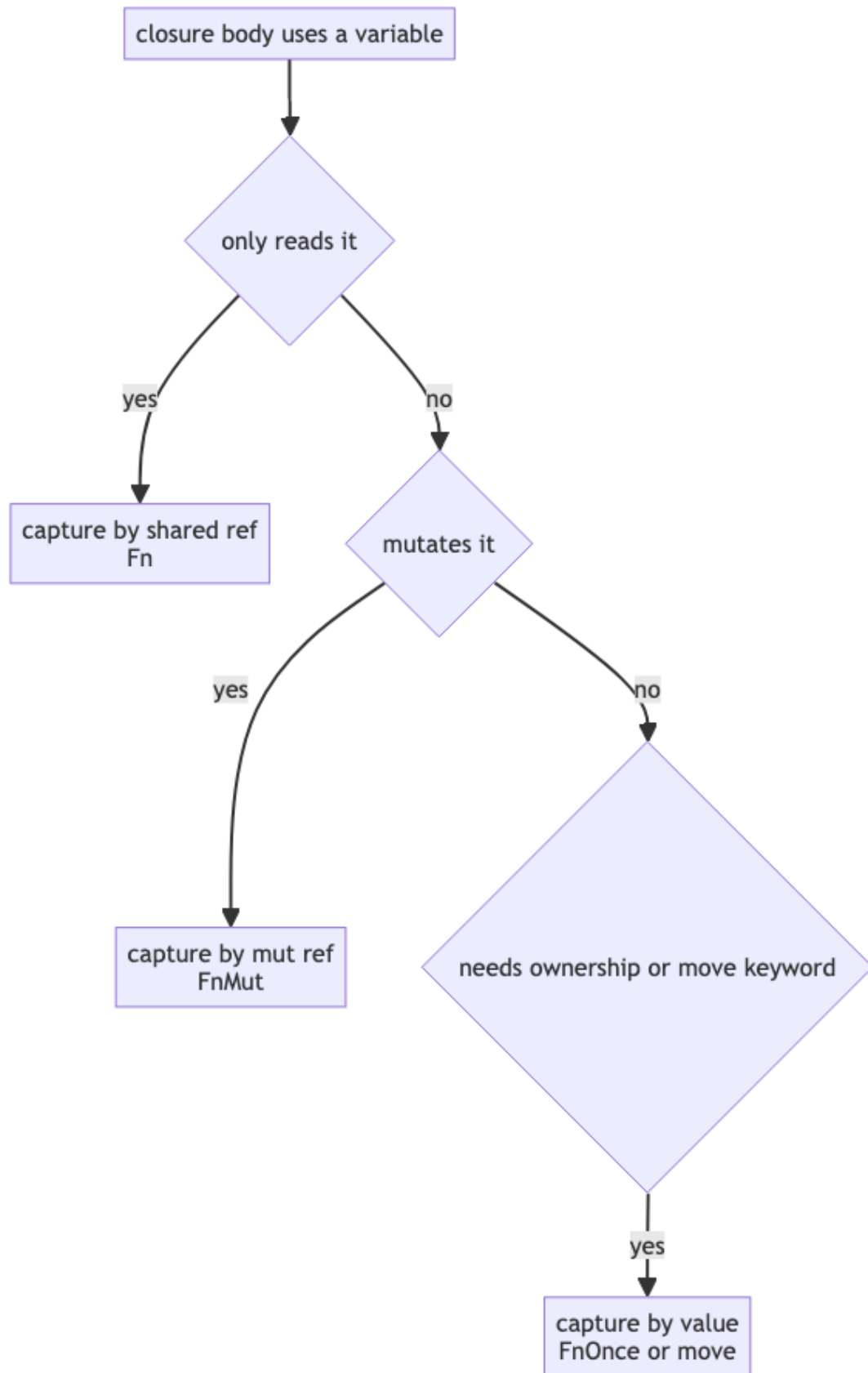
```

fn main() {
    // 1. Capture by shared reference: just reading.
    let name = String::from("Ada");
    let greet = || println!("Hello, {name}");
    greet();
    println!("still usable: {name}");    // name was only borrowed

    // 2. Capture by mutable reference: the closure changes `count`.
    let mut count = 0;
    let mut bump = || count += 1;        // closure must be `mut` to call it
    bump();
    bump();
    println!("count = {count}");        // 2

    // 3. Capture by value with `move`: ownership moves into the closure.
    let owned = String::from("mine");
    let consume = move || println!("took {owned}");
    consume();
    // println!("{owned}");    // would be a COMPILE ERROR: `owned` was moved
}

```



C vs Rust. The borrow checker (Chapter 8 — Borrowing and References) watches closures too. If a closure borrows **name**, you cannot also mutate **name** elsewhere while the closure is alive. C gives you no such protection — a callback holding a pointer to a freed local is a classic crash.

move and ownership

`move` forces every captured variable to be taken **by value** (moved in). You need this when the closure must outlive the current scope — most importantly when you hand it to another thread (see Chapter 19 — Threads and Concurrency), or return it from a function.

```
fn make_counter() -> impl FnMut() -> i32 {
    let mut count = 0;
    move || {
        count += 1;           // `count` is moved into the closure and lives on
        count
    }
}

fn main() {
    let mut next = make_counter();
    println!("{}", next());   // 1
    println!("{}", next());   // 2
    println!("{}", next());   // 3
}
```

Without `move`, the closure would try to borrow `count`, which dies when `make_counter` returns — a dangling reference, which the compiler rejects. `move` moves `count` into the closure so it lives as long as the closure does.

Edition 2024: disjoint field capture

Since edition 2021 (and still the rule in edition 2024), a closure captures **only the fields it actually uses**, not the whole struct. This is called *disjoint capture*.

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let mut p = Point { x: 1, y: 2 };
    let mut bump_x = || p.x += 1;   // captures only p.x, not all of p
    let read_y = p.y;               // reading p.y is fine; only p.x is borrowed
    bump_x();
    println!("{}", p.x, read_y);
}
```

C vs Rust. This is finer-grained than anything C offers. The compiler tracks borrows down to individual struct fields, so two closures can touch different fields of the same

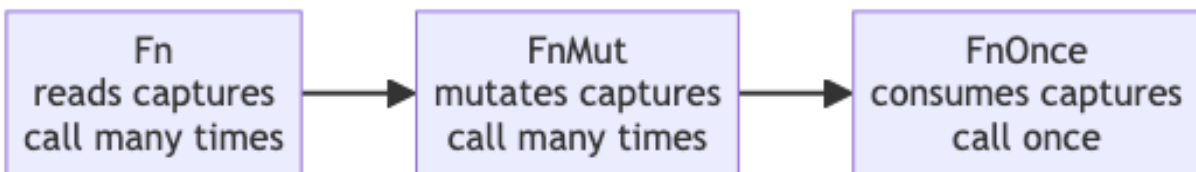
struct at the same time.

The three closure traits: `Fn`, `FnMut`, `FnOnce`

Because every closure has its own hidden type, Rust groups closures by **what they do to their captures** using three traits. A trait is a shared interface (see Chapter 15 — Traits). The three are nested: each one is a stricter version of the next.

Trait	The closure...	Can be called	Implies
<code>FnOnce</code>	consumes captures (moves them out)	once	—
<code>FnMut</code>	mutably borrows captures	many times	<code>FnOnce</code>
<code>Fn</code>	only shares (reads) captures	many times	<code>FnMut</code> + <code>FnOnce</code>

The relationship is a hierarchy: every `Fn` is also `FnMut`, and every `FnMut` is also `FnOnce`. So `Fn` is the most flexible to *receive* (it can do the least harm), and `FnOnce` is the most flexible to *accept from a caller* (it asks the least).



- A closure is `FnOnce` if it **moves a captured value out** of itself (it can run only once, because the value is gone after).
- It is `FnMut` if it **mutates** a capture but does not move anything out.
- It is `Fn` if it only **reads** its captures.

The compiler picks the most capable set automatically. You choose the trait only in a function *signature*, when you say what kind of closure you accept.

Taking closures as parameters

To accept a closure, use a trait bound. The simplest form is `impl Trait` in the parameter position. Pick the loosest trait that your function needs.

```
fn call_twice(f: impl Fn()) {  
    f();  
    f();  
    // Fn can be called many times  
}  
  
fn do_once(f: impl FnOnce() -> String) -> String {  
    f()  
    // FnOnce: we promise to call it at most once  
}
```

```
fn main() {
    let name = String::from("Ada");
    call_twice(|| println!("hi"));

    let owned = String::from("payload");
    let result = do_once(move || owned); // moves `owned` out, so FnOnce
    println!("{result}");
    let _ = name;
}
```

The same with an explicit generic and a **where** clause — identical meaning, useful when the bound is long:

```
fn map_pair<F>(a: i32, b: i32, f: F) -> (i32, i32)
where
    F: Fn(i32) -> i32,
{
    (f(a), f(b))
}

fn main() {
    let (x, y) = map_pair(3, 4, |n| n * n);
    println!("{x} {y}"); // 9 16
}
```

Rule of thumb. Accept the **least restrictive** trait you can: prefer `FnOnce` if you call it once, `FnMut` if you call it repeatedly and it may mutate, and `Fn` if it only needs to read. Accepting `Fn` rejects closures that mutate; accepting `FnOnce` accepts the most callers.

Returning closures

A function can return a closure. Since the closure’s real type has no name, you return `impl Fn...` (when there is a single concrete type) or a boxed trait object `Box<dyn Fn...>` (when the type may vary at runtime).

```
fn adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n // `move` so `n` lives inside the returned closure
}

fn main() {
    let add5 = adder(5);
    println!("{}", add5(10)); // 15
}
```

Use `Box<dyn Fn...>` when different branches return *different* closures, because `impl Trait` requires one single concrete type:

```
fn pick(op: char) -> Box<dyn Fn(i32, i32) -> i32> {
    match op {
        '+' => Box::new(|a, b| a + b),
    }
}
```

```

        _ => Box::new(|a, b| a - b),
    }
}

fn main() {
    let f = pick('+');
    println!("{}", f(2, 3)); // 5
}

```

`Box<dyn Fn...>` stores the closure on the heap and calls it through a pointer — this is dynamic dispatch, the same cost as a C function-pointer call plus one allocation (see Chapter 17 — Smart Pointers for `Box`).

Return form	When to use	Cost
<code>impl Fn...</code>	one concrete closure type	none (static dispatch)
<code>Box<dyn Fn...></code>	type varies (e.g. different match arms)	heap box + dynamic call

Key takeaways

- A function is `fn name(params) -> Ret { ... }`; parameters are always typed; order and headers do not matter.
- A block's value is its **last expression with no semicolon**. That value is the function's return value; `return` is only for early exit.
- A trailing semicolon turns an expression into a statement whose value is `()`, the unit type. This is the most common beginner mistake.
- `()` is Rust's `void`-like value; `!` (the never type) marks diverging functions that panic, loop, or exit.
- There is **no overloading and no default arguments**. Use distinct names, `Option` parameters, or the builder pattern (Chapter 27).
- Function pointers have type `fn(...) -> ...` and carry no state, like in C.
- **Closures** capture their environment by shared reference (`Fn`), mutable reference (`FnMut`), or by value (`move/FnOnce`); the compiler picks the loosest mode. Edition 2024 captures individual struct fields.
- Accept closures with `impl Fn/FnMut/FnOnce` bounds; return them with `impl Fn...` or `Box<dyn Fn...>`.

Watch out (gotchas for C programmers)

- **A trailing semicolon swallows the return value.** `x * x`; returns `()`, not the number. Drop the semicolon on the final expression.
- **`move` moves captures in.** After a `move` closure captures a non-Copy value like a `String`, you cannot use that value outside the closure anymore.
- **Pick the right closure trait.** Accepting `Fn` forbids closures that mutate; accepting `FnMut` forbids ones that move captures out. Take the loosest one your function actually needs.
- **No overloading or default arguments.** Do not try to define two functions with the same name; use distinct names or `Option` parameters.

- **A plain `fn` pointer carries no state.** If your callback needs context, use a closure, not a bare function pointer.
- **A closure borrowing a local cannot outlive it.** Return it with `move` so it owns what it needs, or the borrow checker will reject the dangling reference.

Interview questions

Q: In Rust, how does a function return a value without the `return` keyword? A: The last expression in the function body, written with no trailing semicolon, is the function's return value. A block evaluates to its final expression. `return` is only needed for an early exit before that final expression.

Q: What is the difference between a statement and an expression, and why does a trailing semicolon matter? A: An expression produces a value; a statement performs an action and produces `()`. Adding a semicolon turns an expression into a statement, discarding its value. On a function's last line that changes the return value from the computed value to `()`, which usually causes a type-mismatch error.

Q: What are `Fn`, `FnMut`, and `FnOnce`, and how do you choose one? A: They are traits describing how a closure uses its captures. `Fn` only reads them and can be called many times; `FnMut` mutably borrows them and can be called many times; `FnOnce` may move captures out and can be called once. They form a hierarchy (`Fn` `FnMut` `FnOnce`). In a parameter, accept the loosest one your code needs.

Q: How does a Rust closure differ from a C function pointer? A: A C function pointer is just an address and carries no data; to pass state you pass a separate `void *` context by hand. A Rust closure captures variables from its environment automatically, storing them in a compiler-generated struct, and the capture is type-checked and borrow-checked.

Q: When do you need the `move` keyword on a closure? A: When the closure must own its captures rather than borrow them — for example when it is returned from a function or sent to another thread, so it must outlive the scope where the captured variables were defined. `move` forces every capture to be taken by value.

Try it

1. Write `fn cube(x: i32) -> i32` two ways: once with `return x * x * x`; and once with `x * x * x`. Add a semicolon to the second and read the E0308 error.
2. Write `make_counter` from this chapter and call the returned closure several times. Remove the `move` keyword and read the compiler's explanation.
3. Write a function `fn apply<F: Fn(i32) -> i32>(f: F, v: i32) -> i32`. Pass it both a named function and a closure, and confirm both work.

Chapter 7 — Ownership and Moves

What you'll learn. The single idea that makes Rust different from C: ownership. You will learn the three ownership rules, how values are dropped (freed) automatically and deterministically, what a *move* is and why it invalidates the old name, the difference between `Copy` and `Clone`, and how ownership flows through function calls.

This is the heart of the book. Take your time. Almost everything else in Rust — borrowing, lifetimes, threads, smart pointers — follows from the rules in this one chapter.

The problem ownership solves

In C, *you* manage memory by hand. You call `malloc` to get heap memory and `free` to give it back. Nothing checks that you do this correctly. The result is a familiar list of bugs:

```
char *p = malloc(64);
free(p);
strcpy(p, "oops");    /* use-after-free: undefined behavior */
free(p);               /* double-free: undefined behavior */
```

Each of these is undefined behavior and a common source of crashes and security holes. C trusts you to never make these mistakes. Across a large codebase, nobody ever fully succeeds.

Rust takes a different bet: a small set of rules, checked by the compiler, that make these bugs **impossible to write** in safe code — with no garbage collector and no runtime cost. Those rules are *ownership*.

Mental model. Ownership is the compiler doing the `free` for you, at exactly the right moment, and refusing to compile any program where the timing would be wrong.

The three rules of ownership

Memorize these. Everything else is a consequence.

1. **Each value has exactly one owner** — a variable that is responsible for it.
2. **There is only one owner at a time.** Ownership can be transferred (a *move*), but it is never shared by two owners.
3. **When the owner goes out of scope, the value is dropped** — its memory and other resources are released automatically.

A **scope** is the region between `{` and `}` where a variable is valid, just like in C. The difference is what happens at the closing `}`.

```
fn main() {
    let s = String::from("hello");    // s owns a heap-allocated string
    println!("{s}");
}
```

// scope ends: s is dropped, heap freed here

`String::from("hello")` allocates memory on the heap to hold the text (like `malloc` + `strcpy`). You never call `free`. When `s` goes out of scope at the closing brace, Rust automatically releases that memory. This automatic release is called a **drop**.

C vs Rust. In C you pair every `malloc` with a `free` by hand, and bugs come from getting the pairing wrong. In Rust the `free` is attached to the owner going out of scope, so it always happens exactly once, at a place the compiler can prove.

Drop: automatic, deterministic cleanup (RAII)

When a value is dropped, Rust runs its cleanup code. For a `String` that means freeing the heap buffer. For a file handle it means closing the file. For a lock it means unlocking. This pattern — tie

a resource's lifetime to a variable's scope — is called **RAII** (Resource Acquisition Is Initialization). C++ programmers know it as a destructor; C programmers know it as “the cleanup I always forget in the error path.”

The cleanup is **deterministic**: it happens at a known point (the end of scope), not “sometime later” like a garbage collector. You can see it by implementing the `Drop` trait, which lets you run code when a value is dropped.

```
struct Guard {
    name: &'static str,
}

impl Drop for Guard {
    fn drop(&mut self) {
        println!("dropping {}", self.name);
    }
}

fn main() {
    let _first = Guard { name: "first" };
    let _second = Guard { name: "second" };
    println!("end of main");
}
```

This prints:

```
end of main
dropping second
dropping first
```

Two things to notice. First, the drops happen at the end of `main`, automatically. Second, the drop order is the **reverse** of declaration order: `second` is dropped before `first`. This is the same last-in-first-out order C++ uses for destructors, and it matters when one value depends on another.

C vs Rust. This is the cleanup you would write in a C `goto fail:` chain at the end of a function — close the file, free the buffer, unlock the mutex — except Rust generates it for you and runs it on *every* exit path, including early returns and panics. You cannot forget it.

Dropping early with `drop(x)`

Sometimes you want to release a resource before the end of scope (for example, to unlock a mutex sooner). You cannot call the `drop` *method* directly, but you call the free function `std::drop`, which is in the prelude:

```
struct Guard {
    name: &'static str,
}

impl Drop for Guard {
    fn drop(&mut self) {
```

```

        println!("dropping {}", self.name);
    }
}

fn main() {
    let g = Guard { name: "early" };
    println!("before drop");
    drop(g);                // g is dropped right here
    println!("after drop");
    // g is gone; using it now would be a compile error
}

```

This prints `before drop`, then `dropping early`, then `after drop`. The function `drop` simply takes ownership of `g` and lets it go out of scope immediately. The compiler then knows `g` is gone and forbids any later use — which is our next topic.

Move semantics

Here is where Rust and C part ways most sharply. Consider this C code, which is fine:

```

char *a = strdup("hello");
char *b = a;           /* a and b point at the SAME buffer */
/* both a and b are usable; both point at one allocation */

```

In C, `b = a` copies the pointer. Now two pointers alias one buffer, and you must remember to free it exactly once. The equivalent Rust does something different:

```

// COMPILE ERROR: borrow of moved value: `a`
fn main() {
    let a = String::from("hello");
    let b = a;           // ownership MOVES from a to b
    println!("{a}");     // error[E0382]: borrow of moved value: `a`
    println!("{b}");
}

```

When you write `let b = a;` for a type like `String`, ownership **moves** from `a` to `b`. After the move, `a` is considered *uninitialized* — the compiler will not let you use it. There is now exactly one owner (`b`), so the buffer is freed exactly once, when `b`'s scope ends. Double-free is impossible.

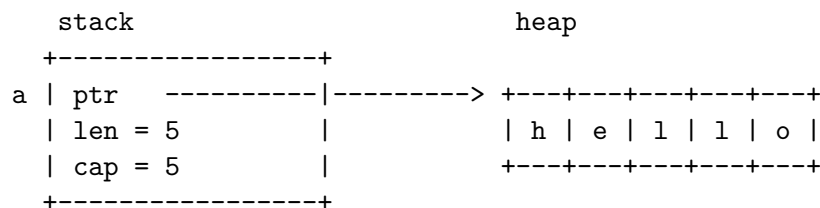
The fix, if you really wanted to keep using `a`, is to use `b` instead, or to make a real copy with `.clone()` (below).

Mental model. Think of ownership like a single physical key to a room. When you give the key to someone else, *you* no longer have it. A move hands over the key; the old variable is left empty-handed.

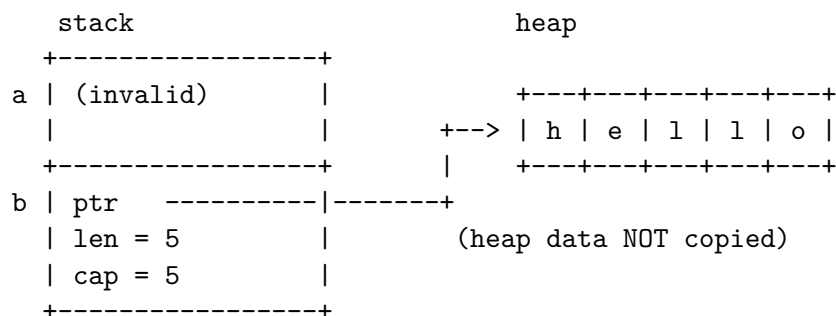
What actually moves: the picture

A `String` is a small three-word **header** stored on the stack: a pointer to the heap buffer, a length, and a capacity. The text itself lives on the heap. A move copies the *header* (three machine words) and marks the source invalid. It does **not** copy the heap text. This is cheap — a few bytes — no matter how long the string is.

Before the move: `let a = String::from("hello");`



After: `let b = a;` (header copied to b; a invalidated)



Only one header now points at the heap buffer, so there is exactly one owner. When `b` is dropped, that single buffer is freed once.

C vs Rust. In C, `b = a` also copies just the pointer header — but C lets you keep using *both* `a` and `b`, which is exactly how double-frees and use-after-free happen. Rust copies the same bytes but then **invalidates the source**, so aliasing-then-freeing-twice cannot occur.

Why “use after move” is a compile error

The error code is **E0382**. The compiler tracks, for every variable, whether it currently owns a value. After a move the source is “moved-out”, and reading it is rejected. This is a compile-time check — there is zero runtime cost and no flag stored at runtime.

```
// COMPILER ERROR: use of moved value: `v`
fn main() {
    let v = vec![1, 2, 3];
    let w = v;           // Vec moved into w
    println!("{}", v.len()); // error[E0382]: borrow of moved value: `v`
    let _ = w;
}
```

The same rule applies to `String`, `Vec<T>`, `Box<T>`, and every type that owns a heap resource. These are **non-Copy** types: assigning or passing them moves them.

Copy types: bit-for-bit duplication

Not every type moves. Small, simple values that live entirely on the stack are **Copy**: assigning them makes a bit-for-bit duplicate, and *both* names stay usable. This matches C’s behavior for plain

values.

```
fn main() {
    let x = 5;
    let y = x;           // i32 is Copy: x is COPIED, not moved
    println!("{x} {y}"); // both still valid: prints 5 5
}
```

There is no heap buffer to worry about, so duplicating the bytes is harmless and cheap. The types that are `Copy` include:

- All integer types (`i32`, `u64`, `usize`, ...).
- Floating-point types (`f32`, `f64`).
- `bool` and `char`.
- Tuples and arrays **if every element is `Copy`** (e.g. (`i32`, `bool`)).
- Shared references `&T` (copying a reference is fine; both can read).

A type can be `Copy` only if it owns no resource that needs cleanup. That is why `String` and `Vec<T>` are **not** `Copy`: duplicating their header bytes would create two owners of one heap buffer — exactly the aliasing Rust forbids.

Type	Copy?	let b = a; does	After, is a usable?
i32, f64, bool, char	yes	bit copy	yes
&T (shared reference)	yes	bit copy	yes
(i32, i32)	yes	bit copy	yes
String, Vec<T>, Box<T>	no	move	no
any type with a Drop impl	no	move	no

Mental model. `Copy` means “this value is just bytes; duplicating the bytes is a perfect, independent copy.” A `String` is not just bytes — it owns a heap buffer — so copying the bytes alone would be wrong.

Clone: an explicit, possibly expensive copy

When you *do* want an independent duplicate of a non-`Copy` value, you ask for one explicitly with `.clone()`. For a `String` or `Vec`, this allocates a new heap buffer and copies the contents — a *deep copy*.

```
fn main() {
    let a = String::from("hello");
    let b = a.clone();           // deep copy: new heap buffer with its own "hello"
    println!("{a} {b}");        // BOTH usable: a was not moved
}
```

After `clone`, `a` and `b` are two separate owners of two separate buffers. Each is freed once, when its own owner goes out of scope.

The key difference from `Copy`: **Clone is explicit and can be expensive.** A move or a `Copy` is always cheap (a few stack bytes). A `clone` may allocate and copy a megabyte. Rust makes you write `.clone()` so the cost is visible in the code, never hidden.

Rule of thumb. Do not reach for `.clone()` just to silence a move error. Most of the time the right answer is to *borrow* the value instead (Chapter 8 — Borrowing and References). Clone when you genuinely need a second independent copy.

Deriving Clone and Copy

For your own types, you opt in with `#[derive(...)]`, which asks the compiler to generate the trait implementation:

```
#[derive(Clone, Copy, Debug)]
struct Point {
    x: i32,
    y: i32,
}

#[derive(Clone, Debug)]
struct Record {
    name: String,           // String is not Copy, so Record cannot be Copy
    points: Vec<Point>,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    let q = p;              // Copy: p still usable
    println!("{p:?} {q:?}");

    let r = Record { name: "a".to_string(), points: vec![p] };
    let s = r.clone();      // explicit deep copy
    println!("{r:?} {s:?}");
}
```

A struct can derive `Copy` only if **every** field is `Copy`. `Record` contains a `String` and a `Vec`, so it can be `Clone` but not `Copy`. `Point` contains only `i32` fields, so it can be both.

Ownership and functions

Passing a value to a function follows the same rules as assignment. For a non-`Copy` type, calling `f(s)` **moves** `s` into `f`. After the call, `s` is gone in the caller — `f` now owns it and will drop it at the end of `f` unless it hands ownership back.

```
fn consume(s: String) {
    println!("consuming {s}");
}                                     // s dropped here

fn main() {
    let text = String::from("hello");
    consume(text);                   // text MOVED into consume
    // println!("{text}");           // COMPILE ERROR: text was moved
}
```

For a `Copy` type, the value is copied in and the original stays usable:

```
fn square(n: i32) -> i32 {
    n * n
}

fn main() {
    let x = 4;
    let y = square(x);    // i32 is Copy: x is copied, still usable
    println!("{x} {y}"); // prints 4 16
}
```

Returning ownership

A function can also **give** ownership by returning a value. The return value moves out of the function and into the caller.

```
fn make_greeting(name: &str) -> String {
    format!("Hello, {name}")    // ownership of the new String moves to the caller
}

fn main() {
    let g = make_greeting("Ada");    // g now owns the String
    println!("{g}");
}                                     // g dropped here
```

The clumsy “give and take back” pattern

Suppose a function needs to *use* a `String` but the caller wants to keep it afterward. With only moves, you must pass ownership in and then return it, threading the value back out by hand:

```
fn count_chars(s: String) -> (String, usize) {
    let n = s.chars().count();
    (s, n)    // hand the String back, plus the answer
}

fn main() {
    let text = String::from("hello");
    let (text, n) = count_chars(text);    // take ownership back
    println!("{text} has {n} chars");
}
```

This works, but it is awkward. Every function that merely *looks at* a value would have to return it. C never makes you do this, because in C you just pass a pointer and the caller still owns the data.

Rust’s real answer is the same idea — pass a reference — but checked by the compiler. You **borrow** the value instead of moving it:

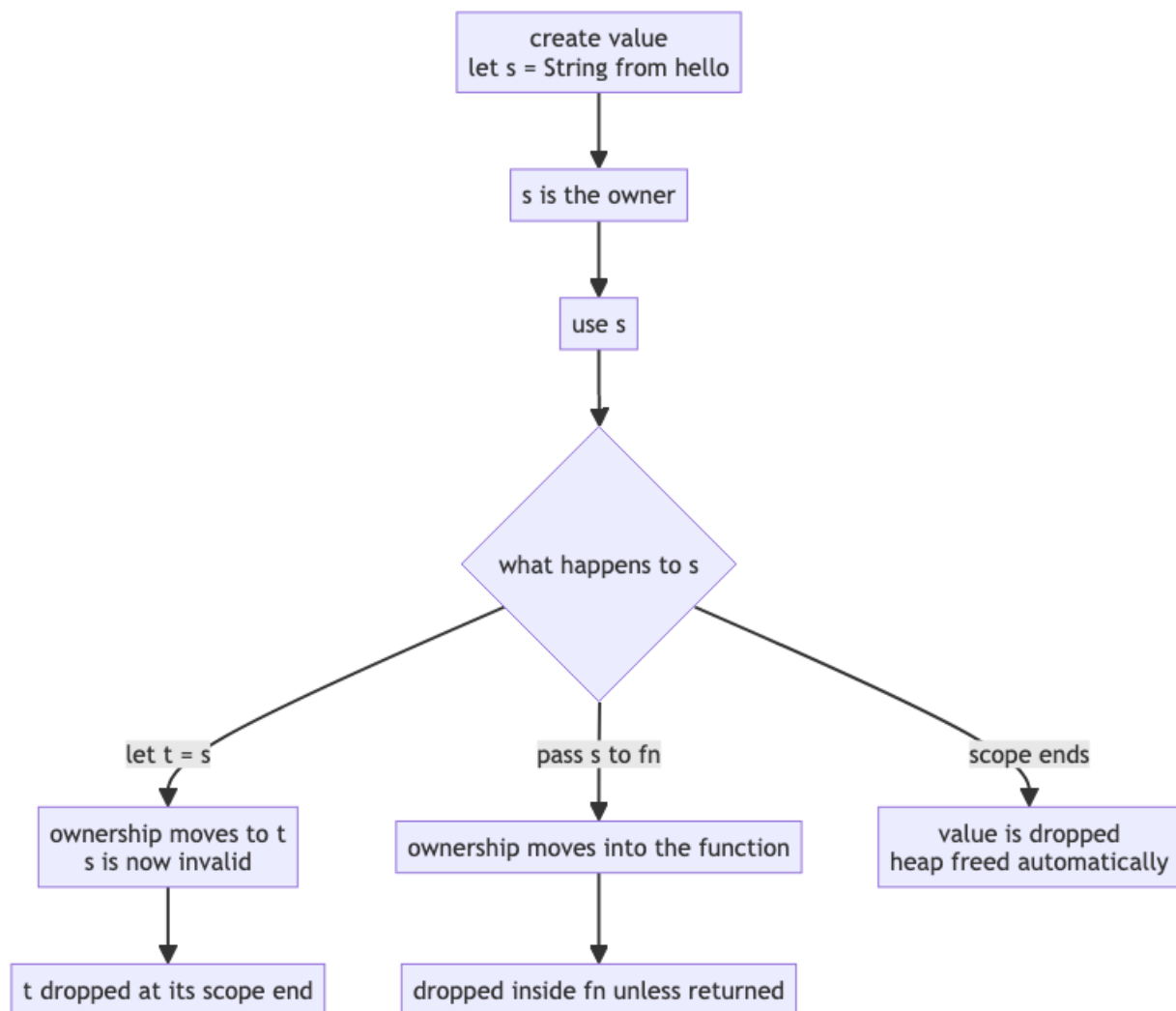
```
fn count_chars(s: &String) -> usize {    // borrow: read without taking ownership
    s.chars().count()
}
```

```
fn main() {
    let text = String::from("hello");
    let n = count_chars(&text);           // lend a reference; text stays owned here
    println!("{text} has {n} chars");     // text still usable
}
```

This is the natural, idiomatic way, and it is the subject of the next chapter. Borrowing lets many parts of your program read (or one part write) a value without moving it, while the compiler guarantees no dangling pointers.

Rule of thumb. Move when a function should *take over* a value’s lifetime. Borrow (Chapter 8 — Borrowing and References) when it only needs to look at or temporarily modify the value. You will borrow far more often than you move.

Putting it together: the lifecycle of an owned value



Every owned value walks this path. It is created with one owner; ownership may move (by assign-

ment or by being passed to a function); and wherever the final owner's scope ends, the value is dropped exactly once. The compiler proves there is always exactly one owner and exactly one drop.

Key takeaways

- **Three rules:** every value has exactly one owner; only one owner at a time; when the owner leaves scope the value is **dropped** (freed) automatically.
- Dropping is **deterministic** — at the end of scope, in reverse declaration order — and runs cleanup (`Drop`), Rust's version of RAII / a destructor. C's `malloc/free` is manual; Rust's free is automatic and always correct.
- Use `drop(x)` to release a value early.
- For non-Copy types (`String`, `Vec`, `Box`), assignment and passing **move** ownership and invalidate the source. Using it afterward is compile error **E0382**. A move copies only the small stack header, not the heap data.
- Copy types (scalars, `bool`, `char`, references, and aggregates of Copy fields) are bit-copied; both names stay usable.
- `.clone()` makes an explicit, possibly deep and expensive copy. Derive traits with `#[derive(Clone, Copy)]`; a type can be Copy only if all fields are Copy and it has no Drop.
- Passing a value to a function moves it in; returning hands ownership back. The clumsy give-and-take-back pattern motivates **borrowing** (Chapter 8).

Watch out (gotchas for C programmers)

- **A move invalidates the source.** Unlike C's `b = a` (where both pointers stay usable), after a Rust move the old name is dead; using it is a compile error.
- **Copy vs move is decided by the type, not the syntax.** `let b = a;` copies an `i32` but moves a `String`. Know which types are Copy.
- **`.clone()` is explicit and can be costly.** Do not sprinkle it to silence the borrow checker; prefer borrowing. Cloning a big `Vec` allocates and copies all of it.
- **Drop runs at scope end, deterministically.** There is no garbage collector and no delay. Order is reverse of declaration — relevant when one value depends on another.
- **You cannot move out of something you only borrowed.** If you have a `&T` (a reference), you may read it but not move the value out — that would leave the real owner with a hole. (Chapter 8 covers references in full.)
- **Passing a non-Copy value to a function gives it away.** If you still need the value afterward, borrow it with `&` instead of passing it by value.

Interview questions

Q: State the three rules of ownership. A: Each value has exactly one owner; there is only one owner at a time (ownership can move but is never shared); and when the owner goes out of scope the value is dropped — its memory and resources released automatically.

Q: What is a “move,” and why can you not use the source variable afterward? A: For a non-Copy type, assigning or passing a value transfers ownership to the destination and marks the source as uninitialized. Only the new owner may use or drop the value, which guarantees a single owner and exactly one free. Using the moved-from variable is compile error E0382.

Q: What is the difference between Copy and Clone? A: Copy is an implicit, cheap, bit-for-bit duplication for simple stack-only types; both the original and the copy stay usable. Clone is an

explicit copy you request with `.clone()`; it can be deep and expensive (e.g. allocating a new heap buffer for a `String`). Copy types are always also `Clone`.

Q: How does Rust free memory without a garbage collector, and when does it happen?

A: Through ownership and the `Drop` trait (RAII). When a value’s owner goes out of scope, the compiler inserts a call to drop it, freeing its heap memory and other resources. This is deterministic — it happens at the closing brace of the owning scope, in reverse order of declaration — with no runtime collector.

Q: Why does `String` move but `i32` copy on assignment? A: `i32` is a small value that lives entirely on the stack and owns no resource, so duplicating its bytes is a perfect independent copy — it is `Copy`. `String` owns a heap buffer; copying just its header bytes would create two owners of one buffer, risking a double-free. So `String` is not `Copy` and assignment moves it instead.

Q: Why is the “take ownership and return it” pattern awkward, and what is the fix?

A: A function that only needs to read a value would have to take ownership and then return it so the caller can keep using it, threading the value in and out by hand. The fix is borrowing: pass a reference (`&T` or `&mut T`) so the function can use the value without owning it, and the caller keeps ownership (Chapter 8).

Try it

1. Implement `Drop` for a small struct that prints in `drop`. Create three of them and watch them drop in reverse order at the end of `main`. Then `drop()` one early and see the order change.
2. Write `let a = String::from("hi"); let b = a; println!("{}", a);` and read the full E0382 message — notice it suggests `.clone()`.
3. Change `a` to an `i32` and confirm the same code now compiles, because `i32` is `Copy`.
4. Write a function that takes a `String` by value, then rewrite it to borrow with `&String`, and observe that the caller keeps the value in the second version.

Chapter 8 — Borrowing and References

What you’ll learn. How to let code use a value without taking ownership of it, using references (`&T` and `&mut T`). You will learn the two borrow rules — “aliasing XOR mutation” — why they exist, and how the borrow checker enforces them. You will see how Rust references differ from C pointers: never null, always valid, checked at compile time.

In Chapter 7 — Ownership and Moves you saw that every value has one owner, and that assigning or passing a value can *move* it. Moving is safe, but it is also heavy: if a function had to take ownership of every value it touched, you would spend all day moving things back and forth.

Borrowing fixes this. A **borrow** lets a function look at, or change, a value *without* taking ownership. The owner keeps the value; the borrower gets a temporary, checked pointer to it. This is the everyday way Rust code passes data around.

Mental model. Ownership is lending someone your car for good — they now own it. Borrowing is lending it for the afternoon — they drive it, then give it back, and it was yours the whole time.

References: &T and &mut T

A **reference** is a value that points to another value without owning it. You make one with the `&` operator. There are two kinds:

- `&T` — a **shared reference** (also called an immutable borrow). You may read the value through it, but not change it. You may have many of these at once.
- `&mut T` — a **mutable reference** (also called an exclusive borrow). You may read *and* change the value through it. You may have only one at a time.

A reference is a real pointer at the machine level — the same size as a C pointer (`usize`, the machine word). The difference is what the compiler guarantees about it, not how it is represented in memory.

```
fn main() {
    let x = 10;
    let r: &i32 = &x; // r borrows x (shared)
    println!("x is {x}, r points to {}", *r);
}
```

C vs Rust. `&x` looks exactly like C’s “address of” operator, and at runtime it *is* a pointer to `x`. The difference is the type system: a Rust `&i32` is guaranteed to point at a live, initialized `i32`, and the compiler proves it. A C `int *` carries no such promise.

Dereferencing with *

The `*` operator follows a reference to the value it points at — the same as in C. You use it to read or write through the reference.

```
fn main() {
    let mut n = 5;
    let r = &mut n; // exclusive borrow of n
    *r += 1; // change n through the reference
    println!("{n}"); // prints 6
}
```

In practice you rarely write `*` by hand. When you call a method, Rust inserts the dereference for you. This is called **auto-deref**: `point.length()` works whether `point` is a `Point`, a `&Point`, or a `&&Point` — the compiler adds as many `*` as needed. (Methods come in Chapter 11 — Structs and Methods.)

Borrowing in functions

The most common use of references is passing data to a function. Compare the two ways of giving a `String` to a function.

Taking ownership (a move) — the caller loses the value:

```
// COMPILER ERROR: borrow of moved value: `s`
fn print_len(s: String) {
    println!("{}", s.len());
} // s is dropped here
```

```
fn main() {
    let s = String::from("hello");
    print_len(s); // s MOVES into the function
    println!("{}", s); // error[E0382]: s was moved away
}
```

Borrowing instead — the caller keeps the value:

```
fn print_len(s: &String) {
    println!("{}", s.len()); // read through the shared reference
}
```

```
fn main() {
    let s = String::from("hello");
    print_len(&s); // lend s; ownership stays in main
    println!("{}", s); // fine: s is still ours
}
```

To let a function change the value, pass `&mut`. The binding must be `mut`, and you write `&mut` at the call site too, so the change is visible at a glance.

```
fn push_world(s: &mut String) {
    s.push_str(", world"); // change the caller's String in place
}

fn main() {
    let mut s = String::from("hello"); // must be `mut` to borrow mutably
    push_world(&mut s);
    println!("{}", s); // prints "hello, world"
}
```

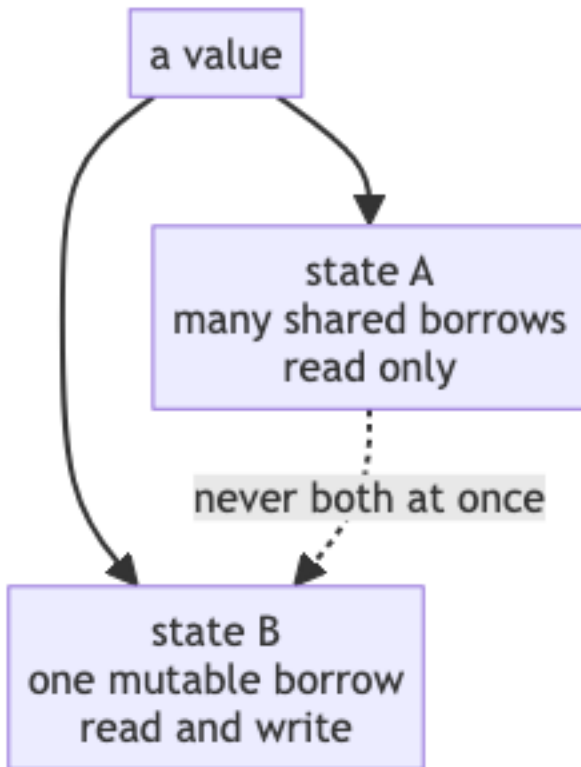
C vs Rust. This is the Rust version of passing a pointer in C. `&String` is like `const char *` (read only), and `&mut String` is like `char *` (you may write). The difference: C does not enforce `const`, and nothing stops two writable pointers from aliasing the same buffer. Rust enforces both.

The borrow rules

Here are the two rules at the center of the whole language. Memorize them.

1. At any given time, for a given value, you may have **either**:
 - any number of shared references (`&T`), **or**
 - exactly one mutable reference (`&mut T`)
 but **never both at once**.
2. A reference must **never outlive** the value it points to.

People summarize rule 1 as “**aliasing XOR mutation**”: you may have aliasing (many readers) or mutation (one writer), but not both at the same time. (“XOR” means exactly one of the two, never both.)



Why these rules exist

These rules are not bureaucracy. They prevent two whole classes of bug that C programmers chase for hours.

Data races. If two pointers can write the same memory at the same time, or one writes while another reads, you have a data race. Aliasing XOR mutation makes that impossible: a writer is always alone. This is the foundation of Rust’s “fearless concurrency” (Chapter 19 — Threads and Concurrency).

Invalidated pointers. This is the one that bites C programmers most. Consider a vector that grows. When a `Vec` runs out of capacity, it allocates a bigger buffer and copies the elements over — exactly like `realloc`. Any pointer into the old buffer is now dangling.

```
/* C: classic dangling-pointer bug */
int *p = &vec.data[0];    /* pointer into the buffer */
vec_push(&vec, 42);        /* may realloc -> old buffer freed */
printf("%d\n", *p);        /* use-after-free: undefined behavior */
```

Rust rejects the equivalent at compile time. `push` needs `&mut` on the vector, but `first` is a live `&` borrow into it — aliasing XOR mutation forbids both:

```
// COMPILE ERROR: cannot borrow `v` as mutable because it is also borrowed as immutable
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0]; // shared borrow into the buffer
```

```

    v.push(4); // error[E0502]: needs &mut while `first` is live
    println!("{first}"); // `first` would be dangling after a realloc
}

```

The borrow checker stops the bug before the program runs. In C, you find it later with a crash or a security report.

The borrow checker in action

The **borrow checker** is the part of the compiler that enforces the two rules. It tracks, for every reference, the region of code where it is used, and checks that no two borrows conflict.

Cannot mix & and &mut

You cannot take a &mut while a & is still in use (this is error E0502):

```

// COMPILER ERROR: cannot borrow `s` as mutable because it is also borrowed as immutable
fn main() {
    let mut s = String::from("hi");
    let r = &s; // shared borrow starts
    s.push('!'); // error[E0502]: needs &mut while `r` is live
    println!("{r}"); // `r` used here, so the shared borrow is still alive
}

```

Cannot have two &mut

You cannot have two mutable references to the same value at once (error E0499):

```

// COMPILER ERROR: cannot borrow `s` as mutable more than once at a time
fn main() {
    let mut s = String::from("hi");
    let a = &mut s;
    let b = &mut s; // error[E0499]: second &mut while `a` is live
    a.push('!');
    b.push('?');
}

```

Non-lexical lifetimes: a borrow ends at its last use

A borrow does **not** last until the end of the enclosing block. It ends at its **last use**. This rule is called **NLL** (non-lexical lifetimes). It makes the two examples above easy to fix: just finish using the first borrow before starting the next.

```

fn main() {
    let mut s = String::from("hi");

    let r = &s; // shared borrow starts
    println!("{r}"); // ...and ends here, at its last use

    s.push('!'); // fine now: no live shared borrow
}

```

```

    let a = &mut s; // a fresh exclusive borrow
    a.push('?'); // last use of a
    println!("{s}"); // fine: a is done
}

```

Mental model. A borrow is alive only between the `&` and the last line that uses it. Think of it as a short window, not a whole scope. Once the window closes, the value is free to be borrowed again — even differently.

time ->

```

&s borrow:    [---- used ----]
&mut s:              [---- used ----]
                  ^ allowed: the first window already closed

```

Dangling references are impossible

Rule 2 says a reference must not outlive its value. The classic C mistake is returning a pointer to a local variable or local buffer. C compilers may warn, but they will build it, and the result is undefined behavior:

```

/* C: returns a pointer to a destroyed local buffer */
char *make_greeting(const char *name) {
    char buf[64];
    snprintf(buf, sizeof buf, "Hello, %s", name);
    return buf; /* buf dies when the function returns; pointer dangles */
}

```

The Rust equivalent simply does not compile (error E0515). Here `make_greeting` builds a new `String` in a local variable and tries to hand back a reference into it:

```

// COMPILER ERROR: cannot return reference to local variable `s`
fn make_greeting(name: &str) -> &str {
    let s = format!("Hello, {name}");
    s.as_str() // error[E0515]: returns a reference to data owned by this function
}

fn main() {
    let g = make_greeting("Ada");
    println!("{g}");
}

```

The return type `&str` borrows from the input `name` (the compiler infers this; see Chapter 9 — Lifetimes), but the body returns a reference into `s`, a local that is dropped when the function ends. The fix is to return the **owned** value, moving it to the caller, instead of a reference into a local:

```

fn make_greeting(name: &str) -> String {
    format!("Hello, {name}") // move the new String out to the caller
}

fn main() {

```

```

let g = make_greeting("Ada");
println!("{g}"); // prints "Hello, Ada"
}

```

C vs Rust. In C, “do not return the address of a local” is a rule *you* must remember. In Rust the compiler proves it for you. When a function returns a reference that the compiler cannot tie to a still-living value, you get a clear error instead of a crash months later. The machinery that lets a function return a reference safely — by naming which input it comes from — is the subject of Chapter 9 — Lifetimes.

References vs C pointers

References look like pointers and compile to pointers, but the guarantees are very different.

Property	C pointer (T *)	Rust reference (&T / &mut T)
Can be null	Yes (NULL)	No, never
Can dangle (point at freed/dead memory)	Yes	No, the compiler prevents it
Can be uninitialized	Yes	No, must point at a live value
Multiple writers to same data	Allowed	Forbidden (one &mut at a time)
const-ness enforced	No (advisory)	Yes (&T cannot write)
Pointer arithmetic	Yes	No (use slices and indices)
Checked by the compiler	No	Yes (the borrow checker)

Mental model. A C pointer is a raw address with no promises. A Rust &T is “a valid, read-only view of a T that is guaranteed to stay alive while I hold it.” A &mut T is the same, but exclusive and writable. The compiler keeps both promises so you do not have to.

Deep dive. Rust *does* have raw pointers, `*const T` and `*mut T`, which are exactly C pointers: nullable, unchecked, no borrow rules. You can only dereference them inside an `unsafe` block. They exist for FFI and low-level code (Chapter 25 — Unsafe and FFI). In normal Rust you never need them.

A preview: &self and &mut self

When you write methods on a type (Chapter 11 — Structs and Methods), the same borrowing rules apply to the receiver. A method that only reads takes `&self` (a shared borrow of the value), and one that changes the value takes `&mut self` (an exclusive borrow). It is the same `&` / `&mut` distinction, applied to the value the method is called on.

```

struct Counter {
    n: i32,
}

impl Counter {
    fn get(&self) -> i32 {
        self.n // reads only: shared borrow
    }
}

```

```

    }
    fn bump(&mut self) {
        self.n += 1; // changes self: exclusive borrow
    }
}

fn main() {
    let mut c = Counter { n: 0 };
    c.bump();
    println!("{}", c.get()); // prints 1
}

```

Key takeaways

- A **reference** lets code use a value without owning it. `&T` is a shared (read-only) borrow; `&mut T` is an exclusive (read-write) borrow.
- The **borrow rules**: either many `&T` or exactly one `&mut T`, never both at once; and a reference must never outlive its value. Summary: **aliasing XOR mutation**.
- These rules prevent **data races** and **invalidated-pointer** bugs (such as a pointer into a `Vec` that then reallocates) at compile time.
- A borrow ends at its **last use**, not at the end of the block (non-lexical lifetimes), so code after it can borrow again.
- Returning a reference to a local variable **fails to compile** (E0515). In C that same code is a classic undefined-behavior bug.
- To take a `&mut`, the binding must be declared `mut`.
- Rust references are **never null**, **never dangling**, **always valid**, and checked by the compiler — unlike C pointers.

Watch out (gotchas for C programmers)

- **No `&mut` while a `&` is live**. You cannot mutate through one path while another path is reading (E0502). End the shared borrow first.
- **Only one `&mut` at a time** (E0499). Two writable references to the same value are forbidden, even in single-threaded code.
- **A borrow’s life ends at its last use, not its scope**. If the compiler complains, moving the last use earlier (or into a smaller block) often fixes it.
- **You cannot return a reference to a local variable** (E0515). Return the value by move instead, or borrow from an input (Chapter 9 — Lifetimes).
- **The binding must be `mut` to borrow `&mut`**. `let s = ...; &mut s` fails; write `let mut s = ...`.
- **`&` is “address of” and a type constructor**. `&T` in a type position means “reference to `T`”; `&x` in an expression means “borrow `x`”. Same symbol, two roles.

Interview questions

Q: What are the two borrow rules in Rust? A: First, for a given value you may have either any number of shared references (`&T`) or exactly one mutable reference (`&mut T`), but never both at the same time. Second, a reference must never outlive the value it points to. Together they are summarized as “aliasing XOR mutation.”

Q: Why does Rust forbid a `&mut` while a `&` is alive? A: To prevent data races and invalidated pointers. If a value could be mutated while another reference reads it, a write could move or free the underlying data (for example, a `Vec` reallocating) and leave the reader dangling. Allowing only one writer, and no readers while it writes, makes that impossible.

Q: How does a Rust reference differ from a C pointer? A: A reference is guaranteed non-null, never dangling, always pointing at a live initialized value, and subject to the borrow rules — all checked by the compiler. A C pointer has none of these guarantees: it can be null, dangling, or uninitialized, and nothing prevents two writable pointers to the same memory.

Q: What does “non-lexical lifetimes” mean, and why does it matter? A: A borrow’s lifetime ends at its last use, not at the end of the enclosing block. This lets you start a new (even conflicting) borrow as soon as the previous one is done being used, so much ordinary code compiles without needing to add extra scopes.

Q: Why does returning a reference to a local variable fail to compile, and what is the C equivalent? A: The local is dropped when the function returns, so the reference would dangle; Rust rejects this with E0515. In C the same pattern (`return &local;`) compiles but is undefined behavior — a use-after-return bug. The Rust fix is to return the value by move.

Try it

1. Write `fn add_one(n: &mut i32) { *n += 1; }` and call it from `main` on a `let mut x = 1;`. Then remove `mut` from `x` and read the error.
2. Make a `let mut v = vec![1, 2, 3];`, take `let first = &v[0];`, then call `v.push(4);` before printing `first`. Read the E0502 message, then move the `println!("{first}")` to *before* the `push` and watch it compile.

Chapter 9 — Lifetimes

What you’ll learn. What a lifetime is, why it scares people more than it should, and how to read and write the `'a` syntax. You will see why some functions need lifetime annotations, why most code needs none (elision), what `'static` means, and how lifetimes are just the compiler proving that references stay valid.

Take a breath. Lifetimes have a reputation for being the hardest part of Rust, but the core idea is small and you already understand it from C. A lifetime is just “how long is this pointer safe to use?” — a question you ask yourself constantly in C. The only new thing is that Rust writes the answer down and checks it.

What a lifetime is

A **lifetime** is a region of the program during which a reference is valid — that is, during which the value it points to is guaranteed to still be alive. Every reference has a lifetime. Most of the time the compiler works it out silently and you never see it.

The key point, and the one that calms most fears:

Mental model. Lifetimes are *labels*, not *lifespans*. A lifetime annotation does not change how long any value lives. It only gives a name to a region the compiler is

already tracking, so the compiler can compare two regions and check that one fits inside the other.

There is **no runtime cost**. Lifetimes are checked at compile time and then erased. They produce no code, no extra data, and no slowdown. They are purely a proof that your references are safe — the same reasoning you do in your head in C, but enforced by the compiler.

C vs Rust. In C, “how long is this pointer good for?” lives only in comments and in your memory. If you get it wrong, you get a dangling pointer and undefined behavior. In Rust, the answer is part of the type, and the compiler refuses to build code where a reference could outlive its data.

Why functions need lifetimes

Most of the time you do not write lifetimes. You need them in one main situation: **a function returns a reference, and the compiler cannot tell which input it borrows from.**

Here is a function that returns the longer of two string slices. (A `&str` is a borrowed view into string data; see Chapter 10 — Slices and Strings.) This version does not compile:

```
// COMPILER ERROR: missing lifetime specifier
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
} // error[E0106]: expected named lifetime parameter
```

The compiler’s problem: the returned reference points into either `x` or `y`, but it cannot tell which. Without that knowledge it cannot check whether the caller’s reference will still be valid. So it asks you to label the connection.

The fix is to add a **lifetime parameter** named `'a` and say “all three references share the lifetime `'a`”:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let a = String::from("longer string");
    let b = String::from("short");
    let result = longest(&a, &b);
    println!("the longest is: {result}");
}
```

Read `<'a>` as “for some lifetime `'a`”. The signature now promises: the returned reference is valid for `'a`, which is the *shorter* of the two input lifetimes. The caller’s value must live at least that long. The compiler can now check every call.

Mental model. The annotation does not *make* anything live longer. It is a contract: “the output borrows from the inputs, so it cannot outlive them.” The compiler enforces the contract at each call site.

Syntax

The pieces are:

- `'a` — a lifetime name. It always starts with a single quote. The name is arbitrary; `'a`, `'b`, `'input` all work. `'a` is the convention.
- `<'a>` after the function name — declares the lifetime parameter, like declaring a generic type parameter.
- `&'a str` — a reference with lifetime `'a`.

C vs Rust. The `'a` syntax looks alien, but it plays the same role as a comment like `/* return value points into one of the arguments; do not use it after either argument is freed */` — except the compiler reads it and enforces it.

Lifetimes on structs

If a struct holds a reference, the struct needs a lifetime parameter. This tells the compiler that the struct must not outlive the data it borrows.

```
struct Excerpt<'a> {
    part: &'a str, // a borrowed view, not owned data
}

fn main() {
    let novel = String::from("Call me Ishmael. Some years ago...");
    let first = novel.split('.').next().expect("no sentence");
    let e = Excerpt { part: first }; // e borrows from novel
    println!("{}", e.part);
} // e must not outlive novel; the compiler checks this
```

The `<'a>` after the struct name declares the lifetime; `part: &'a str` uses it. The meaning: an `Excerpt` value cannot live longer than the string it points into.

C vs Rust. This is the common C pattern of a struct that holds a `char *` borrowed from somewhere else. In C, nothing stops the struct from outliving the buffer, and you get a dangling field. In Rust, the `'a` ties the struct's life to the buffer's, and the compiler enforces it.

Lifetimes on impl blocks

When you write methods on a struct that has a lifetime, you declare the lifetime on the `impl` block too:

```
struct Excerpt<'a> {
    part: &'a str,
}

impl<'a> Excerpt<'a> {
    fn announce(&self, msg: &str) -> &str {
        println!("Attention: {msg}");
        self.part
    }
}
```



```

}

fn main() {
    let text = String::from("Hello. World.");
    let first = text.split('.').next().unwrap();
    let e = Excerpt { part: first };
    println!("{}", e.announce("listen"));
}

```

The `impl<'a> Excerpt<'a>` says “for any lifetime 'a, here are methods on `Excerpt<'a>`”. You usually do not have to annotate the method bodies thanks to elision, described next.

Lifetime elision: why most code needs no lifetimes

If references needed explicit lifetimes everywhere, Rust would be unbearable. They do not. The compiler applies a few rules — **lifetime elision** — that fill in the obvious cases for you. “Elision” means “leaving out something that can be inferred.”

There are three rules the compiler uses on function signatures:

1. Each reference parameter gets its **own** lifetime. `fn f(x: &T, y: &T)` is treated as `fn f<'a, 'b>(x: &'a T, y: &'b T)`.
2. If there is **exactly one** input lifetime, it is assigned to **all** output references. `fn f(x: &T) -> &U` becomes `fn f<'a>(x: &'a T) -> &'a U`.
3. If there is a `&self` or `&mut self` parameter (a method), **self’s lifetime** is assigned to all output references.

If the rules cover the whole signature, you write no lifetimes. If they leave any output reference without a lifetime, the compiler stops and asks you to annotate — which is exactly what happened with `longest`, where rule 2 did not apply because there were two inputs.

```

// Rule 2 applies: one input lifetime flows to the output. No annotation needed.
fn first_word(s: &str) -> &str {
    s.split(' ').next().unwrap_or("")
}

fn main() {
    let phrase = String::from("hello rust world");
    println!("{}", first_word(&phrase));
}

// Rule 3 applies: the output borrows from &self. No annotation needed.
struct Wrapper {
    text: String,
}

impl Wrapper {
    fn get(&self) -> &str {
        &self.text
    }
}

```

```
fn main() {
    let w = Wrapper { text: String::from("hi") };
    println!("{}", w.get());
}
```

Rule of thumb. Write code without lifetimes first. Add 'a only when the compiler asks. Nearly all everyday code compiles with no explicit lifetimes because elision covers it.

The 'static lifetime

There is one special, built-in lifetime: 'static. It means “this reference is valid for the entire run of the program.” The data it points to lives forever.

The most common 'static references are **string literals**. A literal like "hello" is baked into the program's binary, so it is always there. Its type is &'static str.

```
fn greeting() -> &'static str {
    "hello, world" // lives for the whole program; safe to return
}

fn main() {
    let g: &'static str = greeting();
    println!("{}", g);
}
```

Notice this is allowed where returning &local was not: the literal is not a local that dies at the end of the function. It lives in static storage, like a C string literal or a static const global.

C vs Rust. &'static str is close to a C string literal (const char * to a literal in .rodata): it exists for the whole program, so a pointer to it never dangles. Rust just writes that fact into the type.

Watch out. 'static is often misused. When the compiler complains about a lifetime, beginners sometimes “fix” it by sprinkling 'static until it builds. That usually changes the meaning to “this must live forever,” which is rarely what you want and often will not compile anyway, or forces awkward design. The right fix is almost always to relate the output lifetime to an input, or to return an owned value (like String) instead of a reference.

The classic lifetime errors and their fixes

You have already seen the first error. Here are the two you will meet most, with fixes.

“missing lifetime specifier”

A function returns a reference and the elision rules cannot decide its lifetime:

```
// COMPILER ERROR: missing lifetime specifier
fn pick(x: &str, y: &str) -> &str {
```

```

    if x.len() > y.len() { x } else { y }
} // error[E0106]

```

Fix: name a lifetime and tie the output to the inputs.

```

fn pick<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    println!("{}", pick("apple", "fig"));
}

```

“returns a value referencing data owned by the current function”

This one is really the dangling-reference error from Chapter 8 — Borrowing and References, seen through the lens of lifetimes. Here the output lifetime is clear (elision rule 2 ties it to the input `s`), but the body returns a reference into **owned**, a brand-new local that the function owns and then drops:

```

// COMPILER ERROR: returns a value referencing data owned by the current function
fn shout(s: &str) -> &str {
    let owned = s.to_uppercase(); // a new String, owned by this function
    &owned // error[E0515]: `owned` is dropped at the end, so the reference dangles
}

```

No lifetime annotation can save it — the data genuinely dies when the function returns. The fix is to return the **owned** value, not a reference into it:

```

fn shout(s: &str) -> String {
    s.to_uppercase() // move the new String out to the caller; nothing dangles
}

fn main() {
    let loud = shout("hello");
    println!("{}", loud); // prints "HELLO"
}

```

Rule of thumb. If a function builds new data, return it by value (an owned `String`, `Vec`, `struct`, ...). Return a reference only when you are handing back a view into something a caller already owns.

Picture: overlapping vs disjoint lifetimes

A reference is valid only while its target is alive. The borrow checker compares these regions. If the borrow’s region fits inside the owner’s region, it is safe. If the borrow reaches past the owner’s end, it is rejected.

OK: borrow ends before the owner does

```

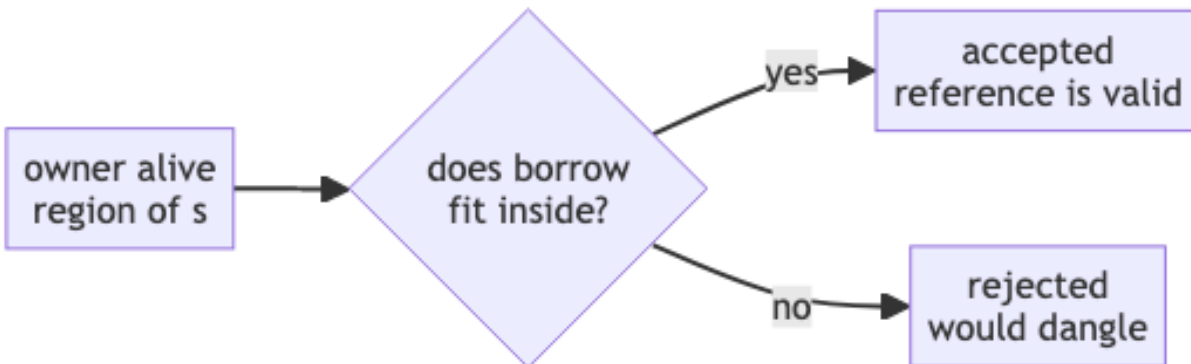
owner s      |=====|
borrow r      |=====|

```

^ last use of r, still inside s

ERROR: borrow would outlive the owner

```
owner s    |=====|
borrow r    |=====|
               ^ r used here, but s is already gone -> rejected
```



Key takeaways

- A **lifetime** is a compile-time region during which a reference is valid. Every reference has one; usually it is inferred.
- Lifetime annotations **describe and check**; they do **not** change how long values live and have **no runtime cost**.
- A function that returns a reference must tie the output's lifetime to an input, e.g. `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`.
- A **struct that holds a reference** needs a lifetime parameter (`struct Excerpt<'a> { part: &'a str }`); so does its `impl` block.
- **Elision** (three rules) fills in lifetimes for the common cases, so most code needs no explicit `'a`.
- `'static` means “valid for the whole program”; string literals are `&'static str`. Do not reach for it to silence errors.
- The two classic errors — “missing lifetime specifier” and “returns a value referencing data owned by the current function” — are fixed by relating the output to an input, or by returning an owned value.

Watch out (gotchas for C programmers)

- **Lifetimes do not extend life.** Adding `'a` never makes a value live longer; it only states a relationship the compiler then checks. To keep data alive, own it.
- **Elision handles most cases.** Do not annotate by reflex. Write code plain, and add `'a` only when the compiler asks.
- **A struct holding a reference needs a lifetime parameter.** If you store a `&T` in a struct and forget `<'a>`, you get E0106. Often the better fix is to store an owned value instead.
- **A returned reference's lifetime must come from an input.** You cannot conjure a valid output reference out of data the function owns; return it by value.

- **'static is usually the wrong fix.** It means “lives forever.” Reach for it only when the data really is static (like a literal), not to quiet the borrow checker.
- **The ' is not a typo.** 'a is a lifetime name; 'a' (with a closing quote) is a `char`. Watch the quotes.

Interview questions

Q: What is a lifetime in Rust, and does it affect runtime? A: A lifetime is a compile-time region during which a reference is valid. It is used only by the borrow checker to prove references do not outlive their data. There is no runtime representation and no runtime cost; lifetimes are erased after checking.

Q: Why does `fn longest(x: &str, y: &str) -> &str` need a lifetime annotation? A: It returns a reference that could point into either argument, and the elision rules cannot decide which, so the compiler cannot verify the result is valid. Writing `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str` ties the output’s lifetime to the inputs, so the compiler can check each call.

Q: Do lifetime annotations change how long a value lives? A: No. They only name and relate regions the compiler already tracks. They are a contract the borrow checker verifies; they never extend or shorten any value’s actual life. To make data live longer, you change ownership, not annotations.

Q: What are the lifetime elision rules, in brief? A: One, each reference parameter gets its own lifetime. Two, if there is exactly one input lifetime, it is given to all output references. Three, if there is a `&self/&mut self`, its lifetime is given to all output references. If these leave an output without a lifetime, you must annotate explicitly.

Q: What is the 'static lifetime, and what is a common mistake with it? A: 'static means a reference is valid for the entire program; string literals are `&'static str`. The common mistake is using 'static to silence a borrow error, which forces the data to live forever and usually does not solve the real problem. The right fix is normally to tie the output to an input lifetime or return an owned value.

Try it

1. Write `longest` without the `<'a>` annotations and read the E0106 error. Add the lifetimes and watch it build.
2. Make a `struct Holder { s: &str }` and try to compile it. Read the error, then add `<'a>` (`struct Holder<'a> { s: &'a str }`). As a second fix, change the field to an owned `String` and notice the lifetime disappears.
3. Write a function that builds a `String` and tries to return `&str` to it. Read “returns a value referencing data owned by the current function,” then change the return type to `String`.

Chapter 10 — Slices and Strings

What you’ll learn. What a slice is — a borrowed, bounds-checked view into a contiguous sequence — and how `Vec<T>` owns the data a slice points into. Then how Rust handles text: `String` versus `&str`, why strings are UTF-8 and not NUL-terminated, and why `s[0]` does not compile.

Slices: a borrowed view into a sequence

A **slice** is a borrowed view into a run of values that sit next to each other in memory. “Borrowed” means a slice does not own the data; it just points at part of something else (see Chapter 8 — Borrowing and References). “Contiguous” means the values are laid out back to back, like a C array.

The type of a slice of `T` is written `&[T]` — “a shared reference to a sequence of `T`.” You make one with a **range** inside square brackets:

```
fn main() {
    let v = vec![10, 20, 30, 40, 50];
    let mid = &v[1..4]; // elements at index 1, 2, 3 -> [20, 30, 40]
    println!("{mid:?}"); // prints [20, 30, 40]
    println!("len = {}", mid.len());
}
```

The range `1..4` includes the start (1) and excludes the end (4). This is the same half-open range you already use in C for loops (`for (i = 1; i < 4; i++)`).

A slice is **not** a pointer like C’s. In C, when you pass an array to a function, it decays to a bare pointer and the length is lost. You must pass the length as a second argument and hope the caller gets it right:

```
/* C: pointer plus a separate length; nothing checks they agree */
int sum(const int *data, size_t len) {
    int total = 0;
    for (size_t i = 0; i < len; i++)
        total += data[i];
    return total;
}
```

In Rust the length travels *with* the slice. A slice is a **fat pointer**: a pair of (data pointer, length) stored together. So the function needs only one argument, and every index is checked against that length.

```
fn sum(data: &[i32]) -> i32 {
    let mut total = 0;
    for &x in data {
        total += x;
    }
    total
}

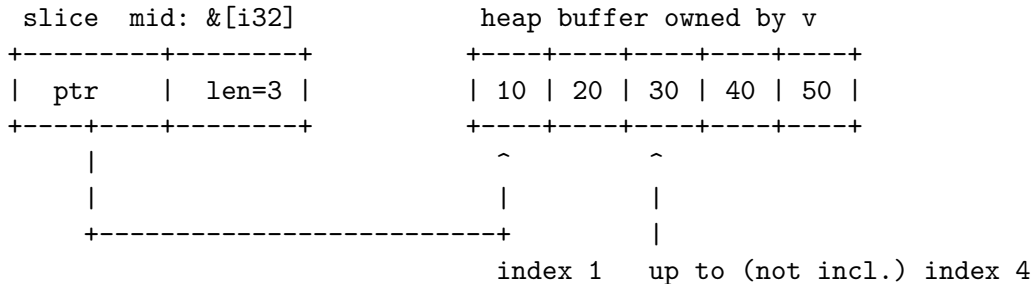
fn main() {
    let v = vec![1, 2, 3, 4];
    println!("{}", sum(&v)); // a &Vec<i32> coerces to &[i32]
    println!("{}", sum(&v[1..3])); // a sub-slice works too
}
```

Mental model. A C array argument is “here is where it starts; trust me about how long it is.” A Rust slice is “here is where it starts **and** exactly how long it is.” The

length is not optional and not separate.

A slice is a fat pointer (ASCII layout)

A slice value is two machine words: a pointer to the first element and a count of elements. It does not copy the data. Here is `&v[1..4]` pointing into the buffer that the `Vec` owns:



The slice borrows; it owns nothing. When `mid` goes out of scope, nothing is freed — the `Vec` still owns the buffer.

Slices are bounds-checked

If a range or index goes past the end, Rust does not read past the buffer the way C would. It **panics** — it stops the program safely instead of causing undefined behavior.

```
fn main() {  
    let v = vec![1, 2, 3];  
    let bad = &v[1..9]; // panics: range end index 9 out of range for slice of length 3  
    println!("{bad:?}");  
}
```

C vs Rust. In C, `data[9]` on a 3-element array is undefined behavior — maybe a crash, maybe silent corruption, maybe an exploit. In Rust the access is checked and the program panics with a clear message. Safety first.

Range shorthand

You can leave out either end of a range:

Syntax	Meaning	C-style loop bounds
<code>&v[2..5]</code>	index 2 up to but not including 5	<code>i = 2; i < 5</code>
<code>&v[..5]</code>	start up to but not including 5	<code>i = 0; i < 5</code>
<code>&v[2..]</code>	index 2 to the end	<code>i = 2; i < len</code>
<code>&v[..]</code>	the whole thing as a slice	<code>i = 0; i < len</code>
<code>&v[2..=5]</code>	index 2 through 5 inclusive	<code>i = 2; i <= 5</code>

Mutable slices

`&[T]` is a read-only view. For a view you can write through, use `&mut [T]`. The same borrow rules apply: while a `&mut` slice exists, no other access to that data is allowed (Chapter 8 — Borrowing and References).

```
fn double_all(xs: &mut [i32]) {
    for x in xs.iter_mut() {
        *x *= 2;
    }
}

fn main() {
    let mut v = vec![1, 2, 3];
    double_all(&mut v);
    println!("{v:?}"); // [2, 4, 6]
}
```

Vec<T>: the owned array a slice borrows from

A slice has to point *into* something. Most often that something is a `Vec<T>` — Rust’s **owned, growable array**. We met it in Chapter 1; here is the part that matters for slices. (Full treatment in Chapter 16 — Collections and Iterators.)

A `Vec<T>` is three words: a pointer to a heap buffer, the **length** (how many elements are in use), and the **capacity** (how many fit before it must grow). It owns its buffer and frees it automatically when the `Vec` is dropped.

Vec<i32> on the stack	heap buffer (capacity 8)
+-----+-----+-----+	+---+---+---+---+---+---+---+
ptr len=3 cap=8 ----->	10 20 30 ?? ??
+-----+-----+-----+	+---+---+---+---+---+---+---+
	_____ in use _____/_free__/\

C vs Rust. A `Vec<T>` is the safe version of the C pattern “a `malloc`’d array plus a length plus a capacity that you grow with `realloc`.” Rust bundles the three together, grows it for you, bounds-checks every access, and calls `free` for you when the `Vec` goes out of scope.

The relationship is simple: the `Vec` **owns** the data; a slice **borrows** a view of it. `&v[..]` (or just passing `&v` where a slice is expected) hands out a (`ptr`, `len`) view without giving up ownership.

Strings: `String` and `&str`

Rust has two main string types, and the split mirrors `Vec<T>` versus `&[T]` exactly:

- **String** — an **owned**, growable, heap-allocated string. It is essentially a `Vec<u8>` that is guaranteed to hold valid UTF-8 text.
- **&str** — a **string slice**: a borrowed view into UTF-8 text. It is a fat pointer of (data pointer, byte length). You read it as “string slice” or “str.”

Mental model. `String` : `&str` is the same relationship as `Vec<T>` : `&[T]`. One owns the heap buffer; the other is a borrowed (`ptr`, `len`) window into it.

A **string literal** like `"hello"` has type `&'static str`. The text is baked into the program’s read-only data, so the slice is valid for the entire run of the program. The `'static` is a lifetime meaning “lives forever” (Chapter 9 — Lifetimes).


```
fn main() {
    let owned: String = String::from("hello"); // owns heap memory
    let literal: &str = "world"; // borrowed view into program data
    let view: &str = &owned[..]; // borrowed view into `owned`

    println!("{owned} {literal} {view}");
}
```

A String points into a heap buffer (ASCII layout)

`String` is the same three words as `Vec`: pointer, length, capacity. The length and capacity are measured in **bytes**, not characters.

String on the stack	heap buffer (UTF-8 bytes)
+-----+-----+-----+	+-----+-----+-----+
ptr len=5 cap=8 ---->	'h' 'e' 'l' 'l' 'o'
+-----+-----+-----+	+-----+-----+-----+
	no NUL terminator at the end

A `&str` is just the first two of those words — a (ptr, len) pair — pointing at some bytes it does not own:

&str fat pointer
+-----+-----+
ptr len=5 ----> into a String, a literal, or any UTF-8 bytes
+-----+-----+

UTF-8 reality (the big difference from C)

This is where Rust text and C text part ways. Read it carefully.

In C, a string is `char *`: a pointer to bytes that ends at a `'\0'` (NUL) byte. The length is *not* stored; you find it by scanning for the NUL with `strlen`, which is $O(n)$. Each `char` is one byte, and the encoding is whatever you decide.

In Rust, a string is **UTF-8 bytes plus a stored length**, with **no NUL terminator**:

- **UTF-8.** Text is encoded as UTF-8. An ASCII character (`'A'`, `'7'`, `'?'`) is one byte. Other characters (`'é'`, `'€'`, `' '`, emoji) take **two to four bytes** each.
- **Length is stored**, so `.len()` is $O(1)$ — no scanning. But `.len()` returns the number of **bytes**, not the number of characters.
- **No NUL terminator.** The end is known from the length, so the bytes can contain a zero byte with no special meaning.

Concept	C <code>char *</code>	Rust <code>String</code> / <code>&str</code>
Storage	bytes ending in NUL	UTF-8 bytes plus a length
End marker	<code>'\0'</code> terminator	none; length is stored
Length cost	<code>strlen</code> scans, $O(n)$	<code>.len()</code> is $O(1)$
<code>.len()</code> unit	bytes (== chars for ASCII)	always bytes
Encoding	up to you	always valid UTF-8
Index <code>s[i]</code>	one byte	does not compile

Concept	C char *	Rust String / &str
Bounds checked	no	yes

Why `s[0]` does not compile

In C, `s[0]` is the first byte and you use it constantly. In Rust, integer indexing of a string is **rejected by the compiler**:

```
// COMPILER ERROR: the type `str` cannot be indexed by `{integer}`
fn main() {
    let s = String::from("héllö");
    let c = s[0]; // error[E0277]: `String` cannot be indexed by integer
    println!("{}", c);
}
```

The reason is the UTF-8 layout. A single index could land in the **middle** of a multi-byte character, which would give you half a character — meaningless. Rust refuses to let you ask an ambiguous question. Consider "héllö":

char:	h	é		l	l	ö	
bytes:	68	C3 A9		6C	6C	6F	(hex)
index:	0	1 2		3	4	5	

```
"héllö".len()           == 6    (bytes, not characters)
"héllö".chars().count() == 5    (characters)
```

The ‘é’ takes two bytes (C3 A9). Byte index 0 is ‘h’, but byte index 1 is only the *first half* of ‘é’. So “give me byte 1 as a character” has no good answer, and Rust will not pretend otherwise.

Iterate with `.chars()` or `.bytes()`

Instead of indexing, you iterate. Pick the unit you actually want:

```
fn main() {
    let s = "héllö";

    for c in s.chars() {
        // each `c` is a `char`: one Unicode scalar value (4 bytes wide)
        print!("{}", c);
    }
    println(); // [h][é][l][l][ö]

    for b in s.bytes() {
        // each `b` is a u8: one raw byte of the UTF-8 encoding
        print!("{}", b);
    }
    println(); // 104 195 169 108 108 111
}
```

A Rust `char` is **not** a C `char`. A C `char` is one byte. A Rust `char` is a **Unicode scalar value** and is always 4 bytes wide, able to hold any single character. Use `.chars()` when you mean “characters” and `.bytes()` when you mean “raw bytes.”

Watch out. `.len()` is the **byte** length. To count characters, use `s.chars().count()` — and know that it costs $O(n)$ because it must walk the UTF-8.

Slicing strings: only at character boundaries

You *can* slice a string with a byte range, and the result is a `&str`. But the range ends must fall on **character boundaries**. A range that splits a multi-byte character panics at runtime.

```
fn main() {
    let s = "hélllo"; // bytes: h | é(2 bytes) | l | l | o

    let head = &s[0..1]; // "h" -> ok, boundary
    let tail = &s[3..6]; // "llo" -> ok, boundary
    println!("{head} {tail}");

    // &s[0..2] would PANIC: byte index 2 is inside 'é'
    // thread 'main' panicked: byte index 2 is not a char boundary
}
```

Rule of thumb. Prefer methods that respect characters (`.chars()`, `.split()`, `.find()`, `.starts_with()`) over manual byte-range slicing. Slice by byte range only when you got the index from such a method (so you know it is a valid boundary).

Conversions between `String` and `&str`

You move between owned and borrowed text constantly. Here are the common moves:

```
fn main() {
    // &str -> String (allocates and copies)
    let a: String = String::from("hi");
    let b: String = "hi".to_string();
    let c: String = "hi".to_owned();

    // String -> &str (no allocation; just a view)
    let s = String::from("hello");
    let v1: &str = &s[..]; // full-range slice
    let v2: &str = s.as_str(); // explicit method
    let v3: &str = &s; // deref coercion (see below)

    // Build a String with format!, like a safe sprintf
    let name = "Ada";
    let msg: String = format!("hello, {name}!");

    println!("{a} {b} {c} {v1} {v2} {v3} {msg}");
}
```

Concatenation

You can join strings with `+`, but note the types: the left side must be an owned `String` and the right side a `&str`. The `+` **consumes** the left `String` (takes ownership) and returns a new `String`.

```
fn main() {
    let hello = String::from("hello, ");
    let world = "world"; // &str
    let greeting = hello + world; // `hello` is moved; cannot use it after
    println!("{greeting}");

    // For more than two pieces, format! is clearer:
    let a = "foo";
    let b = "bar";
    let joined = format!("{a}-{b}-{a}");
    println!("{joined}");

    // push_str / push grow a String in place:
    let mut s = String::from("ab");
    s.push_str("cd"); // append a &str
    s.push('!'); // append one char
    println!("{s}"); // abcd!
}
```

C vs Rust. In C you concatenate with `strcat` into a buffer you must size correctly, risking overflow. Rust's `+`, `format!`, and `push_str` allocate and grow as needed; you cannot overflow a buffer.

API guidance: borrow `&str` and `&[T]`, not `&String` and `&Vec<T>`

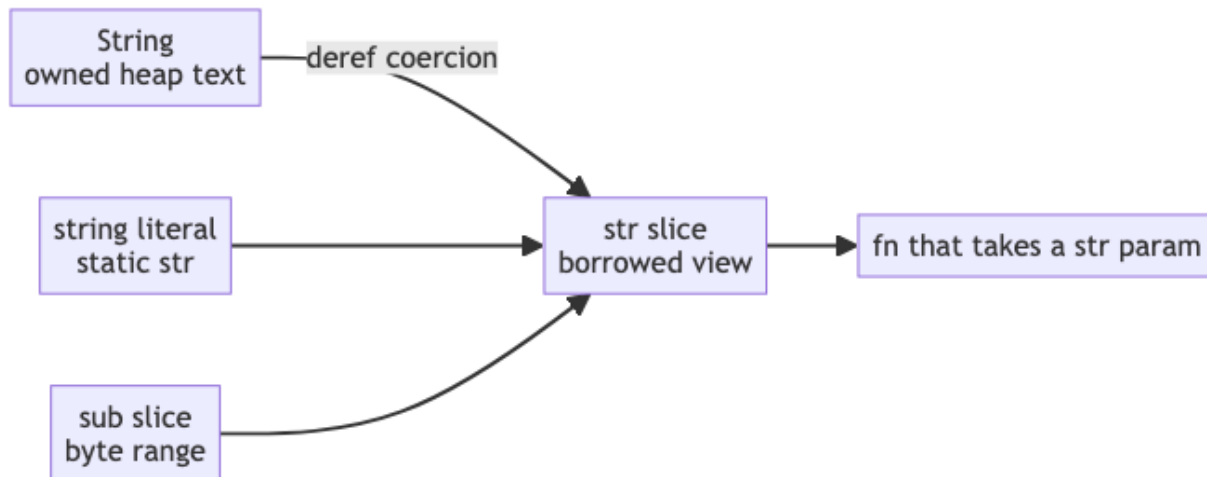
When a function only needs to *read* a string or a sequence, take a slice, not a reference to the owned type. That is, prefer:

- `fn f(s: &str)` over `fn f(s: &String)`
- `fn g(xs: &[i32])` over `fn g(xs: &Vec<i32>)`

A `&str` parameter accepts a `String`, a string literal, and a sub-slice. A `&String` parameter accepts only a `String`. The slice version is strictly more flexible, and it works because of **deref coercion**: the compiler automatically turns a `&String` into a `&str` (and a `&Vec<T>` into a `&[T]`) when a slice is expected.

```
fn shout(s: &str) -> String {
    s.to_uppercase()
}

fn main() {
    let owned = String::from("hi");
    println!("{}", shout(&owned)); // &String coerces to &str
    println!("{}", shout("there")); // a literal is already &str
    println!("{}", shout(&owned[0..1])); // a sub-slice works too
}
```



Rule of thumb. Accept the borrowed slice type (`&str`, `&[T]`) in function signatures; return the owned type (`String`, `Vec<T>`) when you create new data. This is the Rust equivalent of taking `const char *` in C, but type-safe and length-aware.

Key takeaways

- A **slice** `&[T]` is a borrowed, bounds-checked view into a contiguous sequence: a fat pointer of (data pointer, length). Make one with `&v[a..b]`.
- A **Vec**`<T>` is the owned, growable array (pointer, length, capacity). Slices borrow a view of the data a `Vec` (or array) owns.
- **String** is owned, growable, heap, UTF-8 text; `&str` is a borrowed view into UTF-8 text. The analogy is `String : &str :: Vec<T> : &[T]`.
- Rust strings are **UTF-8**, carry a **stored length**, and have **no NUL terminator**. `.len()` is the **byte** length, computed in $O(1)$.
- You cannot index a string by integer (`s[0]`); iterate with `.chars()` or `.bytes()`. Byte-range slicing works only on character boundaries, or it panics.
- Convert with `String::from` / `.to_string()` / `.to_owned()` (to owned) and `&s[..]` / `.as_str()` / `deref` (to borrowed); build with `format!`.
- In function signatures, take `&str` and `&[T]`; `deref coercion` makes `&String` and `&Vec<T>` arguments work automatically.

Watch out (gotchas for C programmers)

- **No `s[i]` indexing on strings.** A byte index could split a character. Use `.chars()`, `.bytes()`, or boundary-safe slicing.
- **`.len()` is bytes, not characters.** For ASCII they match; for any multi-byte character they differ. Count characters with `.chars().count()`.
- **Slicing mid-character panics.** `&s[0..2]` panics if byte 2 is inside a character. Get indices from string methods that respect boundaries.
- **`&str` is not `String`.** One is a borrowed view, the other owns heap memory. Do not store a `&str` longer than the data it borrows lives.
- **String literals are `&'static str`, not `String`.** Call `.to_string()` when you need an owned, growable value.

- **No NUL terminator.** Do not assume C string functions or a trailing `'\0'`. To call C, convert with `CString/CStr` (Chapter 25 — Unsafe Rust and FFI).
- **A Rust `char` is 4 bytes**, a Unicode scalar value — not a C one-byte `char`.

Interview questions

Q: What is a slice, and how does it differ from a C pointer to an array? A: A slice `&[T]` is a borrowed view into a contiguous sequence, stored as a fat pointer of (data pointer, length). Unlike a decayed C array pointer, it carries the length with it and bounds-checks every access, so out-of-range use panics instead of being undefined behavior.

Q: What is the relationship between `String` and `&str`? A: `String` is an owned, growable, heap-allocated UTF-8 buffer; `&str` is a borrowed view (a (ptr, len) fat pointer) into UTF-8 text it does not own. The relationship mirrors `Vec<T>` and `&[T]`: one owns, the other borrows.

Q: Why can't you write `s[0]` to get the first character of a Rust string? A: Rust strings are UTF-8, where characters can be one to four bytes. A single integer index could land inside a multi-byte character and yield half a character, which is meaningless. So the compiler forbids integer indexing; you iterate with `.chars()` or `.bytes()`, or slice on character boundaries.

Q: Does `.len()` return the number of characters in a string? A: No. It returns the number of bytes in the UTF-8 encoding, in $O(1)$. For ASCII that equals the character count, but any multi-byte character makes them differ. To count characters, use `s.chars().count()`, which is $O(n)$.

Q: Why prefer `&str` over `&String` (and `&[T]` over `&Vec<T>`) in function parameters? A: A `&str` accepts a `String`, a literal, and a sub-slice, so it is more general, while `&String` accepts only a `String`. Deref coercion converts `&String` to `&str` automatically, so you lose nothing and gain flexibility. The same reasoning applies to `&[T]` versus `&Vec<T>`.

Try it

1. Make `let s = "naïve";`. Print `s.len()` and `s.chars().count()` and explain why they differ.
2. Try `&s[0..3]` and then a range that splits the “i”. Watch one work and one panic, and read the panic message about the char boundary.
3. Write `fn first_word(s: &str) -> &str` that returns the text up to the first space. Call it with both a `String` (passed as `&s`) and a string literal.

Chapter 11 — Structs and Methods

What you'll learn. How to define and build structs, how Rust lays them out in memory (and why it may reorder fields), and how `impl` blocks attach methods that take `self`, `&self`, or `&mut self` — turning C's “struct plus free functions” into one tidy type.

Structs: grouping related data

A **struct** groups several values into one named type, exactly like a C **struct**. You already know the idea; the syntax and a few rules differ.

Rust has three kinds of struct.

Named-field structs

The common kind. Each field has a name and a type.

```
struct Point {
    x: f64,
    y: f64,
}

fn main() {
    let p = Point { x: 1.0, y: 2.0 }; // build with field: value
    println!("{}", p.x, p.y); // access with a dot
}

/* The C equivalent */
struct Point { double x, y; };
struct Point p = { 1.0, 2.0 };
printf("(%f, %f)\n", p.x, p.y);
```

C vs Rust. Field access is `.` in both, even though `p` is a value, not a pointer. Rust has no `->` operator; you write `p.x` whether `p` is a value or a reference, and the compiler dereferences for you.

Tuple structs

A struct whose fields have **no names**, only positions. Useful for a thin wrapper (“newtype”) or a small fixed group. You access fields by number: `.0`, `.1`.

```
struct Rgb(u8, u8, u8);
struct Meters(f64); // a "newtype": a distinct type around one value

fn main() {
    let red = Rgb(255, 0, 0);
    let dist = Meters(5.0);
    println!("r={} dist={}", red.0, dist.0);
}
```

A newtype like `Meters` is a type-safety trick: `Meters` and `f64` are different types, so you cannot accidentally pass plain meters where seconds are wanted. C has no easy equivalent (a `typedef` does not create a distinct type). More in Chapter 27 — Idioms and Style.

Unit structs

A struct with **no fields at all**. It carries no data; it exists as a type, often to implement a trait (Chapter 15 — Traits).

```
struct Marker; // no fields, no parentheses

fn main() {
```

```
    let _m = Marker;
}
```

Building and updating structs

Field-init shorthand

When a variable has the same name as a field, you can write the name once.

```
struct User {
    name: String,
    age: u32,
}

fn make_user(name: String, age: u32) -> User {
    User { name, age } // short for `name: name, age: age`
}

fn main() {
    let u = make_user(String::from("Ada"), 36);
    println!("{}", u.name, u.age);
}
```

Struct update syntax

To build a new struct from an existing one, changing a few fields, use `..other`. It fills the remaining fields from `other`.

```
#[derive(Debug)]
struct Config {
    width: u32,
    height: u32,
    fullscreen: bool,
}

fn main() {
    let base = Config { width: 800, height: 600, fullscreen: false };
    let hd = Config { width: 1920, height: 1080, ..base };
    println!("{}", hd);
}
```

Watch out. `..base` moves fields out of `base` that are not `Copy` types (Chapter 7 — Ownership and Moves). Here every field is `Copy`, so `base` is still usable. If a field were a `String`, that field would be moved and `base` could no longer be used as a whole.

Mutability is per-binding, not per-field

This trips up C programmers. In C you can mark individual struct members `const`. In Rust there is **no per-field mutability**. Whether you can change fields depends on whether the **binding** (the `let`) is `mut`.


```

struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    // p.x = 5; // COMPILER ERROR: cannot assign; `p` is not declared mutable

    let mut q = Point { x: 1, y: 2 };
    q.x = 5; // ok: the whole binding `q` is mutable
    q.y = 6;
    println!("{}", q.x, q.y);
}

```

C vs Rust. C controls mutability per member (`const int x;`). Rust controls it per variable: `let` means the whole value is immutable, `let mut` means the whole value is mutable. You cannot make just one field immutable.

Memory layout: like C, but fields may be reordered

A struct's fields live next to each other in memory, just like C. Each field has an **alignment** (the address must be a multiple of its size, roughly), so the compiler inserts **padding** bytes to keep fields aligned. So far this is identical to C.

The surprise: **by default Rust may reorder the fields** to pack them with less padding. C guarantees fields appear in declaration order; Rust's default representation (`repr(Rust)`) makes no such promise and optimizes the layout.

Consider this struct, written in a deliberately wasteful order:

```

struct Mixed {
    a: u8, // 1 byte
    b: u32, // 4 bytes, wants 4-byte alignment
    c: u8, // 1 byte
}

```

A naive C-style layout in declaration order wastes space on padding:

```

declaration order (what C must do):
offset: 0   1   2   3   4   5   6   7   8 .. 11
        [a] [pad][pad][pad][ b  b  b  b ][c] [pad pad pad]
        1 used, 3 padding, 4 used, 1 used, 3 padding -> 12 bytes

```

Rust is free to reorder so the big field is first and the two small ones sit together, shrinking the padding:

```

reordered (what Rust may do):
offset: 0   1   2   3   4   5   6   7
        [ b  b  b  b ][a] [c] [pad][pad]
        4 used, 1, 1, 2 padding -> 8 bytes

```

You can ask for the real size with `std::mem::size_of`:

```
struct Mixed {
    a: u8,
    b: u32,
    c: u8,
}

fn main() {
    println!("{}", std::mem::size_of::<Mixed>()); // 8 on a typical target
    println!("{}", std::mem::size_of::<u32>()); // 4
}
```

Watch out. Because the order can change, you must **not** assume field offsets in default Rust structs. Do not write code (or talk to C) that depends on the layout unless you fix it with an attribute.

`#[repr(C)]` for a predictable, C-compatible layout

When you need the layout to match C exactly — for example, to pass the struct to or from C code — put `#[repr(C)]` on it. This forces declaration order and C’s padding rules, so the struct is binary-compatible with the same struct in C.

```
#[repr(C)]
struct Header {
    magic: u32,
    version: u8,
    flags: u8,
}
```

`#[repr(C)]` is essential for the foreign function interface. We cover it fully in Chapter 25 — Unsafe Rust and FFI. For pure-Rust code, leave the default; let the compiler pack fields for you.

Methods with `impl`

In C, a struct is just data. To operate on it, you write **free functions** that take a pointer to the struct:

```
struct Counter { int value; };

void counter_increment(struct Counter *c) { c->value += 1; }
int counter_get(const struct Counter *c) { return c->value; }
```

Rust attaches those functions to the type in an **`impl` block** (short for “implementation”). Inside, the first parameter describes how the method borrows the receiver. The receiver is named `self` (like C’s explicit `c` pointer, but implicit at the call site).

```
struct Counter {
    value: i32,
}

impl Counter {
```

```

fn increment(&mut self) {
    self.value += 1; // needs &mut self to modify
}

fn get(&self) -> i32 {
    self.value // &self is enough to read
}

fn main() {
    let mut c = Counter { value: 0 };
    c.increment(); // method-call syntax; `c` is the receiver
    c.increment();
    println!("{}", c.get()); // 2
}

```

The three receiver forms

The first parameter chooses how the method accesses the value. This maps directly onto borrowing (Chapter 8 — Borrowing and References).

Receiver	Meaning	C analogy	Use when
<code>&self</code>	borrow immutably (read)	<code>const T *self</code>	you only read fields
<code>&mut self</code>	borrow mutably (modify)	<code>T *self</code>	you change fields
<code>self</code>	take ownership (consume)	pass by value, caller's copy gone	you transform or destroy the value

```

struct Buffer {
    data: Vec<u8>,
}

impl Buffer {
    fn len(&self) -> usize {
        self.data.len() // read only
    }

    fn push(&mut self, byte: u8) {
        self.data.push(byte); // modify
    }

    fn into_vec(self) -> Vec<u8> {
        self.data // consume self, hand out the inner Vec
    }
}

```

```
fn main() {
    let mut b = Buffer { data: Vec::new() };
    b.push(1);
    b.push(2);
    println!("len = {}", b.len());
    let v = b.into_vec(); // `b` is consumed here; cannot use it afterward
    println!("{v:?}");
}
```

Mental model. `&self` is “lend me the struct to look at.” `&mut self` is “lend it to me so I can change it.” `self` is “give it to me — you will not need it after.” The `into_` prefix is the convention for methods that consume `self`.

Associated functions and the `new` convention

A function in an `impl` block that does **not** take `self` is an **associated function**. It belongs to the type, not to an instance. You call it with `::`. The most common one is a constructor named `new`. Rust has no built-in constructors; `new` is just a convention.

```
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    fn new(x: f64, y: f64) -> Self {
        Self { x, y } // `Self` means "this type", here Point
    }

    fn origin() -> Self {
        Self::new(0.0, 0.0) // call another associated function
    }
}

fn main() {
    let p = Point::new(3.0, 4.0); // :: not . because there is no receiver yet
    let o = Point::origin();
    println!("{}", p.x, p.y, o.x, o.y);
}
```

`Self` (capital S) is a shorthand for the type the `impl` is for. It saves repeating the name and survives renames.

C vs Rust. A C constructor is just a free function like `point_new` that returns a struct or fills a pointer. Rust’s `Point::new` is the same idea, but namespaced under the type, so the name cannot collide with another type’s `new`.

Multiple impl blocks

You may split methods across several `impl` blocks for the same type. The compiler treats them as one set. This is handy for grouping related methods or adding methods under a `cfg` condition.

```
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    fn new(x: f64, y: f64) -> Self {
        Self { x, y }
    }
}

impl Point {
    fn magnitude(&self) -> f64 {
        (self.x * self.x + self.y * self.y).sqrt()
    }
}

fn main() {
    let p = Point::new(3.0, 4.0);
    println!("{}", p.magnitude()); // 5
}
```

Automatic referencing and dereferencing

In C you must pick `.` or `->` based on whether you have a value or a pointer. Rust does it for you. When you call `obj.method()`, the compiler automatically adds `&`, `&mut`, or `*` as needed to match the method's receiver. So `c.increment()` works whether `c` is a `Counter`, a `&Counter`, or a `&mut Counter`; you do not write the `&`.

```
struct Counter {
    value: i32,
}

impl Counter {
    fn get(&self) -> i32 {
        self.value
    }
}

fn main() {
    let c = Counter { value: 7 };
    let r = &c;
    println!("{}", c.get()); // compiler inserts (&c)
    println!("{}", r.get()); // already a reference; just works
}
```

```
}
```

C vs Rust. No `-> ever`. Rust’s automatic referencing means the method-call dot works on values and references alike, and the compiler inserts the right borrow.

Deriving common traits

A new struct cannot be printed, copied, or compared until it gains the right behavior. The quickest way is `#[derive(...)]`, which asks the compiler to generate standard implementations for you.

```
#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let a = Point { x: 1, y: 2 };
    let b = a.clone(); // Clone: make an explicit deep copy
    println!("{a:?}"); // Debug: Point { x: 1, y: 2 }
    println!("{a:#?}"); // pretty Debug: multi-line, indented
    println!("{}", a == b); // PartialEq: true
}
```

What the common derives give you:

- **Debug** — printing with `{:?}` (compact) and `{:#?}` (pretty, multi-line). This is for programmers, like a `dump` function. Required before you can `{:?}` a struct.
- **Clone** — an explicit `.clone()` that makes a full copy.
- **PartialEq** — `==` and `!=` comparison, field by field.

Watch out. You must derive (or implement) `Debug` before `{:?}` will print your struct. Forgetting it is a very common first error: “Point doesn’t implement `Debug`.”

No default field values

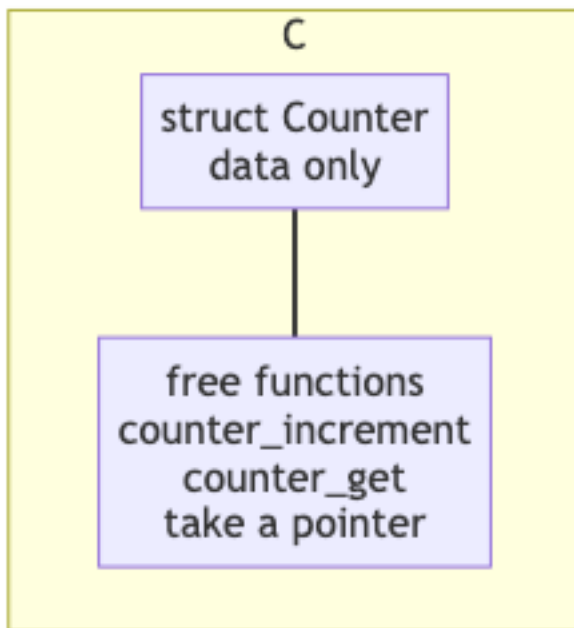
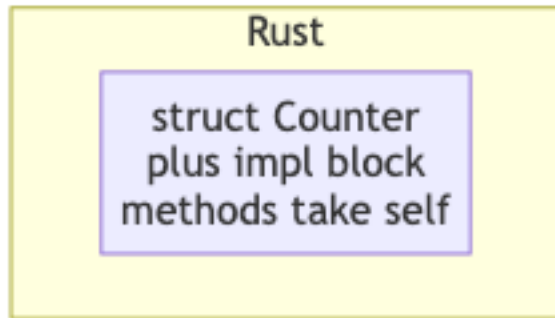
C lets you leave a struct member uninitialized (and read garbage). Rust does not: every field must be given a value when you build the struct, and there is **no syntax for a default value on a field**. To get defaults, implement or derive the `Default` trait, or write a builder.

```
#[derive(Debug, Default)]
struct Settings {
    volume: u32, // defaults to 0
    muted: bool, // defaults to false
}

fn main() {
    let s = Settings::default(); // all fields take their type's default
    let custom = Settings { volume: 5, ..Default::default() };
    println!("{s:?} {custom:?}");
}
```

Builders, for structs with many optional fields, are covered in Chapter 27 — Idioms and Style.

C struct + functions vs Rust struct + methods



Mental model. A C struct plus the free functions that take `struct T *` becomes, in Rust, one struct with an `impl` block whose methods take `self`. The data and the operations live together, and the receiver's `&/&mut`/by-value form states exactly how each method uses the value.

Key takeaways

- Structs come in three forms: named-field, tuple (positional, `.0/.1`), and unit (no fields). Build named structs with `Field { name: value }`.
- Field-init shorthand (`User { name, age }`) and struct update syntax (`..other`) reduce boilerplate; `..other` moves non-Copy fields.
- Mutability is **per binding** (`let mut`), not per field. There is no per-field `const`.

- Fields are laid out like C, **but the compiler may reorder them** to cut padding. Use `#[repr(C)]` for a fixed, C-compatible layout (Chapter 25). Measure with `std::mem::size_of`.
- Methods live in `impl` blocks. The receiver is `&self` (read), `&mut self` (modify), or `self` (consume). Associated functions take no `self` and are called with `::`; `Self::new` is the constructor convention.
- The method-call dot auto-references and auto-dereferences; there is no `->`.
- `#[derive(Debug, Clone, PartialEq)]` generates printing, copying, and comparison. There are no default field values — use `Default` or a builder.

Watch out (gotchas for C programmers)

- **Fields may be reordered.** Default Rust layout is not declaration order. Never assume offsets; use `#[repr(C)]` when the layout must match C.
- **Mutability is per binding, not per field.** `let mut p` makes all of `p` mutable; you cannot mark a single field mutable or `const`.
- **Pick the right receiver.** `&self` to read, `&mut self` to modify, `self` to consume. Taking `self` by value moves the struct; the caller cannot use it after.
- **No default field values.** Every field must be set when you build a struct. Derive `Default` or use a builder for defaults.
- **You must derive `Debug`** before `{:?}` will print your struct. Same for `==` (needs `PartialEq`) and `.clone()` (needs `Clone`).
- **No `->` operator.** Use `.` for everything; the compiler dereferences for you.

Interview questions

Q: In C a struct is data and functions are separate. How does Rust relate the two? A: Rust attaches functions to a type in an `impl` block. Methods take a receiver — `&self`, `&mut self`, or `self` — instead of an explicit struct pointer, and you call them with `value.method()`. The data and its operations live together, and the receiver form states how the method borrows the value.

Q: What is the difference between `&self`, `&mut self`, and `self`? A: `&self` borrows the value immutably to read it; `&mut self` borrows it mutably to modify it; `self` takes ownership and consumes the value, so the caller can no longer use it. They map to C's `const T *`, `T *`, and pass-by-value-then-discard.

Q: Does a Rust struct have the same memory layout as the equivalent C struct? A: Not by default. Rust's default representation may reorder fields to reduce padding, so offsets are unspecified. To get a C-compatible layout in declaration order with C padding rules, annotate the struct with `#[repr(C)]`.

Q: How do you give a struct a constructor, and is `new` special? A: You write an associated function (no `self` parameter) inside an `impl` block, usually named `new`, returning `Self`. It is called as `Type::new(...)`. The name `new` is only a convention, not a keyword; you can have any number of constructor functions with different names.

Q: Why does `println!("{:?}", p)` fail to compile for a new struct, and how do you fix it? A: The `{:?}` format requires the `Debug` trait, which a new struct does not have. Add `#[derive(Debug)]` above the struct (or implement `Debug` by hand). Then `{:?}` prints a compact form and `{:#?}` prints a pretty, indented form.

Try it

1. Define `struct Rectangle { width: f64, height: f64 }` with `Rectangle::new`, an `area(&self)` method, and a `scale(&mut self, factor: f64)` method. Build one, print its area, scale it, and print again.
2. Add `#[derive(Debug)]` and print it with `{:?}` and `{:#?}`. Remove the derive and read the compiler error.
3. Define `struct Mixed { a: u8, b: u64, c: u8 }` and print `std::mem::size_of::<Mixed>()`. Add `#[repr(C)]` and compare the size.

Chapter 12 — Enums and Pattern Matching

What you'll learn. How Rust enums are real *sum types* that can carry data — safe, built-in tagged unions — and how `match` lets you take them apart safely and exhaustively. You will meet `Option<T>` (Rust's answer to null) and `Result<T, E>`, and learn the patterns that make Rust feel different from C.

What a C enum is, and why Rust's is more

In C, an `enum` is just a set of named integer constants. It is a thin label on top of `int`.

```
enum Shape { CIRCLE, RECTANGLE, TRIANGLE };    /* values 0, 1, 2 */

enum Shape s = CIRCLE;
int x = s + 1;                                /* legal: an enum is an int */
s = 999;                                       /* legal: any int fits, even a meaningless one */
```

A C `enum` holds *which* kind, but it cannot hold *data* for that kind. To attach data you reach for a **tagged union**: a `struct` that pairs a tag (the `enum`) with a `union` (overlapping storage for the variants). You must set the tag and read the matching union member by hand. Nothing stops you from reading the wrong one.

```
/* A classic C tagged union. The danger is all on you. */
enum ShapeKind { CIRCLE, RECTANGLE };

struct Shape {
    enum ShapeKind kind;           /* the tag */
    union {
        struct { double r; } circle;
        struct { double w, h; } rectangle;
    } data;
};

double area(const struct Shape *s) {
    switch (s->kind) {
        case CIRCLE:    return 3.14159 * s->data.circle.r * s->data.circle.r;
        case RECTANGLE: return s->data.rectangle.w * s->data.rectangle.h;
    }
    return 0; /* and if someone added a TRIANGLE? the compiler won't warn */
}
```

If you set `kind = CIRCLE` but read `data.rectangle`, the compiler says nothing and you get garbage. This is a whole category of bug.

A Rust **enum** is a *sum type*: a type whose value is exactly one of several **variants**, and each variant can carry its own data. The tag and the data are fused into one safe type. You cannot read the wrong variant, because you can only get at the data by matching the tag first.

C vs Rust. A C **enum** is an **int**. A Rust **enum** is a checked tagged union: tag plus payload, in one type, where the compiler guarantees you read the payload that matches the tag.

Defining an enum with data

Each variant can be a **unit** (no data), a **tuple** variant (positional fields), or a **struct** variant (named fields).

```
enum Shape {
    Circle(f64),                // tuple variant: one f64 (the radius)
    Rectangle { w: f64, h: f64 }, // struct variant: named fields
    Point,                      // unit variant: no data
}

fn main() {
    let shapes = [
        Shape::Circle(2.0),
        Shape::Rectangle { w: 3.0, h: 4.0 },
        Shape::Point,
    ];
    for s in &shapes {
        println!("area = {:.2}", area(s));
    }
}

fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle(r) => std::f64::consts::PI * r * r,
        Shape::Rectangle { w, h } => w * h,
        Shape::Point => 0.0,
    }
}
```

Compare this to the C version above. The Rust enum needs no separate tag field, no **union**, and no manual discipline. The only way to reach a `Circle`'s radius is to `match Shape::Circle(r)`, which can only happen when the value really is a circle.

Mental model. A struct is an **AND** type: a `Point` is an `x` *and* a `y`. An enum is an **OR** type: a `Shape` is a `Circle` *or* a `Rectangle` *or* a `Point`. C has good AND types (structs) but weak OR types (manual tagged unions). Rust has both, first class.

How an enum is stored: tag plus payload

An enum value stores a small integer **tag** (also called the discriminant) that says which variant it is, plus enough space for the **largest** variant’s payload. The whole enum is sized for the biggest case.

enum Shape {	Memory for one Shape value
Circle(f64),	+-----+-----+-----+-----+-----+
Rectangle { w: f64, h: f64 },	tag payload
Point,	+-----+-----+-----+-----+-----+
}	1 byte 16 bytes (two f64s, the (rounded largest variant) + pad up for align)

If the value is Circle(2.0):	tag = 0 r = 2.0 (rest unused)
If it is Rectangle{3.0, 4.0}:	tag = 1 w = 3.0 h = 4.0
If it is Point:	tag = 2 (payload unused)

This is exactly the C `struct { tag; union { ... } }` layout — but the compiler generates it and checks it for you. `std::mem::size_of::<Shape>()` here is the size of the largest variant plus the tag, rounded up for alignment.

C vs Rust. You wrote that tag-plus-union struct by hand in C. Rust writes it for you and makes the tag impossible to lie about.

The niche optimization (C programmers love this)

Sometimes the tag costs zero bytes. If a variant’s payload already has an unused bit pattern — a “niche” — the compiler uses that pattern to mean a different variant, instead of adding a separate tag.

The classic case is `Option<T>`. A reference is never null in Rust, so the all-zero (“null”) pattern is unused. The compiler uses it to represent `None`. The result: `Option<T>` is the same size as a plain pointer, and `None` is literally a null pointer in memory.

```
fn main() {
    use std::mem::size_of;
    assert_eq!(size_of:::<&i32>(), size_of:::<Option<&i32>>()); // both 8 on 64-bit
    assert_eq!(size_of:::<Box<i32>>(), size_of:::<Option<Box<i32>>>());
    println!("Option<&T> and Option<Box<T>> are pointer-sized");
}
```

So `Option<&T>` and `Option<Box<T>>` cost nothing extra over a raw pointer, yet the compiler still forces you to check for `None`. You get C’s representation (a pointer that might be null) with none of C’s danger (forgetting to check).

match: the heart of pattern matching

`match` compares a value against a list of **patterns** and runs the first arm that fits. It is like C’s `switch`, but stronger in three ways: it is **exhaustive**, it has **no fallthrough**, and it can **destructure** data, not just compare integers.

```

fn describe(n: i32) -> &'static str {
    match n {
        0 => "zero",
        1 | 2 | 3 => "small",           // or-pattern: several values, one arm
        4..=9 => "medium",             // inclusive range pattern
        n if n < 0 => "negative",       // guard: an extra `if` condition
        _ => "large",                 // `_` is the catch-all, like `default:`
    }
}

fn main() {
    for n in [-5, 0, 2, 7, 100] {
        println!("{n} is {}", describe(n));
    }
}

```

C vs Rust. A C `switch` falls through from one `case` to the next unless you write `break`, a famous source of bugs. A Rust `match` arm never falls through; each arm is independent. To share one arm between values, use an or-pattern (`1 | 2 | 3`), not `fallthrough`.

Exhaustiveness: the compiler has your back

You must handle every possible case. If you miss one, the program does not compile. This is the opposite of C, where forgetting a `case` is silent.

```

// COMPILER ERROR: non-exhaustive patterns: `Shape::Point` not covered
enum Shape {
    Circle(f64),
    Rectangle { w: f64, h: f64 },
    Point,
}

fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle(r) => 3.14159 * r * r,
        Shape::Rectangle { w, h } => w * h,
        // forgot Shape::Point -> error[E0004]: non-exhaustive patterns
    }
}

fn main() {
    let _ = area(&Shape::Point);
}

```

This is a superpower for maintenance. Add a new variant to an enum later, and the compiler points you at every `match` that must now handle it. In C, adding an enum value silently leaves old `switch` statements wrong.

Rule of thumb. Prefer matching each variant explicitly over using `_`. Then when you

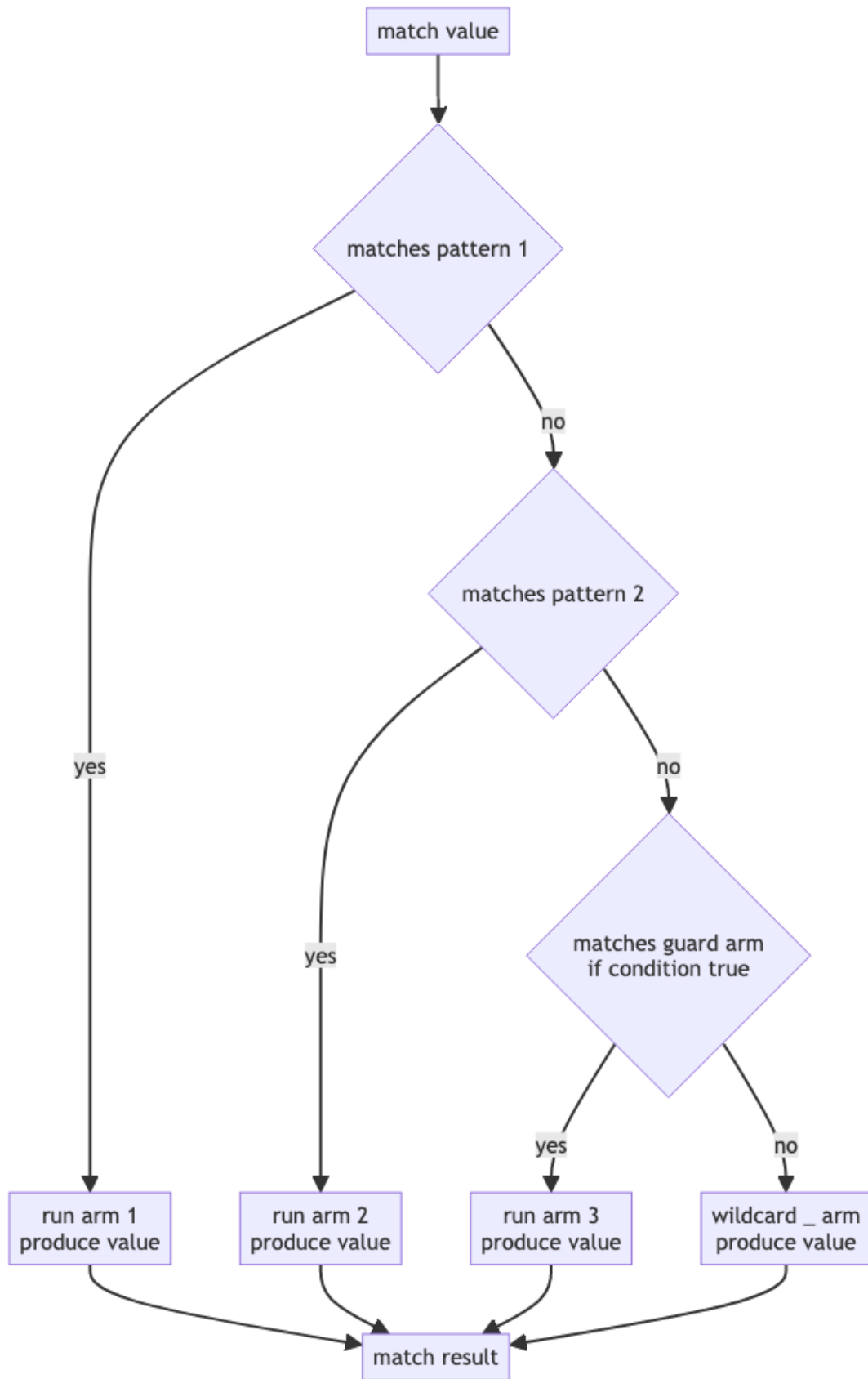
add a variant, the compiler reminds you of every place to update. Use `_` only when you truly mean “everything else.”

match is an expression

Like most things in Rust, `match` produces a value. Every arm must produce the same type. This replaces the common C pattern of declaring a variable, then assigning to it inside each `switch` case.

```
fn main() {  
    let code = 2;  
    let label = match code {  
        0 => "ok",  
        1 => "warning",  
        _ => "error",  
    }; // note the value comes straight out of `match`  
    println!("{label}");  
}
```

A flowchart of how one `match` runs:



More pattern tools

Patterns are a small language of their own. Here are the pieces you will use most.

Destructuring structs and tuples

You can take a struct or tuple apart in a pattern, binding each field to a name.

```
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 3, y: 7 };
    let Point { x, y } = p;           // bind both fields at once
    println!("{x} {y}");

    let pair = (1, "one");
    let (num, word) = pair;           // destructure a tuple
    println!("{num} = {word}");
}
```

The .. rest and @ bindings

Use .. to ignore the fields you do not care about, and @ to bind a value *and* test it against a pattern at the same time.

```
struct Config {
    width: u32,
    height: u32,
    title: String,
}

fn main() {
    let c = Config {
        width: 800,
        height: 600,
        title: String::from("demo"),
    };

    // Take width; ignore the rest of the fields with `..`.
    let Config { width, .. } = &c;
    println!("width = {width}");

    // `@` binds the matched value to a name while also range-checking it.
    let n = 5;
    match n {
        small @ 1..=9 => println!("{small} is a single digit"),
        other => println!("{other} is bigger"),
    }
}
```

```
    }
}
```

.. also works in tuples and slices, for example `let (first, .., last) = tuple;`

Option<T>: Rust has no null

In C, any pointer can be NULL, and dereferencing a null pointer is undefined behavior. The catch is that nothing in the type system tells you which pointers might be null. You just have to remember to check.

Rust has **no null**. Instead, “a value that might be missing” is its own ordinary enum, `Option<T>`, defined in the standard library roughly like this:

```
enum Option<T> {    // <T> means it works for any type T (generics, Chapter 14)
    Some(T),        // a present value, wrapping a T
    None,           // no value
}
```

Because the missing case is a separate variant, the compiler forces you to handle it before you can touch the value inside. You cannot accidentally use a `None` as if it held data.

```
fn first_char(s: &str) -> Option<char> {
    s.chars().next()    // returns Some(c), or None if the string is empty
}

fn main() {
    match first_char("hello") {
        Some(c) => println!("first char is {c}"),
        None => println!("the string was empty"),
    }
}
```

C vs Rust. In C, `char *find(...)` might return NULL and the caller may forget to check, causing a crash. In Rust, `Option<char>` makes “might be missing” part of the type, and `match` makes ignoring `None` a compile error. A whole class of null-dereference crashes is gone.

Common Option methods

Matching is always available, but for simple cases there are shorter helper methods.

```
fn main() {
    let some: Option<i32> = Some(10);
    let none: Option<i32> = None;

    println!("{}", some.unwrap_or(0));    // 10 (value, or a default for None)
    println!("{}", none.unwrap_or(0));    // 0
    println!("{}", some.map(|x| x * 2));  // Some(20) (transform the inner value)
    println!("{}", some.is_some());       // true
    println!("{}", none.is_none());       // true
}
```


`unwrap_or` gives a default for `None`. `map` transforms the inner value if it is present and leaves `None` alone. `is_some/is_none` just ask which variant it is. There is also `.unwrap()`, which returns the inner value but **panics** (crashes the thread) on `None`; use it only when you are sure, and prefer real handling in production code (see Chapter 13 — Error Handling).

Result<T, E>: success or error, as a value

A close cousin of `Option` is `Result<T, E>`, the standard way to report an error. It is also just an enum:

```
enum Result<T, E> {
    Ok(T),          // success, wrapping the produced value
    Err(E),         // failure, wrapping an error value
}
```

Where `Option` says “value or nothing,” `Result` says “value or an explanation of what went wrong.” You handle it the same way — with `match` or helper methods.

```
fn main() {
    let parsed: Result<i32, _> = "42".parse::<i32>();
    match parsed {
        Ok(n) => println!("got {n}"),
        Err(e) => println!("parse failed: {e}"),
    }
}
```

We give `Result` and its `?` operator full treatment in Chapter 13 — Error Handling. The point here is that error handling in Rust is built from the same enum-and-`match` machinery as everything else; it is not a separate feature.

if let, let ... else, and while let

A full `match` is overkill when you care about just one pattern. Rust has three shorthands.

`if let` runs a block only when one pattern matches, with an optional `else`:

```
fn main() {
    let maybe = Some(7);

    if let Some(n) = maybe {
        println!("got {n}");
    } else {
        println!("nothing");
    }
}
```

`let ... else` binds a pattern that *must* match; if it does not, the `else` block runs and must diverge (return, break, or panic). This avoids deep nesting — the “happy path” stays at the top level.

```
fn parse_or_zero(s: &str) -> i32 {
    let Ok(n) = s.parse::<i32>() else {
```

```

        return 0; // the else block must leave the function (or loop/panic)
    };
    n // here `n` is in scope and definitely valid
}

```

```

fn main() {
    println!("{}", parse_or_zero("123")); // 123
    println!("{}", parse_or_zero("xyz")); // 0
}

```

while let keeps looping as long as a pattern matches. It is perfect for draining a collection:

```

fn main() {
    let mut stack = vec![1, 2, 3];
    while let Some(top) = stack.pop() { // pop() returns Option; None ends the loop
        println!("popped {top}");
    }
}

```

C vs Rust. These are the equivalent of a small if/while that both *tests* a tag and *extracts* the payload in one step. In C you would check `kind ==` and then read the union member as two separate, error-prone actions.

Methods on enums with impl

Enums can have methods, just like structs (see Chapter 11 — Structs and Methods). You write an `impl` block and usually `match self` inside.

```

enum Direction {
    North,
    East,
    South,
    West,
}

impl Direction {
    fn turn_right(self) -> Direction {
        match self {
            Direction::North => Direction::East,
            Direction::East  => Direction::South,
            Direction::South => Direction::West,
            Direction::West  => Direction::North,
        }
    }

    fn name(&self) -> &'static str {
        match self {
            Direction::North => "north",
            Direction::East  => "east",
            Direction::South => "south",
        }
    }
}

```

```

        Direction::West => "west",
    }
}

fn main() {
    let d = Direction::North.turn_right();
    println!("after turning right: {}", d.name()); // east
}

```

Make illegal states unrepresentable

This is the design idea that enums unlock, and it is worth saying plainly: **use the type system so that a wrong combination of data cannot even be built.**

In C you often store several fields and rely on a rule like “if `is_connected` is false, ignore `session_id`.” Nothing enforces the rule. Someone will set `is_connected = false` and still read a stale `session_id`.

```

/* Fragile: the fields can disagree. */
struct Connection {
    bool    is_connected;
    int     session_id;   /* only valid when is_connected is true */
    char    *error_msg;   /* only valid when there was an error */
};

```

With an enum, each state carries exactly the data it needs and no more. A disconnected connection has no `session_id` field to misread, because that field only exists inside the `Connected` variant.

```

enum Connection {
    Disconnected,
    Connecting,
    Connected { session_id: u32 },
    Failed { error: String },
}

fn status(c: &Connection) -> String {
    match c {
        Connection::Disconnected => "offline".to_string(),
        Connection::Connecting   => "dialing".to_string(),
        Connection::Connected { session_id } => format!("online, session {session_id}"),
        Connection::Failed { error }      => format!("failed: {error}"),
    }
}

fn main() {
    let c = Connection::Connected { session_id: 42 };
    println!("{}", status(&c));
}

```

Now it is impossible to have a `session_id` without being connected. The bug cannot be written. This is one of the most satisfying habits to bring from C to Rust.

Matching can move the value

One borrow-checker subtlety bites C programmers here. Matching a value by value can **move** its contents out (move ownership; see Chapter 7 — Ownership and Moves). If the inner type is not Copy (like `String`), the original is then gone.

```
// COMPILER ERROR: borrow of partially moved value: `msg`
enum Message {
    Text(String),
    Quit,
}

fn main() {
    let msg = Message::Text(String::from("hi"));
    match msg {
        Message::Text(s) => println!("{s}"), // `s` MOVES the String out of msg
        Message::Quit => {}
    }
    // `msg` is partially moved here; using it again is an error.
    println!("{}", matches!(msg, Message::Quit)); // error[E0382]
}
```

The fix is to match on a **reference** so you borrow instead of move. Match on `&msg` (or rely on Rust’s “match ergonomics,” which bind `s` as `&String` automatically), or use the `ref` keyword to bind by reference.

```
enum Message {
    Text(String),
    Quit,
}

fn main() {
    let msg = Message::Text(String::from("hi"));
    match &msg { // borrow, do not move
        Message::Text(s) => println!("{s}"), // s is &String here
        Message::Quit => {}
    }
    println!("still usable: {}", matches!(msg, Message::Quit)); // ok
}
```

Watch out. `match value` may move ownership out of value. If you need the original afterward, write `match &value` (borrow) or bind with `ref`. The handy `matches!(value, Pattern)` macro returns a `bool` and is great for quick checks.

Key takeaways

- A Rust **enum** is a sum type: a value is exactly one variant, and each variant can carry data (unit, tuple, or struct variants). It is a safe, built-in tagged union — the tag and payload the compiler manages for you.
- C enums are just integers; C tagged unions are manual and unchecked. Rust enums fix both: you cannot read the payload of the wrong variant.
- `Option<T>` (`Some/None`) replaces null pointers; `Result<T, E>` (`Ok/Err`) reports errors. Both are ordinary enums handled with `match`.
- `match` is **exhaustive** (every case handled) and has **no fallthrough**. It is an expression that produces a value.
- Patterns include literals, ranges (`1..=9`), or-patterns (`a | b`), bindings, guards (`if`), struct/tuple/enum destructuring, the `.. rest`, and `@` bindings.
- `if let`, `let ... else`, and `while let` handle a single pattern with less code.
- An enum's size is the largest variant plus a tag. The niche optimization makes `Option<&T>` and `Option<Box<T>>` pointer-sized, with `None` as null.
- Design principle: **make illegal states unrepresentable** by giving each state exactly the data it needs.

Watch out (gotchas for C programmers)

- **match must be exhaustive.** Unlike a C `switch`, missing a case is a compile error, not a silent gap. Adding a variant later flags every `match` to update.
- **No fallthrough.** Arms never fall into the next. Use an or-pattern (`a | b`) to share an arm; never expect C-style fallthrough.
- **Option replaces null.** There is no `NULL`. A maybe-missing value is `Option<T>`, and the compiler forces you to handle `None`.
- **Matching can move.** `match value` can move non-Copy data out of `value`, leaving it unusable. Match on `&value` or use `ref` to borrow instead.
- **Enum size is the largest variant.** A enum with one huge variant makes every value big. Box a large variant (`Box<BigThing>`) to keep the enum small.
- **_ hides future work.** A catch-all arm silences exhaustiveness checks for new variants. Match variants explicitly when you want that safety net.

Interview questions

Q: How is a Rust enum different from a C enum? A: A C enum is a set of named integer constants — just an `int`, with no attached data. A Rust enum is a sum type: each variant can carry its own data, so the enum is a safe tagged union (tag plus payload) managed and checked by the compiler. You can only access a variant's data by matching its tag.

Q: What does it mean that match is exhaustive, and why is that useful? A: Every possible value must be handled by some arm, or the code does not compile. This catches missing cases at compile time. When you add a new enum variant, the compiler flags every `match` that no longer covers all cases, so you cannot forget to update one — unlike a C `switch`, where a missing `case` is silent.

Q: Why does Rust use Option<T> instead of null pointers? A: Null pointers are not visible in C's type system, so the compiler cannot force a check, and forgetting one is undefined behavior. `Option<T>` makes “might be missing” an explicit type with two variants, `Some(T)` and `None`. You

must handle `None` before reaching the value, which eliminates null-dereference bugs. Thanks to the niche optimization, `Option<&T>` is still just a pointer in memory.

Q: When matching an enum, when does the value get moved, and how do you avoid it? A: Binding a variant’s non-Copy payload by value (e.g. `Message::Text(s)`) moves that data out of the matched value, making the original unusable afterward. To avoid it, match on a reference (`match &value`) so the bindings borrow, or use the `ref` keyword to bind by reference.

Q: What is the “make illegal states unrepresentable” principle? A: It is the practice of designing types so that invalid combinations of data cannot be constructed. Instead of separate fields guarded by a rule (e.g. “`session_id` is valid only if connected”), you use an enum where each variant holds exactly the data that state needs. The bad combination then has no representation, so it cannot occur.

Try it

1. Write an enum `Json` with variants `Null`, `Bool(bool)`, `Number(f64)`, `Text(String)`, and `Array(Vec<Json>)`. Write a function that returns a short description of each value using `match`.
2. Make a function return `Option<usize>` for “index of the first even number in a slice.” Handle the result with `if let` and then with `match`.
3. Print `std::mem::size_of::<Json>()` and explain why it equals the largest variant plus a tag. Then wrap the `Array` payload in `Box` and watch the size shrink.

Chapter 13 — Error Handling

What you’ll learn. How Rust handles failure without exceptions: recoverable errors as `Result<T, E>` values, the `?` operator for clean propagation, how to define your own error types, and when `panic!` is the right tool. You will see how this beats C’s return codes and `errno`.

Two kinds of failure

Rust splits failure into two clear categories, and it has **no exceptions** at all.

- **Recoverable errors** — things that can reasonably go wrong: a file is missing, a number will not parse, a network call times out. These are returned as ordinary values of type `Result<T, E>`. The caller decides what to do.
- **Unrecoverable errors** — bugs and broken assumptions: an index is out of bounds, an invariant is violated, “this can never happen” happened. These call `panic!`, which stops the current thread.

C vs Rust. C has neither exceptions (in the C++ sense) nor a `Result` type. It uses return codes and the global `errno`, and *nothing forces the caller to check*. Rust makes the error part of the return type and warns if you ignore it.

`Result<T, E>` is just an enum

As we saw in Chapter 12 — Enums and Pattern Matching, `Result` is an ordinary enum from the standard library:

```
enum Result<T, E> {
    Ok(T),          // success, holding the produced value of type T
    Err(E),         // failure, holding an error value of type E
}
```

A function that can fail returns a `Result`. The caller cannot reach the success value without acknowledging that an error is possible. Here is the most explicit way to handle one — a `match`:

```
use std::num::ParseIntError;

fn double(s: &str) -> Result<i32, ParseIntError> {
    match s.parse::<i32>() {
        Ok(n) => Ok(n * 2),
        Err(e) => Err(e),
    }
}

fn main() {
    match double("21") {
        Ok(n) => println!("doubled: {n}"),
        Err(e) => println!("bad input: {e}"),
    }
}
```

Watch out. `Result` is marked `#[must_use]`. If you call a function that returns a `Result` and ignore the value, the compiler issues a warning: `unused Result that must be used`. In C, ignoring a return code is silent and common. In Rust the tooling nags you to handle it.

The `?` operator: clean propagation

Most of the time a function does not want to *handle* an error from a call it makes; it wants to **pass it up** to its own caller. Doing that with `match` everywhere is noisy. The `?` operator does it in one character.

When you write `expr?`:

- If `expr` is `Ok(v)`, the expression evaluates to `v` and execution continues.
- If `expr` is `Err(e)`, the function **returns early** with `Err(e)` — after converting `e` into the function's own error type via the `From` trait.

Here is a function that reads a file and parses the first line as a number, written the verbose way first:

```
use std::fs::File;
use std::io::{self, Read};

// Verbose: explicit match at every fallible step.
fn read_number_verbose(path: &str) -> Result<i32, io::Error> {
    let mut file = match File::open(path) {
        Ok(f) => f,
```

```

        Err(e) => return Err(e),
    };
    let mut text = String::new();
    match file.read_to_string(&mut text) {
        Ok(_) => {}
        Err(e) => return Err(e),
    }
    // parse() returns a different error type; map it into io::Error.
    match text.trim().parse::<i32>() {
        Ok(n) => Ok(n),
        Err(e) => Err(io::Error::new(io::ErrorKind::InvalidData, e)),
    }
}

fn main() {
    match read_number_verbose("number.txt") {
        Ok(n) => println!("number is {n}"),
        Err(e) => println!("error: {e}"),
    }
}

```

Now the same logic with `?`. Each `?` replaces a whole `match`-and-`return` block:

```

use std::fs;
use std::error::Error;

// Clean: `?` propagates and converts errors automatically.
fn read_number(path: &str) -> Result<i32, Box<dyn Error>> {
    let text = fs::read_to_string(path)?; // io::Error -> Box<dyn Error> via From
    let n = text.trim().parse::<i32>()?; // ParseIntError -> Box<dyn Error> via From
    Ok(n)
}

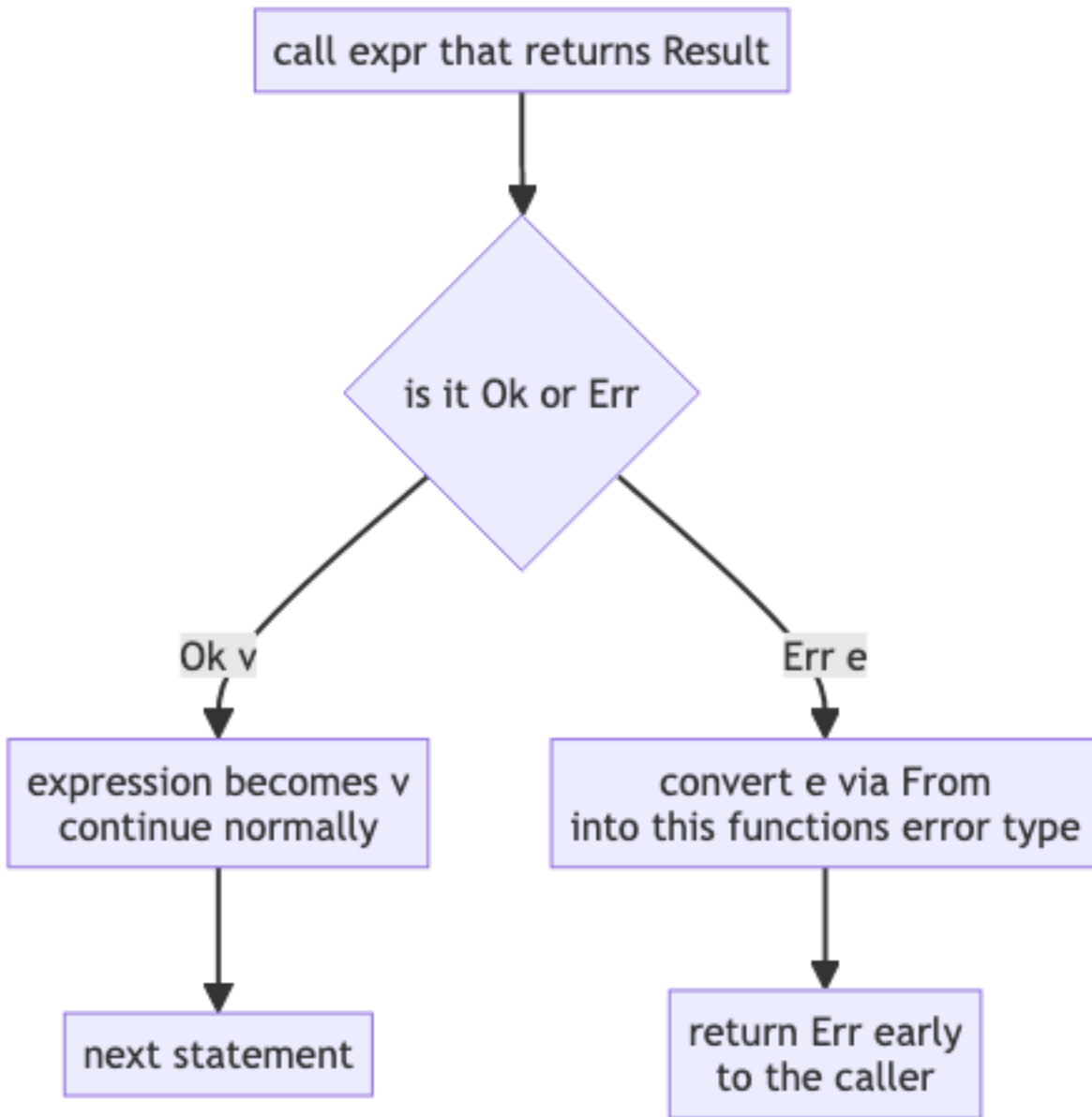
fn main() {
    match read_number("number.txt") {
        Ok(n) => println!("number is {n}"),
        Err(e) => println!("error: {e}"),
    }
}

```

The second version reads top to bottom like the happy path, with `?` marking the spots that may bail out. The two different error types (`io::Error` and `ParseIntError`) are both converted into `Box<dyn Error>` automatically, because the standard library provides the `From` conversions.

C vs Rust. This is the cleanest contrast in the whole book. In C you write `if (rc != 0) return rc;` after almost every call, and you must convert error codes by hand. In Rust `?` is that pattern, built into the language, type-checked, and with the conversion done for you.

How ? flows



Where ? is allowed

? only works inside a function whose return type can express the early exit:

- A function returning `Result<T, E>` — ? on a `Result` works if the error converts into `E`.
- A function returning `Option<T>` — ? on an `Option` returns `None` early.
- You **cannot** mix them: ? on a `Result` in a function returning `Option` (or vice versa) will not compile.

```
// COMPILER ERROR: the `?` operator can only be used in a function that returns Result or Option
fn bad(s: &str) -> i32 {
    let n = s.parse::<i32>()?; // error[E0277]: `i32` is not `Try`/`FromResidual`
```

```

    n
}

fn main() {
    println!("{}", bad("3"));
}

```

? in main

main can return a `Result`, which lets you use `?` directly in it. The common signature is:

```

use std::error::Error;
use std::fs;

fn main() -> Result<(), Box<dyn Error>> {
    let text = fs::read_to_string("number.txt")?;
    let n: i32 = text.trim().parse()?;
    println!("number is {n}");
    Ok(()) // success: the unit value () means "nothing to return, no error"
}

```

If `main` returns `Err`, the program prints the error (using its `Debug` form) and exits with a non-zero status code. `Box<dyn Error>` means “a boxed error of any type that implements the `Error` trait” — a catch-all that lets `?` accept many different error types in one function (more on trait objects in Chapter 15 — Traits).

Handling a `Result` with combinators

Between a full match and a bare `?`, the standard library offers helper methods (“combinators”) for common moves. A few you will reach for often:

```

fn main() {
    let parsed: Result<i32, _> = "x".parse::<i32>();

    // Provide a fallback value computed from the error.
    let n = parsed.unwrap_or_else(|_| -1);
    println!("{n}"); // -1

    // Replace the error with a different type or message.
    let mapped: Result<i32, String> =
        "x".parse::<i32>().map_err(|e| format!("not a number: {e}"));
    println!("{mapped:?}"); // Err("not a number: invalid digit found in string")

    // Throw away the error and get an Option instead.
    let opt: Option<i32> = "5".parse::<i32>().ok();
    println!("{opt:?}"); // Some(5)
}

```

- `unwrap_or_else(f)` — return the `Ok` value, or compute a fallback from the error.

- `map_err(f)` — transform the `Err` value, leaving `Ok` untouched. Useful for converting one error type into another.
- `.ok()` — discard the error and convert `Result<T, E>` into `Option<T>`.

There is also `unwrap_or(default)` (a plain fallback value) and `map(f)` (transform the `Ok` value), which work just like their `Option` versions from Chapter 12.

Defining your own error type

For a real library you usually define one error `enum` that lists everything that can go wrong. To behave like a standard error, it should implement two traits: `std::fmt::Display` (a human-readable message) and `std::error::Error` (the marker trait for errors). Implementing `From` for the underlying errors lets `?` convert into your type automatically.

```
use std::fmt;

#[derive(Debug)]
enum ConfigError {
    Io(std::io::Error),
    Parse(std::num::ParseIntError),
    Empty,
}

// Display: the message a user sees.
impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter<'_,>) -> fmt::Result {
        match self {
            ConfigError::Io(e) => write!(f, "I/O error: {e}"),
            ConfigError::Parse(e) => write!(f, "parse error: {e}"),
            ConfigError::Empty => write!(f, "the config was empty"),
        }
    }
}

// Marker trait: now ConfigError is a real error type.
impl std::error::Error for ConfigError {}

// From conversions let `?` turn the source error into ConfigError automatically.
impl From<std::io::Error> for ConfigError {
    fn from(e: std::io::Error) -> Self {
        ConfigError::Io(e)
    }
}

impl From<std::num::ParseIntError> for ConfigError {
    fn from(e: std::num::ParseIntError) -> Self {
        ConfigError::Parse(e)
    }
}
```

```

fn load(path: &str) -> Result<i32, ConfigError> {
    let text = std::fs::read_to_string(path)?; // io::Error -> ConfigError
    let trimmed = text.trim();
    if trimmed.is_empty() {
        return Err(ConfigError::Empty);
    }
    let value: i32 = trimmed.parse()?; // ParseIntError -> ConfigError
    Ok(value)
}

fn main() {
    match load("config.txt") {
        Ok(v) => println!("loaded {v}"),
        Err(e) => println!("could not load: {e}"), // uses Display
    }
}

```

Notice that `load` uses `?` on two different error types, and each is converted into `ConfigError` by the matching `From` implementation. This is the same machinery the `Box<dyn Error>` example used; here you control the exact error type instead.

C vs Rust. This is like a C library that returns its own `enum ErrorCode`, except the error can carry data (the underlying cause), formats itself, and is impossible to ignore silently.

Use crates in real projects

Writing `Display`, `Error`, and `From` by hand is repetitive. The Rust ecosystem has two standard crates that you should know:

- **thiserror** — for **libraries**. A `#[derive(Error)]` macro generates the `Display` and `From` boilerplate from simple attributes, while keeping a concrete, named error type that callers can match on.
- **anyhow** — for **applications**. It provides `anyhow::Error` (a smart `Box<dyn Error>`) and `anyhow::Result<T>`, so you can propagate any error with `?` and attach context, without defining your own type.

Rule of thumb. In a library, define a precise error type (use `thiserror`); the caller may want to match on specific cases. In an application or CLI, use `anyhow` for easy propagation, because you usually just report the error and exit.

panic!: for unrecoverable errors

`panic!` stops normal execution. Use it for situations that *should never happen* — a violated invariant, an impossible state, a bug. It is not for ordinary errors that a caller could handle.

```

fn main() {
    let config_present = false;
    if !config_present {
        // Truly broken state: better to stop than to limp along with bad data.
        panic!("internal invariant broken: config must be loaded by now");
    }
}

```

```
}
}
```

Two convenience methods panic for you:

- `.unwrap()` — on `Ok(v)/Some(v)` gives `v`; on `Err/None` it **panics**.
- `.expect("message")` — same, but with a custom panic message. Prefer it over `.unwrap()` so the crash explains itself.

```
fn main() {
    let n: i32 = "42".parse().expect("the literal 42 must parse");
    println!("{n}");
    // "abc".parse::

```

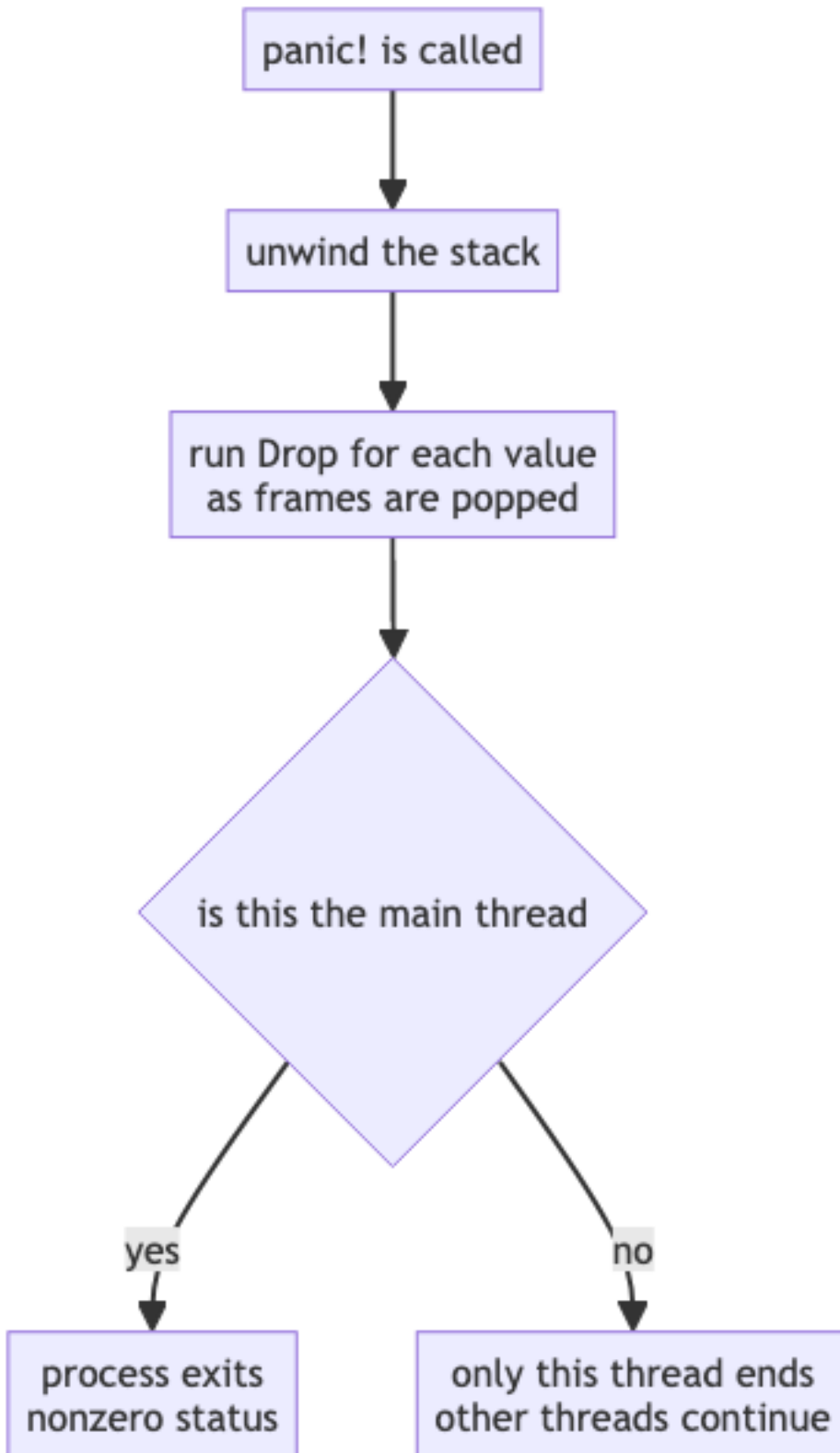
When is panicking acceptable?

- **Truly unrecoverable situations** and broken invariants (“this cannot happen”).
- **Prototypes and examples**, to keep the focus on the main idea (this book uses `.unwrap()` that way, with a caveat each time).
- **Tests**, where a panic correctly marks the test as failed.

Rule of thumb. Never `panic!`, `.unwrap()`, or `.expect()` in library code for errors a caller might recover from. Return a `Result` instead. A library that panics takes the choice away from its user and can crash their whole program.

What a panic actually does

By default a panic **unwinds** the stack: it walks back up the call chain, running each value’s `Drop` (so destructors run and resources are freed — see Chapter 7 — Ownership and Moves), then ends the thread. If the panicking thread is the main thread, the whole program exits with a failure status.



You can switch to **abort** instead of **unwind** by setting `panic = "abort"` in `Cargo.toml`. Then

a panic immediately ends the process without running destructors — smaller binaries and no unwinding machinery, at the cost of no cleanup.

`std::panic::catch_unwind` can catch an unwinding panic at a boundary (for example, to stop a panic from crossing into C code via FFI). It is **not** a general try/catch and must not be used for ordinary control flow.

C vs Rust. A C `abort()` or segfault just dies. A Rust unwinding panic runs destructors on the way out, so files close and locks release. But a panic is still a crash, not a recoverable error — reach for `Result` for anything a caller should handle.

C errno vs Rust Result, side by side

Concept	C	Rust
Report failure	return code and/or <code>errno</code>	<code>Result<T, E></code> value
Ignoring it	silent, very common	<code>#[must_use]</code> warning
Propagate up	<code>if (rc) return rc;</code> by hand	<code>?</code> operator
Convert error type	manual mapping	<code>From</code> trait, automatic with <code>?</code>
Carry detail	an <code>int</code> code, look up meaning	the <code>Err</code> value carries data
Unrecoverable bug	<code>abort()</code> / undefined behavior	<code>panic!</code> (unwind, run <code>Drop</code>)
Exceptions	none (C++ has them)	none — Rust rejects them for errors

Key takeaways

- Rust has **no exceptions**. Recoverable failure is a `Result<T, E>` value; unrecoverable failure is `panic!`.
- `Result` is an enum (`Ok/Err`) and is `#[must_use]`, so ignoring it warns. Handle it with `match`, with combinators (`unwrap_or_else`, `map_err`, `ok`), or with `?`.
- The `?` operator returns the `Err` (or `None`) early and converts the error via the `From` trait. It works in functions returning `Result` or `Option`.
- `fn main() -> Result<(), Box<dyn Error>>` lets you use `?` in `main`. `Box<dyn Error>` is the easy catch-all for propagation.
- Define a custom error enum implementing `Display + Error (+ From for ?)`, or use `thiserror` for libraries and `anyhow` for applications.
- `panic!`, `.unwrap()`, `.expect()` are for truly unrecoverable situations, prototypes, and tests — not for library errors. A panic unwinds the stack (running `Drop`) by default, or aborts; an unrecovered panic ends the thread.

Watch out (gotchas for C programmers)

- **Do not ignore a `Result`.** It is `#[must_use]`; the compiler warns. Use the value, propagate with `?`, or explicitly discard with `let _ =` if you really mean to.
- `.unwrap()/expect()` **panic**. On `Err` or `None` they crash the thread. Fine for prototypes and tests; risky in shipping code. Prefer `?` or a `match`.
- `?` **needs a compatible return type**. It only works where the function returns `Result` or `Option`, and the error must convert into the function's error type via `From`. Otherwise you get a compile error.

- **Do not panic! in library code** for recoverable errors. Return a `Result` and let the caller decide. Panicking can crash an application that depended on you.
- **Error conversion is via `From`**. To let `?` turn error type A into your error type B, implement `From<A> for B` (or derive it with `thiserror`).
- **A panic is not try/catch**. `catch_unwind` exists for boundaries (like FFI), not for normal error flow. Treat panics as crashes, not as a control structure.

Interview questions

Q: What are the two kinds of error in Rust, and which type goes with each? A: Recoverable errors use `Result<T, E>` — a returned value the caller must handle, for expected failures like a missing file. Unrecoverable errors use `panic!` — for bugs and broken invariants. Rust has no exceptions; both are explicit and distinct.

Q: What exactly does the `?` operator do? A: For an expression of type `Result`, `?` evaluates to the inner value if it is `Ok`, or returns `Err` early from the current function if it is `Err`, converting the error into the function's error type via the `From` trait. It also works on `Option`, returning `None` early. It is only allowed in functions that return `Result` or `Option`.

Q: How is Rust's error handling safer than C's return codes and `errno`? A: In C nothing forces a caller to check a return code, and `errno` is a separate global that is easy to misuse. In Rust the error is part of the return type `Result<T, E>`, which is `#[must_use]`, so ignoring it warns. The `?` operator gives clean, type-checked propagation, and the `Err` value can carry rich detail instead of a bare integer code.

Q: When is it acceptable to call `.unwrap()` or `panic!`? A: When the failure is truly unrecoverable or represents a bug (a violated invariant, an impossible state), and in prototypes, examples, and tests where stopping is the right behavior. In library code that handles recoverable errors, you should return a `Result` instead, so callers keep control.

Q: What happens when a panic occurs, by default? A: By default the stack unwinds: each stack frame is popped and the `Drop` of each value runs, so resources are released cleanly, then the thread ends. If it is the main thread, the process exits with a non-zero status. You can instead configure `panic = "abort"`, which ends the process immediately without running destructors.

Try it

1. Write `fn parse_sum(a: &str, b: &str) -> Result<i32, std::num::ParseIntError>` that parses both strings and returns their sum, using `?` for both parses.
2. Change `parse_sum` to return `Result<i32, Box<dyn std::error::Error>>` and call it from a `main` that also returns `Result<(), Box<dyn std::error::Error>>`, using `?` throughout. Feed it `"3"` and `"x"` and watch the error print.
3. Define a small error enum with two variants, implement `Display` and `Error`, add the `From` conversions, and rewrite the function to use your type. Then look up `thiserror` and rewrite it again with `#[derive(Error)]`.

Chapter 14 — Generics

What you'll learn. How to write one piece of code that works for many types, without giving up type safety or speed. You will see generic functions, structs, enums, and

methods; trait bounds (a preview of Chapter 15 — Traits); and *monomorphization*, the trick that makes Rust generics cost nothing at run time — unlike C macros and `void *`.

The problem: writing the same code twice

In C, suppose you want a function that returns the larger of two values. You write one for `int`:

```
int max_int(int a, int b) {
    return a > b ? a : b;
}
```

Now you need it for `double`. C has no real way to reuse the body, so you have three bad choices:

- **Copy and paste.** Write `max_double`, `max_long`, and so on. The code drifts and bugs multiply.
- **A macro.** `#define MAX(a, b) ((a) > (b) ? (a) : (b))`. This is *untyped* text substitution. It works on anything, but the compiler checks nothing, it can evaluate its arguments twice (`MAX(i++, j)` is a bug), and the error messages are terrible.
- **`void *` and a comparison callback**, like `qsort`. This works for any type but throws away type information. You cast pointers, you pass element sizes by hand, and a single wrong cast is undefined behavior — at run time, not compile time.

Rust solves this with **generics**: code written once, with a *placeholder* for the type, that the compiler checks fully and specializes for each concrete type you actually use. You get the flexibility of a macro, the safety of a normal function, and the speed of hand-written code.

Mental model. A generic is a template with a hole for a type. The compiler fills the hole separately for every type you use, and type-checks each filled copy. It is C++ templates done with full checking, or a C macro that the compiler actually understands.

Generic functions

A **type parameter** is a name (by convention a single capital letter like `T`) that stands for “some type, chosen later.” You declare it in angle brackets after the function name.

```
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut biggest = &list[0];
    for item in list {
        if item > biggest {
            biggest = item;
        }
    }
    biggest
}

fn main() {
    let numbers = vec![34, 50, 25, 100, 65];
    let chars = vec!['y', 'm', 'a', 'q'];

    println!("largest number: {}", largest(&numbers));
}
```

```
println!("largest char:  {}", largest(&chars));
}
```

Read the signature piece by piece:

- `<T: PartialOrd>` declares a type parameter `T`. The `: PartialOrd` part is a **trait bound** (more below): it says “`T` must be a type you can compare with `<` and `>`.”
- `list: &[T]` takes a slice of `T` (a borrowed view of a contiguous array — see Chapter 10 — Slices and Strings).
- `-> &T` returns a reference to one element.

You call it with no special syntax: `largest(&numbers)`. The compiler **infers** that `T` is `i32` from the argument. For `chars`, it infers `T` is `char`. One function body, many types, all checked.

C vs Rust. In C, `qsort` takes a `void *` array, an element size, and a comparison function pointer. Nothing stops you from passing the wrong size or a comparison meant for a different type — the mistake shows up as a crash or garbage at run time. The Rust generic carries the real type `T` through, so the wrong type is a compile error.

Why you need a trait bound

This is the first surprise for C programmers. You **cannot** do just anything to a value of a generic type. The compiler must know the operation is valid for *every* type `T` might be. Without a bound, `T` is a complete unknown, so almost nothing is allowed.

```
// COMPILE ERROR: binary operation `>` cannot be applied to type `&T`
fn largest<T>(list: &[T]) -> &T {
    let mut biggest = &list[0];
    for item in list {
        if item > biggest { // error[E0369]: `T` has no `>` unless bounded
            biggest = item;
        }
    }
    biggest
}
```

The fix is the bound `T: PartialOrd`, which promises “`T` supports comparison.” A trait bound is a *contract*: it lists the abilities `T` must have, so the compiler can verify the body once and trust it for all valid types.

C vs Rust. A C macro never checks this. `MAX(a, b)` expands to text that uses `>`; if `a` and `b` are structs, you get a confusing error deep inside the expansion, far from the call. Rust reports the missing ability right at the bound, in plain language.

Generic structs

A struct can hold fields of a generic type. The type parameter goes after the struct name.

```
struct Point<T> {
    x: T,
    y: T,
}
```

```
fn main() {
    let integer = Point { x: 5, y: 10 };           // Point<i32>
    let float = Point { x: 1.0, y: 4.0 };         // Point<f64>

    println!("{}", integer.x, float.y);
}
```

`Point<T>` is like a C struct where the field type is a parameter you pick when you create the value. Note both fields here share **one** parameter `T`, so `x` and `y` must be the *same* type. `Point { x: 5, y: 1.0 }` would not compile.

Multiple type parameters

If two fields may differ, use two parameters.

```
struct Pair<T, U> {
    first: T,
    second: U,
}

fn main() {
    let mixed = Pair { first: 5, second: "hello" }; // Pair<i32, &str>
    println!("{}", mixed.first, mixed.second);
}
```

Rule of thumb. Add only as many type parameters as you truly need. Each one is another thing the caller (and the reader) must track. Two or three is common; more usually means the design needs rethinking.

Generic enums

You have already used generic enums, even if you did not notice. The two most important types in Rust are generic enums from the standard library:

```
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

`Option<T>` means “maybe a `T`” (Chapter 12 — Enums and Pattern Matching), and `Result<T, E>` means “a `T` on success or an `E` on error” (Chapter 13 — Error Handling). They are ordinary generic enums; nothing about them is magic. You can write your own the same way:

```
enum Tree<T> {
    Leaf(T),
}
```

```
    Node(Box<Tree<T>>, Box<Tree<T>>),
}
```

Generic methods

You implement methods on a generic type with `impl<T>`. The `<T>` right after `impl` declares the parameter so the rest of the block can use it.

```
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

fn main() {
    let p = Point { x: 5, y: 10 };
    println!("{}", p.x());
}
```

You can also write methods for **one** concrete type only. This block adds a method that exists just for `Point<f64>`:

```
struct Point<T> {
    x: T,
    y: T,
}

impl Point<f64> {
    fn distance_from_origin(&self) -> f64 {
        (self.x * self.x + self.y * self.y).sqrt()
    }
}

fn main() {
    let p = Point { x: 3.0, y: 4.0 };
    println!("{}", p.distance_from_origin());    // 5.0
}
```

A `Point<i32>` would not have `distance_from_origin`, because `sqrt` is a method on floating-point numbers only. This lets you give extra abilities to specific versions of a generic type.

Trait bounds in depth

A **trait** is a named set of behaviors (the full topic of Chapter 15 — Traits). For now, treat a trait as an *interface*: a list of methods or operators a type provides. A **trait bound** restricts a type

parameter to types that implement a given trait.

You need a bound to use any operation on a generic value, because the compiler must guarantee the operation exists for every possible T.

Multiple bounds with +

Require several traits at once by joining them with +.

```
use std::fmt::Display;

fn show_largest<T: PartialOrd + Display>(list: &[T]) {
    let mut biggest = &list[0];
    for item in list {
        if item > biggest {
            biggest = item;
        }
    }
    println!("largest is {biggest}");
}

fn main() {
    show_largest(&[3, 7, 2, 9, 4]);
}
```

Here T must be both comparable (PartialOrd) and printable (Display).

where clauses for readability

When a function has several parameters with several bounds, the signature gets crowded. A where clause moves the bounds below the signature, where they are easier to read.

```
use std::fmt::{Debug, Display};

// Hard to read:
fn report<T: Display + Clone, U: Clone + Debug>(t: &T, u: &U) {
    println!("{t}");
    let _t2 = t.clone();
    let _u2 = u.clone();
    println!("{u:?}");
}

// Same thing, easier to read with `where`:
fn report2<T, U>(t: &T, u: &U)
where
    T: Display + Clone,
    U: Clone + Debug,
{
    println!("{t}");
    let _t2 = t.clone();
}
```

```

    let _u2 = u.clone();
    println!("{u:?}");
}

fn main() {
    report(&"hi", &vec![1, 2, 3]);
    report2(&"hi", &vec![1, 2, 3]);
}

```

Rule of thumb. Use inline bounds (`<T: Clone>`) for one or two simple bounds; switch to `where` once the line gets long or there are several parameters. Both mean exactly the same thing.

Monomorphization: generics with zero runtime cost

Here is the key idea, and the reason Rust generics are not like C `void *` callbacks.

When you call a generic function with a concrete type, the compiler generates a **separate, specialized copy** of the function for that type, with `T` replaced by the real type. This process is called **monomorphization** (turning one “poly-typed” function into many “mono-typed” ones).

Take this code:

```

fn id<T>(x: T) -> T {
    x
}

fn main() {
    let a = id(5);           // T = i32
    let b = id(2.5);        // T = f64
}

```

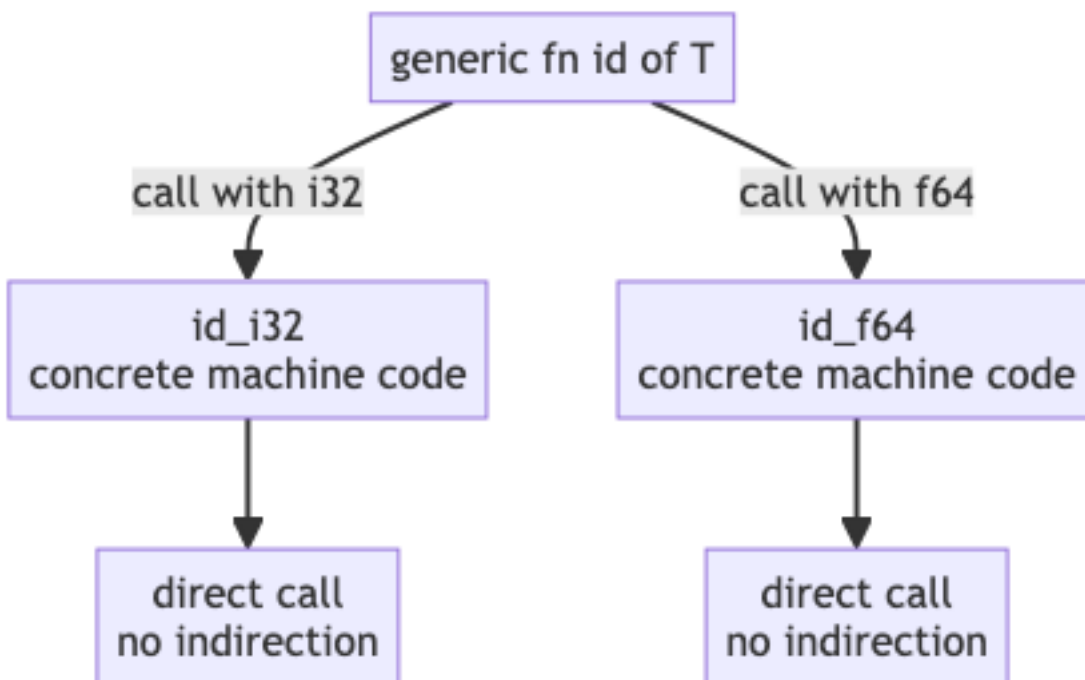
The compiler effectively rewrites it into two ordinary, non-generic functions, as if you had written:

```

fn id_i32(x: i32) -> i32 { x }
fn id_f64(x: f64) -> f64 { x }

fn main() {
    let a = id_i32(5);
    let b = id_f64(2.5);
}

```



Because each copy is a normal function with concrete types, there is:

- **No run-time type lookup.** The type is fixed at compile time.
- **No pointer indirection.** Unlike a `void *` callback, there is no function pointer to follow; calls can be inlined.
- **Full optimization.** The compiler optimizes each copy as if you wrote it by hand. This is what “zero-cost abstraction” means.

C vs Rust. A `qsort`-style design pays at run time: every comparison is an indirect call through a function pointer, and the compiler cannot inline it. Rust’s monomorphized generic has no indirection at all — it runs exactly as fast as a version you wrote for that one type by hand.

The trade-off: code size

Monomorphization is not entirely free. Each concrete type gets its own copy of the machine code. If you use a big generic function with ten different types, the compiler emits ten copies. This can grow the binary — an effect known as **code bloat** — and can lengthen compile times.

Concept	C macro	C <code>void *</code> callback	Rust generic
Type checked	No (text substitution)	Weakly (casts)	Fully, at compile time
Run-time cost	None	Indirection per call	None (monomorphized)
Code size	One copy per use site (inlined)	One shared copy	One copy per concrete type
Error messages	Poor, inside expansion	Crash at run time	Precise, at the bound
Can be inlined	Yes	No	Yes

Rule of thumb. Reach for generics by default; the speed is worth it. If binary size matters a lot (embedded targets) and a function is large, you can trade speed for size by using a trait object (`dyn Trait`, Chapter 15 — Traits), which keeps a single shared copy.

Const generics

So far the parameter has been a *type*. A **const generic** lets the parameter be a *value* known at compile time — almost always an array length. This is how Rust talks about fixed-size arrays `[T; N]` in generic code.

```
struct Buffer<const N: usize> {
    data: [u8; N],
}

impl<const N: usize> Buffer<N> {
    fn new() -> Self {
        Buffer { data: [0; N] }
    }

    fn len(&self) -> usize {
        N
    }
}

fn main() {
    let small: Buffer<8> = Buffer::new();
    let big: Buffer<1024> = Buffer::new();

    println!("{}", small.len(), big.len());    // 8 1024
}
```

`Buffer<8>` and `Buffer<1024>` are different types with different sizes, both generated from one definition. The length lives in the type, so it costs nothing at run time and the size is known to the optimizer.

C vs Rust. In C you would pass the size as a separate `size_t` argument, or bake it in with a macro. A const generic ties the size to the type itself, so the compiler checks it and the wrong size cannot slip through.

The turbofish: helping inference

Usually the compiler infers type parameters from the arguments. Sometimes it cannot — there are no arguments to look at, or several types would fit. Then you spell out the type with the **turbofish** syntax: `::<Type>` after the name.

```
fn main() {
    // `parse` could produce many number types. Which one? Tell it.
    let n = "42".parse::<i32>().unwrap();    // turbofish picks i32
    println!("{}", n);
}
```



```

// `collect` can build many collections. Annotate the target type.
let v = (1..=5).collect::

```

The name “turbofish” comes from the shape `::<>`. You need it whenever the result type cannot be inferred from context. The two common cases are `parse` and `collect`, because both can produce many different types.

Watch out. Forgetting the turbofish on `parse/collect` gives an error like “type annotations needed.” The compiler is not confused about your logic; it just cannot guess the target type. Add `::<T>` or annotate the variable.

Static dispatch (a look ahead)

Everything in this chapter uses **static dispatch**: the exact function to call is chosen at *compile time*, because monomorphization produces a concrete function for each type. There is no run-time decision.

The opposite is **dynamic dispatch**, where the type is decided at *run time* and the call goes through a table of function pointers (a vtable). Rust does that with trait objects (`dyn Trait`), the subject of Chapter 15 — Traits. Generics are fast and inline-able but can grow the binary; trait objects keep one copy but pay a small run-time cost. Keep this contrast in mind; the next chapter makes it concrete.

Key takeaways

- **Generics** let you write code once for many types, with full compile-time type checking — unlike C macros (untyped text) and `void *` (unchecked casts).
- A **type parameter** like `<T>` is a placeholder filled in per call. Structs, enums, functions, and methods can all be generic. `Option<T>` and `Result<T, E>` are just generic enums.
- You need a **trait bound** (`T: SomeTrait`) to use any operation on a generic value; the compiler must know the operation is valid for every possible type. Combine bounds with `+` and tidy them with `where`.
- **Monomorphization** generates a specialized copy per concrete type, so generics have **zero run-time cost** and can be inlined — the trade-off is larger code size.
- **Const generics** (`<const N: usize>`) parameterize over a compile-time value, usually an array length.
- Use the **turbofish** (`name::<Type>()`) when type inference needs help, most often with `parse` and `collect`.

Watch out (gotchas for C programmers)

- **A generic type can do almost nothing without a trait bound.** Unlike a C macro that just substitutes text, the compiler refuses operations it cannot prove valid for all `T`. Add the

bound that names the ability you need.

- **Monomorphization can bloat the binary.** Many concrete types means many code copies. If size matters, consider a trait object (`dyn`, Chapter 15 — Traits).
- **Generics are resolved at compile time (static dispatch).** There is no run-time flexibility here; for run-time polymorphism you need `dyn Trait`.
- **`Point<T>` with one parameter forces both fields to the same type.** Use two parameters (`Point<T, U>`) when they may differ.
- **Inference is not magic.** When the result type is ambiguous (`parse`, `collect`), you must use the turbofish or annotate the variable.

Interview questions

Q: What is monomorphization, and what does it cost? A: Monomorphization is the compiler generating a separate, specialized copy of a generic function or type for each concrete type it is used with, replacing the type parameter with the real type. The benefit is zero run-time cost: no indirection, full inlining, code as fast as hand-written. The cost is larger binary size and potentially longer compile times, because each concrete type gets its own copy.

Q: Why must a generic function declare trait bounds to use operators or methods on its parameters? A: Inside a generic function the type is unknown, so the compiler must prove that every operation is valid for *every* type the function could be called with. A trait bound (for example `T: PartialOrd`) is the contract that guarantees the needed ability exists, letting the compiler check the body once and trust it for all valid types.

Q: How do Rust generics differ from C macros and from `void *` callbacks? A: A C macro is untyped text substitution with no checking and can evaluate arguments multiple times. A `void *` callback throws away type information and pays a run-time indirection on every call, with mistakes surfacing as crashes. Rust generics are fully type-checked at compile time and monomorphized, so they are both type-safe and as fast as specialized hand-written code.

Q: What is the turbofish, and when do you need it? A: The turbofish is the `::<Type>` syntax used to specify a generic type parameter explicitly, as in `"42".parse::<i32>()`. You need it when the compiler cannot infer the type from context — most commonly with `parse` and `collect`, which can each produce many different result types. Annotating the variable’s type is an equivalent alternative.

Q: What is a `const` generic? A: A `const` generic is a generic parameter that is a compile-time *value* rather than a type, declared like `<const N: usize>`. It is most often used for array lengths, as in `struct Buffer<const N: usize> { data: [u8; N] }`. The value lives in the type, so different lengths are different types and there is no run-time cost.

Try it

1. Write `fn first<T: Clone>(list: &[T]) -> T { list[0].clone() }` and call it on a `Vec<i32>` and a `Vec<String>`. Remove the `Clone` bound and read the error.
2. Make a generic `struct Wrapper<T>(T);` with a method `fn get(&self) -> &T`. Then add an `impl Wrapper<i32>` block with a method that only `i32` wrappers have, and confirm a `Wrapper<&str>` does not get it.
3. Call `let v = (1..10).collect();` with no type annotation, read the “type annotations needed” error, then fix it two ways: with a turbofish and with a variable annotation.

Chapter 15 — Traits

What you’ll learn. How Rust expresses *shared behavior* with traits — the rough equivalent of C interfaces built from structs of function pointers, but type-checked. You will learn how to define and implement traits, the difference between static and dynamic dispatch (trait objects and their vtables), the orphan rule, and the standard-library traits every Rust programmer must know.

What a trait is

A **trait** is a named set of methods that a type can promise to provide. It describes *behavior*, not data. If you know other languages, a trait is close to an *interface* (Java, Go) or a *typeclass* (Haskell). For a C programmer, the closest familiar thing is a **struct of function pointers** that several types agree to fill in — except a trait is checked by the compiler, so you can never forget a method or get a signature wrong.

```
trait Summary {  
    fn summarize(&self) -> String;  
}
```

This says: “Any type that *is* `Summary` must have a method `summarize` returning a `String`.” It is a contract. By itself it does nothing; types must opt in.

Mental model. A trait is a checklist of abilities. A type that wants those abilities implements the trait and fills in every item on the checklist. Code can then accept “any type that ticks this checklist” instead of one fixed type.

Implementing a trait

You connect a trait to a type with an `impl Trait for Type` block.

```
trait Summary {  
    fn summarize(&self) -> String;  
}  
  
struct Article {  
    title: String,  
    body: String,  
}  
  
struct Tweet {  
    user: String,  
    text: String,  
}  
  
impl Summary for Article {  
    fn summarize(&self) -> String {  
        format!("{}", self.title, &self.body[..10.min(self.body.len())])  
    }  
}
```

```

impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("@{}: {}", self.user, self.text)
    }
}

fn main() {
    let a = Article {
        title: String::from("Rust 1.96"),
        body: String::from("A new release with many improvements."),
    };
    let t = Tweet {
        user: String::from("rustlang"),
        text: String::from("we shipped"),
    };

    println!("{}", a.summarize());
    println!("{}", t.summarize());
}

```

Two unrelated types now share one behavior. Note that the method is called with ordinary dot syntax, `a.summarize()`, exactly like a normal method.

C vs Rust. In C you would define `struct SummaryOps { char *(*summarize)(void *self); }` and give each type its own ops table plus a `void *self`. You must wire the pointers up by hand and a wrong cast is undefined behavior. A trait does the same job, but the compiler builds and checks everything, and there is no `void *`.

Default method implementations

A trait may supply a **default** body for a method. Types that implement the trait get the default for free and may override it.

```

trait Summary {
    fn summarize(&self) -> String {
        String::from("(no summary)")    // default body
    }
}

struct Photo;

impl Summary for Photo {}    // empty: takes the default

fn main() {
    let p = Photo;
    println!("{}", p.summarize());    // prints (no summary)
}

```

A default method may call other methods of the same trait, even ones with no default. This is

how a trait defines a lot of behavior in terms of a few required methods — the `Iterator` trait does exactly this.

Using traits to write generic code

Traits become powerful when combined with generics (Chapter 14 — Generics). A **trait bound** lets a generic function accept any type that implements a trait.

```
trait Summary {
    fn summarize(&self) -> String;
}

fn notify<T: Summary>(item: &T) {
    println!("Breaking! {}", item.summarize());
}
```

`notify` works for any `T` that implements `Summary`. Inside the body, the only thing you may do with `item` is call `Summary` methods — that is exactly what the bound promised.

`impl Trait` in argument position

For the common case of “some type that implements this trait,” there is a shorter syntax: write `impl Trait` as the parameter type.

```
trait Summary {
    fn summarize(&self) -> String;
}

fn notify(item: &impl Summary) {
    println!("Breaking! {}", item.summarize());
}
```

`&impl Summary` means “a reference to some single type that implements `Summary`.” It is sugar for the generic version above; it still uses static dispatch and monomorphization. Use it when you do not need to name the type parameter.

`impl Trait` in return position

You can also return `impl Trait`, meaning “I return some concrete type that implements this trait, but I will not spell out which one.” This is essential for returning closures and iterators, whose real types are unnameable.

```
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}

fn main() {
    let add5 = make_adder(5);
    println!("{}", add5(10));    // 15
}
```

Watch out. `-> impl Trait` returns **one** concrete type, fixed at compile time. You cannot return a **Tweet** from one branch and an **Article** from another with `-> impl Summary`; for that you need a trait object (`Box<dyn Summary>`, below).

Static vs dynamic dispatch

This is the central idea of the chapter. There are two ways to call a method through a trait, and a C programmer should understand both clearly.

Static dispatch is what generics and `impl Trait` use. The compiler monomorphizes: it generates a specialized copy of the code for each concrete type, so the exact function is known at compile time. Calls can be inlined. There is no run-time cost. The downside is more machine code (see Chapter 14 — Generics).

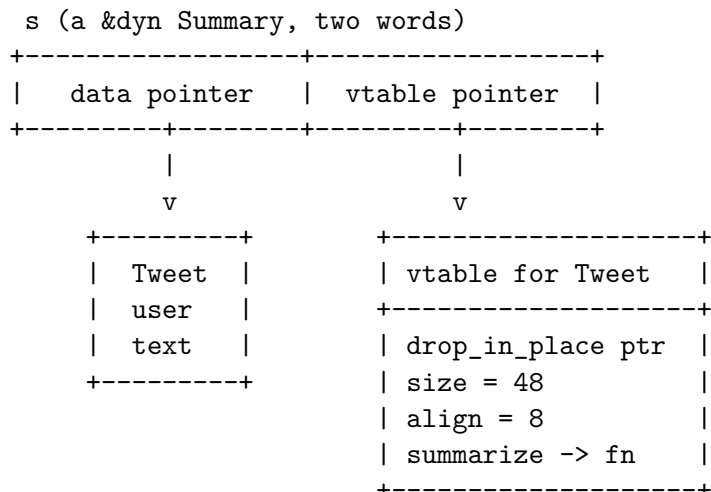
Dynamic dispatch uses a **trait object**, written `dyn Trait`. The concrete type is *not* known at compile time. Instead, the call goes through a table of function pointers — a **vtable** (virtual method table) — chosen at run time. This keeps one copy of the code and lets a single collection hold many different types, at the cost of an indirect call that cannot be inlined.

Aspect	Static (<code>impl Trait</code> / generics)	Dynamic (<code>dyn Trait</code>)
When the method is chosen	Compile time	Run time
Mechanism	Monomorphized copies	Vtable (function pointers)
Speed	Fastest; inlinable	One indirect call per method
Code size	One copy per type	One shared copy
Can mix types in a collection	No	Yes
Pointer size	Normal	Fat pointer (data + vtable)

What a trait object looks like in memory

A reference to a trait object is a **fat pointer**: two machine words. The first points to the data; the second points to the vtable for that type. The vtable holds the function pointers for the trait's methods (plus the type's size, alignment, and destructor).

```
let s: &dyn Summary = &tweet;
```



Calling `s.summarize()` reads the `summarize` slot from the `vtable` and calls it, passing the data pointer as `self`. This is **exactly** the C pattern of a struct of function pointers plus a `void *self` — but the compiler builds the `vtable` and guarantees the function signatures match.

C vs Rust. A C “object” is often `struct { const Ops *vtable; ... data; }` where `Ops` is a struct of function pointers. Rust’s trait object is the same machine idea, split into a (data, `vtable`) pair, with full type checking and no manual wiring.

Using trait objects

Trait objects appear behind a pointer: `&dyn Trait`, `&mut dyn Trait`, or `Box<dyn Trait>` (an owning heap pointer — see Chapter 17 — Smart Pointers). The classic use is a collection of mixed types that share a trait.

```
trait Summary {
    fn summarize(&self) -> String;
}

struct Article { title: String }
struct Tweet { user: String }

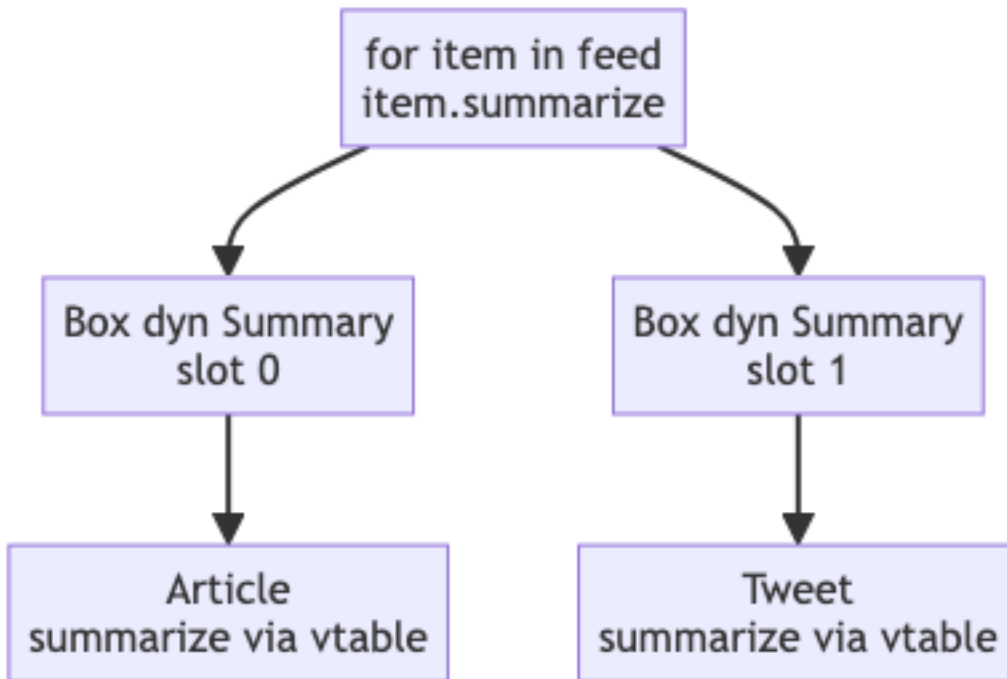
impl Summary for Article {
    fn summarize(&self) -> String {
        format!("article: {}", self.title)
    }
}

impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("tweet by @{}", self.user)
    }
}

fn main() {
    // A Vec of different concrete types, unified by `dyn Summary`.
    let feed: Vec<Box<dyn Summary>> = vec![
        Box::new(Article { title: String::from("Rust news") }),
        Box::new(Tweet { user: String::from("rustlang") }),
    ];

    for item in &feed {
        println!("{}", item.summarize());    // dynamic dispatch through the vtable
    }
}
```

You cannot do this with generics: a `Vec<T>` holds one type `T`. The trait object erases the concrete type and lets one vector hold both.



Rule of thumb. Reach for generics / `impl Trait` by default — they are faster. Use `dyn Trait` when you need to store mixed types together, when the set of types is open-ended (plugins), or when monomorphized code size is a problem.

Object safety

Not every trait can become a `dyn` trait object. A trait is **object safe** only if its methods can be dispatched through a vtable. The two main rules:

- Methods must take `self` by reference (`&self` or `&mut self`), not by value or as a bare generic, so the vtable can call them through a pointer.
- Methods must not be **generic** (no `fn foo<U>(&self)`), because each `U` would need its own vtable slot, which cannot exist.
- The trait must not use `Self` as a return type in a way that needs the concrete type (for example returning `Self`), because the concrete type is erased.

```
// COMPILER ERROR: the trait `Cloneable` cannot be made into an object
trait Cloneable {
    fn duplicate(&self) -> Self;    // returns Self by value -> not object safe
}

fn store(_x: Box<dyn Cloneable>) {} // error[E0038]: not object safe

fn main() {}
```

If a trait is not object safe, you simply cannot use `dyn` with it; use generics instead. Most everyday traits (`Display`, your own service traits) are object safe.

The orphan rule (coherence)

You may write `impl Trait for Type` only if **you define the trait, or you define the type** (or both) in your crate. You cannot implement a trait you do not own for a type you do not own. This is the **orphan rule**, part of what Rust calls *coherence*.

```
// COMPILER ERROR: only traits defined in the current crate can be implemented
//                  for types defined outside of the crate (orphan rule)
impl std::fmt::Display for Vec<i32> {    // both Display and Vec are foreign
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "len {}", self.len())
    }
}

fn main() {}
```

The reason: if two different crates could both add `impl Display for Vec<i32>`, the compiler would not know which one to use, and your program’s meaning could change just by adding a dependency. The orphan rule keeps each (trait, type) pair implemented in exactly one place.

Rule of thumb. To add behavior to a foreign type, wrap it in your own type (the *newtype* pattern — Chapter 27 — Idioms and Style): `struct MyVec(Vec<i32>);`, then `impl Display for MyVec`. Now the type is yours, so the rule is satisfied.

The standard-library traits you must know

Rust’s standard library defines a small set of traits that appear everywhere. Knowing them is most of “knowing Rust.” Many can be **derived** automatically.

Trait	What it gives you	Usually obtained by
Debug	<code>{:?}</code> developer-facing formatting	<code>#[derive(Debug)]</code>
Display	<code>{}</code> user-facing formatting	hand-written
Clone	explicit deep copy via <code>.clone()</code>	<code>#[derive(Clone)]</code>
Copy	implicit bitwise copy on assignment	<code>#[derive(Copy, Clone)]</code>
PartialEq / Eq	<code>==</code> and <code>!=</code>	<code>#[derive(PartialEq, Eq)]</code>
PartialOrd / Ord	<code><</code> , <code>></code> , sorting	<code>#[derive(PartialOrd, Ord)]</code>
Default	a default value via <code>Default::default()</code>	<code>#[derive(Default)]</code>
Hash	use as a <code>HashMap</code> key	<code>#[derive(Hash)]</code>
From / Into	type conversions	hand-written
Iterator	the <code>for</code> loop and iterator adapters	hand-written
Drop	run code when a value is freed (RAII)	hand-written

Deriving traits

`#[derive(...)]` tells the compiler to generate a standard implementation for you. It works when every field also implements the trait.

```
#[derive(Debug, Clone, PartialEq, Default)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    let q = p.clone();

    println!("{p:?}");           // Debug: Point { x: 1, y: 2 }
    println!("{}", p == q);      // PartialEq: true
    let origin = Point::default(); // Default: Point { x: 0, y: 0 }
    println!("{origin:?}");
}
```

C vs Rust. In C you write `point_equals`, `point_copy`, and a `print_point` by hand for every struct. `#[derive]` generates the correct, field-by-field versions for you, and updates them automatically when you add a field.

Display vs Debug

This pair confuses newcomers, so be precise:

- **Debug** (`{:?}`) is for *programmers*. It shows the structure for debugging and logging. You almost always derive it. There is also `{:#?}` for a pretty-printed, multi-line form.
- **Display** (`{}`) is for *end users*. It is the polished, human-readable form. It cannot be derived, because only you know how your type should look to a user. You implement it by hand.

```
use std::fmt;

struct Temperature {
    celsius: f64,
}

impl fmt::Display for Temperature {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{:.1} C", self.celsius)
    }
}

fn main() {
    let t = Temperature { celsius: 21.345 };
    println!("{}", t);           // 21.3 C    (Display)
}
```

Implementing `Display` also gives you `.to_string()` for free, because the standard library provides a blanket implementation of `ToString` for every `Display` type (more on blanket impls below).

From and Into

`From` defines how to build one type from another. Implement `From`, and you get `Into` automatically — they are two views of the same conversion.

```
struct Celsius(f64);
struct Fahrenheit(f64);

impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) -> Self {
        Fahrenheit(c.0 * 9.0 / 5.0 + 32.0)
    }
}

fn main() {
    let f = Fahrenheit::from(Celsius(100.0));
    let f2: Fahrenheit = Celsius(0.0).into(); // Into comes for free
    println!("{}", f.0, f2.0);               // 212 32
}
```

`From/Into` also power the `?` operator's error conversion (Chapter 13 — Error Handling): a `?` can turn one error type into another if a `From` impl exists.

Associated types

Some traits attach a **type** to the implementation, not just methods. This is an **associated type**. The standard example is `Iterator`, whose associated type `Item` names what the iterator yields.

```
struct Counter {
    count: u32,
}

impl Iterator for Counter {
    type Item = u32; // associated type: this iterator yields u32

    fn next(&mut self) -> Option<Self::Item> {
        if self.count < 3 {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let total: u32 = Counter { count: 0 }.sum();
}
```

```
println!("{total}");           // 1 + 2 + 3 = 6
}
```

An associated type is like a type parameter, but the implementer fixes it once per type, so callers do not have to specify it. `Counter` yields `u32` and nothing else. Once you implement `next`, you get dozens of methods (`map`, `filter`, `sum`, ...) for free from default methods on `Iterator` — covered in Chapter 16 — Collections and Iterators.

Supertraits

A trait can require that types also implement another trait. The required trait is a **supertrait**. Here `Summary` requires `Display`, so any `Summary` type can be printed with `{}`, and the trait's methods may rely on that.

```
use std::fmt::Display;

trait Summary: Display {
    fn summarize(&self) -> String {
        format!("summary of: {self}")    // allowed: Display is guaranteed
    }
}

struct Note(String);

impl Display for Note {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "{}", self.0)
    }
}

impl Summary for Note {}

fn main() {
    let n = Note(String::from("buy milk"));
    println!("{}", n.summarize());
}
```

Blanket implementations

A **blanket impl** implements a trait for *every* type that satisfies some bound. The standard library uses this heavily. The classic example: every type that implements `Display` automatically implements `ToString`.

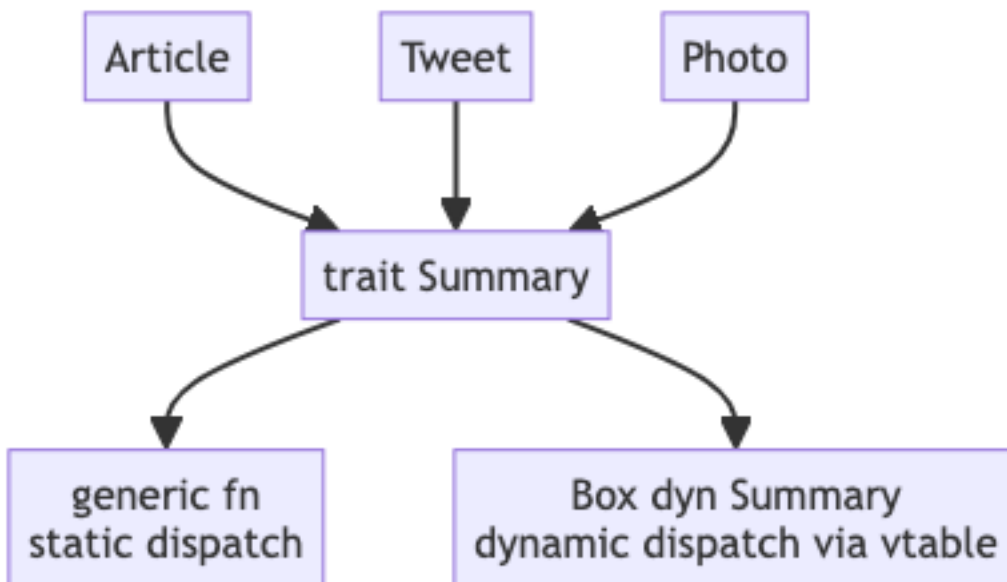
```
// From the standard library (paraphrased):
// impl<T: Display> ToString for T {
//     fn to_string(&self) -> String { /* uses Display */ }
// }

fn main() {
    let s: String = 42.to_string();    // i32 is Display, so it is ToString too
}
```

```
println!("{}", s);
}
```

You can write your own blanket impls for traits you define. They are a powerful way to give behavior to a whole family of types at once.

How traits map to C, summarized



C vs Rust. C has interfaces only “by convention”: a struct of function pointers, an agreement about a `void *self`, and discipline to keep them in sync. A trait makes that convention a real, compiler-checked thing. A generic with a trait bound is the *static* version (resolved at compile time); a `dyn Trait` is the *dynamic* version (a real vtable at run time) — and the compiler builds the vtable for you.

Key takeaways

- A **trait** defines shared behavior — a checklist of methods. Types opt in with `impl Trait for Type`. Traits may provide **default** method bodies.
- Use traits through **trait bounds** on generics, `impl Trait` arguments and return types, and `where` clauses.
- **Static dispatch** (generics, `impl Trait`) is monomorphized and fast. **Dynamic dispatch** (`dyn Trait`) uses a **fat pointer** (data + vtable) and an indirect call, but lets one collection hold mixed types.
- **Object safety** limits which traits can be `dyn`: no by-value `self`, no generic methods, no returning `Self`.
- The **orphan rule** lets you implement a trait for a type only if you own the trait or the type. Wrap foreign types in a newtype to get around it.
- Know the standard traits: `Debug/Display`, `Clone/Copy`, `PartialEq/Eq`, `PartialOrd/Ord`, `Default`, `Hash`, `From/Into`, `Iterator`, `Drop`. Many are **derived**; `Display` is hand-written.
- **Associated types** (`Iterator::Item`), **supertraits**, and **blanket impls** round out the system.

Watch out (gotchas for C programmers)

- **Static (`impl Trait`/generics) vs dynamic (`dyn`) dispatch is a real choice.** Generics are faster but grow the binary and cannot mix types; `dyn` shares one copy and mixes types but adds an indirect call.
- **-> `impl Trait` returns one concrete type.** To return different types from different branches, use `Box<dyn Trait>`.
- **Object safety limits `dyn`.** Traits with generic methods or by-value `self` cannot be made into trait objects.
- **The orphan rule blocks `impl ForeignTrait for ForeignType`.** Use a newtype wrapper when you must.
- **`derive` needs every field to implement the trait too.** You cannot derive `Clone` for a struct containing a non-`Clone` field.
- **`Display` is user-facing and not derivable; `Debug` is developer-facing and is derivable.** Do not confuse `{}` with `{:?}`.

Interview questions

Q: What is the difference between static and dynamic dispatch in Rust? A: Static dispatch (generics and `impl Trait`) resolves the called method at compile time by monomorphizing a specialized copy per type; calls can be inlined and there is no run-time cost, but binary size grows. Dynamic dispatch (`dyn Trait`) resolves the method at run time through a vtable; there is one shared copy and you can store mixed types together, at the cost of an indirect, non-inlinable call.

Q: What is a trait object, and what does it look like in memory? A: A trait object (`&dyn Trait` or `Box<dyn Trait>`) is a value whose concrete type is erased and accessed through a trait. In memory it is a *fat pointer*: two words, one pointing to the data and one pointing to the vtable. The vtable holds the trait method pointers plus the type's size, alignment, and destructor. It is the same idea as a C struct of function pointers with a `void *self`, but built and checked by the compiler.

Q: What is the orphan rule and why does it exist? A: The orphan rule says you may implement a trait for a type only if you define the trait or the type in your crate. It exists for coherence: it guarantees a given (trait, type) pair has exactly one implementation across all crates, so two dependencies cannot provide conflicting impls and adding a dependency cannot silently change behavior. The newtype pattern works around it.

Q: When would you choose `dyn Trait` over a generic with a trait bound? A: Use `dyn Trait` when you need to store values of different concrete types in one collection, when the set of types is open-ended (for example a plugin or callback system), or when monomorphization would bloat the binary too much. Otherwise prefer generics, which are faster because they are statically dispatched and inlinable.

Q: What is the difference between `Display` and `Debug`? A: `Debug` (the `{:?}` format) is developer-facing output for debugging and logging, and it can be derived with `#[derive(Debug)]`. `Display` (the `{}` format) is polished, user-facing output and must be implemented by hand, because only the author knows how the type should look to a user. Implementing `Display` also provides `.to_string()` via a blanket impl.

Q: What is an associated type, and how does it differ from a generic parameter?

A: An associated type is a type declared inside a trait (for example `Iterator::Item`) that each implementer fixes once. A generic parameter is chosen by the *caller* and can vary per call, while an associated type is chosen by the *implementer* and is fixed for that type, so callers do not have to specify it. This is why you write `Iterator<Item = u32>` results without passing the item type at every call site.

Try it

1. Define `trait Shape { fn area(&self) -> f64; }`, implement it for a `Circle` and a `Rectangle`, and put both in a `Vec<Box<dyn Shape>>`. Sum their areas in a loop.
2. Add a default method `fn describe(&self) -> String` to `Shape` that calls `self.area()`. Confirm both types get it without writing it twice.
3. Implement `Display` for `Circle` so `println!("{circle}")` prints its radius, then call `.to_string()` on it and notice it works for free.

Chapter 16 — Collections and Iterators

What you’ll learn. Rust’s standard collections — `Vec`, `String`, `HashMap`, and friends — and the lazy, zero-cost **iterator** pipeline that replaces the hand-written index loops and `qsort` calls you know from C.

The problem these solve

In C, the standard library gives you almost no data structures. You either use a fixed-size array, hand-roll a dynamic array with `malloc/realloc/free`, or write your own hash table (or pull in a third-party one). You track the length in a separate variable, you check bounds yourself, and you free the memory by hand.

Rust ships a small set of well-tested, generic collections in the standard library, all of which manage their own memory and free it automatically when they go out of scope. On top of them sits the **iterator**: a uniform way to walk over any collection, transform it, and collect the result, all checked at compile time and compiled down to a plain loop.

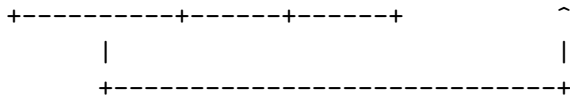
C vs Rust. A C dynamic array is a struct you write: `{ T *data; size_t len; size_t cap; }` plus a pile of functions and manual `free`. Rust’s `Vec<T>` is that exact struct, written once, generic, bounds-checked, and freed for you.

`Vec<T>`: the growable array

A `Vec<T>` (read “vector”) is a growable array of values of type `T`, stored on the heap. It is the collection you will reach for most. It owns its buffer, and when the `Vec` is dropped, the buffer is freed.

A `Vec` is three machine words: a pointer to the heap buffer, the **length** (how many elements are in use), and the **capacity** (how many fit before it must grow).

Vec<i32> on the stack				heap buffer (capacity = 4)			
+-----+-----+-----+				+-----+-----+-----+			
ptr	len	cap		10	20	30	??
0x6000..	3	4		+-----+-----+-----+			



`len` is how many elements you have; `cap` is how many the buffer holds. When you push past `cap`, the `Vec` allocates a bigger buffer, copies the elements over, and frees the old one — exactly what you would do with `realloc`, done for you.

Creating and growing a `Vec`

```
fn main() {
    // The vec! macro builds and fills a Vec in one step.
    let primes = vec![2, 3, 5, 7, 11];

    // Build an empty Vec and push onto it. `mut` is required to change it.
    let mut stack: Vec<i32> = Vec::new();
    stack.push(1);
    stack.push(2);
    stack.push(3);

    let top = stack.pop(); // pop() returns Option<i32>: Some(3) here, None if empty
    println!("{primes:?} top={top:?} stack={stack:?}");
}
```

- `vec![a, b, c]` is a macro that creates a `Vec` with those elements.
- `Vec::new()` makes an empty `Vec`. The type is inferred from later use, or you annotate it (`Vec<i32>`).
- `push` appends one element; `pop` removes the last and returns an `Option<T>` (`Some(value)`, or `None` if the `Vec` is empty). There is no “popped off an empty stack” undefined behavior.

Indexing: `v[i]` vs `.get(i)`

There are two ways to read an element, and the difference matters.

```
fn main() {
    let v = vec![10, 20, 30];

    let a = v[1]; // direct index: fast, but PANICS if i is out of bounds
    println!("a={a}");

    // .get(i) returns Option<&T>: Some(&value) in range, None out of range.
    match v.get(5) {
        Some(x) => println!("got {x}"),
        None => println!("index 5 is out of range"),
    }
}
```

- `v[i]` gives you the value directly but **panics** (cleanly aborts) if `i` is out of bounds. It never reads past the end — there is no buffer overrun.
- `v.get(i)` returns an `Option<&T>`, so you handle the out-of-range case yourself.

C vs Rust. In C, `v[i]` with a bad `i` reads or writes random memory: silent, dangerous, exploitable. In Rust, `v[i]` checks the bound and panics instead of corrupting memory. Use `.get(i)` when an out-of-range index is a normal, expected case rather than a bug.

with_capacity: avoid repeated reallocation

If you know roughly how many elements you will add, reserve the space up front, the same way you would size a `malloc` once instead of calling `realloc` in a loop.

```
fn main() {
    let mut v = Vec::with_capacity(1000); // one allocation, room for 1000
    for i in 0..1000 {
        v.push(i); // no reallocation happens during this loop
    }
    println!("len={} cap={}", v.len(), v.capacity());
}
```

`with_capacity(n)` allocates room for `n` elements but leaves the length at 0. It is a performance hint, not a size limit; the `Vec` still grows if you exceed it.

String: a UTF-8 Vec<u8> (recap)

A `String` is, under the hood, a `Vec<u8>` whose bytes are guaranteed to be valid **UTF-8** (the standard variable-width text encoding). It owns its heap buffer and grows the same way a `Vec` does. A `&str` is a borrowed view into that text, like a checked `const char *` that also knows its length.

```
fn main() {
    let mut s = String::new();
    s.push_str("Rust"); // append a &str
    s.push('!');        // append one char
    println!("{s} has {} bytes", s.len()); // len is BYTES, not characters
}
```

Strings are covered in full in Chapter 10 — Slices and Strings. The key point for this chapter: a `String` is just a specialized `Vec`, so it grows and frees the same way, and you can build one from an iterator with `collect` (shown below).

HashMap<K, V>: the key-value store

A `HashMap<K, V>` maps keys of type `K` to values of type `V` using a hash table. It is Rust's equivalent of the hash table you would hand-write or import in C. The keys must implement two traits: **Eq** (can be compared for equality) and **Hash** (can be hashed). All the primitive types and `String` already do.

`HashMap` is not in the prelude, so you import it.

```
use std::collections::HashMap;

fn main() {
    let mut scores: HashMap<String, i32> = HashMap::new();
    scores.insert("blue".to_string(), 10);
}
```

```

scores.insert("red".to_string(), 50);

// get() returns Option<V>: Some(&value) or None if the key is absent.
if let Some(s) = scores.get("blue") {
    println!("blue = {s}");
}

// Iteration order is UNSPECIFIED - do not rely on it.
for (team, score) in &scores {
    println!("{team}: {score}");
}
}

```

The entry API

A very common need is “update the value for a key, inserting a default if it is missing.” In C you would look up, check for null, and branch. Rust gives you the **entry** API, which does the lookup once.

```

use std::collections::HashMap;

fn main() {
    let text = "the cat the dog the bird";
    let mut counts: HashMap<&str, i32> = HashMap::new();

    for word in text.split_whitespace() {
        // or_insert returns a &mut to the value, inserting 0 if the key is new.
        *counts.entry(word).or_insert(0) += 1;
    }
    println!("{counts:?}"); // {"the": 3, "cat": 1, ...} in some order
}

```

`entry(key)` returns a handle to that slot. `or_insert(default)` ensures a value is present and hands you a mutable reference to it. The `*` dereferences that reference so you can add to the value in place.

HashSet, BTreeMap, BTreeSet, VecDeque

The standard library has a small family of collections. Pick by what you need.

Collection	What it is	C analogy	Order
<code>Vec<T></code>	growable array	malloc'd array + len	insertion order
<code>VecDeque<T></code>	double-ended queue	ring buffer	insertion order
<code>HashMap<K, V></code>	hash table	hand-rolled hash table	none
<code>HashSet<T></code>	set of unique keys	hash table of keys	none
<code>BTreeMap<K, V></code>	sorted map (B-tree)	balanced tree	sorted by key
<code>BTreeSet<T></code>	sorted set	balanced tree of keys	sorted

```

use std::collections::{HashSet, BTreeMap, VecDeque};

fn main() {
    // A HashSet stores unique values; inserting a duplicate is ignored.
    let mut seen = HashSet::new();
    seen.insert(3);
    seen.insert(3); // ignored
    println!("unique count = {}", seen.len()); // 1

    // A BTreeMap keeps keys sorted, so iteration is in key order.
    let mut sorted: BTreeMap<i32, &str> = BTreeMap::new();
    sorted.insert(2, "two");
    sorted.insert(1, "one");
    for (k, v) in &sorted {
        println!("{k} => {v}"); // 1 then 2, always
    }

    // A VecDeque pushes and pops cheaply at BOTH ends.
    let mut q: VecDeque<i32> = VecDeque::new();
    q.push_back(1);
    q.push_front(0);
    println!("{:?}", q); // [0, 1]
}

```

Rule of thumb. Use `HashMap/HashSet` when you only need fast lookup and do not care about order. Use `BTreeMap/BTreeSet` when you need keys kept sorted. Use `VecDeque` for a queue or a ring buffer.

Iterators: walking data without index loops

An **iterator** is any value that produces a sequence of items, one at a time. In Rust this is captured by the `Iterator` **trait**, whose core method is:

```

// Simplified from the standard library.
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}

```

`next` returns `Some(item)` for each element and `None` when the sequence is finished. That is the whole protocol. Everything else is built on it.

C vs Rust. In C you write `for (size_t i = 0; i < n; i++) use(a[i]);` and manage the index, the bound, and off-by-one risk yourself. A Rust iterator hides the index entirely: there is no `i` to get wrong, and no way to read past the end.

Iterators are lazy

This is the surprise for newcomers. Iterator **adaptors** — methods like `map` and `filter` that transform one iterator into another — do **nothing** on their own. They just build a description of

the work. The work runs only when a **consumer** (like `collect` or `sum`) pulls items through by calling `next`.

```
fn main() {
    // This line does NOT square anything yet. It builds a lazy iterator.
    let lazy = (1..=5).map(|x| {
        println!("squaring {x}");
        x * x
    });

    println!("nothing has printed yet");
    let squares: Vec<i32> = lazy.collect(); // NOW the closure runs, 5 times
    println!("{squares:?}");
}
```

Watch out. A pipeline that ends without a consumer does no work. If you write `v.iter().map(|x| do_something(x))`; and expect side effects, nothing happens — the compiler even warns that the iterator is unused. Add a consumer like `.for_each(...)` or `.collect()`.

The pipeline: source, adaptors, consumer

A typical iterator expression has three parts. Think of it as a pipe.



```
fn main() {
    let nums = vec![1, 2, 3, 4, 5, 6];

    let result: Vec<i32> = nums
        .iter()                // source: yields &i32
        .map(|&x| x * x)        // adaptor: square each
        .filter(|&x| x % 2 == 0) // adaptor: keep even squares
        .collect();            // consumer: gather into a Vec

    println!("{result:?}"); // [4, 16, 36]
}
```

Common adaptors

Adaptors take an iterator and return a new iterator. They are lazy.

Adaptor	What it does
<code>map(f)</code>	apply <code>f</code> to each item
<code>filter(pred)</code>	keep items where <code>pred</code> is true
<code>take(n)</code>	yield at most the first <code>n</code> items

Adaptor	What it does
<code>skip(n)</code>	drop the first <code>n</code> items
<code>enumerate()</code>	pair each item with its index: <code>(0, a)</code> , <code>(1, b)</code> , ...
<code>zip(other)</code>	pair items from two iterators together
<code>chain(other)</code>	yield all of one, then all of the other
<code>rev()</code>	reverse the order (needs a double-ended iterator)

```
fn main() {
    let names = ["Ada", "Linus", "Grace"];

    // enumerate gives you the index without a manual counter.
    for (i, name) in names.iter().enumerate() {
        println!("{i}: {name}");
    }

    let a = [1, 2, 3];
    let b = [10, 20, 30];
    // zip walks two iterators in lockstep, stopping at the shorter one.
    let sums: Vec<i32> = a.iter().zip(b.iter()).map(|(x, y)| x + y).collect();
    println!("{sums:?}"); // [11, 22, 33]

    let first_two: Vec<i32> = (1..).take(2).collect(); // (1..) is infinite; take bounds it
    println!("{first_two:?}"); // [1, 2]
}
```

Common consumers

A **consumer** drives the iterator to completion (or until it decides to stop) and produces a final value.

Consumer	Returns
<code>collect()</code>	build a collection (<code>Vec</code> , <code>String</code> , <code>HashMap</code> , ...)
<code>sum()</code> / <code>product()</code>	the total / product of the items
<code>count()</code>	how many items
<code>fold(init, f)</code>	reduce to one value (like a left fold / accumulator)
<code>for_each(f)</code>	run <code>f</code> for its side effects
<code>find(pred)</code>	first item matching <code>pred</code> , as <code>Option</code>
<code>position(pred)</code>	index of first match, as <code>Option</code>
<code>any(pred)</code> / <code>all(pred)</code>	does any / do all match? <code>bool</code>
<code>max()</code> / <code>min()</code>	largest / smallest, as <code>Option</code>

```
fn main() {
    let v = vec![3, 1, 4, 1, 5, 9, 2, 6];

    let total: i32 = v.iter().sum();
    let biggest = v.iter().max();           // Option<i32>
}
```

```

let evens = v.iter().filter(|&x| x % 2 == 0).count();
let has_big = v.iter().any(|&x| x > 8);    // true
let first_even = v.iter().find(|&x| x % 2 == 0); // Some(4)

// fold is the general reducer: start at 0, add each item.
let folded = v.iter().fold(0, |acc, &x| acc + x);

println!("{total} {biggest:?} {evens} {has_big} {first_even:?} {folded}");
}

```

iter vs iter_mut vs into_iter

There are three ways to get an iterator from a collection, and they differ by **how they touch the elements** — read, mutate, or take ownership.

Method	Yields	Effect on the collection
<code>iter()</code>	<code>&T</code> (shared reference)	borrows it; collection still usable after
<code>iter_mut()</code>	<code>&mut T</code> (mutable reference)	borrows it mutably; you can edit in place
<code>into_iter()</code>	<code>T</code> (owned value)	consumes it; collection is moved away

```

fn main() {
    let v = vec![1, 2, 3];
    let doubled: Vec<i32> = v.iter().map(|&x| x * 2).collect();
    // v is still usable here, because iter() only borrowed it.
    println!("{v:?} -> {doubled:?}");

    let mut w = vec![1, 2, 3];
    for x in w.iter_mut() {
        *x += 100; // edit each element in place through &mut
    }
    println!("{w:?}"); // [101, 102, 103]

    let owned = vec![String::from("a"), String::from("b")];
    for s in owned.into_iter() {
        // s is an owned String here, moved out of the Vec.
        println!("took {s}");
    }
    // owned is gone now - into_iter() consumed it.
}

```

Watch out. `for x in v` uses `into_iter()` and **moves** `v` (unless `T` is `Copy`). If you want to keep `v`, write `for x in &v` (which calls `iter()`) or `for x in &mut v` (which calls `iter_mut()`).

collect: choosing the target type

collect can build many different collections, so it must be told **which** one. You tell it with a type annotation on the variable, or with the **turbofish** syntax `::<Type>` on the call.

```
use std::collections::HashMap;

fn main() {
    // 1. Type annotation on the binding tells collect the target.
    let squares: Vec<i32> = (1..=4).map(|x| x * x).collect();

    // 2. Turbofish on collect itself. The _ lets Rust infer the inner type.
    let doubled = (1..=4).map(|x| x * 2).collect::
```

Watch out. `collect` is generic over the return type. If Rust cannot tell which collection you want, you get an error like “type annotations needed.” Add a type on the variable or a turbofish on `collect`.

Iterators are zero-cost: a for loop is sugar

A Rust `for` loop is **syntactic sugar** for using an iterator. These two are the same program:

```
fn main() {
    let v = vec![10, 20, 30];

    // What you write:
    for x in &v {
        println!("{x}");
    }

    // What it desugars to (roughly):
    let mut it = (&v).into_iter();
    while let Some(x) = it.next() {
        println!("{x}");
    }
}
```

Because adaptors are inlined and the index is hidden, an iterator pipeline compiles to the same tight machine code as a hand-written C loop — often **better**, because the compiler can prove there are no out-of-bounds accesses and skip some checks. This is what “zero-cost abstraction” means: you write it at a high level and pay nothing extra at runtime.

C vs Rust. To sort in C you call `qsort` with a `void *` comparator and cast pointers by hand, with no type checking. In Rust you call `v.sort()` (or `v.sort_by(|a, b| ...)`), which is type-checked and needs no casts.

```
fn main() {
    let mut v = vec![3, 1, 4, 1, 5];
    v.sort(); // ascending
    v.sort_by(|a, b| b.cmp(a)); // descending, via a comparator closure
    v.dedup(); // remove consecutive duplicates
    println!("{v:?}");
}
```

A complete example: word frequencies

This pulls the chapter together: a `HashMap` for counts, the `entry` API to update, and an iterator pipeline to find the top words.

```
use std::collections::HashMap;

fn main() {
    let text = "the quick brown fox the lazy dog the fox";

    let mut counts: HashMap<&str, i32> = HashMap::new();
    for word in text.split_whitespace() {
        *counts.entry(word).or_insert(0) += 1;
    }

    // Move the pairs into a Vec so we can sort them.
    let mut ranked: Vec<(&str, i32)> = counts.into_iter().collect();
    ranked.sort_by(|a, b| b.1.cmp(&a.1)); // by count, descending

    for (word, n) in ranked.iter().take(3) {
        println!("{word}: {n}");
    }
}
```

Key takeaways

- `Vec<T>` is a growable heap array: a `(ptr, len, cap)` header, like a C dynamic array but generic, bounds-checked, and freed automatically.
- `v[i]` panics on a bad index (no buffer overrun); `v.get(i)` returns `Option`. Use `with_capacity` to pre-allocate.
- `String` is a UTF-8 `Vec<u8>` (Chapter 10 — Slices and Strings).
- `HashMap<K, V>` needs keys that are `Eq + Hash`; `get` returns `Option`; the `entry` API updates-or-inserts; iteration order is unspecified.
- `BTreeMap`/`BTreeSet` keep keys sorted; `HashSet` stores unique values; `VecDeque` is a double-ended queue.
- An iterator is any type with `next() -> Option<Item>`. Adaptors (`map`, `filter`, `take`, ...) are **lazy**; consumers (`collect`, `sum`, `fold`, ...) drive them.

- `iter()` borrows (`&T`), `iter_mut()` borrows mutably (`&mut T`), `into_iter()` takes ownership (`T`). A `for` loop is sugar for an iterator.
- Iterators are zero-cost — they compile to the same code as a hand-written loop.

Watch out (gotchas for C programmers)

- **Indexing panics.** `v[i]` aborts cleanly on an out-of-range index instead of reading garbage. Use `.get(i)` when out-of-range is an expected case.
- **HashMap order is unspecified.** Never assume insertion or sorted order; use `BTreeMap` if you need order. Keys must implement `Eq + Hash`.
- **Iterators are lazy.** Adaptors do nothing until a consumer runs. A pipeline with no consumer does no work (and the compiler warns).
- **iter vs into_iter.** `for x in v` moves `v`; write `for x in &v` to keep it. `into_iter()` consumes the collection.
- **collect needs a target type.** Annotate the variable or use a turbofish (`collect::<Vec<_>>()`), or you get “type annotations needed.”

Interview questions

Q: What three fields make up a `Vec<T>`, and how does it relate to a C dynamic array?

A: A pointer to a heap buffer, a length (elements in use), and a capacity (elements the buffer can hold). It is exactly the hand-rolled `{ data, len, cap }` struct C programmers write, but generic, bounds-checked, and freed automatically when the `Vec` is dropped.

Q: What is the difference between `v[i]` and `v.get(i)`? A: `v[i]` returns the element directly but panics if `i` is out of bounds. `v.get(i)` returns `Option<&T>` — `Some(&value)` in range, `None` otherwise — so you handle the missing case without panicking. Neither can read past the buffer.

Q: What does it mean that iterators are “lazy,” and why does it matter? A: Adaptors like `map` and `filter` only build a description of the work; they do not touch any elements. The work runs only when a consumer such as `collect`, `sum`, or `for_each` pulls items through `next`. It matters because a pipeline without a consumer does nothing, and because laziness lets adaptors compose into a single efficient pass.

Q: When does `for x in collection` move the collection, and how do you avoid it? A: `for x in collection` calls `into_iter()`, which takes ownership and moves the collection (unless its elements are `Copy`). To iterate without moving, use `for x in &collection` (borrows, calls `iter()`) or `for x in &mut collection` (mutably borrows, calls `iter_mut()`).

Q: Why must you sometimes annotate the result of `collect`? A: `collect` is generic over the collection it produces, so the compiler needs to know the target type. You provide it with a type annotation on the binding (`let v: Vec<i32> = ...`) or a turbofish on the call (`.collect::<Vec<_>>()`).

Try it

1. Build a `Vec<i32>` of `1..=20`, then use one pipeline to keep the even numbers, square them, and `collect` into a new `Vec`. Print it.
2. Count the letters of a word with a `HashMap<char, i32>` and the `entry` API, then switch to a `BTreeMap` and notice the output is now in sorted order.

3. Replace a C-style `for (i = 0; i < n; i++)` loop you have written with `.iter().enumerate()` and see that the index variable disappears.

Chapter 17 — Smart Pointers

What you’ll learn. What a smart pointer is, and the four you will use most: `Box` (single heap owner), `Rc` (shared ownership by counting), `RefCell` (borrow rules checked at runtime), and how `Deref`/`Drop` make them feel like plain pointers that clean up after themselves.

What is a smart pointer?

A **smart pointer** is a value that behaves like a pointer but also owns the data it points to and runs cleanup code when it is dropped. In Rust, a type is a smart pointer when it implements two traits:

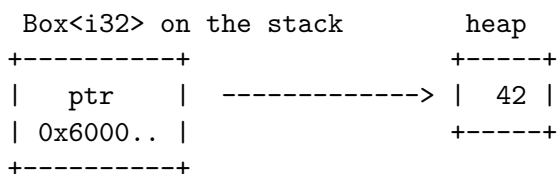
- **Deref** — so `*p` and method calls reach through to the pointed-to value, and `&p` can be used where a plain reference is expected.
- **Drop** (usually) — so the data is cleaned up automatically when the pointer goes out of scope.

You have already used smart pointers without naming them: `Vec<T>` and `String` are smart pointers — they own a heap buffer and free it on drop.

C vs Rust. In C, a pointer is “dumb”: it is just an address, with no idea who owns the memory or when to free it. A Rust smart pointer bundles the address with ownership and an automatic destructor, so there is no separate `free` call to forget.

`Box<T>`: a single owner on the heap

A `Box<T>` holds a value of type `T` on the heap and owns it. The `Box` itself is one pointer on the stack; the value lives in a heap allocation. When the `Box` is dropped, the heap value is dropped and the memory is freed — automatically.



```
fn main() {
    let b = Box::new(42); // allocate an i32 on the heap, b owns it
    println!("{b}");      // Deref lets us use b like the value
    println!("{}", *b + 1); // *b reaches the i32
} // b is dropped here; the heap memory is freed automatically
```

C vs Rust. `Box::new(x)` is like `malloc(sizeof x)` plus storing `x` into it — but you never call `free`. The `Box` frees the memory when it goes out of scope. One owner, one allocation, one automatic free. No leaks, no double-free.

There are three main reasons to reach for a `Box`.

Reason 1: put large data on the heap

If a value is large and you want to move it around cheaply, box it. Moving a `Box` copies one pointer, not the whole payload.

```
fn main() {
    let big = Box::new([0u8; 1_000_000]); // a megabyte lives on the heap
    let moved = big; // moves one pointer, not a megabyte
    println!("first byte = {}", moved[0]);
}
```

Reason 2: recursive types need a known size

This is the classic use. A struct that contains itself — a linked list node, a tree node — has no finite size, because the compiler would have to add up an infinite chain. A `Box` breaks the cycle: a `Box` is always one pointer wide, regardless of what it points to, so the size becomes known.

```
// COMPILER ERROR: recursive type `List` has infinite size
enum List {
    Cons(i32, List), // List contains a List contains a List ... no finite size
    Nil,
}
```

The fix is to box the recursive part, exactly as a C linked list stores a *pointer* to the next node, not the node itself:

```
enum List {
    Cons(i32, Box<List>), // a pointer-sized link; size is now known
    Nil,
}

use List::{Cons, Nil};

fn main() {
    let list = Cons(1, Box::new(Cons(2, Box::new(Nil))));
    if let Cons(head, _) = &list {
        println!("head = {head}");
    }
}
```

C vs Rust. A C list node is `struct node { int v; struct node *next; };` — the `next` field is a pointer because a struct cannot contain itself by value. `Box<List>` is that pointer, with automatic freeing of the whole chain on drop.

Reason 3: trait objects (`Box<dyn Trait>`)

A **trait object** lets a value be “any type that implements this trait,” chosen at runtime. Because such values can have different sizes, they must live behind a pointer. A `Box<dyn Trait>` is the owning version (traits are covered in Chapter 15 — Traits).

```
trait Shape {
    fn area(&self) -> f64;
}
```

```

}

struct Circle { r: f64 }
struct Square { s: f64 }

impl Shape for Circle { fn area(&self) -> f64 { 3.14159 * self.r * self.r } }
impl Shape for Square { fn area(&self) -> f64 { self.s * self.s } }

fn main() {
    // A Vec of different concrete types, unified behind a trait object.
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(Circle { r: 1.0 }),
        Box::new(Square { s: 2.0 }),
    ];
    let total: f64 = shapes.iter().map(|s| s.area()).sum();
    println!("total area = {total:.2}");
}

```

C vs Rust. `Box<dyn Shape>` is like a C struct holding a `void *` data pointer plus a struct of function pointers (a vtable). Rust builds and type-checks that pair for you.

Deref, DerefMut, and deref coercion

The **Deref** trait defines what `*p` produces. **DerefMut** is the mutable version. Because smart pointers implement these, you use them just like the value inside.

A handy consequence is **deref coercion**: when you pass a `&SmartPointer` where a `&Target` is expected, the compiler automatically inserts the deref for you. This is why `&String` works where `&str` is wanted, and `&Box<T>` works where `&T` is.

```

fn greet(name: &str) { // wants &str
    println!("hi {name}");
}

fn main() {
    let owned = String::from("Ada");
    greet(&owned); // &String coerces to &str automatically

    let boxed = Box::new(String::from("Linus"));
    greet(&boxed); // &Box<String> -> &String -> &str, two coercions
}

```

Mental model. Deref coercion is the compiler quietly adding the `*` and `&` you would otherwise write by hand, so a smart pointer feels transparent.

Drop: automatic cleanup (recap)

The **Drop** trait lets a type run code when it goes out of scope — Rust’s version of a C++ destructor, and the mechanism behind RAII (covered in Chapter 7 — Ownership and Moves). This is how every smart pointer frees its memory.

```

struct Guard {
    name: String,
}

impl Drop for Guard {
    fn drop(&mut self) {
        println!("dropping {}", self.name); // runs automatically at scope end
    }
}

fn main() {
    let _a = Guard { name: "first".into() };
    let _b = Guard { name: "second".into() };
    println!("end of main");
} // prints: end of main, dropping second, dropping first (reverse order)

```

C vs Rust. In C you write the cleanup (`free`, `fclose`, `unlock`) by hand at every exit path and hope you covered them all. Drop runs the cleanup automatically, on every path, in reverse order of creation.

Rc<T>: shared ownership by reference counting

Sometimes one value needs **several owners** — for example, many parts of a graph pointing at the same node. A `Box` cannot do this; it has exactly one owner. The answer is `Rc<T>` (“reference counted”).

`Rc<T>` keeps a count of how many owners exist. `Rc::clone` does not copy the data; it makes another pointer to the same allocation and **bumps the count by one**. When an `Rc` is dropped, the count drops by one. When it reaches **zero**, the data is freed. `Rc` gives **shared, immutable** access — you get `&T`, not `&mut T`.

a: Rc	b: Rc	heap allocation
+-----+	+-----+	+-----+-----+
ptr	ptr	strong = 2 "hello"
+-----+	+-----+	+-----+-----+
		^
+-----+	+-----+	+-----+-----+

```
use std::rc::Rc;
```

```

fn main() {
    let a = Rc::new(String::from("shared"));
    println!("count = {}", Rc::strong_count(&a)); // 1

    let b = Rc::clone(&a); // not a deep copy: same data, count is now 2
    let c = Rc::clone(&a); // count is now 3
    println!("count = {}", Rc::strong_count(&a)); // 3
    println!("{a} {b} {c}");

    drop(c); // count goes back to 2
}

```

```
println!("count = {}", Rc::strong_count(&a)); // 2
} // a and b dropped here; count hits 0 and the String is freed
```

C vs Rust. This is the manual reference counting C programmers write by hand: an `int refcount` you `++` on share and `--` on release, with a `free` when it hits zero. The bugs there — forgetting to increment, decrementing twice, freeing early — are exactly what `Rc` removes by doing the counting for you.

Watch out. `Rc<T>` is **not thread-safe**. It is for single-threaded sharing. For sharing across threads, use `Arc<T>` (atomic reference counted), which does the same job with atomic counter operations. See Chapter 19 — Threads and Concurrency and Chapter 20 — Channels and Shared State.

RefCell<T>: interior mutability

`Rc` gives you shared but **immutable** access. What if shared owners need to *change* the value? Normally the borrow checker forbids mutating through a shared reference. `RefCell<T>` provides **interior mutability**: it lets you mutate the value through a shared reference, and it moves the borrow checking from compile time to **runtime**.

You borrow through methods:

- `borrow()` returns a shared guard (`Ref<T>`), like `&T`.
- `borrow_mut()` returns a mutable guard (`RefMut<T>`), like `&mut T`.

`RefCell` enforces the same rule as the compiler — many readers **or** one writer, never both — but it checks at runtime and **panics** if you break it.

```
use std::cell::RefCell;

fn main() {
    let cell = RefCell::new(vec![1, 2, 3]);

    cell.borrow_mut().push(4); // mutate through a shared handle
    println!("{:?}", cell.borrow()); // [1, 2, 3, 4]

    let count = cell.borrow().len();
    println!("len = {count}");
}
```

This next example compiles fine but **panics at runtime**, because it holds a mutable borrow and a second borrow at the same time:

```
use std::cell::RefCell;

fn main() {
    let cell = RefCell::new(5);
    let a = cell.borrow_mut();
    let b = cell.borrow(); // PANIC: already mutably borrowed: BorrowMutError
    println!("{a} {b}");
}
```

Watch out. With `RefCell` you trade a compile error for a possible runtime panic. The rules are identical; only *when* they are checked changes. Keep borrow guards short-lived to avoid surprises.

`Cell<T>` is a lighter cousin for Copy values (like integers). Instead of handing out references, it lets you `get` a copy and `set` a new value, so there is no borrow to track and no panic.

```
use std::cell::Cell;

fn main() {
    let counter = Cell::new(0);
    counter.set(counter.get() + 1);
    counter.set(counter.get() + 1);
    println!("{}", counter.get()); // 2
}
```

Rule of thumb. Interior mutability is a deliberate **escape hatch**, not the default. Reach for `RefCell`/`Cell` only when you genuinely need to mutate through a shared reference and the compiler cannot prove it is safe.

`Rc<RefCell<T>>`: shared mutable data

Combine the two and you get the standard Rust recipe for **shared, mutable** data, common in trees and graphs: `Rc` provides many owners, and `RefCell` lets those owners mutate the value.

```
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    let shared = Rc::new(RefCell::new(0));

    let a = Rc::clone(&shared);
    let b = Rc::clone(&shared);

    *a.borrow_mut() += 10; // mutate through one owner
    *b.borrow_mut() += 5;  // mutate through another

    println!("value = {}", shared.borrow()); // 15
}
```

Here is a tiny tree where every node is a shared, mutable owner of its children:

```
use std::rc::Rc;
use std::cell::RefCell;

struct Node {
    value: i32,
    children: Vec<Rc<RefCell<Node>>>,
}

fn main() {
```

```

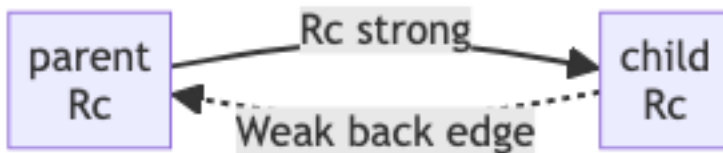
let leaf = Rc::new(RefCell::new(Node { value: 3, children: vec![] }));
let root = Rc::new(RefCell::new(Node { value: 1, children: vec![Rc::clone(&leaf)] }));

leaf.borrow_mut().value = 99; // mutate the shared leaf
println!("root's child = {}", root.borrow().children[0].borrow().value); // 99
}

```

Weak<T>: breaking reference cycles

Rc has one danger. If two Rcs point at each other — a **cycle** — their counts never reach zero, so the data is never freed. That is a memory leak, the one kind of leak safe Rust still allows.



The fix is **Weak<T>**: a non-owning reference that does **not** raise the count. A **Weak** can point at data without keeping it alive. To use it, call **upgrade**, which returns **Option<Rc<T>>** — **Some** if the data still exists, **None** if it has been freed. Use a strong **Rc** for the direction that should own (parent to child) and a **Weak** for the back-edge (child to parent).

```

use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct Node {
    value: i32,
    parent: RefCell<Weak<Node>>, // Weak: does NOT keep the parent alive
    children: RefCell<Vec<Rc<Node>>>, // Rc: owns the children
}

fn main() {
    let parent = Rc::new(Node {
        value: 1,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });

    let child = Rc::new(Node {
        value: 2,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });

    // Link both ways: parent owns child (strong), child points back (weak).
    parent.children.borrow_mut().push(Rc::clone(&child));
    *child.parent.borrow_mut() = Rc::downgrade(&parent);
}

```



```

    // upgrade() turns the Weak back into an Option<Rc>.
    if let Some(p) = child.parent.borrow().upgrade() {
        println!("child {}'s parent is {}", child.value, p.value);
    }
} // no cycle of strong counts, so both nodes are freed

```

Watch out. Rc cycles leak memory because the counts never hit zero. Break any cycle by making one direction a **Weak**. This is the doubly linked list / parent-pointer problem, and **Weak** is the idiomatic fix.

Picking the right pointer

Need	Use	C analogy
One owner on the heap	<code>Box<T></code>	<code>malloc</code> + auto <code>free</code>
Recursive type / trait object	<code>Box<T></code> / <code>Box<dyn Trait></code>	pointer in a struct / vtable
Many owners, single-threaded	<code>Rc<T></code>	manual <code>int refcount</code>
Many owners, across threads	<code>Arc<T></code>	atomic refcount
Mutate through a shared ref	<code>RefCell<T></code> (or <code>Cell<T></code>)	casting away <code>const</code>
Shared and mutable	<code>Rc<RefCell<T>></code>	refcounted, mutable struct
Break an Rc cycle	<code>Weak<T></code>	a non-owning back-pointer

Key takeaways

- A smart pointer owns data and acts like a pointer; it implements `Deref` and usually `Drop`, so it cleans up automatically.
- `Box<T>` is a single heap owner. Use it for large data, recursive types (a known-size link), and trait objects (`Box<dyn Trait>`). It is `malloc` with an automatic `free`.
- `Deref`/`DerefMut` and deref coercion make smart pointers transparent: `&String` becomes `&str`, `&Box<T>` becomes `&T`.
- `Rc<T>` is single-threaded shared ownership by reference counting; cloning bumps the count, and the data is freed when the count hits zero. `Arc<T>` is the thread-safe version (Chapters 19–20).
- `RefCell<T>` gives interior mutability by checking the borrow rules at **runtime** (and panicking on violation). `Cell<T>` is the `Copy`-value version.
- `Rc<RefCell<T>>` is the standard recipe for shared, mutable data. `Weak<T>` breaks reference cycles, which otherwise leak.

Watch out (gotchas for C programmers)

- **RefCell moves borrow checks to runtime.** A violation is a panic, not a compile error. Keep borrow guards short.
- **Rc is not thread-safe.** Use `Arc<T>` to share across threads; the compiler will stop you from sending an `Rc` to another thread.
- **Rc cycles leak.** If `Rcs` point at each other, their counts never reach zero and the memory is never freed. Use `Weak<T>` for back-edges.
- **Box is for recursive types.** A struct cannot contain itself by value; box the recursive field so the size is known, just like a C `next` pointer.

- **Interior mutability is an escape hatch.** Prefer ordinary `&mut` and let the compiler check at compile time; reach for `RefCell/Cell` only when you must.

Interview questions

Q: What makes a type a “smart pointer,” and how does it differ from a C pointer? A: A smart pointer owns its data and acts like a pointer by implementing `Deref` (and usually `Drop`). Unlike a raw C pointer, which is just an address with no ownership or cleanup, a smart pointer frees its data automatically when it goes out of scope, so there is no manual `free` to forget.

Q: Why do recursive types like a linked list need `Box`? A: A type that contains itself by value would have infinite size, so the compiler rejects it. A `Box<T>` is a single pointer of known size regardless of what it points to, so boxing the recursive field gives the type a finite, known size — exactly like storing a `next` pointer in a C struct.

Q: How does `Rc<T>` differ from `Box<T>`, and when would you use each? A: `Box<T>` has exactly one owner; `Rc<T>` allows multiple owners by counting references. Cloning an `Rc` bumps the count and shares the same data; the data is freed when the count reaches zero. Use `Box` for sole ownership and `Rc` when several parts of a single-threaded program must share read-only ownership.

Q: What is interior mutability, and what does `RefCell` trade for it? A: Interior mutability is mutating a value through a shared reference, which the compiler normally forbids. `RefCell<T>` allows it by enforcing the borrow rules at runtime instead of compile time: `borrow/borrow_mut` return guards, and the program panics if you ever have a writer alongside another borrow.

Q: How can safe Rust leak memory, and how do you prevent it? A: A cycle of `Rcs` pointing at each other keeps every reference count above zero, so the data is never freed — a leak. You prevent it by making at least one edge in the cycle a `Weak<T>`, a non-owning reference that does not raise the count; you call `upgrade()` to access the data if it still exists.

Try it

1. Define the `enum List { Cons(i32, Box<List>), Nil }` and build a three-element list. Then remove the `Box` and read the “recursive type has infinite size” error.
2. Make an `Rc<String>`, clone it twice, and print `Rc::strong_count` after each clone and after a `drop`. Watch the count rise and fall.
3. Wrap an `i32` in `RefCell`, then deliberately hold a `borrow_mut()` and a `borrow()` at the same time. Run it and read the `BorrowMutError` panic.

Chapter 18 — Memory Without a Garbage Collector

What you’ll learn. Where Rust puts your data (stack vs heap), how it frees memory with no garbage collector and no `free`, whether you can still leak memory in safe code, and how to control performance the way you do in C.

The big picture: same speed as C, no GC, no `free`

C programmers manage memory by hand. You call `malloc` to get heap memory and `free` to give it back. If you forget to free, you leak. If you free twice, you crash. If you use memory after freeing it, you get undefined behavior.

Garbage-collected languages (Go, Java, C#, Python) take the opposite approach. A **garbage collector** (GC) is a part of the runtime that periodically scans for memory you no longer use and frees it for you. You never call `free`, but you pay for it: the GC runs at unpredictable times, which causes pauses and uses extra CPU and memory.

Rust takes a third path. There is **no GC** and you almost never call `free` yourself. Instead, the compiler frees memory **deterministically**: each value is freed at a known point in the code — when its owner’s scope ends. This gives you C-level performance and predictable timing, with the safety of automatic cleanup.

Approach	Who frees memory	When	Cost
C	You, by hand (free)	Whenever you write the call	Bugs: leaks, double-free, use-after-free
Go / Java	The garbage collector	Later, at unpredictable times	GC pauses, extra CPU and memory
Rust	The compiler (ownership)	Deterministically, at end of scope	None at runtime; some thinking at compile time

Mental model. Rust’s memory management is C++ RAII enforced by the compiler. “RAII” means *Resource Acquisition Is Initialization*: a value owns a resource, and when the value is destroyed, the resource is released. Rust does this for every owning value, automatically, and checks that you got it right.

Stack vs heap (a quick recap)

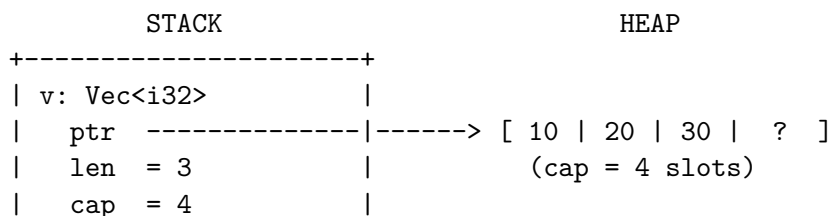
You know this from C, but the terms matter for the rest of the chapter.

- The **stack** is fast memory that grows and shrinks with function calls. A local variable with a size known at compile time lives here. Pushing and popping is just moving a pointer.
- The **heap** is a large pool of memory you ask for at runtime when you need a size that is not known up front, or data that must outlive the current function. In C you reach the heap with `malloc`.

In Rust, plain values like `i32`, `f64`, `bool`, arrays `[i32; 4]`, and structs of those live on the **stack**. Three common types put their *data* on the **heap**:

- `Box<T>` — one heap value of type `T` (like a single `malloc`’d object).
- `Vec<T>` — a growable array; the elements live on the heap.
- `String` — growable UTF-8 text; the bytes live on the heap.

Each of these is a small struct on the stack that holds a pointer to the heap.



```

+-----+
| s: String          |
|   ptr  -----> [ 'h' 'i' ]
|   len   = 2
|   cap   = 2
+-----+
| n: i32 = 7          |    (no heap; the value is right here)
+-----+

```

This is the same picture as a C struct holding a pointer, a length, and a capacity, except Rust writes it for you and frees the buffer automatically.

C vs Rust. A `Vec<T>` is a `malloc`'d array plus a length and a capacity, bundled into one type. When the `Vec` goes out of scope, Rust calls the allocator's `free` for you. In C you track the pointer, length, and capacity in separate variables and free the buffer by hand.

Moves: transferring ownership, not copying data

When you assign one owning value to another, Rust **moves** it. A move copies the small stack part (the pointer, length, capacity) and marks the old variable as invalid. It does **not** copy the heap data. This is a cheap, shallow operation — the same cost as copying a C pointer.

```

fn main() {
    let a = String::from("hello"); // a owns a heap buffer
    let b = a;                      // MOVE: b now owns it; a is invalid
    println!("{b}");               // ok
    // println!("{a}");             // would be a compile error: a was moved
}

```

Compare with C, where copying a pointer makes two pointers to **one** buffer — the setup for a double-free:

```

char *a = strdup("hello");
char *b = a;    /* two pointers, one buffer */
free(a);
free(b);        /* double free: undefined behavior */

```

Rust avoids this by making `a` unusable after the move, so the buffer has exactly one owner and is freed exactly once. (Moves and ownership are Chapter 7 — Ownership and Moves.)

How freeing happens: Drop and scope

When an owning value goes out of scope, Rust runs its destructor — a method from the `Drop` trait — which frees any heap memory it owns. You do not call it; the compiler inserts the call. You can also write your own `Drop` to release other resources (files, sockets, locks), just like a C++ destructor.

```

struct Logger {
    name: String,
}

```

```

impl Drop for Logger {
    fn drop(&mut self) {
        println!("dropping {}", self.name);
    }
}

fn main() {
    let _first = Logger { name: String::from("first") };
    let _second = Logger { name: String::from("second") };
    println!("end of main");
}

```

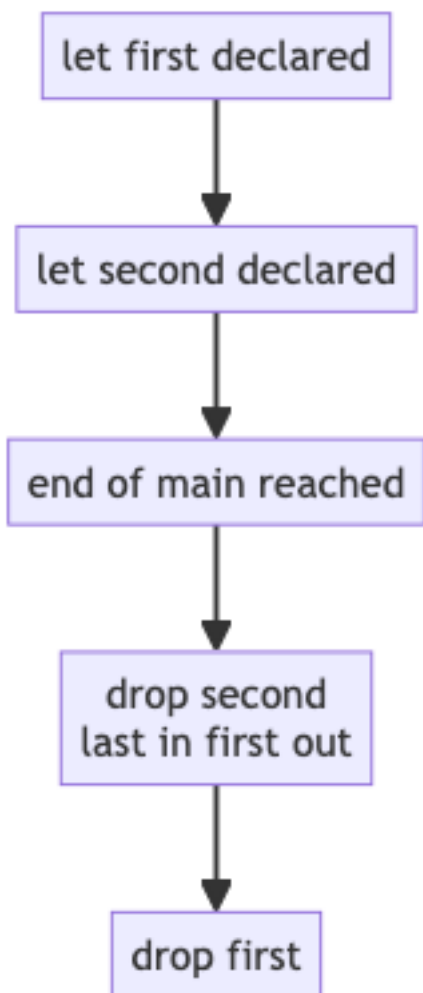
This prints:

```

end of main
dropping second
dropping first

```

Notice the order. Values are dropped in **reverse order of declaration**, like popping a stack: the last one declared is freed first. This matters when one value depends on another being alive.



C vs Rust. In C you decide where every `free` goes and run it yourself. In Rust the `free` is tied to scope and runs automatically in reverse declaration order. You get RAII cleanup without writing the cleanup code.

The allocator: who actually calls `malloc`

Under the hood, `Box`, `Vec`, and `String` ask the **global allocator** for heap memory and return it when dropped. The default global allocator calls the system allocator (the same `malloc/free` family C uses). You rarely touch it directly.

A few facts worth knowing:

- You can replace the global allocator with `#[global_allocator]` (for example, to use `jemalloc` or a custom arena). This is advanced and uncommon.
- Manual, raw allocation exists through the `std::alloc` module, but it is `unsafe` and you almost never need it. (See Chapter 25 — Unsafe and FFI.)
- Growing a `Vec` past its capacity reallocates: it asks for a bigger buffer, copies the elements over, and frees the old one — exactly like writing `realloc` in C.

Can you leak memory in safe Rust?

Yes. This surprises newcomers. Rust prevents use-after-free, double-free, and data races in safe code, but it does **not** promise to prevent **leaks**. A leak means memory is never freed. Leaking is considered **safe** — it cannot corrupt memory or cause undefined behavior — but it is still a bug if you did not mean to do it.

Watch out. “Memory safe” does not mean “leak free.” Safe Rust guarantees you never touch freed or invalid memory. It does not guarantee every allocation is eventually freed.

There are three common ways to leak in safe code:

1. **Reference-count cycles with Rc/Arc.** An `Rc<T>` frees its value when the count reaches zero. If two `Rc` values point at each other, the count never reaches zero, so neither is freed. (See Chapter 17 — Smart Pointers for `Rc` and the **Weak** fix.)
2. **`std::mem::forget`.** This takes a value and *skips* its destructor on purpose. The memory is never freed.

```
fn main() {  
    let data = vec![1, 2, 3];  
    std::mem::forget(data); // destructor never runs; the heap buffer leaks  
}
```

3. **`Box::leak`.** This turns an owned `Box<T>` into a plain reference that lives for the rest of the program (`&'static mut T`). It is sometimes done on purpose to create data that must never be freed.

```
fn main() {  
    let boxed: Box<i32> = Box::new(42);  
    let r: &'static mut i32 = Box::leak(boxed); // never freed; lives forever  
    println!("{}", r);  
}
```

These functions are safe because a leak harms only your memory budget, not memory correctness. The borrow checker is about *safety*, not *thrift*.

Performance: how Rust stays in C’s class

Rust’s runtime cost is the same class as C. Here is why, and what you control.

Why it is fast by default:

- **No runtime, no GC.** There is no background collector and no managed heap. Memory frees happen at compile-time-known points.
- **Zero-cost abstractions.** Iterators, generics, and traits compile to the same machine code you would write by hand. You do not pay extra at runtime for using them.
- **Monomorphization.** A generic function is compiled into a specialized copy for each concrete type it is used with — like the C compiler expanding a macro — so there is no dynamic dispatch cost. (See Chapter 14 — Generics.)
- **Bounds checks, often free.** Indexing a slice checks the index against the length, which C does not do. The optimizer removes many of these checks when it can prove the index is in

range (for example, in a `for` loop over the slice).

What you control (just like in C):

- **Preallocate with `with_capacity`.** If you know how many items are coming, allocate once instead of growing repeatedly. Each growth is a `realloc` plus a copy.

```
fn main() {  
    let mut v = Vec::with_capacity(1000); // one allocation up front  
    for i in 0..1000 {  
        v.push(i); // no reallocation: capacity is already 1000  
    }  
    println!("len = {}", v.len());  
}
```

- **Pass references, do not clone.** Passing `&T` or `&mut T` hands over a pointer with no copy, the same as passing a pointer in C. Calling `.clone()` on a `Vec` or `String` does a **deep copy** — a fresh allocation and a byte-for-byte copy. Clone when you need a separate owned copy; borrow when you only need to read or modify in place.
- **Prefer cheap Copy types.** Small types like `i32`, `f64`, and `char` implement `Copy`: assigning them duplicates the bits, like copying an `int` in C. There is no heap and no move bookkeeping. Use them freely.
- **Choose stack vs heap on purpose.** A fixed-size array `[u8; 64]` lives on the stack with no allocation. `Box`/`Vec`/`String` allocate. For small, fixed data, stay on the stack.

Measuring it:

- The same tools you use in C still work: `valgrind`, `heaptrack`, and `perf` all understand Rust binaries because the output is native machine code.
- **Debug and release builds differ a lot.** `cargo build (debug)` keeps bounds checks for integer overflow, does little optimization, and is much slower. `cargo build --release` turns on optimizations. Always benchmark the release build.

Rule of thumb. When something is slow, look first for accidental `.clone()` calls and `Vectors` that grow in a loop without `with_capacity`. These are the Rust equivalents of needless `memcpy` and missing `realloc` sizing in C.

Key takeaways

- Rust has **no garbage collector**. Memory is freed deterministically when its owner’s scope ends, via `Drop` (RAII), giving predictable performance with no GC pauses — ideal for systems and real-time work.
- Sized locals live on the **stack**; `Box`, `Vec`, and `String` keep their data on the **heap**. A **move** transfers ownership of the heap data with no deep copy.
- The **global allocator** backs `Box`/`Vec`/`String`. You rarely call it directly. Values drop in **reverse order of declaration**.
- You **can** leak in safe Rust (`Rc` cycles, `std::mem::forget`, `Box::leak`). Leaks are **safe**, not undefined behavior, but still bugs. “Safe” does not mean “leak free.”
- Performance is C’s class: no runtime, zero-cost abstractions, monomorphization, bounds checks usually optimized away. Control it with `with_capacity`, passing references instead of

cloning, cheap `Copy` types, and stack-vs-heap choices.

- `valgrind/heaptrack/perf` still work; always measure the `--release` build.
- Manual allocation and raw memory exist behind `unsafe` (Chapter 25 — Unsafe and FFI), but you almost never need them.

Watch out (gotchas for C programmers)

- **Leaks are possible but safe.** Safe Rust stops use-after-free and double-free, not leaks. `Rc` cycles and `mem::forget` will leak quietly.
- **Drop order is reverse of declaration.** The last value declared is freed first. This matters when one value must outlive another.
- **`.clone()` has a real cost.** On `Vec/String` it is a deep copy with a fresh allocation. Borrow with `&/&mut` when you do not need a separate copy.
- **Use `with_capacity` for hot loops.** A `Vec` that grows by `push` may reallocate and copy many times. Preallocate when you know the size.
- **No GC means deterministic frees.** Cleanup happens at end of scope, not “sometime later.” Do not expect a background collector to save you from a cycle.
- **Always benchmark `--release`.** Debug builds are far slower and behave differently (for example, integer overflow panics in debug).

Interview questions

Q: Rust has no garbage collector. How is heap memory freed? A: By ownership and the `Drop` trait (RAII). Each heap value has a single owner. When the owner goes out of scope, the compiler inserts a call to its destructor, which frees the memory. This happens at compile-time-known points, so timing is deterministic with no GC pauses.

Q: Can safe Rust leak memory? A: Yes. Reference-count cycles with `Rc/Arc`, `std::mem::forget`, and `Box::leak` all leak in safe code. Leaks are considered safe because they cannot cause undefined behavior — but they are still bugs. Memory safety does not mean leak freedom.

Q: What is the difference between a move and a clone? A: A move transfers ownership by copying only the small stack part (pointer, length, capacity) and invalidating the source; the heap data is not copied, so it is cheap. A clone makes a full independent copy, which for `Vec/String` means a new allocation and a deep copy of the data.

Q: How do you avoid repeated reallocations when filling a `Vec`? A: Use `Vec::with_capacity(n)` to allocate enough room once before the loop. Otherwise the `Vec` grows on demand, and each growth reallocates a larger buffer and copies the existing elements, like calling `realloc` repeatedly in C.

Q: Why might the same Rust program run much faster after one change to the build command? A: Because of debug vs release builds. `cargo build` is a debug build with few optimizations and extra checks; `cargo build --release` enables optimizations. Always benchmark the release build, since they can differ by an order of magnitude.

Try it

1. Write a struct with a `Drop` impl that prints its name, declare three of them in `main`, and confirm they drop in reverse order.

2. Fill a `Vec` with one million integers two ways: with `Vec::new()` and with `Vec::with_capacity(1_000_000)`. Time both with `cargo run --release` and compare.
3. Call `std::mem::forget` on a `Vec` and run the program under `valgrind`. Watch it report the leaked bytes — proof that a safe program can still leak.

Chapter 19 — Threads and Fearless Concurrency

What you'll learn. How to start OS threads in Rust, why the same ownership rules that prevent memory bugs also prevent data races at compile time, the `Send/Sync` traits, and how to share data safely with `Arc<Mutex<T>>` and scoped threads — all compared to C and pthreads.

Starting a thread

In C you start a thread with `pthread_create`, passing a function pointer and a `void *` argument, and you wait for it with `pthread_join`. You manage the argument's lifetime by hand, and a mistake there is silent undefined behavior.

In Rust you call `std::thread::spawn`, passing a **closure** (an anonymous function that can capture variables). It returns a `JoinHandle`. Calling `.join()` on that handle waits for the thread to finish and gives you its return value inside a `Result`.

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        // this runs on a new OS thread
        let mut sum = 0;
        for i in 1..=100 {
            sum += i;
        }
        sum // the closure's return value becomes the thread's result
    });

    // do other work here on the main thread...

    let result = handle.join().unwrap(); // wait, then take the result
    println!("sum = {result}");
}
```

These are **OS threads**, one Rust thread per one operating-system thread (a “1:1” model), exactly like pthreads. They are real, preemptively scheduled kernel threads, not lightweight green threads.

C vs Rust. Go's goroutines are lightweight tasks managed by Go's runtime; thousands are cheap. Rust's `thread::spawn` gives you a full OS thread, the same heavyweight thing as `pthread_create`. For massive concurrency Rust uses `async` instead (Chapter 21 — Async/Await). This chapter is about real threads.

Watch out. If you drop a `JoinHandle` without calling `.join()`, the thread is **detached** — it keeps running on its own, and the program may exit before it finishes. Always

`.join()` threads whose work you care about.

move closures: giving data to a thread

A spawned thread may outlive the function that created it. So the closure cannot just borrow local variables — the borrow might dangle. Rust requires the closure to **own** what it uses, by writing `move` before it. `move` transfers ownership of the captured variables into the closure (and thus into the thread).

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3];

    let handle = thread::spawn(move || {
        // `data` is moved into this thread; main can no longer use it
        let total: i32 = data.iter().sum();
        println!("thread computed {total}");
    });

    handle.join().unwrap();
}
```

Without `move`, this would be a compile error: the closure would borrow `data`, but the compiler cannot prove `data` lives long enough, because the thread might run after `main`'s variables are gone.

C vs Rust. In C you pass a `void *` to `pthread_create` and promise that the data stays alive long enough. The compiler does not check this. In Rust, `move` transfers ownership into the thread, and the compiler proves nothing dangles.

Fearless concurrency: data races caught at compile time

A **data race** happens when two threads access the same memory at the same time, at least one of them writes, and there is no synchronization. In C this is silent undefined behavior — it may work for years and then corrupt data under load.

Rust calls its approach **fearless concurrency**: the *same* ownership and borrowing rules you already learned (Chapter 8 — Borrowing and References) prevent data races at compile time. The key rule is “aliasing XOR mutation”: you may have many shared readers (`&T`) **or** one exclusive writer (`&mut T`), never both. That single rule, applied across threads, makes data races impossible in safe code.

So you cannot simply hand a `&mut` to two threads, and you cannot share a value that is not safe to share. The compiler enforces this using two marker traits.

Send and Sync: the thread-safety markers

A **marker trait** is a trait with no methods; it just labels a type with a property. Two of them govern threading:

- **Send** — a type is **Send** if it is safe to **move** to another thread. Almost everything is **Send**. The famous exception is `Rc<T>` (the single-threaded reference-counted pointer, Chapter 17 — Smart Pointers).
- **Sync** — a type is **Sync** if it is safe to **share** by reference (`&T`) across threads. `T` is **Sync** exactly when `&T` is **Send**.

These are **auto-implemented**: the compiler figures out whether your type is **Send**/**Sync** from its fields. You normally never write them yourself.

What is **not** **Send**?

- `Rc<T>` — its reference count is updated without atomics, so two threads could corrupt the count. Use `Arc<T>` (atomic `Rc`) instead.
- Raw pointers (`*const T`, `*mut T`) — the compiler cannot reason about them.

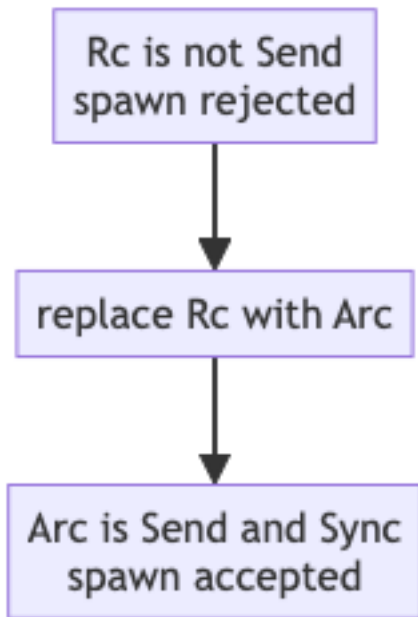
Here is the compiler stopping you from moving an `Rc` into a thread:

```
// COMPILER ERROR: `Rc<i32>` cannot be sent between threads safely
use std::rc::Rc;
use std::thread;

fn main() {
    let counter = Rc::new(5);
    let c = Rc::clone(&counter);

    thread::spawn(move || {
        println!("{}", c); // error[E0277]: `Rc<i32>` cannot be sent between threads safely
    })
    .join()
    .unwrap();
}
```

The fix is `Arc`, shown below. In C there is no such check; nothing stops you from sharing a non-thread-safe object, and the bug surfaces only at runtime.



Sharing ownership: Arc

When several threads need to read the **same** heap value, you give each one shared ownership with `Arc<T>` — an **A**tomically **R**eference **C**ounted pointer. It is like `Rc<T>` but uses atomic operations for the count, so it is safe across threads (`Arc` is `Send` and `Sync` when `T` is). Cloning an `Arc` bumps the count and hands out another handle to the same data; the value is freed when the last handle drops.

`Arc` alone gives **shared, read-only** access. To **mutate** shared data, you also need a lock.

Shared mutation: `Mutex` and `RwLock`

You cannot hand a `&mut` to several threads — that breaks the borrow rule. To allow shared mutation, wrap the data in a `Mutex<T>` (mutual exclusion lock). Calling `.lock()` blocks until the lock is free, then returns a **guard**. The guard lets you read and write the inner value, and it **unlocks automatically when it drops** (RAII). You never forget to unlock.

The standard pattern for “shared and mutable across threads” is `Arc<Mutex<T>>`: `Arc` for shared ownership, `Mutex` for safe mutation.

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0)); // shared, lockable integer
    let mut handles = Vec::new();

    for _ in 0..10 {
        let counter = Arc::clone(&counter); // a new handle for this thread
        let handle = thread::spawn(move || {
```

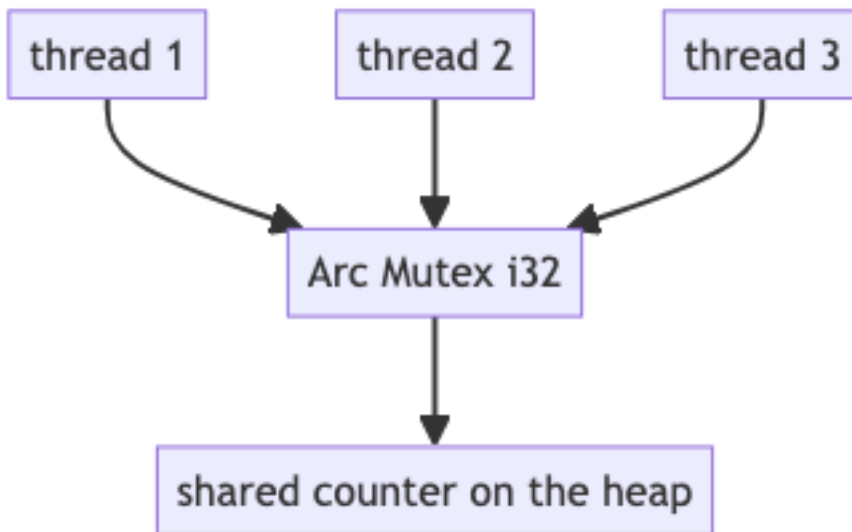
```

        let mut guard = counter.lock().unwrap(); // lock; unlocks on drop
        *guard += 1;
    });
    handles.push(handle);
}

for handle in handles {
    handle.join().unwrap(); // wait for all threads
}

println!("counter = {}", *counter.lock().unwrap()); // prints 10
}

```



Here is the same idea in C with pthreads. Notice the manual lock/unlock pair you must never forget, and that nothing checks you actually took the lock:

```

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *worker(void *arg) {
    pthread_mutex_lock(&lock);
    counter += 1;
    pthread_mutex_unlock(&lock); /* forget this and you deadlock or leak */
    return NULL;
}

```

C vs Rust. In C the lock and the data are separate; the compiler never checks that you hold the lock before touching `counter`. In Rust the data lives *inside* the `Mutex`, so the only way to reach it is through `.lock()`, and the guard unlocks on drop. You cannot touch the data without locking, and you cannot forget to unlock.

`RwLock<T>` is a variant that allows **many readers or one writer** at a time (`.read()` and `.write()`), useful when reads vastly outnumber writes — like a pthread read-write lock.

Scoped threads: borrowing local data

`thread::spawn` requires the closure to be `'static`, meaning it may not borrow local variables — that is why we used `move` and `Arc` above. But often you just want a few threads to read local data and finish before the function returns. For that, use `std::thread::scope`. Threads spawned inside the scope are guaranteed to finish before the scope ends, so they may safely **borrow** locals with no `Arc` and no `'static` requirement.

```
use std::thread;

fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6];

    thread::scope(|s| {
        s.spawn(|| {
            let sum: i32 = numbers.iter().sum(); // borrows `numbers`, no Arc
            println!("sum = {sum}");
        });
        s.spawn(|| {
            let max = numbers.iter().max().unwrap();
            println!("max = {max}");
        });
    }); // all scoped threads are joined here, automatically

    println!("still own numbers: {} items", numbers.len());
}
```

The scope joins every spawned thread at its closing brace, so the borrows can never outlive `numbers`. This is the modern, ergonomic way to fan out work over borrowed data.

Rule of thumb. If the threads finish within the current function, use `thread::scope` and `borrow`. If a thread must outlive the function or own the data, use `thread::spawn` with `move` and `Arc`.

What Rust does *not* prevent

Fearless concurrency stops data races, but not every concurrency bug:

- **Deadlocks are still possible.** If thread A holds lock 1 and waits for lock 2 while thread B holds lock 2 and waits for lock 1, both block forever. Rust does not detect this. Always acquire multiple locks in a consistent order.
- **Mutex poisoning.** If a thread panics while holding a `Mutex`, the lock becomes **poisoned**. Later `.lock()` calls return an `Err` to warn that the protected data may be in a bad state. That is why `.lock()` returns a `Result` and we call `.unwrap()` on it in the examples. Real code decides whether to recover or propagate the error.

Watch out. `.lock()` returning a `Result` is not about the lock failing to acquire — it is about poisoning. A poisoned lock means some other thread panicked mid-update.

Key takeaways

- `thread::spawn(closure)` starts an **OS thread** (1:1) and returns a `JoinHandle`; `.join()` waits and returns the thread's result. Use a `move` closure to transfer owned data into the thread.
- The **same ownership and borrowing rules** that prevent memory bugs prevent **data races** at compile time. This is “fearless concurrency.”
- **Send** (safe to move to another thread) and **Sync** (safe to share `&T` across threads) are auto-implemented marker traits. `Rc` and raw pointers are **not Send**; the compiler rejects sharing them across threads.
- For shared **read** access use `Arc<T>`; for shared **mutation** use `Arc<Mutex<T>>`. `.lock()` returns a guard that unlocks on drop (RAII). `RwLock<T>` allows many readers or one writer.
- `thread::scope` lets threads **borrow** local data without `Arc` or `'static`, joining them automatically at the end of the scope.
- Rust does **not** prevent deadlocks, and a panic while locked **poisons** the `Mutex`, which is why `.lock()` returns a `Result`.

Watch out (gotchas for C programmers)

- Use **move closures** to give data to `thread::spawn`; otherwise the borrow may dangle and the code will not compile.
- **Rc is not Send**. Use `Arc` to share ownership across threads. The compiler will stop you if you try `Rc`.
- **Shared mutation needs a Mutex** (or `RwLock`). You cannot give `&mut` to two threads. Wrap the data: `Arc<Mutex<T>>`.
- **Deadlocks are not prevented**. Lock ordering is still your responsibility, the same as with `pthread`s.
- **Remember to .join()**. A dropped `JoinHandle` detaches the thread, and the program may exit before it finishes.
- **Mutex poisoning** makes `.lock()` return `Result`. A poisoned lock means another thread panicked while holding it.
- **Prefer thread::scope** when threads only need to borrow locals and finish before the function returns — it avoids `Arc` entirely.

Interview questions

Q: What does “fearless concurrency” mean in Rust? A: It means the compiler uses the ordinary ownership and borrowing rules to guarantee there are no data races in safe code. Because you may have many shared readers or one exclusive writer but never both, two threads cannot mutate the same data without synchronization. Data races become compile-time errors instead of silent runtime undefined behavior.

Q: What are Send and Sync? A: They are auto-implemented marker traits. **Send** means a value is safe to move to another thread; **Sync** means `&T` is safe to share across threads (equivalently, `&T` is **Send**). The compiler derives them from a type's fields. `Rc` and raw pointers are not **Send**, so they cannot cross thread boundaries.

Q: Why use Arc<Mutex<T>>, and what does each part do? A: `Arc` gives several threads shared ownership of one heap value using an atomic reference count. `Mutex` allows safe shared

mutation: `.lock()` returns a guard that gives exclusive access and unlocks automatically when dropped. `Arc` handles sharing; `Mutex` handles mutation.

Q: How does Rust’s mutex differ from a pthread mutex? A: In Rust the data lives inside the `Mutex`, so you can only reach it by locking, and the returned guard unlocks automatically on drop (RAII). In C the mutex and the data are separate; nothing checks that you locked before access or that you unlocked afterward. Rust also poisons the mutex if a holder panics, surfaced as an `Err` from `.lock()`.

Q: Does Rust prevent deadlocks? A: No. Rust prevents data races, but deadlocks from inconsistent lock ordering are still possible and not detected at compile time. You must still acquire locks in a consistent order, just as in C.

Try it

1. Change the counter example to use `RwLock` and have some threads `.read()` the value while others `.write()` to it. Confirm the final count is still correct.
2. Replace the `Arc<Mutex<T>>` counter with a `thread::scope` version that borrows a local `Mutex` directly, with no `Arc`.
3. Try moving an `Rc` into `thread::spawn` and read the full compiler error. Notice how it mentions `Send`, then switch to `Arc` to fix it.

Chapter 20 — Channels and Shared State

What you’ll learn. Two ways for threads to work together: sending messages through *channels*, and sharing memory behind a lock or an atomic. You will see when to pick each, and how Rust makes both safe by default.

In Chapter 19 — Threads and Concurrency you learned how to start threads and why the compiler will not let two threads touch the same mutable data without protection. This chapter is about the two safe ways to make threads cooperate:

- **Message passing.** One thread *sends* a value to another through a channel. Ownership moves with the message. This is the focus of the first half.
- **Shared state.** Threads share one piece of memory, guarded by a lock (`Mutex`, `RwLock`) or an atomic. This is the second half.

Mental model. A channel is a one-way pipe between threads, like a Unix pipe or a POSIX message queue — but type-safe, and it moves ownership of whatever you put in. Shared state is the classic C model: one buffer, one `pthread_mutex_t`, everybody locks before touching it.

Channels: `std::sync::mpsc`

The standard library channel lives in `std::sync::mpsc`. The name `mpsc` means **multi-producer, single-consumer**: many threads may send, but only one thread receives.

You create a channel with `channel()`. It returns a pair: a **sender** (`tx`, for “transmit”) and a **receiver** (`rx`, for “receive”).

```
use std::sync::mpsc;
use std::thread;
```

```
fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        tx.send(42).unwrap(); // send moves 42 into the channel
    });

    let value = rx.recv().unwrap(); // block until a value arrives
    println!("got {value}");
}
```

Three things to notice:

- `tx.send(v)` **moves** `v` into the channel. After you send a value you no longer own it; the receiving thread does. For a non-Copy type like `String`, the sender cannot use the value again. This is the borrow checker enforcing the same single-owner rule you already know.
- `rx.recv()` **blocks** the calling thread until a value is ready (or the channel closes). It returns a `Result`, so we `unwrap()` here; real code would handle the error case.
- The `move` keyword on the closure moves `tx` into the new thread, so the thread owns its sender. (See Chapter 19 — Threads and Concurrency for `move` closures.)

C vs Rust. In C you would build this yourself: a buffer, a `pthread_mutex_t`, and a `pthread_cond_t` to wake the reader. You must remember to lock, signal, and unlock in the right order. The Rust channel hands you all of that, correct, in two lines.

Ownership moves through the channel

Because `send` moves the value, this does **not** compile:

```
// COMPILER ERROR: borrow of moved value: `msg`
use std::sync::mpsc;

fn main() {
    let (tx, _rx) = mpsc::channel();
    let msg = String::from("hello");
    tx.send(msg).unwrap(); // msg moves into the channel
    println!("{msg}");      // error[E0382]: msg was moved
}
```

The receiver now owns the `String`. The sender giving it away means there is never a moment when two threads both own it — so there is no data race to worry about.

Multiple producers: clone the sender

To have several threads send, **clone** the sender. Each clone is another handle to the same channel. The single receiver collects everything.

```
use std::sync::mpsc;
use std::thread;
```

```
fn main() {
    let (tx, rx) = mpsc::channel();

    for id in 0..3 {
        let tx = tx.clone();           // each thread gets its own sender
        thread::spawn(move || {
            tx.send(id).unwrap();
        });
    }
    drop(tx); // drop the original sender so the channel can close later

    for value in rx {                  // iterate until the channel closes
        println!("received {value}");
    }
}
```

The order of the three values is not fixed; the threads race. The `for value in rx` loop receives values until the channel closes (explained below).

Receiving: `recv`, `try_recv`, and iterating

There are three ways to take values out:

- `rx.recv()` — block until a value arrives. Returns `Err` if the channel closed.
- `rx.try_recv()` — return immediately. `Ok(v)` if a value was ready, `Err` otherwise. Use this in a poll loop when you must not block.
- `for v in rx` (or `rx.iter()`) — loop, blocking for each value, and stop when the channel closes. This is the cleanest pattern for a consumer.

```
use std::sync::mpsc;

fn main() {
    let (tx, rx) = mpsc::channel();
    tx.send(1).unwrap();

    match rx.try_recv() {
        Ok(v) => println!("ready: {v}"),
        Err(_) => println!("nothing yet"),
    }
}
```

The channel closes when all senders drop

A channel stays open as long as at least one sender exists. When the **last** sender is dropped, the channel **closes**. After that, `recv()` returns an `Err` (`RecvError`), and a `for v in rx` loop ends. This is how the consumer knows “no more messages will ever come.”

```
use std::sync::mpsc;

fn main() {
```

```

let (tx, rx) = mpsc::channel::<i32>();
drop(tx); // no senders remain

match rx.recv() {
    Ok(v) => println!("got {v}"),
    Err(e) => println!("channel closed: {e}"), // this branch runs
}
}

```

Watch out. If you clone the sender into threads but also keep the original `tx` alive in `main`, the channel never closes and a `for v in rx` loop blocks forever. Drop the extra sender (or let it go out of scope) so the channel can close. This is the most common channel bug for newcomers.

Bounded channels: `sync_channel`

`channel()` is **unbounded**: `send` never blocks, and the queue can grow without limit. If a fast producer outruns a slow consumer, memory grows.

`sync_channel(n)` makes a **bounded** channel with room for `n` buffered values. When the buffer is full, `send` **blocks** until the consumer takes one out. This is *backpressure*: the producer is forced to slow down.

```

use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::sync_channel(2); // buffer holds 2 values

    let producer = thread::spawn(move || {
        for i in 0..5 {
            tx.send(i).unwrap(); // blocks when the buffer is full
        }
    });

    for value in rx {
        println!("got {value}");
    }
    producer.join().unwrap();
}

```

A `sync_channel(0)` is a *rendezvous* channel: every `send` waits for a matching `recv`, so the two threads meet exactly.

A producer/consumer example

Here is a complete worker pool: one producer sends 100 jobs, four workers square each number, and the main thread sums the results. It joins every thread and the total is deterministic (the sum of squares of 1 to 100 is 338350).

```

use std::sync::mpsc;
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let (job_tx, job_rx) = mpsc::channel::<u64>();
    let (result_tx, result_rx) = mpsc::channel::<u64>();

    // std mpsc has ONE receiver. To share it among workers, wrap it
    // in Arc<Mutex<...>> so each worker can lock and take one job.
    let job_rx = Arc::new(Mutex::new(job_rx));

    let mut workers = Vec::new();
    for _ in 0..4 {
        let job_rx = Arc::clone(&job_rx);
        let result_tx = result_tx.clone();
        workers.push(thread::spawn(move || loop {
            // Lock, take one job, then unlock before doing work.
            let job = job_rx.lock().unwrap().recv();
            match job {
                Ok(n) => result_tx.send(n * n).unwrap(),
                Err(_) => break, // job channel closed: stop
            }
        }));
    }
    drop(result_tx); // workers hold the only remaining result senders

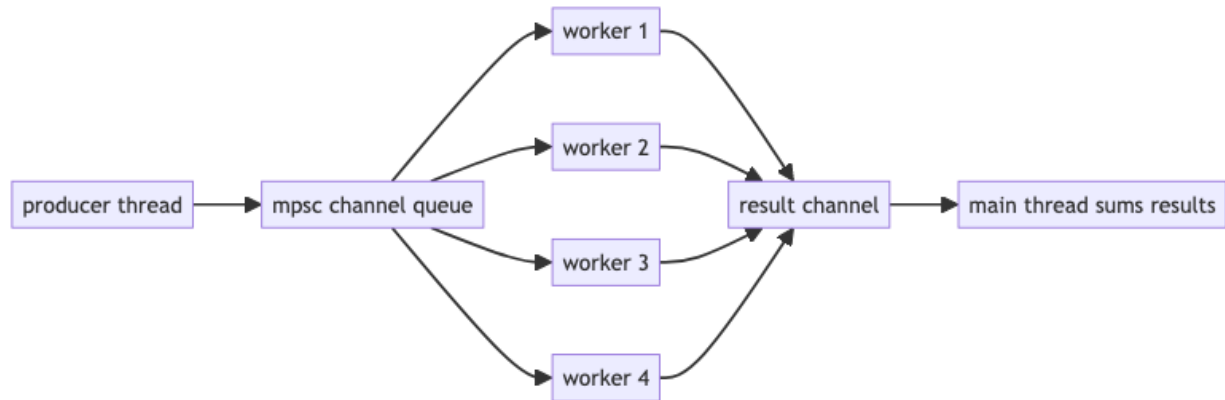
    for n in 1..=100 {
        job_tx.send(n).unwrap();
    }
    drop(job_tx); // close the job channel so workers stop

    let total: u64 = result_rx.iter().sum(); // ends when all workers exit

    for w in workers {
        w.join().unwrap();
    }
    println!("sum of squares 1..=100 = {total}"); // 338350
}

```

Deep dive. Wrapping the receiver in `Arc<Mutex<...>>` works, but a worker holds the lock while it calls `recv()`. If `recv()` had to wait, the other workers would wait too. For a real multi-consumer queue, reach for a crate (see below) that supports many receivers directly.



The C version: a hand-built queue

To do the same in C you write the thread-safe queue yourself: a buffer, a mutex, and two condition variables (one for “not empty”, one for “not full”).

```

/* A bounded thread-safe queue: mutex plus two condition variables. */
#include <pthread.h>
#define CAP 64

typedef struct {
    int buf[CAP];
    int head, tail, count;
    pthread_mutex_t lock;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
} queue_t;

void q_push(queue_t *q, int v) {
    pthread_mutex_lock(&q->lock);
    while (q->count == CAP)
        pthread_cond_wait(&q->not_full, &q->lock);
    q->buf[q->tail] = v;
    q->tail = (q->tail + 1) % CAP;
    q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
}

int q_pop(queue_t *q) {
    pthread_mutex_lock(&q->lock);
    while (q->count == 0)
        pthread_cond_wait(&q->not_empty, &q->lock);
    int v = q->buf[q->head];
    q->head = (q->head + 1) % CAP;
    q->count--;
    pthread_cond_signal(&q->not_full);
}
  
```

```
pthread_mutex_unlock(&q->lock);
return v;
}
```

That whole file is one `mpsc::sync_channel(64)` in Rust — and the Rust version cannot deadlock from a forgotten unlock or signal the wrong condition variable.

Channels vs shared state

There is a famous slogan, from Go but true in Rust too:

Rule of thumb. “Do not communicate by sharing memory; share memory by communicating.” When in doubt, send a message and move ownership. It is easier to reason about than locks.

Channels shine when you want to **transfer ownership** of work or results between threads, or to **coordinate** stages of a pipeline. There is no lock to hold and no shared mutable state to reason about.

But sometimes shared state is simpler. If many threads need to read or update **one counter** or **one configuration value**, passing messages around is awkward. A `Mutex` or an atomic is the direct tool. Use shared state when:

- the data is small and naturally lives in one place (a counter, a cache, a map),
- many threads read the same thing, or
- the cost of copying or moving the data through a channel is too high.

Shared state: `Arc<Mutex<T>>`

You met `Arc<Mutex<T>>` in Chapter 19 — Threads and Concurrency. Here is the recap.

- `Arc<T>` is an **atomically reference-counted** pointer. It lets many threads share ownership of the same value; the value is dropped when the last `Arc` goes away. (It is the thread-safe version of `Rc` from Chapter 17 — Smart Pointers.)
- `Mutex<T>` guards the value. `lock()` returns a *guard*; while you hold the guard you have exclusive access, and the lock is released automatically when the guard drops. No manual unlock, ever.

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = Vec::new();

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut n = counter.lock().unwrap(); // lock; guard frees it
            *n += 1;
        }));
    }
}
```

```

    for h in handles {
        h.join().unwrap();
    }
    println!("total = {}", *counter.lock().unwrap()); // always 10
}

```

C vs Rust. In C the data and its mutex are two separate variables; nothing stops you from touching the data without locking. In Rust the data lives *inside* the `Mutex`, so you literally cannot reach it without locking first. The type system enforces the discipline you have to remember by hand in C.

RwLock: many readers or one writer

A `Mutex` lets exactly one thread in at a time, even for reading. An **RwLock** (reader-writer lock) is smarter: it allows **many readers at once**, or **one writer alone**. Use it when reads are common and writes are rare.

```

use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));

    let mut handles = Vec::new();
    for _ in 0..3 {
        let data = Arc::clone(&data);
        handles.push(thread::spawn(move || {
            let guard = data.read().unwrap(); // many readers allowed
            println!("sum = {}", guard.iter().sum::<i32>());
        }));
    }

    {
        let mut guard = data.write().unwrap(); // one writer, exclusive
        guard.push(4);
    }

    for h in handles {
        h.join().unwrap();
    }
}

```

This is exactly C's `pthread_rwlock_t`, but the read guard and write guard release the lock automatically when they drop.

Atomics: lock-free counters

For a single integer, a full `Mutex` is overkill. An **atomic** type performs reads and writes that cannot be interrupted, with no lock at all. `AtomicUsize`, `AtomicU64`, `AtomicBool`, and friends live


```
in std::sync::atomic.
```

Every atomic operation takes an **Ordering**, which tells the compiler and CPU how strictly this operation must be ordered against others. This is the same idea as C11's `<stdatomic.h>` memory orders (`memory_order_relaxed`, `memory_order_seq_cst`, and so on).

```
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

fn main() {
    let counter = Arc::new(AtomicUsize::new(0));
    let mut handles = Vec::new();

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            counter.fetch_add(1, Ordering::Relaxed); // atomic increment
        }));
    }

    for h in handles {
        h.join().unwrap();
    }
    println!("total = {}", counter.load(Ordering::Relaxed)); // 10
}
```

For a plain counter where you only care about the final count, `Ordering::Relaxed` is correct and fast: it guarantees the increments do not get lost, but makes no promise about ordering relative to other variables. When an atomic is used as a flag that *guards* other data, you need a stronger ordering.

Rule of thumb. If you are unsure which `Ordering` to use, choose `Ordering::SeqCst` (sequentially consistent). It is the strongest and easiest to reason about. Optimize to `Relaxed` only for independent counters where order does not matter.

Faster channels in the ecosystem

The standard `mpsc` is fine, but two crates are popular when you need more:

- **crossbeam** — fast channels, including **multi-consumer** channels (many receivers), plus other lock-free data structures. Add it with `cargo add crossbeam`.
- **flume** — a drop-in, faster channel that is both multi-producer and multi-consumer, and works in both sync and async code. Add it with `cargo add flume`.

If you found yourself wrapping a receiver in `Arc<Mutex<...>>` (as in the worker example), that is a sign you want one of these instead.

Key takeaways

- A channel from `std::sync::mpsc` is **multi-producer, single-consumer**. `channel()` returns `(tx, rx)`.
- `tx.send(v)` **moves** `v` to the receiver. Clone `tx` to get multiple producers.
- Receive with `rx.recv()` (blocking), `rx.try_recv()` (non-blocking), or `for v in rx` (until closed).
- The channel **closes when all senders are dropped**; then `recv` returns `Err`.
- `sync_channel(n)` is bounded and applies backpressure; `sync_channel(0)` is a rendezvous.
- Prefer **message passing** to transfer ownership; prefer **shared state** (`Arc<Mutex<T>>`, `RwLock`, `atomics`) for a small value many threads touch.
- `RwLock` allows many readers or one writer; `atomics` give lock-free counters with an explicit `Ordering`.
- `crossbeam` and `flume` offer faster and multi-consumer channels.

Watch out (gotchas for C programmers)

- **send moves ownership.** You cannot use a value after sending it (unless it is `Copy`). The receiver owns it now.
- **The channel closes only when every sender drops.** Keep an extra `tx` alive and the consumer loop blocks forever. Drop senders you do not need.
- **std mpsc has a single consumer.** To fan work out to many receivers, share the receiver behind a lock or use `crossbeam/flume`.
- **Channel or mutex?** Moving data through a channel avoids shared mutable state. Reach for a `Mutex` only when several threads genuinely share one value.
- **Atomic orderings matter.** `Relaxed` is fine for an independent counter, but not when the atomic guards other data. Use `SeqCst` when in doubt.
- **A Mutex guard holds the lock until it drops.** Holding it across a blocking call (like `recv()`) can serialize your threads. Keep critical sections short.

Interview questions

Q: What does “mpsc” mean, and how do you get multiple producers? A: It means multi-producer, single-consumer. `channel()` gives one sender and one receiver. To have multiple producers, you `clone()` the sender; each clone is another handle to the same channel, and the single receiver collects all messages.

Q: When does a `std::sync::mpsc` channel close, and how does the receiver find out? A: The channel closes when the last sender is dropped. After that, `rx.recv()` returns `Err(RecvError)` and a `for v in rx` loop ends. This is how the consumer knows no more messages will arrive.

Q: What is the difference between `channel()` and `sync_channel(n)`? A: `channel()` is unbounded — `send` never blocks and the queue can grow without limit. `sync_channel(n)` is bounded to `n` buffered messages; `send` blocks when the buffer is full, giving backpressure. `sync_channel(0)` is a rendezvous: each `send` waits for a matching receive.

Q: When would you use shared state instead of a channel? A: When several threads need to read or update one small piece of data — a counter, a cache, a config value — that naturally lives in one place. Passing it through a channel would be awkward. Use `Arc<Mutex<T>>`, `RwLock`, or an atomic instead.

Q: Why does `Mutex<T>` in Rust hold the data inside it, and why is that better than C's separate mutex and buffer? A: Because the only way to reach the data is to call `lock()`, which returns a guard giving exclusive access. You cannot forget to lock — the type system requires it. In C the mutex and the data are separate variables, so nothing stops you from touching the data without locking, which causes data races.

Try it

1. Build the multi-producer example, but keep the original `tx` alive in `main` (remove the `drop(tx)`). Watch the `for v in rx` loop hang, then fix it.
2. Replace the `Mutex<i32>` counter with an `AtomicUsize` and `fetch_add`. Confirm you get the same total of 10 with less code.
3. Change `channel()` in the worker example to `sync_channel(4)` and add a small `thread::sleep` in the consumer. Notice the producer slowing down to match.

Chapter 21 — Async/Await

What you'll learn. What `async/.await` is, why it exists, and how it lets one program handle thousands of network connections without one thread each. You will also learn why `async` needs a *runtime* (almost always Tokio), and when to reach for `async` versus plain threads.

Note on crates. The standard library has the `async/.await` *syntax* and the `Future` type, but **no runtime to run futures**. So almost every example in this chapter needs an external crate, **Tokio**. Before running them, add it:

```
cargo add tokio --features full
```

Examples that need Tokio are marked. They will **not** compile with the standard library alone. The one std-only example is marked as such.

The problem: thousands of connections

Imagine a network server that handles 10,000 clients at once. Most of the time each client is just *waiting* — for a packet to arrive, for the disk to respond. The work is **I/O-bound**: limited by waiting, not by the CPU.

The classic C approach is **one thread (or process) per connection**. It is simple to write, but every thread costs memory (a stack, often megabytes) and the operating system must switch between them. At 10,000 threads this becomes expensive and slow.

The other classic C approach is a single-threaded **event loop** with `epoll` (or `kqueue`, or `select`). One thread watches all the sockets and reacts when one is ready. This scales beautifully but is painful to write: you hand-build a *state machine* for each connection and store its progress in a struct.

```
/* A classic C event loop: one thread, epoll, manual state per connection. */
int ep = epoll_create1(0);
/* register sockets ... */
for (;;) {
    int n = epoll_wait(ep, events, MAX, -1);
```

```

for (int i = 0; i < n; i++) {
    conn_t *c = events[i].data.ptr;
    switch (c->state) {
        case READING_HEADER: /* ... */ break;
        case READING_BODY:  /* ... */ break;
        case WRITING_REPLY:  /* ... */ break;
    }
}
}

```

Async/await gives you the **scalability of the event loop** with code that **reads like the simple one-thread-per-connection version**. The compiler builds the state machine for you.

Mental model. `async/.await` is an automatic version of the `epoll` state machine. You write straight-line code with `.await` where it would block; the compiler turns each function into a state machine that pauses and resumes at those points.

`async fn` returns a future

Mark a function `async` and it becomes an **asynchronous function**. Calling it does **not** run its body. Instead it returns a **future**: a value that represents a computation that will finish *later*.

A future is **lazy**. It does nothing until you `.await` it or hand it to the runtime. This surprises everyone coming from threads, where starting work is immediate.

```

// This example compiles with the standard library alone (no Tokio).
async fn say_hello() {
    println!("hello");
}

fn main() {
    let _future = say_hello(); // NOTHING prints: the future is lazy
    println!("the future was created but not run");
}

```

Running this prints only the second line. The `say_hello` body never runs, because nobody awaited the future. (Edition 2024 puts `Future` in the prelude, so you do not need to import it.)

C vs Rust. A C function call runs the body right away. A Rust `async fn` call builds a state machine and returns it, frozen at the start. It is more like getting a function pointer plus its saved state than calling the function.

`.await` runs it and yields

Inside another `async` function, `.await` drives a future to completion. While the future is waiting (for a timer, a socket, a lock), `.await` **yields control** back to the runtime so other tasks can run on the same thread. When the future is ready, your function resumes right after the `.await`.

```

// needs: cargo add tokio --features full
use tokio::time::{sleep, Duration};

#[tokio::main]

```

```

async fn main() {
    println!("start");
    sleep(Duration::from_millis(100)).await; // yield for 100 ms
    println!("done after 100 ms");
}

```

While `sleep(...).await` waits, the thread is free to run other tasks. It is **not** blocked the way `std::thread::sleep` blocks an OS thread.

Futures need a runtime

A future is just a state machine. Something has to **poll** it — call it repeatedly, advancing it past each `.await` as data becomes ready. That something is the **runtime** (also called the **executor**).

The standard library has the future *machinery* but **no executor**. So you pull in a crate. The dominant choice is **Tokio**. Others exist (`async-std`, `smol`), but Tokio is the default for most of the ecosystem.

The `#[tokio::main]` attribute is a small macro that sets up the runtime and runs your `async fn main` on it. It rewrites your `main` into a normal `fn main` that starts Tokio and blocks on your async code.

```

// needs: cargo add tokio --features full
#[tokio::main]
async fn main() {
    println!("running on the Tokio runtime");
}

```

C vs Rust. The runtime is your `epoll` loop, written once and reused. In C you write that loop yourself in every event-driven program. In Rust you depend on Tokio and let it own the loop.

Spawning tasks

`tokio::spawn` hands a future to the runtime as a **task** — an independent unit the runtime drives, much like a lightweight thread. Unlike `tokio::spawn`, simply creating a future does nothing; spawning is one of the two ways (with `.await`) to make a future actually run. `spawn` returns a handle you can `.await` to get the task's result.

```

// needs: cargo add tokio --features full
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        sleep(Duration::from_millis(50)).await;
        "task result"
    });

    let result = handle.await.unwrap(); // wait for the task, get its value
}

```

```
    println!("{result}");
}
```

A task is far cheaper than an OS thread. Tokio happily runs hundreds of thousands of tasks on a small pool of threads, because a waiting task costs almost nothing — just its saved state, no separate stack.

Tokio channels

Tokio provides its own channels in `tokio::sync`, whose `send/recv` are `async` (they `.await` instead of blocking a thread). They mirror the `std::sync::mpsc` channels from Chapter 20 — Channels and Shared State.

```
// needs: cargo add tokio --features full
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<i32>(8); // bounded, capacity 8

    tokio::spawn(async move {
        for i in 0..5 {
            tx.send(i).await.unwrap(); // async send
        }
    });

    while let Some(value) = rx.recv().await { // ends when all senders drop
        println!("got {value}");
    }
}
```

Concurrency within a task

Spawning makes separate tasks. But you can also run several futures **inside one task** and wait for them together.

`tokio::join!` runs futures concurrently and waits for **all** of them. Two 100-millisecond waits finish in about 100 milliseconds total, not 200, because they overlap.

```
// needs: cargo add tokio --features full
use tokio::time::{sleep, Duration};

async fn fetch(name: &str, ms: u64) -> String {
    sleep(Duration::from_millis(ms)).await;
    format!("{name} done")
}

#[tokio::main]
async fn main() {
    let (a, b) = tokio::join!(fetch("a", 100), fetch("b", 100));
}
```

```

    println!("{a}, {b}"); // about 100 ms total, not 200
}

```

`tokio::select!` waits for the **first** of several futures to finish and drops the rest. It is perfect for timeouts and racing two operations.

```

// needs: cargo add tokio --features full

```

```

use tokio::time::{sleep, Duration};

```

```

#[tokio::main]

```

```

async fn main() {

```

```

    tokio::select! {

```

```

        _ = sleep(Duration::from_millis(50)) => {
            println!("the 50 ms timer won");

```

```

        }

```

```

        _ = sleep(Duration::from_millis(100)) => {
            println!("the 100 ms timer won");

```

```

        }

```

```

    }

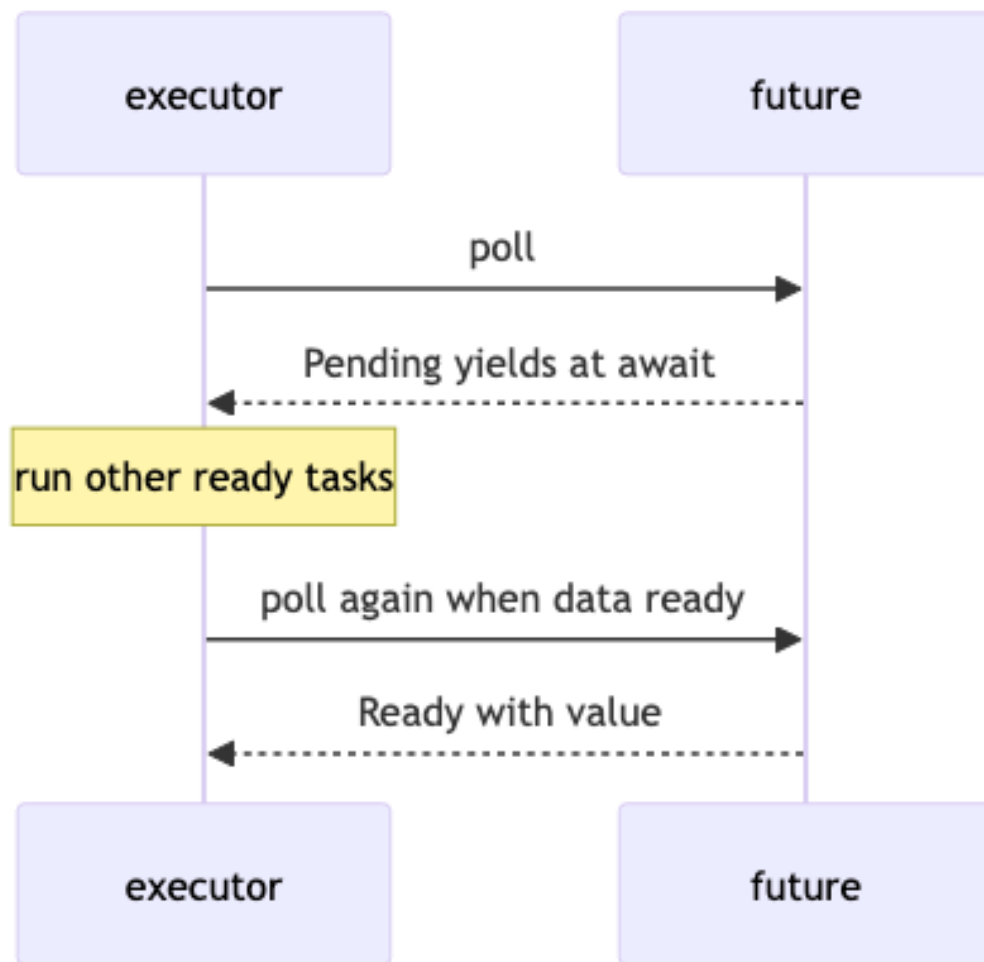
```

```

}

```

Here is the state-machine view of one future pausing and resuming at its `.await` points. The executor polls it, gets `Pending`, runs other work, and polls again when the data is ready.



async and Send

Tokio's default runtime is **multi-threaded**: it may move a task from one thread to another at an `.await` point. So any data a future holds **across** an `.await` must be safe to send between threads — it must be `Send` (see Chapter 19 — Threads and Concurrency).

This future does not compile, because it holds an `Rc` (which is not `Send`) across an `.await`:

```
// COMPILE ERROR: future cannot be sent between threads safely
// (also needs: cargo add tokio --features full)
use std::rc::Rc;
use tokio::time::{sleep, Duration};

async fn bad() {
    let x = Rc::new(5); // Rc is not Send
    sleep(Duration::from_millis(10)).await; // x is held across .await
    println!("{x}");
}
```



```
#[tokio::main]
async fn main() {
    tokio::spawn(bad()).await.unwrap(); // spawn requires a Send future
}
```

The fix is to use a `Send` type (`Arc` instead of `Rc`), or to make sure the non-`Send` value is dropped *before* the `.await`.

Do not block inside async

A future shares its thread with many other tasks. If you do **blocking** work inside an `async` function — a long CPU computation, `std::thread::sleep`, a synchronous file read — you freeze that thread. Every other task scheduled on it is **starved** until you finish. This is the most common async performance bug.

For blocking or CPU-heavy work, hand it to `tokio::task::spawn_blocking`, which runs it on a separate thread pool meant for blocking jobs.

```
// needs: cargo add tokio --features full
#[tokio::main]
async fn main() {
    let result = tokio::task::spawn_blocking(|| {
        // CPU-heavy or blocking work belongs here, off the async threads.
        (1..=1_000_000u64).sum::<u64>()
    })
    .await
    .unwrap();

    println!("sum = {result}");
}
```

Watch out. Use `tokio::time::sleep(...).await`, not `std::thread::sleep(...)`, inside async code. The first yields to the runtime; the second blocks the whole thread and starves other tasks.

When async, when threads

Both async and threads give concurrency. Choose by the kind of work:

Situation	Best tool	Why
Many I/O-bound tasks waiting on the network	async (Tokio)	Cheap tasks, no thread per connection
A few CPU-bound jobs	threads	Real parallelism across cores
Lots of CPU-bound data parallelism	<code>rayon</code> crate	Easy parallel iterators over cores
Mixed: async server doing occasional heavy work	async + <code>spawn_blocking</code>	Keep the event loop free

Rule of thumb. I/O-bound and many connections → async. CPU-bound → threads or `rayon`. Do not pick async just because it is new; for CPU work it adds complexity

with no benefit.

Function coloring

`async` “colors” your functions. You can only `.await` a future inside an `async` function. A normal function cannot `.await`; it must either block on a runtime or become `async` itself. So `async` tends to spread upward through your call stack — callers of async code often must be `async` too. People call this the “what color is your function” problem. It is the main ergonomic cost of `async`, and a reason to keep plain synchronous code synchronous unless you need `async`.

C vs Rust. In a C event loop, “async-ness” spreads too: any function that might wait must return to the event loop instead of blocking, so it splits into callbacks or state transitions. Rust’s `async/.await` is the same constraint, but the compiler writes the state machine and the code still reads top to bottom.

Key takeaways

- Async exists to handle **many I/O-bound tasks** cheaply, without one OS thread per connection.
- An `async fn` returns a **future**: a lazy state machine that does nothing until it is `.awaited` or spawned.
- `.await` drives a future and **yields** control to the runtime while waiting, so other tasks run.
- Futures need a **runtime**; the std library has none, so you use a crate — almost always **Tokio** (`#[tokio::main]`, `tokio::spawn`, `tokio::time::sleep`, `tokio::sync` channels).
- Run futures together with `tokio::join!` (wait for all) and `tokio::select!` (take the first).
- A future held across an `.await` must be **Send** on a multi-threaded runtime.
- **Never block** inside `async`; use `tokio::task::spawn_blocking` for CPU or blocking work.
- I/O-bound → `async`; CPU-bound → threads or `rayon`. `async` “colors” functions.

Watch out (gotchas for C programmers)

- **Futures are lazy.** Calling an `async fn` runs nothing. You must `.await` it or spawn it. A “missing await” often shows as a compiler warning that a future is unused.
- **You need a runtime.** Plain std cannot run a future. Add Tokio and use `#[tokio::main]` or build a runtime yourself.
- **Blocking starves the executor.** A long CPU loop or `std::thread::sleep` inside `async` freezes every task on that thread. Use `spawn_blocking`.
- **Send across .await.** Holding a non-Send value (like `Rc`) across an `.await` fails to compile on the multi-threaded runtime. Use `Arc`, or drop the value before the `.await`.
- **Async colors your functions.** `.await` is only allowed in `async` contexts, so `async` tends to spread up the call stack. Do not make code `async` unless it benefits.

Interview questions

Q: What does an `async fn` return, and what does “lazy” mean here? A: It returns a future — a state machine representing a computation that finishes later. “Lazy” means the body does not run when you call the function; nothing happens until the future is `.awaited` or handed to a runtime to be spawned.

Q: Why does `async Rust` need an external crate when threads do not? A: The standard library provides the `async/.await` syntax and the `Future` trait, but no executor to poll futures to completion. You need a runtime such as `Tokio` (or `async-std`, `smol`) to actually drive the futures. Threads, by contrast, are provided by the OS through `std::thread`.

Q: What happens if you do blocking work inside an `async` task, and how do you fix it?

A: You block the thread the executor is using, so every other task scheduled on that thread is starved until you finish. Fix it by moving CPU-heavy or blocking work to `tokio::task::spawn_blocking`, which uses a separate thread pool, and by using `async` equivalents (like `tokio::time::sleep`) instead of blocking ones.

Q: Why must a future be `Send` to be spawned on `Tokio`’s multi-threaded runtime? A:

The runtime may move a task between worker threads at an `.await` point. Any data the future keeps alive across that point therefore crosses threads, so it must be `Send`. Holding a non-`Send` type like `Rc` across an `.await` is a compile error.

Q: When would you choose threads over `async`? A: For CPU-bound work that needs real parallelism across cores. `Async` helps with I/O-bound work where tasks mostly wait. For heavy computation, threads (or the `rayon` crate for data parallelism) are simpler and just as fast, with none of `async`’s function-coloring overhead.

Try it

1. Run the lazy-future example and confirm “hello” never prints. Then add `Tokio`, make `main` `async` with `#[tokio::main]`, and `say_hello().await`. Now it prints.
2. Build the `tokio::join!` example and time it. Change one wait to 300 ms and watch the total become about 300 ms — the `join` waits for the slowest.
3. Put a `std::thread::sleep(Duration::from_secs(1))` inside a spawned task and start several tasks. Notice how they no longer overlap, then switch to `tokio::time::sleep` and see them run concurrently again.

Chapter 22 — Cargo, Crates, and Workspaces

What you’ll learn. How Cargo manages dependencies: the full `Cargo.toml` manifest, version requirements and the lockfile, the crates.io ecosystem, optional features, and workspaces that hold many crates in one repository. This is the dependency and packaging story C never standardized.

Chapter 2 — Installing Rust and the Cargo Toolchain gave you the command tour: `cargo new`, `build`, `run`, `add`. This chapter goes deeper. It is about the **package model** — how Cargo decides which versions to download, how it makes builds reproducible, and how you split a large project into several crates.

The problem C never solved

C has no standard package manager. To use a library in C, you do this by hand:

- Find the library and install a `-dev` package with your operating system’s package manager (`apt`, `dnf`, `brew`), or copy its source into your tree.
- Tell the compiler where the headers are with `-I/path/to/include`.

- Tell the linker where the library file is with `-L/path/to/lib`.
- Name the library to link with `-lfoo`.
- Hope the version installed on the build machine matches what your code expects. There is no built-in record of “exactly which version did we build against.”

```
# C: a typical manual build against zlib
cc main.c -I/usr/include -L/usr/lib -lz -o main
```

Every project reinvents this in a Makefile or CMake file. There is no central registry, no standard version syntax, and no lockfile.

Cargo replaces all of it. You declare what you need in one file; Cargo fetches it from a central registry, picks compatible versions, records the exact result, and passes the right flags to the compiler.

C vs Rust. In C, dependencies live in your operating system and your build script. In Rust, dependencies live **in the project**: named in `Cargo.toml`, pinned in `Cargo.lock`, cached under `~/.cargo`. Clone the repo, run `cargo build`, and it just works.

Cargo.toml in depth

`Cargo.toml` is the **manifest**: the file Cargo reads to learn your project’s name, edition, and dependencies. It is written in TOML, a simple key-value format where `[name]` starts a table (a section) and `key = value` sets a field. Here is a fuller example than the one `cargo new` gives you, annotated section by section.

```
[package]
name = "imageproc"           # the crate name (lowercase, dashes ok in the name)
version = "0.3.1"            # this crate's own version (semantic versioning)
edition = "2024"             # the language edition; this book uses 2024
description = "Tiny image filters" # shown on crates.io if you publish
license = "MIT OR Apache-2.0" # standard dual license for Rust crates
```

```
[dependencies]
# crates needed to BUILD AND RUN your library or program
serde = { version = "1.0", features = ["derive"] }
rand = "0.9"
```

```
[dev-dependencies]
# crates needed ONLY to build tests, examples, and benchmarks.
# These are NOT compiled into your shipped library or binary.
criterion = "0.5"
```

```
[build-dependencies]
# crates needed ONLY by your build script (build.rs), if you have one
cc = "1.0"
```

```
[features]
# named on/off switches; see the Features section below
default = ["png"]
png = []
```

```

simd = []

[profile.release]
# tune the optimized build
opt-level = 3          # optimization level 0..3 (like -O0 .. -O3 in C)
lto = true             # link-time optimization across crates
codegen-units = 1      # fewer units = better optimization, slower compile

```

The four dependency tables are the important distinction:

Table	Used when	C analogy
[dependencies]	building your library or binary	a library you <code>-l</code> link at runtime
[dev-dependencies]	building tests, examples, benchmarks	a test-only library, never shipped
[build-dependencies]	running <code>build.rs</code> before the build	a code generator you run at build time
[features]	turning optional parts on or off	<code>#ifdef</code> configuration macros

Watch out. A dev-dependency is invisible to anyone who *uses* your crate as a library. It is compiled only when you build *your* crate’s own tests and examples. Put your test-only helpers (like `criterion`) there, not in `[dependencies]`, so you do not force them on your users.

Profiles: `[profile.*]`

A **profile** is a named set of compiler settings. Cargo has built-in profiles: `dev` (used by `cargo build`) and `release` (used by `cargo build --release`). You override their settings in `[profile.dev]` and `[profile.release]`.

```

[profile.dev]
opt-level = 0          # no optimization: fast compile, slow runtime (default)

[profile.release]
opt-level = 3          # full optimization: slow compile, fast runtime

```

C vs Rust. In C you pass `-O0 -g` or `-O2` on the command line, or scatter them through a Makefile. In Rust the optimization settings live declaratively in `Cargo.toml`, one place, applied to the whole dependency graph at once.

Cargo.lock: reproducible builds

`Cargo.toml` says what you *want* (“serde version 1.x”). `Cargo.lock` records what you actually *got* (“serde 1.0.210, which pulls in `serde_derive` 1.0.210, which pulls in `syn` 2.0.77 ...”). Cargo writes and updates `Cargo.lock` automatically; you never edit it by hand.

The lockfile pins the **exact** version of every dependency, including transitive ones (dependencies of your dependencies). Anyone who checks out your code and runs `cargo build` gets byte-for-byte the same versions you built and tested with.

`Cargo.toml` → "I want `serde 1.x`" (your requirement, you edit it)
`Cargo.lock` → "`serde = 1.0.210 exactly`" (the resolved result, Cargo writes it)

Rule of thumb. Commit `Cargo.lock` to version control. The modern guidance (current Cargo) is to commit it for **both applications and libraries**. For an application it guarantees reproducible builds. For a library it gives your own contributors and CI a known-good version set; consumers of your library still ignore your lockfile and resolve their own.

C vs Rust. C has nothing like `Cargo.lock` built in. To get reproducible dependency versions in C you pin Git submodules, vendor source, or maintain a manifest yourself. Cargo gives you this for free, automatically, on every build.

Version requirements and semver

Versions on crates.io follow **semantic versioning** (semver): `MAJOR.MINOR.PATCH`, for example `1.4.2`. The rules:

- **MAJOR** changes when there is a breaking change (old code may stop compiling).
- **MINOR** changes when features are added in a backward-compatible way.
- **PATCH** changes for backward-compatible bug fixes.

When you write a dependency version in `Cargo.toml`, you are writing a **requirement** — a range of acceptable versions, not one exact version. The syntax:

Requirement	Meaning	Allows
" <code>1.4.2</code> " or " <code>^1.4.2</code> "	caret (the default)	<code>>=1.4.2</code> and <code><2.0.0</code>
" <code>~1.4.2</code> "	tilde	<code>>=1.4.2</code> and <code><1.5.0</code>
" <code>=1.4.2</code> "	exact	only <code>1.4.2</code>
" <code>*</code> "	wildcard	any version (avoid this)
" <code>1</code> "	caret on a partial version	<code>>=1.0.0</code> and <code><2.0.0</code>

Watch out. A bare version string like `serde = "1.0"` is **not** an exact pin. It is the **caret** requirement: “any `1.x` version at or above `1.0`, excluding `2.0`.” This surprises C programmers who expect “`1.0`” to mean exactly `1.0`. The reason it is safe is semver: anything below `2.0` promises to stay backward compatible. The `Cargo.lock` file pins the exact version actually used.

Updating dependencies

`cargo update` recomputes `Cargo.lock`, moving each dependency to the newest version still allowed by your requirements in `Cargo.toml`.

```
cargo update           # update everything within the semver rules
cargo update -p serde  # update only serde (and what it needs)
cargo add serde@1.0    # add or change a requirement, then resolve
```

`cargo update` never crosses a major version boundary on its own, because your caret requirement forbids it. To move from `1.x` to `2.x` you edit `Cargo.toml` and accept the breaking change deliberately.

The crates.io ecosystem

crates.io is the central public registry — one place where the whole community publishes and finds libraries. **docs.rs** automatically builds and hosts the API documentation for every published crate, so every library has browsable docs at a predictable URL.

C vs Rust. There is no crates.io for C. You search the web, your distro’s repositories, and GitHub, and documentation quality varies wildly. In Rust, `cargo add foo` plus `https://docs.rs/foo` is the universal workflow.

A C programmer will meet these crates constantly. It helps to know them by name:

Crate	Job	Why you reach for it
<code>serde</code>	serialization	turn structs into JSON, TOML, etc. and back
<code>tokio</code>	async runtime	the engine that runs <code>async/await</code> code (Chapter 21)
<code>clap</code>	command-line args	parse <code>argv</code> into typed options, with <code>--help</code> for free
<code>rand</code>	random numbers	there is no <code>rand()</code> in std
<code>anyhow</code>	easy error handling	one error type for applications (Chapter 13)
<code>thiserror</code>	custom error types	derive clean error enums for libraries (Chapter 13)
<code>rayon</code>	data parallelism	turn <code>.iter()</code> into <code>.par_iter()</code> and use all cores
<code>regex</code>	regular expressions	no regex in std
<code>reqwest</code>	HTTP client	make web requests, sync or async

Rule of thumb. Before writing low-level code, search crates.io. The things a C programmer often hand-rolls — argument parsing, JSON, logging, random numbers — are mature, well-documented crates you add in one line.

Cargo features

A **feature** is a named, compile-time switch. Features let one crate ship optional functionality that callers turn on or off. This is Rust’s structured answer to C’s `#ifdef` configuration macros.

You declare features in the `[features]` table. Each feature lists what it enables: other features, or **optional dependencies**.

```
[dependencies]
serde = { version = "1.0", optional = true }    # optional: off unless a feature turns it on

[features]
default = ["fast"]          # features enabled when the caller says nothing
fast = []                   # a plain switch, enables no extra deps
json = ["dep:serde"]        # the "json" feature pulls in the optional serde dependency
```

In code, you compile blocks conditionally with the `cfg` attribute, testing for a feature by name:

```
// With no "json" feature defined, the not(...) branch is the one that compiles.
#[cfg(feature = "json")]
pub fn to_json(value: &i32) -> String {
    // only compiled when the "json" feature is enabled
    format!("{}", value)
}

#[cfg(not(feature = "json"))]
pub fn to_json(_value: &i32) -> String {
    String::from("json feature disabled")
}

fn main() {
    println!("{}", to_json(&42));
}
```

C vs Rust. `#[cfg(feature = "json")]` is like wrapping code in `#ifdef HAVE_JSON` ... `#endif`. The difference: features are declared in the manifest and resolved by Cargo across the whole dependency graph, instead of being passed as ad hoc `-D` flags that each build script sets differently.

Callers choose features when they depend on you:

```
cargo add imageproc --features json          # turn on a feature
cargo add imageproc --no-default-features    # start from nothing
```

Feature unification

Here is the subtle part. If two different crates in your build both depend on `serde`, and one asks for `serde` with the `derive` feature while the other does not, Cargo enables `derive` for **both**. Features are **additive** and **unified** across the whole dependency graph: a feature is on if *anything* turns it on.

Watch out. Feature unification means you cannot assume a feature is off just because *you* did not enable it. Another crate in the graph may have. So a feature must only **add** capability, never remove or change behavior. A feature that makes a function behave differently when enabled can break a sibling crate that did not expect it.

Workspaces: many crates, one repository

As a project grows you split it into several crates: a core library, a command-line front end, maybe a separate crate for plugins. A **workspace** is a set of related crates that share one `Cargo.lock` and one `target/` build directory. They build together and stay version-consistent.

You create a workspace with a top-level `Cargo.toml` that has a `[workspace]` table instead of (or in addition to) a `[package]` table.

```
# top-level Cargo.toml (the workspace root)
[workspace]
resolver = "3"          # the edition-2024 feature resolver
```



```
members = [
    "core",
    "cli",
]
```

```
[workspace.dependencies]
```

```
# versions declared once here, inherited by members
```

```
serde = { version = "1.0", features = ["derive"] }
```

A member crate inherits a shared dependency by writing `.workspace = true`:

```
# core/Cargo.toml
```

```
[package]
```

```
name = "core"
```

```
version = "0.1.0"
```

```
edition = "2024"
```

```
[dependencies]
```

```
serde = { workspace = true } # use the version from the workspace root
```

Path dependencies

Inside a workspace, one crate depends on a sibling by **path**, not by a registry version. A **path dependency** points at a directory on disk.

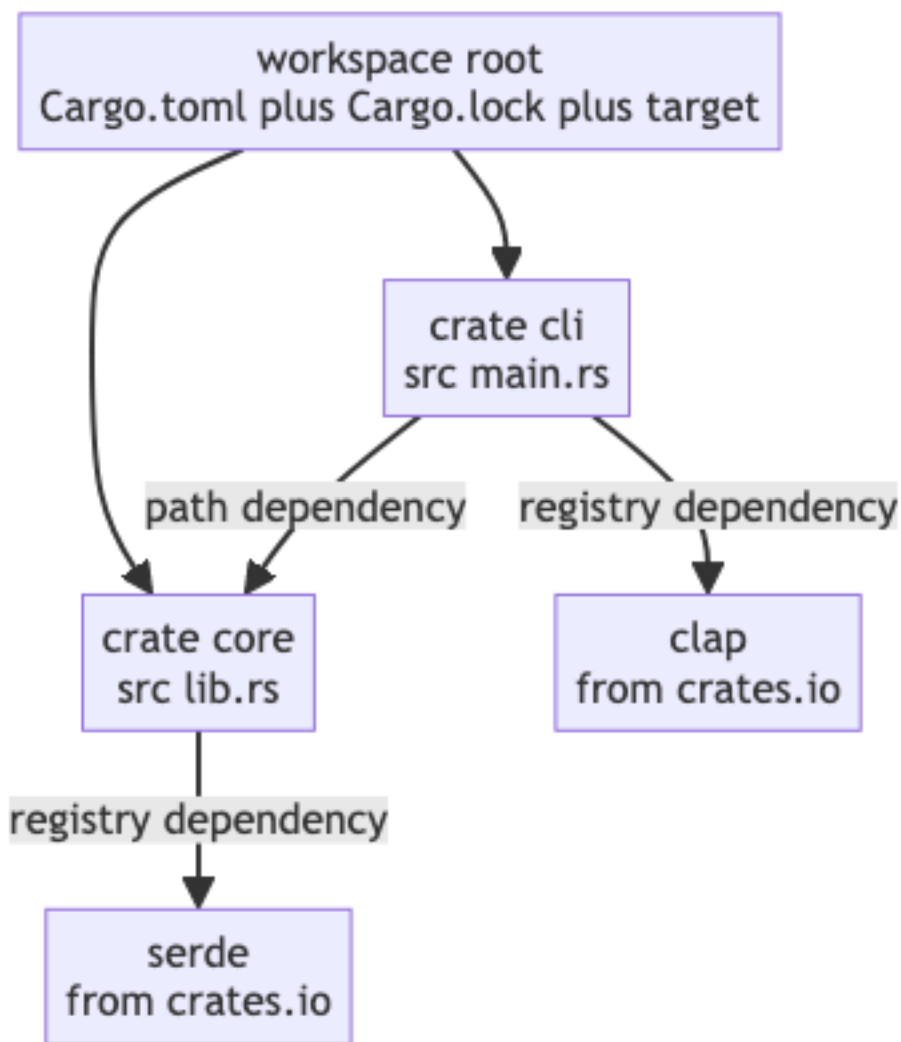
```
# cli/Cargo.toml
```

```
[dependencies]
```

```
core = { path = "../core" } # depend on the sibling crate in ../core
```

Now the `cli` crate can use `core::...` to call into the `core` library. When you change `core`, rebuilding `cli` picks up the change immediately — no publishing step.

Here is how the pieces fit together:



Watch out. All members of a workspace share one `Cargo.lock`. That means one resolved version of `serde` is used everywhere in the workspace — you cannot have `core` on `serde 1.0.50` and `cli` on `serde 1.0.99` at the same time. This is usually what you want: consistency. Declare shared versions once in `[workspace.dependencies]`.

Splitting `lib.rs` and `main.rs`

Even inside a single package, the idiomatic split is a library crate (`src/lib.rs`) holding the logic and a thin binary crate (`src/main.rs`) that calls it. The binary depends on the library by name. This keeps the logic testable and reusable (see Chapter 3 — Program Structure: Crates, Modules, and Visibility).

A package can also hold standard extra directories that Cargo recognizes:

Directory	What it holds	Run with
<code>src/</code>	the library and/or binary code	<code>cargo build</code>

Directory	What it holds	Run with
<code>tests/</code>	integration tests, each file its own crate	<code>cargo test</code>
<code>examples/</code>	small runnable example programs	<code>cargo run --example name</code>
<code>benches/</code>	benchmarks	<code>cargo bench</code>

Chapter 23 — Testing covers `tests/` in detail.

Publishing and build scripts (briefly)

To share a crate on crates.io you publish it:

```
cargo login          # paste your crates.io API token, once
cargo publish        # upload the current crate
```

Once a version is published it is **permanent** — you cannot delete or overwrite it (you can only “yank” it to stop new projects from selecting it). So follow semver discipline: bump MAJOR for breaking changes, MINOR for new features, PATCH for fixes.

A **build script** is a file named `build.rs` at the crate root. Cargo compiles and runs it **before** building the crate. It is used to generate code, compile a bundled C library (with the `cc` crate), or detect platform features. It is the Rust answer to the configure step in a C project.

```
// build.rs - runs before the crate is compiled
fn main() {
    // tell Cargo to re-run this script only if build.rs itself changes
    println!("cargo::rerun-if-changed=build.rs");
}
```

C vs Rust. `build.rs` plays the role of `./configure` or a code-generation Makefile rule, but it is ordinary Rust, runs automatically, and its dependencies go in `[build-dependencies]`.

Key takeaways

- C has no standard package manager; you wire up `-I/-L/-l` by hand with no central registry and no lockfile. Cargo provides all of this.
- `Cargo.toml` is the manifest: `[package]`, `[dependencies]`, `[dev-dependencies]`, `[build-dependencies]`, `[features]`, and `[profile.*]`.
- `Cargo.lock` pins exact versions for reproducible builds. **Commit it** — for both applications and libraries (current guidance).
- Version requirements default to **caret**: `serde = "1.0"` means `>=1.0, <2.0`. `~` is tighter, `=` is exact, `*` is anything. `cargo update` moves within the rules.
- crates.io is the registry and docs.rs hosts the docs. Learn the common crates: `serde`, `tokio`, `clap`, `rand`, `anyhow/thiserror`, `rayon`, `regex`, `request`.
- Features are compile-time switches and optional dependencies, tested with `#[cfg(feature = "...")]`. They are **additive and unified** across the graph.
- A workspace holds many crates sharing one `Cargo.lock` and one `target/`. Members use path dependencies on siblings and can inherit `[workspace.dependencies]`.

Watch out (gotchas for C programmers)

- **The default version requirement is a caret, not an exact pin.** "1.0" allows any 1.x. Use `=1.0.0` only when you truly need to lock one version.
- **Commit `Cargo.lock`.** It is the difference between “builds on my machine” and “builds everywhere, forever.”
- **Feature unification is global.** A feature you did not enable may still be on because another crate in the graph enabled it. Features must only add behavior.
- **Workspace members share one resolved version of each dependency.** You cannot pin different versions of the same crate in different members of one workspace.
- **dev-dependencies are not shipped.** They build only for your tests and examples, and are invisible to crates that depend on you. Do not put runtime dependencies there.

Interview questions

Q: What is the difference between `Cargo.toml` and `Cargo.lock`, and which do you commit? A: `Cargo.toml` records your *requirements* (version ranges, features) and you edit it by hand. `Cargo.lock` records the *exact* resolved version of every dependency, including transitive ones, and Cargo writes it for you. Commit `Cargo.lock` (for both applications and libraries today) so builds are reproducible.

Q: What does the dependency requirement `serde = "1.0"` actually allow? A: It is a caret requirement, equivalent to `>=1.0.0, <2.0.0`. It allows any 1.x version because semver promises 1.x releases stay backward compatible. It is not an exact pin; the exact version chosen is recorded in `Cargo.lock`.

Q: What is feature unification and why does it matter? A: Cargo features are additive: if any crate in the dependency graph enables a feature of a shared crate, that feature is enabled for every user of that crate in the build. It matters because you cannot assume a feature is off, and because features must only *add* capability — a feature that changes behavior can break another crate that did not expect it.

Q: What is a Cargo workspace and what do its members share? A: A workspace is a set of related crates in one repository, declared with a `[workspace]` table. The members share one `Cargo.lock` and one `target/` directory, so they build together with a single consistent set of dependency versions. Members depend on each other by path and can inherit shared dependency versions from `[workspace.dependencies]`.

Q: How do Rust dependencies differ from linking a C library? A: In C you install a library system-wide, then pass `-I`, `-L`, and `-l` flags to find headers and link, with no standard version record. In Rust you name the crate and a version requirement in `Cargo.toml`; Cargo fetches it from crates.io, resolves compatible versions for the whole graph, pins them in `Cargo.lock`, and passes the right flags to the compiler automatically.

Try it

1. Run `cargo new demo`, then `cargo add rand`. Open `Cargo.toml` and note the caret requirement. Open `Cargo.lock` and find the exact `rand` version chosen.
2. Run `cargo tree` to see the full dependency graph `rand` pulled in. Then `cargo update` and `diff Cargo.lock`.

3. Turn `demo` into a workspace: create a `core` library crate and make the binary depend on it with `core = { path = "../core" }`. Move a function into `core` and call it from `main`.

Chapter 23 — Testing

What you'll learn. How Rust builds testing into the language and the tool: unit tests in the same file, integration tests in `tests/`, and documentation tests that compile your examples. One command, `cargo test`, runs all three.

Testing is built in, not bolted on

C has no standard test framework. Every project picks one — CUnit, Check, Unity, Google Test for C++, or a pile of hand-written `assert()` calls and a custom runner. There is no agreed way to write a test, find tests, or run them. Each project teaches you its own setup.

Rust builds testing into the toolchain. A test is a function marked `#[test]`. The compiler collects them, and `cargo test` compiles and runs them with a built-in test runner. There is nothing to install and nothing to choose.

C vs Rust. In C, “how do we run tests?” is a project decision with many answers. In Rust the answer is always `cargo test`. Tests live next to the code or in a `tests/` directory, and the tool finds them for you.

Unit tests

A **unit test** checks one small piece of code — usually one function — in isolation. In Rust you put unit tests in the **same file** as the code they test, inside a module marked `#[cfg(test)]`.

The `#[cfg(test)]` attribute means “compile this module only when running tests.” So your tests add nothing to the normal build and ship nothing in your binary.

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;           // bring the parent module's items into scope

    #[test]
    fn adds_positive_numbers() {
        assert_eq!(add(2, 3), 5);
    }

    #[test]
    fn adds_negative_numbers() {
        assert_eq!(add(-2, -3), -5);
    }
}
```

The pieces:

- `#[cfg(test)] mod tests { ... }` — a module compiled only under test. By convention it is named `tests`.
- `use super::*;` — `super` is the parent module (the file’s top level). The glob `*` imports everything from it, so the tests can call `add` directly.
- `#[test]` — marks a function as a test. It takes no arguments. If it returns normally, the test passes. If it **panics**, the test fails.

The assertion macros

A **panic** is Rust’s “abort this thread with an error message” (Chapter 13 — Error Handling). The assertion macros panic when a check fails:

Macro	Passes when	C analogy
<code>assert!(cond)</code>	<code>cond</code> is <code>true</code>	<code>assert(cond)</code>
<code>assert_eq!(a, b)</code>	<code>a == b</code>	<code>assert(a == b)</code> , but prints both values
<code>assert_ne!(a, b)</code>	<code>a != b</code>	<code>assert(a != b)</code> , but prints both values

`assert_eq!` and `assert_ne!` print the actual left and right values when they fail, which `assert(a == b)` in C cannot do. You can add a custom message:

```
#[test]
fn check_with_message() {
    let result = 2 + 2;
    assert_eq!(result, 4, "math broke: got {result}");
}
```

Testing that code panics: `#[should_panic]`

Sometimes the correct behavior *is* to panic. Add `#[should_panic]` so the test passes only if the function panics.

```
pub fn divide(a: i32, b: i32) -> i32 {
    if b == 0 {
        panic!("division by zero");
    }
    a / b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    #[should_panic(expected = "division by zero")]
    fn panics_on_zero() {
```

```

        divide(10, 0);
    }
}

```

The optional `expected = "..."` string must appear in the panic message, so the test does not pass for the wrong panic.

Tests that return Result

A test can return `Result<(), E>` instead of nothing. Then you can use the `?` operator inside it (Chapter 13). The test passes on `Ok` and fails on `Err`.

```

fn parse_port(s: &str) -> Result<u16, std::num::ParseIntError> {
    s.parse::<u16>()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_a_valid_port() -> Result<(), std::num::ParseIntError> {
        let port = parse_port("8080")?; // `?` returns Err on failure -> test fails
        assert_eq!(port, 8080);
        Ok(())
    }
}

```

Rule of thumb. Use `assert_eq!` for checking values and a `Result`-returning test when the code under test uses `?` itself. Returning `Result` keeps the test readable and avoids a pile of `.unwrap()` calls.

Running tests with cargo test

```

cargo test                # build and run ALL tests (unit + integration + doc)
cargo test adds           # run only tests whose name contains "adds"
cargo test --release      # run tests with optimizations on
cargo test -- --nocapture  # show stdout/stderr from passing tests too
cargo test -- --test-threads=1 # run tests one at a time (no parallelism)

```

Two things to understand about how the runner behaves:

- **Tests run in parallel by default**, each on its own thread, to finish faster. This is why tests must not depend on shared global state or on running in a certain order. Use `--test-threads=1` to force sequential runs.
- **Output is captured.** Anything a *passing* test prints is hidden, so the results stay readable. A failing test's output is shown. Pass `--nocapture` to see output from every test.

Watch out. Because tests run in parallel, two tests that both write the same file or environment variable can interfere. Give each test its own data, or serialize them with `--test-threads=1`.

Note the `--` separator, the same idea as in `cargo run`: arguments before `--` go to Cargo; arguments after `--` go to the test runner.

Integration tests

A **unit test** lives beside the code and can see private items. An **integration test** lives in a separate `tests/` directory and tests your crate the way a real user would — through its **public API only**.

Each file in `tests/` is compiled as **its own separate crate**. It does not have access to your crate's internals; it can only **use** what you marked `pub`.

```
mypkg/  
  Cargo.toml  
  src/  
    lib.rs          # pub fn add(...) lives here  
  tests/  
    api.rs          # an integration test crate  
  
// tests/api.rs - a separate crate that uses mypkg's public API  
use mypkg::add;      // only PUBLIC items are reachable here  
  
#[test]  
fn add_works_through_public_api() {  
    assert_eq!(add(2, 2), 4);  
}
```

There is no `#[cfg(test)]` here — the whole file is already test-only, because Cargo compiles `tests/` only when testing.

C vs Rust. An integration test is like writing a separate `.c` program that links against your library and calls only its public header functions. Rust does this automatically: each `tests/*.rs` file is a small client program of your crate.

Watch out. Integration tests see only `pub` items. If a test in `tests/` cannot find a function, check that the function (and its module path) is public. A unit test in `src/` would have seen it; an integration test will not.

Documentation tests

This is the feature C has no equivalent for. Code inside a documentation comment is **compiled and run** by `cargo test`. That means the examples in your docs can never silently rot — if an example stops compiling or its assertion fails, the test suite fails.

A **doc comment** starts with `///` and documents the item right below it. Anything inside a ````rust` fenced block in that comment is a **doc test**.

```
/// Adds two numbers.  
///  
/// # Examples  
///  
/// ```  
/// use mypkg::add;
```



```

///
/// let sum = add(2, 3);
/// assert_eq!(sum, 5);
/// ...
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

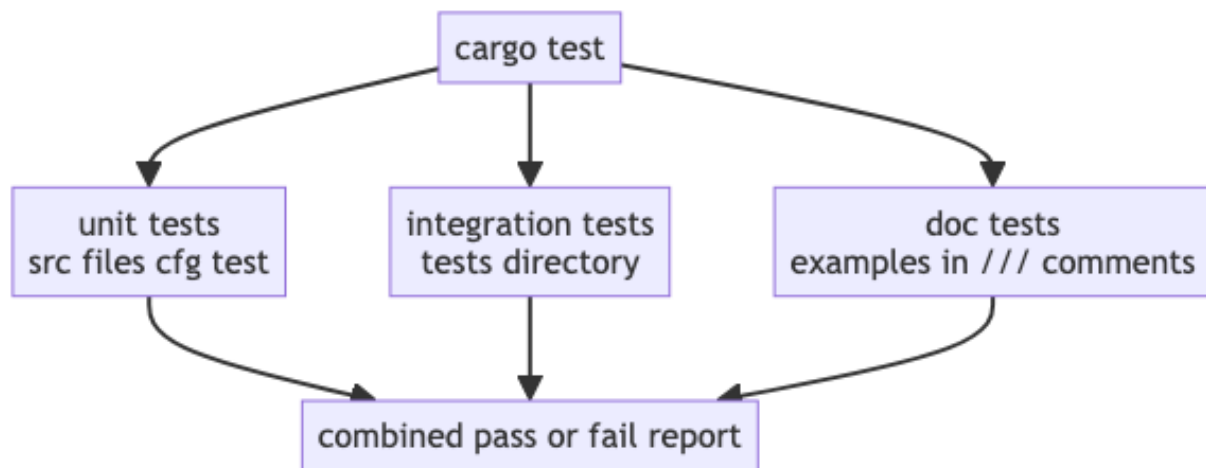
```

When you run `cargo test`, Cargo extracts that example, wraps it in a `main` function, compiles it as a tiny standalone program, and runs it. The `assert_eq!` makes the test meaningful: it proves the documented result is actually correct.

Watch out. Doc tests really compile and run, and they run as if from *outside* your crate, so they can use only public items (just like an integration test). If you change a function’s behavior, its doc-test assertion may start failing — that is the point.

If you want to *show* a line in the docs but not run it, or hide setup from the reader, prefix lines with `#` inside the code block (the `#` lines are compiled but hidden from the rendered docs). To show code that should not run at all, tag the block ```no_run` (compiled but not executed) or ```ignore` (skipped).

What `cargo test` runs, all at once



One command compiles and runs all three kinds and prints one summary. You see a line per kind, for example “test result: ok. 5 passed” for unit tests, then again for the doc tests.

Organizing tests and fixtures

A few practical patterns:

- `#[ignore]` marks a slow or special test that is skipped by default. Run it on demand with `cargo test -- --ignored`.

```

#[test]
#[ignore = "slow: hits the network"]
fn talks_to_server() {

```

```

    // run only with: cargo test -- --ignored
}

```

- **Fixtures via Default or a builder.** A *fixture* is a known starting value for a test. Rust needs no special fixture framework — derive `Default` or write a small constructor, and build your test input from it.

```

#[derive(Default)]
struct Config {
    retries: u32,
    verbose: bool,
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn uses_a_fixture() {
        let cfg = Config { retries: 3, ..Default::default() };
        assert_eq!(cfg.retries, 3);
        assert!(!cfg.verbose);
    }
}

```

- **Mocking via traits — no framework needed.** To test code that talks to the outside world (a clock, a database, the network), make it depend on a **trait** (Chapter 15 — Traits) instead of a concrete type. In tests, pass a fake implementation.

```

trait Clock {
    fn now(&self) -> u64;
}

fn is_expired(clock: &dyn Clock, deadline: u64) -> bool {
    clock.now() > deadline
}

#[cfg(test)]
mod tests {
    use super::*;

    struct FakeClock {
        fixed: u64,
    }

    impl Clock for FakeClock {
        fn now(&self) -> u64 {
            self.fixed // a fixed time we control
        }
    }
}

```

```

#[test]
fn detects_expiry() {
    let clock = FakeClock { fixed: 100 };
    assert!(is_expired(&clock, 50));
    assert!(!is_expired(&clock, 200));
}
}

```

C vs Rust. In C you might mock by swapping function pointers or using the linker to override symbols. In Rust you swap a trait implementation, checked by the compiler, with no framework and no link tricks.

Ecosystem crates for testing

The built-in tools cover most needs. A few crates fill the gaps:

Crate	Job	Why
<code>criterion</code>	benchmarking	the built-in <code>#[bench]</code> harness is nightly-only; <code>criterion</code> works on stable and gives statistical results
<code>proptest</code>	property testing	generates many random inputs and checks that a property always holds, instead of one hand-picked case
<code>assert_cmd</code>	CLI testing	runs your compiled binary and asserts on its exit code, stdout, and stderr

These go in `[dev-dependencies]` (Chapter 22 — Cargo, Crates, and Workspaces), so they are compiled only for your tests and never shipped.

Key takeaways

- Testing is built into Rust. A test is a `#[test]` function; `cargo test` finds, compiles, and runs them. No framework to choose, unlike C.
- Unit tests live in the same file inside `#[cfg(test)] mod tests { use super::*; }` and can see private items. They are excluded from normal builds.
- Use `assert!`, `assert_eq!`, `assert_ne!`; `#[should_panic]` for code that should panic; and a `Result`-returning test when you want to use `?`.
- Integration tests live in `tests/`, each file its own crate, and can use only the **public** API.
- Doc tests are code in `///` comments; `cargo test` compiles and runs them, so documentation stays correct.
- `cargo test` runs unit, integration, and doc tests together. Tests run in parallel and their output is captured unless you pass `--nocapture`.
- Mock with traits, build fixtures with `Default` or a builder, and reach for `criterion`, `proptest`, and `assert_cmd` when you need them.

Watch out (gotchas for C programmers)

- **Tests behind `#[cfg(test)]` are not shipped.** They compile only under `cargo test`, so they add nothing to your release binary.
- **Doc tests really compile and run.** A broken example in a `///` comment fails the build of your test suite — treat doc examples as real code.
- **Integration tests (and doc tests) see only the public API.** If something is not `pub`, only an in-file unit test can reach it.
- **Output is captured by default.** A passing test's `println!` is hidden. Pass `-- --nocapture` to see it.
- **Tests run in parallel by default.** Do not let two tests share mutable global state or a file; isolate them or use `-- --test-threads=1`.
- **A `#[test]` function passes by not panicking.** A failed `assert_eq!` panics, which marks the test failed; that is the whole mechanism.

Interview questions

Q: Where do unit tests and integration tests live in a Rust project, and what can each access? A: Unit tests live in the same source file, inside a `#[cfg(test)] mod tests` block, and can access private items via `use super::*`. Integration tests live in the `tests/` directory, each file compiled as its own crate, and can access only the crate's public (`pub`) API.

Q: What is a documentation test and why is it valuable? A: It is a code example inside a `///` doc comment's ````rust` block. `cargo test` compiles and runs it, so the examples in your documentation are proven to compile and produce the asserted results. It keeps docs from drifting out of sync with the code.

Q: How does `cargo test` behave by default regarding parallelism and output? A: It runs tests in parallel, each on its own thread, and captures their standard output and error, showing output only for failing tests. You can disable parallelism with `-- --test-threads=1` and show all output with `-- --nocapture`.

Q: How do you test a function that is expected to panic? A: Mark the test with `#[should_panic]`. The test passes only if the function panics. Add `#[should_panic(expected = "...")]` to require that the panic message contains a specific string, so the test does not pass for the wrong panic.

Q: How do you mock a dependency in Rust without a mocking framework? A: Make the code depend on a trait rather than a concrete type. In production, pass the real implementation; in tests, pass a small fake struct that implements the same trait with controlled behavior. The compiler checks the fake satisfies the trait, so no framework or linker tricks are needed.

Try it

1. In a `cargo new --lib mathlib`, add a `pub fn square(x: i32) -> i32`, write a `#[cfg(test)]` unit test, and run `cargo test`.
2. Add a doc example with an `assert_eq!` above `square`, run `cargo test`, and watch the doc test run. Then change the asserted answer to a wrong value and see it fail.
3. Add a `tests/api.rs` integration test that calls `mathlib::square`. Make `square` private (remove `pub`) and confirm the integration test stops compiling, proving it only sees the public API.

Chapter 24 — Tooling

What you’ll learn. The tools that ship around the Rust compiler: the formatter, the linter, the editor language server, the documentation generator, the debuggers, and the analysis tools that find undefined behavior in `unsafe` code. Each one maps to a C tool you already know.

One ecosystem, not a pile of separate tools

In C, you assemble your own toolbox. You pick a formatter (`clang-format`), a linter (`clang-tidy`, `cppcheck`), a debugger (`gdb`, `lldb`), a memory checker (Valgrind), and a profiler (`gprof`, `perf`). Each tool comes from a different project, has its own config file, and may or may not agree with the others.

Rust ships a single, coordinated set of tools. Most are installed by `rustup` (Chapter 2 — Installing Rust and the Cargo Toolchain) and run as `cargo` subcommands. They share one style, one project model, and one source of truth. This chapter is the deeper tour: Chapter 2 introduced the commands; here we focus on the tools themselves and what they do for you.

Here is the map from your C toolbox to Rust’s:

Job	C tool	Rust tool	Run with
Format code	<code>clang-format</code>	<code>rustfmt</code>	<code>cargo fmt</code>
Lint / static analysis	<code>clang-tidy</code> , <code>cppcheck</code>	<code>clippy</code>	<code>cargo clippy</code>
Editor intelligence	<code>clangd</code>	<code>rust-analyzer</code>	(your editor)
API documentation	Doxygen	<code>rustdoc</code>	<code>cargo doc</code>
Debugger	<code>gdb</code> , <code>lldb</code>	<code>rust-gdb</code> , <code>rust-lldb</code>	<code>rust-gdb ./bin</code>
Detect undefined behavior	Valgrind (memcheck)	Miri	<code>cargo +nightly miri run</code>
Profiler	<code>gprof</code> , <code>perf</code>	<code>perf</code> , <code>cargo-flamegraph</code>	<code>cargo flamegraph</code>
Benchmark	hand-rolled timers	<code>criterion</code>	<code>cargo bench</code>
Audit dependencies	(manual)	<code>cargo-audit</code> , <code>cargo-deny</code>	<code>cargo audit</code>

Mental model. In C you choose and wire up tools yourself. In Rust the tools come pre-wired and agree with each other. Less choice, less friction.

`rustfmt`: one canonical style

`rustfmt` is the official code formatter. You run it with `cargo fmt`. It rewrites your source to the single, community-wide Rust style: 4-space indentation, braces on the same line, a 100-column width, and consistent spacing everywhere.

```
cargo fmt           # reformat the whole project in place
cargo fmt --check    # report unformatted files but change nothing (use in CI)
```

The key difference from C: there is **one** style, defined by the Rust project, and almost everyone uses it unchanged. There is no argument about brace placement or tabs versus spaces, because the answer is “whatever `rustfmt` does.”

C vs Rust. `clang-format` has dozens of presets (LLVM, Google, GNU, your team’s custom `.clang-format`). Every C team picks a different one. `rustfmt` has defaults that nearly everyone accepts, so Rust code looks the same across the whole ecosystem.

You *can* tune `rustfmt` with a `rustfmt.toml` file in the project root, but most projects do not, and the set of stable options is intentionally small:

```
# rustfmt.toml (optional; most projects leave this out entirely)
max_width = 100           # the default; lines wrap at 100 columns
edition = "2024"         # match your crate's edition
```

Rule of thumb. Do not fight `rustfmt`. Run `cargo fmt` before every commit and add `cargo fmt --check` to CI. Diffs stay small because formatting never changes by accident.

clippy: the linter you should always run

`clippy` is Rust’s linter. It runs the real compiler front end and then applies **over 700 extra checks** for likely bugs, slow patterns, and code that is correct but not idiomatic. It is far more helpful than a typical C linter, and beginners should treat its output as free tutoring.

```
cargo clippy                # lint the project
cargo clippy --fix          # apply the auto-fixable suggestions
cargo clippy -- -D warnings # turn every warning into an error (CI)
```

`Clippy` groups its lints into **categories**, so you can decide how strict to be:

Category	What it catches	Default
correctness	Code that is almost certainly a bug	deny (hard error)
suspicious	Probably wrong; worth a second look	warn
style	Works, but not idiomatic Rust	warn
complexity	Needlessly complicated code	warn
perf	A faster way exists	warn
pedantic	Strict, opinionated lints	allow (opt in)
nursery	New, still-being-tuned lints	allow (opt in)

Real clippy examples

`Clippy` does not just say “this is wrong.” It names the lint, explains why, and usually shows the fix. Here are two common ones.

First, a needless comparison to `true`:

```
fn main() {
    let ready = true;
    if ready == true { // clippy: `if ready` is clearer
        println!("go");
    }
}
```

Clippy reports the `needless_bool / bool_comparison` lint and suggests `if ready`. The code compiles either way; clippy makes it idiomatic.

Second, a manual loop that should be an iterator:

```
fn main() {
    let v = vec![1, 2, 3, 4];
    let mut sum = 0;
    for i in 0..v.len() {    // clippy: `needless_range_loop`
        sum += v[i];
    }
    println!("{sum}");
}
```

Clippy flags `needless_range_loop` and suggests iterating directly: `for &x in &v { sum += x; }`. The iterator version also skips the bounds check on every index, so it can be faster.

Allowing and denying lints

You silence a specific lint where you really mean to, with an attribute. This is like a `// NOLINT` comment in clang-tidy, but type-checked and scoped.

```
#[allow(clippy::needless_range_loop)]
fn manual_copy(src: &[u8], dst: &mut [u8]) {
    for i in 0..src.len() {
        dst[i] = src[i];
    }
}
```

You can also turn the dial the other way and *deny* whole categories at the top of a crate, which is common in serious projects:

```
// at the top of src/lib.rs or src/main.rs
#![deny(clippy::all)]           // make all default clippy lints hard errors
#![warn(clippy::pedantic)]     // opt in to the strict set, as warnings
```

Rule of thumb. Run `cargo clippy` as often as `cargo check`. In CI, run `cargo clippy -- -D warnings` so a warning fails the build. Clippy is the single highest-value tool for a C programmer learning idiomatic Rust.

rust-analyzer: the editor language server

`rust-analyzer` is the **language server**: a background program your editor talks to over the Language Server Protocol (LSP). It is the Rust counterpart of `clangd`. It gives your editor:

- **Inlay type hints** — the inferred type shown next to a `let` with no annotation, so you can see what the compiler inferred.
- **Completion** — method and field suggestions that respect traits in scope.
- **Go to definition / find references** — jump around the crate and its dependencies.
- **Quick fixes** — one-key actions like “import this”, “fill match arms”, or “add the missing trait method”.
- **Inline errors** — the same diagnostics `cargo check` produces, shown as you type.

Install it as a `rustup` component (and the matching extension in your editor):

```
rustup component add rust-analyzer
rustup component add rust-src          # lets it read the std library source
```

C vs Rust. `clangd` needs a `compile_commands.json` to know your flags and include paths. `rust-analyzer` reads `Cargo.toml` directly, so there is nothing to generate — it understands the project the moment you open it.

cargo doc: documentation from comments

`rustdoc` turns your code and its comments into browsable HTML, the way Doxygen does for C. You run it through Cargo:

```
cargo doc --open          # build docs for your crate and its deps, open a browser
cargo doc --no-deps      # only your crate's docs
```

Rust has two kinds of documentation comment, both written in Markdown:

- `///` documents the **item that follows** it (a function, struct, or module).
- `//!` documents the **enclosing item** — usually placed at the top of a file to document the whole module or crate.

```
//! A tiny math helper module.
//! This text becomes the module's front page.

/// Returns the greatest common divisor of `a` and `b`.
///
/// # Examples
/// ```
/// let g = mymath::gcd(12, 18);
/// assert_eq!(g, 6);
/// ```
pub fn gcd(mut a: u64, mut b: u64) -> u64 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
```

The fenced code block inside the doc comment is not just for show. `cargo test` compiles and runs it as a **doc-test** (see Chapter 23 — Testing). That means your examples cannot rot: if the API changes and the example stops compiling, the test suite fails. C has no built-in equivalent; a Doxygen example is just text.

C vs Rust. A Doxygen `@code` block is never checked. A Rust doc example is a real test. Documentation and tests are the same artifact.

Debugging

rust-gdb and rust-lldb

Rust compiles to native code with standard debug info (DWARF), so `gdb` and `lldb` work directly. But `rustup` also installs thin wrappers — `rust-gdb` and `rust-lldb` — that load Rust-aware “pretty printers”. These show a `Vec`, `String`, or `Option` in a readable form instead of raw struct fields.

```
cargo build                # build with debug info (the default for `build`)
rust-gdb ./target/debug/myprog
rust-lldb ./target/debug/myprog
```

Many people debug from inside an editor instead. In VS Code, the **CodeLLDB** extension gives you breakpoints, watches, and stepping with the same Rust pretty printers. Set a breakpoint, hit run, and inspect variables as usual.

C vs Rust. The debugger is the same family of tool you already use; the only new thing is the `rust-` wrapper that teaches it how Rust’s standard types are laid out.

The `dbg!` macro

For quick “printf debugging”, Rust has the `dbg!` macro. It prints the file, line, the expression text, and its value to standard error, then **returns the value**, so you can wrap it around an expression without restructuring code.

```
fn main() {
    let x = 5;
    let y = dbg!(x * 2) + 1;    // prints: [src/main.rs:3:13] x * 2 = 10
    println!("y = {y}");
}
```

Because `dbg!` returns its argument, `let y = dbg!(x * 2) + 1;` still computes `y == 11`. Remove the `dbg!` calls before committing; clippy can be configured to flag them.

RUST_BACKTRACE for panics

When Rust hits an unrecoverable error it **panics** (Chapter 13 — Error Handling). By default a panic prints one line. Set the `RUST_BACKTRACE` environment variable to get a full stack trace, like a crash dump in C:

```
RUST_BACKTRACE=1 cargo run    # show a backtrace on panic
RUST_BACKTRACE=full cargo run # include std/runtime frames too

thread 'main' panicked at src/main.rs:4:30:
index out of bounds: the len is 3 but the index is 9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Watch out. Without `RUST_BACKTRACE=1` you only get the panic message and location, not the call chain. Set it the moment a panic puzzles you.

Miri: finding undefined behavior in unsafe code

This tool matters most to a C programmer. **Miri** is an interpreter for Rust’s mid-level IR. It runs your program (and tests) while watching for **undefined behavior** (UB): out-of-bounds pointer

access, use-after-free, reading uninitialized memory, data races, misaligned accesses, and violations of Rust's pointer-aliasing rules. It is the spiritual cousin of Valgrind's memcheck, but it understands Rust's *rules*, not just raw memory.

Miri only matters when you write `unsafe` (Chapter 25 — Unsafe Rust and FFI). Safe Rust cannot have UB, so Miri has nothing to catch there. It runs on nightly:

```
rustup +nightly component add miri
cargo +nightly miri test          # run the test suite under Miri
cargo +nightly miri run           # run the program under Miri

fn main() {
    let v = vec![1, 2, 3];
    let p = v.as_ptr();
    unsafe {
        // Reading p.add(3) is one past the end: out of bounds.
        // Normal runs may "work"; Miri reports this as undefined behavior.
        let _bad = *p.add(3);    // Miri error: out-of-bounds pointer use
    }
}
```

A normal `cargo run` of code like this might appear to work and print garbage, exactly like the matching C bug. Miri catches it deterministically.

C vs Rust. Valgrind checks what the hardware did. Miri checks what the Rust abstract machine *allows*. Miri is slower (it is an interpreter) and cannot run code that calls into C, but for pure-Rust `unsafe` it is the gold standard.

More analysis and inspection tools

These are extra `cargo` subcommands you install once with `cargo install`. They are not in the default toolchain but are widely used.

`cargo expand`: see macro output

Macros (Chapter 26 — Macros) and `derive` generate code you never see. `cargo expand` prints the source *after* macro expansion, so you can read what `#[derive( Debug)]` or `println!` actually produced.

```
cargo install cargo-expand
cargo expand          # print the whole crate with macros expanded
```

This is the rough equivalent of running the C preprocessor with `gcc -E`, but for Rust's much more powerful macros.

`cargo bench`: benchmarks

`cargo bench` runs functions marked as benchmarks. The built-in harness needs nightly, so most projects use the **criterion** crate on stable, which gives statistical timing and protects against the compiler optimizing your test away.

```
cargo add --dev criterion
cargo bench
```

cargo tree: the dependency graph

`cargo tree` prints your dependencies as a tree, which helps you understand why some transitive crate is in your build.

```
cargo tree           # show the dependency tree
cargo tree -i log    # invert: show what pulls in the `log` crate
cargo tree -d        # show duplicate versions of the same crate
```

cargo audit and cargo-deny: supply-chain checks

`cargo audit` checks your `Cargo.lock` against the RustSec advisory database and reports dependencies with known security vulnerabilities. `cargo-deny` goes further: it can also block disallowed licenses, banned crates, and duplicate versions, all from one config file. There is no standard C equivalent — you would track CVEs by hand.

```
cargo install cargo-audit cargo-deny
cargo audit           # report vulnerable dependencies
cargo deny check      # check advisories, licenses, and bans
```

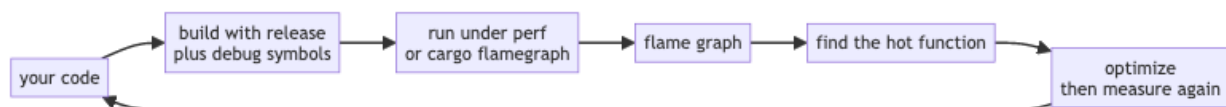
Rule of thumb. Run `cargo audit` in CI for any project with dependencies. A vulnerable transitive crate is easy to miss otherwise.

Profiling

When you need speed, measure first. Rust profiling reuses the platform’s native profilers, because Rust binaries are ordinary native code with symbols.

- **perf** (Linux) — sample where time is spent, exactly as you would for a C binary. Build with `--release` and keep debug symbols for readable names.
- **cargo flamegraph** — a `cargo install` tool that drives `perf` (or `dtrace` on macOS) and renders a flame graph SVG in one step.
- **criterion** — for micro-benchmarks of a single function, with statistics.

```
cargo install flamegraph
cargo flamegraph --release # produces flamegraph.svg
```



Watch out. Always profile and benchmark a `--release` build. A debug build is unoptimized and adds overflow checks, so its timings are meaningless for performance work. This is the same trap as profiling a C binary built with `-O0`.

Sanitizers (nightly)

Rust can also use the LLVM **sanitizers** you may know from `clang`: AddressSanitizer (ASan), ThreadSanitizer (TSan), and others. They are nightly-only and turned on with a compiler flag. They are most useful for **unsafe** code and FFI boundaries, where Miri cannot reach (for example, when C code is involved).

```
RUSTFLAGS="-Zsanitizer=address" cargo +nightly run --target x86_64-unknown-linux-gnu
```

Rule of thumb. Use **Miri** for pure-Rust **unsafe**, and **sanitizers** when the bug may cross into C through FFI. Together they cover what Valgrind does for C.

Key takeaways

- Rust’s tools ship together and agree with each other. Most run as **cargo** subcommands and map cleanly onto C tools you know.
- **rustfmt** (**cargo fmt**) enforces one canonical style; tune it with an optional **rustfmt.toml**, but most projects do not.
- **clippy** (**cargo clippy**) is a 700+ lint checker grouped into categories. Silence a lint with **#[allow(...)]**; in CI run **cargo clippy -- -D warnings**.
- **rust-analyzer** is the editor language server: inlay type hints, completion, go-to-definition, and quick fixes, with no **compile_commands.json** to generate.
- **rustdoc** (**cargo doc**) builds HTML from **///** and **//!** comments, and the examples are run as doc-tests.
- Debug with **rust-gdb/rust-lldb** or VS Code + CodeLLDB, the **dbg!** macro, and **RUST_BACKTRACE=1** for panic backtraces.
- **Miri** detects undefined behavior in **unsafe** Rust; **perf/cargo flamegraph/criterion** profile and benchmark; **cargo audit/cargo-deny** guard dependencies.

Watch out (gotchas for C programmers)

- **Clippy is gold — run it.** It teaches idiomatic Rust faster than anything else. Make it part of your edit loop and your CI.
- **RUST_BACKTRACE=1 for panics.** Without it you only see the message and location, never the call chain.
- **Miri finds unsafe UB.** Safe Rust has none to find. If you write **unsafe**, run **cargo +nightly miri test**. Miri cannot cross into C — use sanitizers there.
- **Debug vs release performance.** Profile and benchmark only **--release** builds; debug builds are unoptimized and add overflow checks.
- **rustfmt is non-negotiable on teams.** Run **cargo fmt** before committing and **cargo fmt --check** in CI, so formatting never shows up in diffs or reviews.
- **dbg! returns its argument.** Handy, but remove the calls before committing.

Interview questions

Q: What is the difference between cargo check, cargo clippy, and cargo fmt? A: **cargo check** runs the compiler front end (parse, type-check, borrow-check) to report errors without building a binary. **cargo clippy** runs that same front end plus 700+ extra lints for likely bugs and un-idiomatic code. **cargo fmt** does not analyze meaning at all; it only reformats source to the canonical style.

Q: When would you use Miri, and why can it find bugs that a normal run cannot? A: Use Miri when you write **unsafe** code. It interprets the program against Rust’s abstract machine and flags undefined behavior — out-of-bounds pointers, use-after-free, uninitialized reads, aliasing violations — that a normal run might silently “get away with”. Safe Rust has no UB, so Miri is unnecessary there.

Q: How do /// and //! doc comments differ, and what makes Rust docs special? A: **///** documents the item that follows it; **//!** documents the enclosing item, typically a module or

crate at the top of a file. Both are Markdown. The special part is that fenced code examples in doc comments are compiled and run as tests by `cargo test`, so documentation cannot silently fall out of date.

Q: A C colleague asks how Rust replaces Valgrind, gprof, and clang-tidy. What do you say? A: clang-tidy maps to clippy (`cargo clippy`). gprof maps to native profilers like perf and cargo flamegraph. Valgrind splits in two: Miri detects undefined behavior in pure-Rust `unsafe` code, and the LLVM sanitizers (nightly) cover cases that cross into C through FFI.

Q: Why does Rust not have the brace-style and tabs-versus-spaces arguments that C teams have? A: Because rustfmt defines one canonical style that nearly the whole ecosystem adopts unchanged. Teams run `cargo fmt` automatically, so formatting is mechanical and never debated in review, unlike `clang-format` where each project picks a different preset.

Try it

1. In any project, run `cargo clippy` and read every suggestion. Apply the auto-fixable ones with `cargo clippy --fix`.
2. Write a function with a `for i in 0..v.len()` loop over a slice and watch clippy suggest the iterator form. Then add `#[allow(clippy::needless_range_loop)]` and confirm the warning disappears.
3. Add a `/// # Examples` doc-test to a function, then run `cargo test` and find the doc-test in the output. Break the example on purpose and watch the test fail.

Chapter 25 — Unsafe Rust and FFI

What you'll learn. What the `unsafe` keyword really turns on (and what it does not), how raw pointers work, and how Rust and C call each other across the foreign function interface. This is the chapter where Rust looks most like the C you already write.

Why this chapter matters to a C programmer

Everything you have learned so far is **safe Rust**: the compiler proves there are no dangling pointers, no data races, no buffer overruns. But sometimes you must step outside those proofs — to talk to the operating system, to call a C library, to build a data structure the borrow checker cannot understand, or to squeeze out the last bit of performance.

`unsafe` is how you do that. It does not make Rust into a different language. It hands you a small set of extra powers and says: “the compiler can no longer prove this is correct, so *you* must guarantee it, the way you always have in C.”

Mental model. Safe Rust is driving with every safety system on. `unsafe` is the override switch for five specific maneuvers. The car still has brakes and a steering wheel — you are just responsible for these five things by hand.

What `unsafe` actually allows: the five superpowers

This is the single most misunderstood part of Rust. `unsafe` unlocks exactly **five** abilities, and nothing else:

1. **Dereference a raw pointer** (`*const T` or `*mut T`).

2. **Call an unsafe function** (including any function from C through FFI).
3. **Access or modify a mutable static** variable (a global).
4. **Implement an unsafe trait** (such as `Send` or `Sync`).
5. **Access the fields of a union.**

That is the whole list. Critically:

Watch out. `unsafe` does **not** turn off the borrow checker. References are still checked, moves still apply, lifetimes still hold, and ownership still frees your memory. `unsafe` only permits the five actions above. Everything else is exactly as strict as before.

So `unsafe` is much narrower than “C mode”. It is a promise you make to the compiler: “I have checked, by hand, that the invariants these five powers require are upheld.” If you are wrong, you get **undefined behavior** (UB) — the same class of bug you fight in C.

```
fn main() {
    let mut x = 10;
    let r: *mut i32 = &mut x;    // creating a raw pointer is SAFE
    unsafe {
        *r += 1;                // dereferencing it needs `unsafe`
    }
    println!("{}", x);           // prints 11
}
```

Notice the split: making the raw pointer is safe, because nothing bad happens until you *use* it. The dereference is the dangerous step, so that is what needs an `unsafe` block.

Rule of thumb. Keep `unsafe` blocks as small as possible — wrap only the one operation that needs the power, not the whole function. A small block is a small thing to audit.

Raw pointers: C pointers, back again

A **raw pointer** is Rust’s version of a plain C pointer. There are two kinds:

- `*const T` — a pointer to a `T` you intend to only read (like `const T *`).
- `*mut T` — a pointer to a `T` you may write (like `T *`).

Unlike references (`&T` / `&mut T`), raw pointers carry **no guarantees**. They may be null, may dangle, may be unaligned, and may alias each other freely. The compiler tracks nothing about them — exactly like a C pointer.

Property	C pointer <code>T *</code>	Rust reference <code>&T</code>	Rust raw pointer <code>*const T</code>
Can be null	yes	no	yes
Can dangle	yes	no (checked)	yes
Bounds known	no	yes (slices)	no
Aliasing checked	no	yes	no
Needs <code>unsafe</code> to deref	no	no	yes

You create raw pointers in **safe** code, from a reference or from an address:

```
fn main() {
    let x = 42;
    let p: *const i32 = &x;           // from a reference: always valid here
    let q = 0x1234 as *const i32;     // from an address: probably invalid!

    unsafe {
        println!("{}", *p);           // 42 - p points at real data
        // println!("{}", *q);         // would be UB: q is a bogus address
    }
    let _ = q;
}
```

Pointer arithmetic

In C you write `p + 3` to move a pointer forward by three elements. In Rust you use the `.add` and `.offset` methods, which advance by *elements*, not bytes — the compiler multiplies by `size_of::<T>()` for you, just as C does:

```
fn main() {
    let arr = [10, 20, 30, 40];
    let base: *const i32 = arr.as_ptr();
    unsafe {
        let third = *base.add(2);     // base[2] == 30
        println!("{}", third);
    }
}
```

`.add(n)` moves forward by `n` elements; `.offset(n)` takes a signed count and can move backward. Both are `unsafe` because the compiler cannot check you stayed in bounds — going past the end is UB, the same as in C.

Here is the picture, the same one you would draw for a C array:

```
arr:  [ 10 | 20 | 30 | 40 ]
      ^       ^
      |       |
      base    base.add(2)  -> *base.add(2) == 30
```

each cell is `size_of::<i32>() == 4` bytes
`base.add(n)` advances `n * 4` bytes

C vs Rust. Raw pointers behave like C pointers in every way. The only difference is that Rust forces the dangerous operations — dereference and arithmetic-then-deref — into an `unsafe` block, so they are easy to find and review.

Calling C from Rust

This is the most common reason to use `unsafe`: calling an existing C library. You declare the C functions in an `extern` block. In **edition 2024** the block itself is marked `unsafe`, because the compiler cannot verify a foreign function honors Rust's rules:

```
// edition 2024 form: the extern block is `unsafe`
unsafe extern "C" {
    fn abs(input: i32) -> i32;    // from the C standard library
}

fn main() {
    let n = -7;
    let a = unsafe { abs(n) };    // calling C is an unsafe operation
    println!("abs({n}) = {a}");
}
```

Two layers of `unsafe` appear here, and they mean different things:

- `unsafe extern "C" { ... }` — the **declaration** is unsafe: you are vouching that these signatures match the real C functions.
- `unsafe { abs(n) }` — the **call** is unsafe: every foreign function is treated as an `unsafe fn`.

`extern "C"` means “use the C **ABI**” — the calling convention and layout rules that C uses on this platform. It is what lets the two languages agree on how arguments are passed.

C types in Rust

Rust integers like `i32` have fixed sizes, but C types like `int` and `long` vary by platform. To match C exactly, use the C type aliases from `core::ffi` (also re-exported in `std::os::raw`):

C type	Rust type (<code>core::ffi</code>)
<code>int</code>	<code>c_int</code>
<code>unsigned int</code>	<code>c_uint</code>
<code>long</code>	<code>c_long</code>
<code>char</code>	<code>c_char</code>
<code>void *</code>	<code>*mut c_void</code>
<code>size_t</code>	<code>usize</code> (commonly)

```
use core::ffi::c_int;

unsafe extern "C" {
    fn abs(input: c_int) -> c_int;    // matches C's `int abs(int)`
}

fn main() {
    let a = unsafe { abs(-5) };
    println!("{a}");
}
```

Strings across the boundary: CStr and CString

This trips up everyone. A Rust `String` is UTF-8 and stores its length; it is **not** NUL-terminated. A C string is a `char *` that ends at the first `\0`. They are not interchangeable. Rust gives you two bridge types:

- `CString` — an owned, NUL-terminated byte buffer you build to *pass into* C.
- `CStr` — a borrowed view of a NUL-terminated string you got *back from* C.

```
use core::ffi::{c_char, c_int};
use std::ffi::CString;

unsafe extern "C" {
    fn puts(s: *const c_char) -> c_int;    // C's puts: prints a NUL-terminated string
}

fn main() {
    let msg = CString::new("hello from C's puts").unwrap();
    unsafe {
        puts(msg.as_ptr());    // pass a `*const c_char` into C
    }
    // `msg` owns the buffer and frees it at end of scope; keep it alive
    // for as long as C might read the pointer.
}
```

`CString::new` returns an error if the input contains an interior `\0`, because that byte would prematurely terminate the C string. It adds the trailing `\0` for you.

Watch out. Keep the `CString` alive while C holds the pointer. If you write `puts(CString::new("hi").unwrap().as_ptr())`, the `CString` is dropped at the end of that statement and C reads freed memory — a classic dangling pointer, exactly as in C.

Linking to a library

The `#[link]` attribute names the C library to link against, like `-l` on the C compiler command line. The C standard library is linked automatically, so the examples above need nothing extra; a third-party library does:

```
#[link(name = "z")]    // link libz (zlib), like cc ... -lz
unsafe extern "C" {
    fn crc32(crc: u32, buf: *const u8, len: u32) -> u32;
}
```

Calling Rust from C

The reverse direction lets C code call your Rust function. You must do three things: use the C ABI, keep the symbol name unmangled, and build the right kind of library.

```
#[unsafe(no_mangle)]
pub extern "C" fn add_one(x: i32) -> i32 {
    x + 1
}
```

- `extern "C"` — export with the C calling convention.
- `#[unsafe(no_mangle)]` — keep the symbol name exactly `add_one` so the C linker can find it. Rust normally **mangles** names (encodes them for uniqueness). In edition 2024 this attribute

is wrapped in `unsafe(...)`, because a clashing unmangled name can cause incorrect linking — another invariant you promise to uphold.

- `pub` — make it visible outside the crate.

On the C side, you write the matching declaration by hand (or generate it; see below) and call it like any C function:

```
/* main.c */
#include <stdio.h>

extern int add_one(int x); /* the Rust function */

int main(void) {
    printf("%d\n", add_one(41)); /* prints 42 */
    return 0;
}
```

To produce something C can link, set the crate type in `Cargo.toml`:

```
[lib]
crate-type = ["staticlib"] # a .a archive to link statically
# or:
# crate-type = ["cdylib"] # a .so / .dll / .dylib shared library
```

- `staticlib` builds a `.a` you link into a C program statically.
- `cdylib` builds a shared library (`.so`, `.dll`, or `.dylib`) you load at runtime.

cbindgen: generate the C header

Writing the C declarations by hand is error-prone. The **cbindgen** tool reads your Rust source and generates a matching C/C++ header file, so the two sides cannot drift apart.

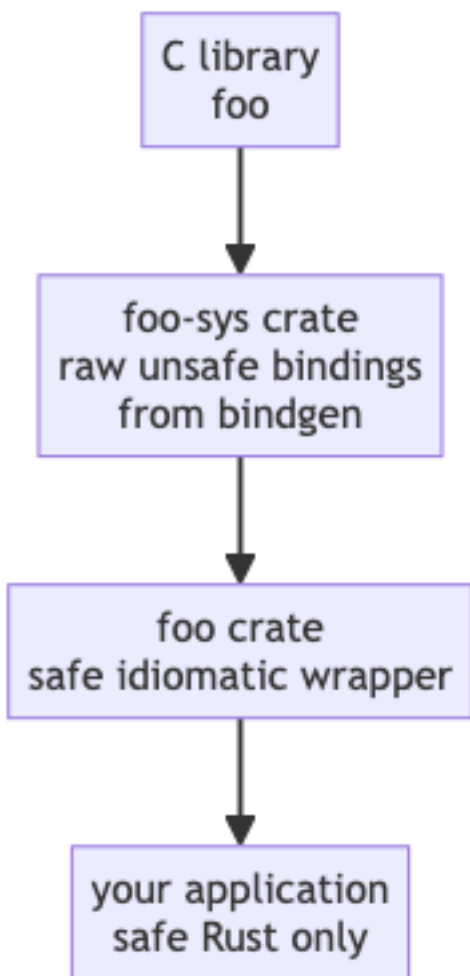
```
cargo install cbindgen
cbindgen --output mylib.h # generate a header from your Rust exports
```

bindgen and the *-sys convention

Going the other way — wrapping a big C library — you do not want to hand-type hundreds of `extern` declarations. **bindgen** reads a C header and generates the Rust `extern` blocks, struct definitions, and constants automatically.

```
cargo install bindgen-cli
bindgen /usr/include/somelib.h -o src/bindings.rs
```

The Rust ecosystem has a strong convention here. A crate named `foo-sys` contains **only** the raw, generated, `unsafe` bindings to the C library `foo`. A separate crate named `foo` then builds a **safe**, idiomatic wrapper on top. This split — raw bindings below, safe API above — is the standard pattern.



Wrapping `unsafe` in a safe abstraction

This is the most important idea in the whole chapter. The goal is not to avoid `unsafe` — it is to **contain** it. You write a small `unsafe` core, check its invariants carefully, and expose a **safe** function that cannot be misused. This is exactly how the standard library is built: `Vec`, `String`, and `Mutex` are all safe wrappers around `unsafe` internals.

A safe wrapper means: no matter what arguments a caller passes, they can never trigger UB. Here is the shape of the pattern, splitting one slice into two halves without copying:

```
fn split_at_mut(slice: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = slice.len();
    let ptr = slice.as_mut_ptr();
    assert!(mid <= len);    // upholds the invariant the unsafe code relies on

    unsafe {
        // SAFETY: `mid <= len` is checked above, so both ranges stay in bounds,
        // and the two resulting slices cover disjoint regions, so the aliasing
        // rule (no two overlapping &mut) is not violated.
    }
}
```

```

        (
            std::slice::from_raw_parts_mut(ptr, mid),
            std::slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}

fn main() {
    let mut v = [1, 2, 3, 4, 5, 6];
    let (a, b) = split_at_mut(&mut v, 3);
    a[0] = 100;
    b[0] = 200;
    println!("{v:?}");    // [100, 2, 3, 200, 5, 6]
}

```

The function signature is fully safe: callers see only `&mut [i32]`. The `assert!` turns a would-be UB (an out-of-bounds `mid`) into a clean panic. The borrow checker alone could not prove the two `&mut` slices are non-overlapping, which is why `unsafe` is needed — but we, the authors, can prove it, and we say so.

Write `// SAFETY:` comments

Every `unsafe` block should carry a comment, conventionally starting with `// SAFETY:`, that states *why* the invariants hold. This is the audit trail. When someone reviews or changes the code, the comment tells them exactly what must stay true. Treat a missing `// SAFETY:` comment the way you would treat a `free()` with no clear owner — a smell.

`#[repr(C)]` for layout compatibility

By default Rust may reorder struct fields for efficiency. C does not. So any struct you pass across the FFI boundary must be annotated `#[repr(C)]`, which forces C’s field order and padding rules (see Chapter 11 — Structs and Methods):

```

#[repr(C)]
pub struct Point {
    pub x: i32,
    pub y: i32,
}

unsafe extern "C" {
    fn translate(p: Point, dx: i32, dy: i32) -> Point;
}

```

Without `#[repr(C)]`, Rust’s field layout is unspecified, and C would read the wrong bytes. This is the same care you take in C when matching a struct to a binary format.

The big picture: Rust and C coexisting

FFI is a two-way door. The boundary is where safe Rust meets unchecked C.

SAFE RUST | the FFI boundary | C

-----	extern C -----	
&T &mut T Vec String	<----->	int char * struct
checked, no UB possible	unsafe required	you uphold invariants
	repr C c_int	
	CString CStr	

Because of this door, you do not have to rewrite a C program all at once. You can replace one module at a time: build a Rust `staticlib`, link it into the existing C binary, and move functionality across function by function. Many real migrations (including parts of Firefox and the Linux kernel) work exactly this way.

C vs Rust. Inside `unsafe` and across FFI, you are essentially writing C-level code again — same pointers, same responsibility, same UB if you get it wrong. The win is that this dangerous code is small, clearly marked, and wrapped in a safe API, instead of being your *entire* program.

Key takeaways

- `unsafe` unlocks exactly five powers: dereference a raw pointer, call an `unsafe` function, access a mutable `static`, implement an `unsafe` trait, and read a `union` field. Nothing else.
- `unsafe` does **not** disable the borrow checker. You still get ownership, borrowing, and lifetime checks; you only gain those five actions and the duty to uphold their invariants by hand.
- Raw pointers `*const T/*mut T` are C pointers: no null check, no bounds, no aliasing rules. Creating them is safe; dereferencing and `.add/.offset` need `unsafe`.
- Call C with `unsafe extern "C" { ... }` (edition 2024), use `core::ffi` types like `c_int`, link with `#[link(name = "...")]`, and bridge strings with `CString/CStr`.
- Expose Rust to C with `#[unsafe(no_mangle)] pub extern "C" fn`, build a `staticlib` or `cdylib`, and generate headers with `cbindgen`.
- Use `bindgen` and the `*-sys/safe-wrapper` convention to wrap C libraries. Always wrap `unsafe` in a safe API, mark FFI structs `#[repr(C)]`, and write `// SAFETY:` comments.

Watch out (gotchas for C programmers)

- **`unsafe` does not disable the borrow checker.** It is not “C mode”. It only permits five specific operations.
- **Breaking an invariant is UB.** A wrong `unsafe` block is just as dangerous as a C memory bug. Run `cargo +nightly miri test` (Chapter 24 — Tooling) to catch it.
- **`CString` NUL handling.** A Rust `String` is not NUL-terminated. Build a `CString` to pass into C, and never let it drop while C still holds the pointer.
- **`#[repr(C)]` on FFI structs.** Without it, Rust may reorder fields and C reads garbage.
- **Pointer lifetimes cross the boundary blindly.** The compiler cannot track a pointer once it enters C. Keep the owning Rust value alive yourself.
- **`#[unsafe(no_mangle)]` is the edition-2024 spelling.** Older code writes plain `#[no_mangle]`; on edition 2024 it must be wrapped in `unsafe(...)`.

Interview questions

Q: Does `unsafe` turn off the borrow checker? A: No. `unsafe` enables exactly five extra operations: dereferencing raw pointers, calling `unsafe` functions, accessing mutable statics, implementing `unsafe` traits, and reading `union` fields. The borrow checker, ownership, moves, and lifetimes all

still apply. You take on the duty to uphold the invariants those five operations require, but every other rule is unchanged.

Q: What is the difference between a reference and a raw pointer? A: A reference (`&T`/`&mut T`) is guaranteed by the compiler to be non-null, properly aligned, pointing at live data, and to obey the aliasing rule. A raw pointer (`*const T`/`*mut T`) has none of these guarantees — it is a C pointer. Creating a raw pointer is safe, but dereferencing one requires an `unsafe` block.

Q: Why can't you pass a Rust String directly to a C function expecting char *? A: A Rust `String` is UTF-8 and stores its length; it is not NUL-terminated, and it may contain interior NUL bytes. C expects a NUL-terminated buffer. You convert with `CString::new`, which appends the terminating NUL (and errors on interior NULs), then pass `as_ptr()` while keeping the `CString` alive.

Q: What does `#[repr(C)]` do, and when do you need it? A: It forces a struct (or enum/union) to use C's field order, alignment, and padding instead of Rust's default, which is unspecified and may reorder fields. You need it on any type passed across the FFI boundary so that Rust and C agree on the exact byte layout.

Q: How do you safely expose unsafe code to other programmers? A: Wrap it in a safe abstraction: a function with a fully safe signature whose body contains a small `unsafe` block, guarded by checks (like an `assert!`) so that no input can trigger undefined behavior. Document each block with a `// SAFETY:` comment explaining why the invariants hold. This is how `Vec`, `String`, and the rest of the standard library are built.

Try it

1. Declare `unsafe extern "C" { fn sqrt(x: f64) -> f64; }`, call it inside an `unsafe` block from `main`, and print the result. (The C math library is linked automatically.)
2. Build the `split_at_mut` example, then change the `assert!` to allow `mid > len` and run it under `cargo +nightly miri run`. Watch Miri report the out-of-bounds access.
3. Write `#[unsafe(no_mangle)] pub extern "C" fn triple(x: i32) -> i32 { x * 3 }`, set `crate-type = ["staticlib"]`, build it, and call it from a small C program.

Chapter 26 — Macros

What you'll learn. What macros are and why Rust has them, how `macro_rules!` writes code from patterns, why the macros you already use (`println!`, `vec!`, `assert!`) *must* be macros, and a tour of procedural macros like `#[derive(...)]`. Throughout, how all of this beats the C preprocessor.

What a macro is, and why a language needs one

A **macro** is code that writes code. The compiler runs the macro at compile time, the macro produces more source code, and that produced code is then compiled as if you had typed it yourself. This is called **metaprogramming**: a program that generates a program.

You already know one macro system: the **C preprocessor**. In C you write `#define`, `#include`, and `#ifdef`, and a separate pass edits your text before the real compiler ever sees it.

Why have macros at all? A normal function cannot do some things:

- **Variadic, type-checked interfaces.** `println!("{}", a, b)` takes any number of arguments of different types and checks the format string against them *at compile time*. A normal Rust function cannot take a variable number of arguments like C's `printf`.
- **New syntax (small DSLs).** A **DSL** (domain-specific language) is a mini language for one job. `vec![1, 2, 3]` and `vec![0; 100]` are tiny DSLs for building a vector.
- **Removing boilerplate.** Generating repetitive code (trait implementations, table entries) without copy-paste.

C vs Rust. The C preprocessor does **raw text substitution**. It does not understand C: not types, not scopes, not even that your tokens form valid expressions. Rust macros are different. They operate on **structured tokens** (the compiler's view of the code), they respect scope, and they are **hygienic** (defined below). They cannot silently corrupt your program the way a careless `#define` can.

Rust has two kinds of macro:

- **Declarative macros**, written with `macro_rules!`. You match patterns of tokens and say what to expand them into. This is most of what you write.
- **Procedural macros**, which are Rust functions that take tokens and return tokens. They include `#[derive(...)]`, attribute macros, and function-like macros. They live in their own special crate.

The C preprocessor, and its traps

First, a refresher on why C macros are dangerous, so you can see what Rust fixes. Here is the classic broken MAX:

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main(void) {
    int i = 5;
    int m = MAX(i++, 3);    /* i++ is evaluated TWICE: i becomes 7, not 6 */
    return m;
}
```

The preprocessor pasted `i++` into the body twice. There is no concept of “evaluate the argument once” because there is no concept of an *expression* — only text. Other classic traps:

```
#define SQUARE(x) x * x
int a = SQUARE(2 + 3);    /* expands to 2 + 3 * 2 + 3 = 11, not 25 */

#define DOUBLE(x) ((x) + (x))
int n = 0;
int d = DOUBLE(n++);      /* n++ twice again: undefined behavior */
```

And the **hygiene** problem. **Hygiene** means a macro's own names cannot accidentally collide with names at the call site. C macros are *not* hygienic:

```
#define SWAP(a, b) { int tmp = a; a = b; b = tmp; }

int main(void) {
    int x = 1, tmp = 2;
```

```

    SWAP(x, tmp);           /* the macro's `tmp` clashes with your `tmp` */
    return x;               /* broken: the names collide */
}

```

Rust macros do not have any of these problems, as we will now see.

Declarative macros with `macro_rules!`

A declarative macro looks like a **match**: you write **rules**, each with a **matcher** (a pattern of tokens) on the left and a **transcriber** (the code to produce) on the right.

Here is the smallest useful example — **square!**, which evaluates its argument exactly once and is correctly parenthesized:

```

macro_rules! square {
    ($x:expr) => {
        $x * $x
    };
}

fn main() {
    let n = square!(2 + 3);    // expands to (2 + 3) * (2 + 3) = 25, correctly
    println!("{n}");
}

```

`$x:expr` is a **metavariable**. The `$x` is the name; `expr` is a **fragment specifier** that says “match a whole expression here.” Because Rust captured a real expression — not raw text — `2 + 3` stays together as one unit. The **SQUARE** bug from C cannot happen.

C vs Rust. A C macro argument is text and you must defensively wrap it in parentheses. A Rust `$x:expr` is a complete, parsed expression. It already behaves as one value, so `square!(2 + 3)` is never re-parsed wrongly.

Fragment specifiers (the `:kind` part)

The fragment specifier tells the macro what kind of token to match. The common ones:

Specifier	Matches	Example match
<code>expr</code>	an expression	<code>2 + 3</code> , <code>foo()</code> , <code>x</code>
<code>ident</code>	an identifier (a name)	<code>count</code> , <code>Widget</code>
<code>ty</code>	a type	<code>i32</code> , <code>Vec<u8></code>
<code>literal</code>	a literal value	<code>42</code> , <code>"hi"</code> , <code>true</code>
<code>pat</code>	a pattern (for <code>match/let</code>)	<code>Some(x)</code> , <code>_</code>
<code>block</code>	a <code>{ ... }</code> block	<code>{ do_it(); }</code>
<code>stmt</code>	a statement	<code>let y = 1</code>
<code>path</code>	a path	<code>std::mem::swap</code>
<code>tt</code>	a single token tree (most flexible)	almost anything

Repetition: matching many arguments

To accept a variable number of arguments, you use a **repetition**. The syntax is `$( ... )sep rep`, where `sep` is an optional separator and `rep` is `*` (zero or more), `+` (one or more), or `?` (zero or one).

`$( $x:expr ),*` means “zero or more expressions, separated by commas.” You then repeat the expansion with the same `$( ... )*` shape. Here is a small `my_vec!` that mimics the real `vec!`:

```
macro_rules! my_vec {
    // empty: my_vec![]
    () => {
        Vec::new()
    };
    // one or more elements: my_vec![a, b, c]
    ( $( $x:expr ),+ $(,)? ) => {{
        let mut v = Vec::new();
        $(
            v.push($x);
        )+
        v
    }};
}

fn main() {
    let a: Vec<i32> = my_vec![];
    let b = my_vec![1, 2, 3];
    let c = my_vec![10, 20,]; // trailing comma allowed by $(,)?
    println!("{a:?} {b:?} {c:?}");
}
```

Three things to notice:

- The matcher `$( $x:expr ),+` captures the list. The transcriber `$( v.push($x); )+` repeats once per captured expression. The repetition on the right must use the same metavariable as the left.
- `$(,)?` allows an optional trailing comma, like `vec![1, 2, 3,]`.
- The body uses **double braces** `{{ ... }}`. The outer braces are the macro’s transcriber; the inner braces make a **block expression** so the macro expands to a single value (the `v` on the last line).

A `hashmap!` macro follows the same shape and shows a `key => value` separator:

```
use std::collections::HashMap;

macro_rules! hashmap {
    ( $( $key:expr => $val:expr ),* $(,)? ) => {{
        let mut m = HashMap::new();
        $(
            m.insert($key, $val);
        )*
        m
    }}
```

```

    }
}

fn main() {
    let scores = hashmap!{
        "alice" => 10,
        "bob"   => 7,
    };
    println!("{:?}", scores.get("alice"));
}

```

Hygiene: the macro's own names do not leak

Remember the broken C `SWAP` where `tmp` collided. Watch the Rust version. The macro introduces a variable called `tmp`, and the caller *also* has a `tmp`. They do **not** clash:

```

macro_rules! swap {
    ($a:expr, $b:expr) => {{
        let tmp = $a;
        $a = $b;
        $b = tmp;
    }};
}

fn main() {
    let mut x = 1;
    let mut tmp = 2;           // same name the macro uses internally
    swap!(x, tmp);             // works correctly; no collision
    println!("x={x} tmp={tmp}"); // x=2 tmp=1
}

```

The `tmp` inside the macro and the `tmp` in `main` are treated as different names, because the compiler tracks where each identifier came from. This is **hygiene**, and C macros do not have it.

Mental model. Think of each macro expansion as getting its own private namespace painted a different color. The macro's `tmp` is “blue”; your `tmp` is “red.” They never mix, even though they are spelled the same.

Watch out. Hygiene is helpful but it has an edge: a macro normally cannot *invent* a name and hand it back for you to use, because that name is in the macro's color, not yours. If you need a name to cross the boundary, the caller must pass it in as an **ident**.

Where macros are visible

A `macro_rules!` macro is in scope **after** the point where it is defined (textually), which is unlike normal items. To use it across modules or export it from a library, mark it `#[macro_export]`:

```

#[macro_export]
macro_rules! my_vec {
    // ...
}

```

```
() => { Vec::new() };
}
```

This puts the macro at the crate root so other crates can use `your_crate::my_vec;`.

The macros you already use (and why they must be macros)

You have been using macros since Chapter 1. Now you can see *why* each one is a macro and not a function.

Macro	Why it cannot be a plain function
<code>println! / print!</code>	Variable number of arguments; checks the format string against the arguments at compile time.
<code>format!</code>	Same as <code>println!</code> , but builds a <code>String</code> instead of printing.
<code>vec!</code>	Takes any number of elements, or the <code>vec! [x; n]</code> form; builds a <code>Vec</code> .
<code>assert! / assert_eq!</code>	On failure, prints the source text of the failed expression and the file/line — only a macro can see that text.
<code>panic!</code>	Variadic, format-string aware, and records the call location.
<code>dbg!</code>	Prints the expression’s source text <i>and</i> its value, then returns the value.

The format-checking point is worth stressing. This is caught at **compile time**, before the program runs:

```
// COMPILE ERROR: 1 positional argument in format string, but no arguments were given
fn main() {
    let _ = format!("value = {}"); // error: missing argument for `{}`
}
```

A C `printf` cannot do this in general; a wrong format specifier is at best a warning and at worst undefined behavior at runtime. The Rust macro turns it into a hard compile error.

```
/* C: this compiles; it is undefined behavior at runtime */
printf("%d\n", "not a number"); /* wrong type for %d */
```

Procedural macros (an overview — you will mostly *use* them)

Procedural macros (“proc macros”) are the more powerful kind. A proc macro is a Rust function that receives a stream of tokens and returns a new stream of tokens. Because it is ordinary Rust code, it can do arbitrary work — parse the input, inspect a struct’s fields, and build new code.

Three forms exist:

- **Derive macros** — `#[derive(Debug, Clone, Serialize)]`. They generate a trait implementation for the type they are attached to. This is by far the most common form you will meet.

- **Attribute macros** — `#[tokio::main]`, `#[test]`, web route attributes like `#[get("/users")]`. They wrap or transform the item they decorate.
- **Function-like macros** — `sql!(SELECT * FROM users)`. They look like `macro_rules!` calls but are backed by a proc-macro function.

You will **use** proc macros far more often than you write them. The everyday example is `serde`, which turns a plain struct into something that can be converted to and from JSON with one line:

```
// needs: cargo add serde --features derive and cargo add serde_json
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
struct Config {
    name: String,
    retries: u32,
}

fn main() {
    let c = Config { name: "db".into(), retries: 3 };
    let json = serde_json::to_string(&c).unwrap(); // real code handles the error
    println!("{}", json);                        // {"name": "db", "retries": 3}
}
```

The `#[derive(Serialize, Deserialize)]` line ran a proc macro that generated all the serialization code for `Config` at compile time. You wrote none of it.

How proc macros are built (just so you recognize it)

Two crates do the heavy lifting, and you will see them in any proc-macro project:

- **syn** parses the incoming tokens into a syntax tree you can inspect.
- **quote** builds the outgoing tokens from a template, with `#var` interpolation.

A proc macro must live in **its own crate**, marked in `Cargo.toml`:

```
[lib]
proc-macro = true
```

This separate-crate rule exists because a proc macro is compiled and run *by the compiler* while it compiles your program. It is a compiler plugin, so it cannot sit in the same crate as the code that uses it.

Deep dive. A minimal derive macro signature looks like this. You do not need to memorize it; just recognize the shape.

```
// in a crate with proc-macro = true
use proc_macro::TokenStream;

#[proc_macro_derive(Hello)]
pub fn derive_hello(input: TokenStream) -> TokenStream {
    // parse `input` with syn, build output with quote, return tokens
}
```

```

    TokenStream::new()    // (a real one returns generated code)
}

```

C macros vs Rust macros, side by side

Feature	C preprocessor	macro_rules!	Rust proc macros
Operates on	raw text	structured tokens	structured tokens / syntax tree
Hygienic	no	yes	mostly (you control spans)
Understands types	no	partly (fragments)	yes (can inspect a struct)
Argument evaluated once	no (you must be careful)	yes (\$x:expr is one value)	yes
Scoped / namespaced	no (global text)	yes (module-scoped)	yes
Can read a struct's fields	no	no	yes (derive macros do)
Where it lives	inline #define	inline or #[macro_export]	its own proc-macro crate
Debuggable	cpp -E shows text	cargo expand	cargo expand

The X-macro: a C trick Rust does better

C programmers use the **X-macro** pattern to keep a list and its generated code in sync — for example, an enum and its name table:

```

/* the single list */
#define COLORS  X(RED) X(GREEN) X(BLUE)

enum Color {
#define X(name) name,
    COLORS
#undef X
};

const char *color_name(enum Color c) {
    switch (c) {
#define X(name) case name: return #name;
        COLORS
#undef X
    }
    return "?";
}

```

It works, but it is fragile text trickery with **#define**/**#undef** dances. In Rust you write a **macro_rules!** macro that builds both the enum and the name function from one list — hygienically, with no **#undef**:

```
macro_rules! colors {
    ( $( $name:ident ),+ $(,)? ) => {
        #[derive(Debug, Clone, Copy)]
        enum Color { $( $name ),+ }

        impl Color {
            fn name(self) -> &'static str {
                match self {
                    $( Color::$name => stringify!($name) ),+
                }
            }
        }
    };
}

colors!(Red, Green, Blue);

fn main() {
    println!("{}", Color::Green.name()); // "Green"
}
```

`stringify!` is a built-in macro that turns tokens into a string literal — the clean equivalent of C’s `#name` stringizing operator. For a struct-by-struct job (like generating one trait impl per type), a proc macro is even better, because it can read the type’s real fields.

Seeing the expansion: `cargo expand`

Macros expand before type checking, so when one misbehaves the error can point at generated code you never wrote. The fix is to **look at the expansion**. The `cargo expand` tool (Chapter 24 — Tooling) prints your code with every macro already expanded:

```
cargo install cargo-expand    # one-time
cargo expand                  # prints the whole crate, macros expanded
cargo expand main              # just one module/function
```

Run it on the `my_vec!` or `colors!` example above and you will see the plain code the macro produced. This is your `cpp -E` for Rust, but it shows real expanded Rust, not raw text.

Rule of thumb. Reach for a macro only when a function or a generic cannot do the job. Macros are harder to read, harder to debug, and worse for IDE support. Variable argument counts, new syntax, and per-type code generation are good reasons. “I want a default argument” or “I want to avoid typing a generic” usually are not.

Key takeaways

- A macro is code that writes code at compile time. Rust has **declarative** macros (`macro_rules!`) and **procedural** macros.
- Unlike C’s text-substituting preprocessor, Rust macros work on **structured tokens**, are **hygienic** (their names do not collide with yours), and capture whole expressions so an argument is evaluated as one unit.

- `macro_rules!` matches **fragments** (`$x:expr`, `$x:ident`, `$x:ty`, ...) and supports **repetition** (`$( ... ),*`). Use double braces `{{ }}` to expand to a single block expression.
- `println!`, `format!`, `vec!`, `assert!`, `panic!`, and `dbg!` are macros because they are variadic and/or check the format string at compile time.
- Procedural macros power `#[derive(...)]`, attribute macros (`#[tokio::main]`), and function-like macros. They live in their own `proc-macro` crate and are built with `syn` and `quote`. You will mostly *use* them (e.g. `serde`).
- Use `cargo expand` to see what a macro produced when debugging.

Watch out (gotchas for C programmers)

- **Hygiene is not C’s behavior.** A Rust macro’s internal `tmp` will not clash with yours, and a macro cannot leak a new variable name to the caller unless the caller passes the name in as an `ident`.
- **Macros expand before type checking.** A macro can produce code that *looks* fine but fails to type-check later, with errors that point at generated code. Run `cargo expand` to see the real source.
- **Procedural macros need a separate crate** (`proc-macro = true`). You cannot define a `derive` macro in the same crate that uses it.
- **`macro_rules!` is order-sensitive.** A macro is only usable after its definition, unlike normal functions and types. Use `#[macro_export]` to share it.
- **Prefer functions and generics when they suffice.** A macro is a last resort, not a first reach. Macros hurt readability, debugging, and tooling.
- **The `!` is part of the call.** `vec!`, `println!`, and friends always need the `!`; without it the compiler looks for a function of that name.

Interview questions

Q: How do Rust macros differ from C preprocessor macros? A: The C preprocessor does raw text substitution and understands nothing about the language — not types, not scopes, not expressions. Rust macros operate on structured tokens, are hygienic (their identifiers do not collide with the caller’s), and capture whole fragments such as expressions, so an argument is evaluated once rather than pasted as text. This removes the classic C macro bugs like double evaluation and name capture.

Q: What does “hygienic” mean for a macro? A: It means names introduced inside the macro live in their own scope and cannot accidentally collide with names at the call site. In C, a macro that declares `int tmp;` breaks if the caller also has a `tmp`. In Rust the two `tmp`s are treated as distinct, so the macro is safe to use anywhere.

Q: What is the difference between a declarative macro and a procedural macro? A: A declarative macro (`macro_rules!`) matches token patterns and expands to new code; it is good for variadic syntax and small DSLs. A procedural macro is a Rust function that takes tokens and returns tokens; it can inspect a type’s structure and generate code from it. Proc macros power `#[derive(...)]`, attribute macros, and function-like macros, and they live in their own `proc-macro` crate.

Q: Why is `println!` a macro instead of a function? A: It takes a variable number of arguments of different types and checks the format string against those arguments at compile time.

A normal Rust function cannot be variadic like that, and only a macro can validate the format string before the program runs, turning what would be a C runtime bug into a compile error.

Q: A macro you wrote produces a confusing type error. How do you debug it? A: Run `cargo expand`, which prints your source with all macros already expanded. You can then read the generated Rust directly and see where it goes wrong, since macros expand before type checking and the error often points at code you did not type by hand.

Try it

1. Write the `my_vec!` macro from this chapter and run `cargo expand` to see the `push` calls it generates.
2. Add a rule to `my_vec!` for the `vec![x; n]` form (hint: match `$elem:expr ; $count:expr` and call a loop or `vec!` internally).
3. Write the broken C `SQUARE(x) x * x` in your head, then confirm the Rust `square!($x:expr)` version returns 25 for `square!(2 + 3)`. Change `$x:expr` thinking about why text substitution would have failed.

Chapter 27 — Idioms and Style

What you'll learn. How experienced Rust programmers actually write code: making bad states impossible with types, the newtype and builder patterns, the conversion traits, error style, naming conventions, and how each C habit maps to a cleaner Rust idiom.

Rust gives you tools C does not have. Using them well is the difference between “C with semicolons moved around” and real, idiomatic Rust. This chapter collects the patterns that matter most, all built on chapters you have already read.

Make illegal states unrepresentable

This is the most important idea in the chapter. In C you often encode state with **sentinels** (magic values) and **flags** (booleans), and you rely on the programmer to keep them consistent. The type system does not stop a bad combination.

```
/* C: a value, plus a flag saying whether the value is valid */
struct Reading {
    int value;
    bool has_value; /* if false, `value` is garbage */
};
```

Nothing stops code from reading `value` when `has_value` is false. In Rust you use a **type** that cannot be in a bad state. `Option<T>` is exactly this: it is either `Some(value)` or `None`, and you cannot read the value without handling the `None` case (Chapter 12 — Enums and Pattern Matching).

```
struct Reading {
    value: Option<i32>, // no separate flag; "no value" is built in
}

fn main() {
```



```

let r = Reading { value: None };
match r.value {
    Some(v) => println!("got {v}"),
    None => println!("no reading"),
}
}

```

The same idea replaces a pile of booleans with an **enum**. Consider a connection that is either disconnected, connecting, or connected with a session id. With booleans, you can represent nonsense like “disconnected but has a session.” With an enum, you cannot:

```

// This compiles fine; the point is that bad states simply cannot be built.
enum Connection {
    Disconnected,
    Connecting,
    Connected { session_id: u64 },
}

fn describe(c: &Connection) -> String {
    match c {
        Connection::Disconnected => "offline".to_string(),
        Connection::Connecting => "dialing".to_string(),
        Connection::Connected { session_id } => format!("online #{session_id}"),
    }
}

fn main() {
    println!("{}", describe(&Connection::Connected { session_id: 42 }));
}

```

Rule of thumb. When you reach for a `bool` field or a “magic” sentinel value, ask: can an enum or `Option` make the bad combination impossible? If so, use it. The compiler then checks your logic for free.

C vs Rust. In C the invariant (“if `has_value` is false, do not read `value`”) lives in comments and discipline. In Rust the invariant lives in the type, and `match` forces you to handle every case. The bug class disappears.

The newtype pattern

A **newtype** is a tuple struct with one field that wraps an existing type: `struct Meters(f64)`. It costs nothing at runtime (it compiles away) but gives you a distinct type the compiler can tell apart.

It solves two problems.

1. Type safety. In C, a `double` is a `double`; nothing stops you from adding meters to seconds. Newtypes make the units distinct:

```

struct Meters(f64);
struct Seconds(f64);

```

```
fn stopping_distance(speed_mps: f64, t: Seconds) -> Meters {
    Meters(speed_mps * t.0)           // `.0` reaches the inner field
}

fn main() {
    let d = stopping_distance(30.0, Seconds(2.0));
    println!("{}", d.0);
}
```

Now passing a `Meters` where a `Seconds` is expected is a compile error, not a silent bug.

2. Implementing external traits (the orphan rule). Rust’s **orphan rule** says you may implement a trait for a type only if you own the trait or the type. You cannot `impl Display` for `Vec<T>`, because you own neither. A newtype gives you a type you *do* own, so you can wrap and implement:

```
use std::fmt;

struct CsvRow(Vec<String>);           // a type we own, wrapping a Vec we do not

impl fmt::Display for CsvRow {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}", self.0.join(","))
    }
}

fn main() {
    let row = CsvRow(vec!["a".into(), "b".into(), "c".into()]);
    println!("{}", row);               // a,b,c
}
```

Mental model. A newtype is like a C typedef that the compiler actually enforces. C’s typedef makes a name; a Rust newtype makes a genuinely distinct type.

Conversions: From, Into, TryFrom, AsRef

Rust has standard traits for converting between types. Use them instead of inventing your own `to_x` functions.

- **From<T>** says “I can be built from a T.” Implement **From**, and you get **Into** for free — the standard library provides it automatically. So implement **From**, and call `.into()` whenever convenient.

```
struct Celsius(f64);
struct Fahrenheit(f64);

impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) -> Self {
        Fahrenheit(c.0 * 9.0 / 5.0 + 32.0)
    }
}
```

```

}

fn main() {
    let f = Fahrenheit::from(Celsius(100.0));
    let f2: Fahrenheit = Celsius(0.0).into();    // free, from the From impl
    println!("{}", f.0, f2.0);
}

```

- `TryFrom<T>` / `TryInto<T>` are the fallible versions: the conversion can fail, so they return `Result`. Use them when not every input is valid:

```

use std::convert::TryFrom;

struct Percent(u8);    // must be 0..=100

impl TryFrom<i32> for Percent {
    type Error = String;
    fn try_from(n: i32) -> Result<Self, Self::Error> {
        if (0..=100).contains(&n) {
            Ok(Percent(n as u8))
        } else {
            Err(format!("{n} is out of range"))
        }
    }
}

fn main() {
    println!("{}", Percent::try_from(50).map(|p| p.0));    // Ok(50)
    println!("{}", Percent::try_from(200).map(|p| p.0));    // Err(...)
}

```

- `AsRef<T>` means “I can be viewed as a `&T` cheaply.” It is how you write a function that accepts “anything string-like” without forcing the caller to convert first:

```

use std::path::Path;

fn open_log(path: impl AsRef<Path>) {
    let p: &Path = path.as_ref();
    println!("opening {}", p.display());
}

fn main() {
    open_log("app.log");    // &str works
    open_log(String::from("err.log"));    // String works too
}

```

C vs Rust. C conversions are casts `((int)x)` or hand-written functions with names you must remember. Rust standardizes them: `From/Into` for conversions that always succeed, `TryFrom/TryInto` for ones that can fail, and the `?` operator even uses `From` to convert error types automatically (Chapter 13 — Error Handling).

Default, struct-update syntax, and the builder pattern

Rust has **no default function arguments**. C does not either, but C programmers often fake them with many functions or with a flags struct. Rust has cleaner tools.

Default gives a type a “zero value.” Derive it, then build a value and override just the fields you care about with **struct-update syntax** (`..`):

```
#[derive(Debug, Default)]
struct ServerConfig {
    host: String,           // defaults to ""
    port: u16,              // defaults to 0
    tls: bool,              // defaults to false
}

fn main() {
    let cfg = ServerConfig {
        host: "localhost".into(),
        port: 8080,
        ..Default::default()    // fill the rest with defaults
    };
    println!("{cfg:?}");
}
```

When there are many optional settings, use the **builder pattern**: a separate type whose methods set one field each and return `self`, ending in a `build()` call. This reads well and avoids a constructor with ten arguments.

```
#[derive(Debug)]
struct Server {
    host: String,
    port: u16,
    threads: u32,
}

struct ServerBuilder {
    host: String,
    port: u16,
    threads: u32,
}

impl ServerBuilder {
    fn new(host: &str) -> Self {
        ServerBuilder { host: host.into(), port: 80, threads: 1 }
    }
    fn port(mut self, p: u16) -> Self {
        self.port = p;
        self
    }
    fn threads(mut self, n: u32) -> Self {
```

```

        self.threads = n;
        self
    }
    fn build(self) -> Server {
        Server { host: self.host, port: self.port, threads: self.threads }
    }
}

fn main() {
    let s = ServerBuilder::new("example.com")
        .port(443)
        .threads(8)
        .build();
    println!("{s:?}");
}

```

Rule of thumb. A few arguments: a plain `new`. Many optional arguments, or arguments that need validation: a builder. The chained calls make each option self-documenting at the call site.

Function signatures: borrow on the way in, own on the way out

Idiomatic signatures **borrow** their inputs and **return owned** values. This gives the caller the most freedom and avoids needless copies.

- Take `&str`, not `String`, when you only read text. A `&str` accepts both a string literal and a borrowed `String`.
- Take `&[T]`, not `&Vec<T>`, when you only read a sequence. A `&[T]` (a slice) accepts arrays, vectors, and sub-ranges.

```

// Good: accepts &str and &String; accepts &[T] from arrays and Vecs.
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}

fn sum(values: &[i32]) -> i32 {
    values.iter().sum()
}

fn main() {
    let owned = String::from("hello world");
    println!("{}", first_word(&owned));           // pass a &String as &str
    println!("{}", first_word("from a literal"));

    let v = vec![1, 2, 3];
    println!("{}", sum(&v));                       // pass a &Vec as &[i32]
    println!("{}", sum(&[10, 20]));                // and an array slice
}

```

C vs Rust. In C you pass `const char *` and a length, and a `T *` plus a count. `&str`

and `&[T]` bundle the pointer and length into one safe, bounds-checked value (Chapter 10 — Slices and Strings). The idiom is the same as C’s “pass a borrowed view,” but checked.

Prefer iterators, combinators, and ?

Three small habits make Rust code cleaner and often faster:

- **Iterators over index loops.** They are bounds-safe and express intent.
- **Combinators (`map`, `filter`, `sum`, ...)** over manual loops *where it is clearer*. Do not force a one-liner that hurts readability.
- **? over `.unwrap()`** for propagating errors (Chapter 13).

```
fn even_squares(nums: &[i32]) -> Vec<i32> {
    nums.iter()
        .filter(|&n| n % 2 == 0)
        .map(|&n| n * n)
        .collect()
}

fn main() {
    println!("{:?}", even_squares(&[1, 2, 3, 4, 5, 6])); // [4, 16, 36]
}
```

Error style: `thiserror` for libraries, `anyhow` for applications

Two community crates are the standard choice (Chapter 13 — Error Handling):

- **`thiserror` for libraries.** It derives a clean, typed error enum so callers can `match` on the specific error.
- **`anyhow` for applications.** Its `anyhow::Error` holds any error and adds context; perfect for a `main` that just needs to report and exit.

```
// library style: a specific, matchable error type
// needs: cargo add thiserror
use thiserror::Error;

#[derive(Error, Debug)]
enum ParseError {
    #[error("empty input")]
    Empty,
    #[error("not a number: {0}")]
    NotANumber(String),
}

fn parse_count(s: &str) -> Result<u32, ParseError> {
    if s.is_empty() {
        return Err(ParseError::Empty);
    }
    s.parse().map_err(|_| ParseError::NotANumber(s.to_string()))
}
```

```

}

fn main() {
    println!("{:?}", parse_count("42"));
    println!("{:?}", parse_count("oops"));
}

```

Rule of thumb. Never `.unwrap()` in library code. A library should return its error to the caller, who decides what to do. In a small example, in tests, or in an application's top level, `.unwrap()`/`.expect()` is fine.

Naming conventions

Rust has firm, tool-enforced naming rules. `clippy` and `rustfmt` nudge you toward them.

Item	Convention	Example
Functions, variables, modules	<code>snake_case</code>	<code>read_line</code> , <code>byte_count</code>
Types, traits, enum variants	<code>CamelCase</code>	<code>ServerConfig</code> , <code>Display</code> , <code>NotFound</code>
Constants and statics	<code>SCREAMING_SNAKE_CASE</code>	<code>MAX_RETRIES</code>

Getters have no `get_` prefix. A getter for a `name` field is just `name()`, not `get_name()`. The setter, if any, is `set_name()`.

```

struct User {
    name: String,
}

impl User {
    fn name(&self) -> &str {      // getter: no `get_` prefix
        &self.name
    }
    fn set_name(&mut self, name: String) {
        self.name = name;
    }
}

fn main() {
    let u = User { name: "Ada".into() };
    println!("{}", u.name());
}

```

Conversion method prefixes follow a strict convention that signals cost and ownership:

Prefix	Meaning	Cost	Example
as_	cheap borrow-to-borrow view	free	<code>str::as_bytes</code>
to_	clone or expensive conversion, keeps original	allocates	<code>str::to_string</code>
into_	consumes <code>self</code> , gives owned result	moves	<code>String::into_bytes</code>

C vs Rust. C has no enforced naming; styles vary by project. Rust's conventions are uniform across the whole ecosystem, so any Rust code looks familiar, and `as_/to_/into_` tell you the cost of a call before you read its body.

Display and Debug

Two traits control how a value prints (Chapter 15 — Traits):

- **Debug** (`{:?}`) is for programmers: derive it on almost everything.
- **Display** (`{}`) is for users: implement it by hand on types a person will read, because only you know the right human-facing format.

```
use std::fmt;

#[derive(Debug)]           // derive Debug everywhere
struct Point { x: i32, y: i32 }

impl fmt::Display for Point { // implement Display for user-facing output
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

fn main() {
    let p = Point { x: 3, y: 4 };
    println!("{p}");           // (3, 4)           - Display
    println!("{p:?}");         // Point { x: 3, y: 4 } - Debug
}
```

Keep the API small, run clippy, follow the guidelines

- **Minimal public surface.** Everything is private by default (Chapter 3 — Program Structure). Mark pub only what callers truly need. A small API is easier to keep stable.
- **Run clippy.** `cargo clippy` catches non-idiomatic code and common mistakes; treat its lints as a teacher (Chapter 24 — Tooling).
- **Follow the Rust API Guidelines.** The official checklist (naming, common traits to derive, error types) keeps your crate consistent with the ecosystem.

Translating C idioms to Rust idioms

This table summarizes the moves you will make again and again:

C idiom	Rust idiom	Why
Error code / <code>errno</code>	<code>Result<T, E> + ?</code>	Errors are values you cannot ignore.
Out-parameter (<code>int *out</code>)	Return a tuple or <code>Result</code>	No pointers needed; the return value carries it.
<code>void * callback</code> + context	Closure or generic <code>Fn</code>	Type-safe, captures state automatically.
Opaque struct + accessors	Private fields / newtype	Invariants enforced by visibility.
Magic sentinel value	<code>Option<T></code> / enum	“No value” is a real, checked case.
Flags struct of bools	A state enum	Bad combinations cannot be built.
Manual <code>free</code>	Ownership and <code>Drop</code>	Cleanup is automatic and exact.
Macro <code>#define</code> for a constant	<code>const</code>	Typed, scoped, no preprocessor.

C error code



Rust Result plus question mark

C out parameter



Rust tuple or Result return

C void star callback



Rust closure or generic

C opaque struct



Rust private fields or newtype

Key takeaways

- **Make illegal states unrepresentable.** Use `Option` instead of a sentinel and an enum instead of a bag of booleans; let `match` enforce your logic.
- The **newtype** (`struct Meters(f64)`) gives unit safety for free and lets you implement external traits despite the orphan rule.
- Use the standard conversion traits: implement **From** and get **Into** free; use **TryFrom**/**TryInto** when conversion can fail; accept **AsRef** for flexible inputs.
- Rust has no default arguments: use **Default** plus struct-update syntax for a few overrides, and the **builder pattern** for many optional settings.
- Signatures should **borrow inputs** (`&str`, `&[T]`) and **return owned** values. Prefer iterators and combinators over index loops, and `?` over `.unwrap()`.
- Error style: **thiserror** for libraries, **anyhow** for applications, and never `.unwrap()` in library code.
- Follow naming conventions; getters have no `get_` prefix; `as_/to_/into_` signal cost. Implement **Display** for users, derive **Debug** for everything. Keep the public API small and run clippy.

Watch out (gotchas for C programmers)

- **Overusing `.clone()` and `.unwrap()`.** Reaching for `.clone()` to silence the borrow checker, or `.unwrap()` to skip error handling, is a code smell. Borrow instead of cloning; propagate errors with `?`.
- **Fighting the borrow checker.** If you cannot satisfy it, the design usually needs restructuring (split a function, narrow a borrow, change ownership), not more `clones` or `unsafe`.
- **“Stringly-typed” code.** Passing meaning around as strings (“status = ”active”) instead of enums loses all compile-time checking. Use a type.
- **Skipping `?`.** Chains of `match` on `Result` where one `?` would do are un-idiomatic and noisy.
- **A `get_` prefix on getters.** It is `name()`, not `get_name()`. Tools and reviewers will flag it.
- **Premature `unsafe` or optimization.** Write safe, clear code first; measure; reach for `unsafe` only with a real, proven need (Chapter 25 — `Unsafe` and FFI).

Interview questions

Q: What does “make illegal states unrepresentable” mean, and how do you do it in Rust? A: It means designing your types so that an invalid combination cannot even be constructed, instead of relying on runtime checks and discipline. In Rust you use `Option<T>` instead of a value-plus-valid-flag, and an `enum` instead of several booleans. The compiler and exhaustive `match` then enforce that every real state is handled and no impossible state exists.

Q: What is the newtype pattern and what two problems does it solve? A: A newtype is a single-field tuple struct, like `struct Meters(f64)`, that wraps an existing type with no runtime cost. It gives type safety (you cannot mix `Meters` and `Seconds`), and it lets you implement an external trait on an external type by wrapping it, working around the orphan rule that otherwise forbids `impl ForeignTrait for ForeignType`.

Q: If you implement `From`, why do you not also implement `Into`? A: The standard library provides a blanket implementation that gives you `Into` automatically whenever `From` exists. So you implement `From<T>` for `U`, and you get `T: Into<U>` for free. Implementing `Into` directly is discouraged.

Q: How does Rust handle the absence of default function arguments? A: It does not have them, so idiomatic Rust uses the `Default` trait plus struct-update syntax (`..Default::default()`) to override just a few fields, and the builder pattern for types with many optional settings — chained methods that each set one field and return `self`, ending in `build()`.

Q: When should you use `thiserror` versus `anyhow`? A: Use `thiserror` in libraries, where callers benefit from a specific, typed error enum they can match on. Use `anyhow` in applications, where you just want one easy error type with context for reporting at the top level. Library code should return errors rather than calling `.unwrap()`.

Q: Why prefer `&str` and `&[T]` over `String` and `Vec<T>` in function parameters? A: Borrowed slices accept more callers and avoid copies. `&str` accepts both string literals and borrowed `Strings`; `&[T]` accepts arrays, vectors, and sub-ranges. Taking the owned type forces the caller to give up ownership or clone, which is rarely necessary when the function only reads the data.

Try it

1. Take a struct with a `value: i32` and a `valid: bool` field and rewrite it to use `Option<i32>`. Notice the `valid` flag disappears.
2. Make `struct UserId(u64)` a newtype and write a function that takes a `UserId` and not a bare `u64`. Try passing a plain `u64` and read the error.
3. Write a builder for a `Request` type with `url`, `method`, and `timeout` fields, giving sensible defaults in `new`. Compare the call site to a five-argument constructor.

Chapter 28 — Gotchas for C Programmers (the checklist)

What you'll learn. The traps that bite C programmers most when they first write Rust, gathered in one place. For each one: the trap, why it surprises a C programmer, and the fix. Use this as a checklist when the compiler fights you.

This chapter does not teach new ideas. It collects the surprises from the rest of the book into a single list you can scan fast. Each gotcha points to the chapter where it is taught in full. If you read only one chapter twice, make it this one.

The format is always the same:

- **The trap** — what you wrote.
- **Why it surprises you** — what C taught you to expect.
- **The fix** — the Rust way, with a tiny snippet.

Intentionally broken examples start with `// COMPILER ERROR:` so you know the failure is on purpose.

Variables and types

Variables are immutable by default

The trap. You declare a variable and then try to change it.

```
// COMPILER ERROR: cannot assign twice to immutable variable `x`
fn main() {
    let x = 5;
```

```

    x = 6; // error[E0384]
}

```

Why it surprises you. In C, every variable is mutable unless you write `const`. In Rust it is the other way around: `let` makes a value that cannot change. This is the single most common first-day error.

The fix. Add `mut` (“mutable”).

```

fn main() {
    let mut x = 5;
    x = 6; // fine now
    println!("{x}");
}

```

C vs Rust. C: mutable by default, `const` to lock. Rust: immutable by default, `mut` to unlock. (Chapter 4 — Variables, Types, and Constants.)

No implicit numeric conversions

The trap. You mix integer and float, or `i32` and `i64`, and expect C’s automatic promotion.

```

// COMPILE ERROR: mismatched types (expected i64, found i32)
fn main() {
    let a: i32 = 5;
    let b: i64 = 10;
    let c = a + b; // error[E0308]
}

```

Why it surprises you. C silently promotes and converts numbers in arithmetic. This is convenient but hides bugs (truncation, sign changes). Rust never converts number types for you.

The fix. Convert on purpose with `as` (a cast) or `From/into` (a safe widening conversion).

```

fn main() {
    let a: i32 = 5;
    let b: i64 = 10;
    let c = a as i64 + b; // explicit cast
    let d = i64::from(a) + b; // safe widening, no data loss
    println!("{c} {d}");
}

```

Rule of thumb. Use `From/into` when the conversion cannot lose data (small to big). Use `as` when you accept truncation. (Chapter 4.)

Integer overflow panics in debug, wraps in release

The trap. You add two numbers and the result is too big for the type.

```

use std::hint::black_box;

```

```

fn main() {
    // `black_box` keeps these out of the compiler's constant folding; in real

```

```

    // code the values would come from input. With plain literals the compiler
    // rejects the overflow at compile time instead (even nicer).
    let x: u8 = black_box(250);
    let y: u8 = black_box(10);
    let z = x + y; // debug build: panics "attempt to add with overflow"
    println!("{z}"); // release build: wraps to 4
}

```

Why it surprises you. In C, unsigned overflow wraps silently and signed overflow is undefined behavior. Many C programmers rely on wrapping. Rust treats accidental overflow as a bug: it panics in debug builds so you find it.

The fix. Say what you mean with the explicit methods.

```

fn main() {
    let x: u8 = 250;
    let a = x.wrapping_add(10); // wraps on purpose -> 4
    let b = x.checked_add(10); // Option: None on overflow
    let c = x.saturating_add(10); // clamps to u8::MAX (255)
    let (d, overflowed) = x.overflowing_add(10); // value + a bool flag
    println!("{a} {b:?} {c} {d} {overflowed}");
}

```

Watch out. Behavior differs between debug and release builds. Never depend on the default wrap. (Chapter 4; performance in Chapter 18 — Memory Without a GC.)

usize is the type for indexing and sizes

The trap. You index a collection with an `i32`.

```

// COMPILE ERROR: the type `[i32]` cannot be indexed by `i32`
fn main() {
    let v = vec![10, 20, 30];
    let i: i32 = 1;
    let x = v[i]; // error[E0277]: indexing wants usize
    println!("{x}");
}

```

Why it surprises you. In C you index with any integer. Rust uses `usize` (an unsigned, pointer-sized integer, like C's `size_t`) for all lengths and indices.

The fix. Use `usize`, or cast.

```

fn main() {
    let v = vec![10, 20, 30];
    let i: usize = 1;
    println!("{}", v[i]);
}

```

Ownership, moves, and borrows

Assignment and passing MOVE non-Copy values

The trap. You use a value after assigning it elsewhere or passing it to a function.

```
// COMPILER ERROR: borrow of moved value: `s`
fn main() {
    let s = String::from("hi");
    let t = s; // ownership MOVES to t
    println!("{s}"); // error[E0382]: s is no longer valid
    println!("{t}");
}
```

Why it surprises you. In C, `t = s` copies the pointer; both names work (and that is exactly how you get double-free bugs). In Rust, assigning a non-Copy value *moves* ownership. The old name becomes invalid. Small Copy types (`i32`, `bool`, `char`, fixed arrays of Copy) are duplicated instead, so they keep working.

The fix. Borrow instead of moving, or clone if you truly need a second copy.

```
fn main() {
    let s = String::from("hi");
    let t = &s; // borrow: s still owns the data
    println!("{s} {t}"); // both work

    let u = s.clone(); // deep copy if you need an independent String
    println!("{s} {u}");
}
```

C vs Rust. A move is like `memcpy` of the handle plus a promise that the old handle will never be touched again — enforced by the compiler. (Chapter 7 — Ownership and Moves.)

Returning a reference to a local fails to compile

The trap. You return a pointer to a local variable.

```
// COMPILER ERROR: cannot return reference to local variable `s`
fn dangling() -> &String {
    let s = String::from("oops");
    &s // error[E0515]: `s` is dropped at the end of the function
}
```

Why it surprises you. This is the *opposite* of C. In C this compiles fine and gives you a dangling pointer that crashes later. In Rust it is a compile error, so the dangling pointer can never exist.

The fix. Return the owned value (move it out), not a reference to it.

```
fn not_dangling() -> String {
    let s = String::from("fine");
    s // move ownership to the caller
}
```

```
fn main() {
    println!("{}", not_dangling());
}
```

Mental model. A reference is a borrowed key, not a copy. You cannot hand out a key to a room that is about to be demolished. (Chapters 8 and 9.)

The borrow rules: shared XOR mutable

The trap. You hold a shared reference and a mutable reference at the same time.

```
// COMPILER ERROR: cannot borrow `v` as mutable because it is also borrowed as immutable
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0]; // shared borrow
    v.push(4); // mutable borrow while `first` is alive -> error[E0502]
    println!("{}", first);
}
```

Why it surprises you. In C you can read through one pointer and write through another at the same time. That is how iterator-invalidation bugs happen (push might reallocate and free the buffer `first` points into). Rust forbids it.

The rule. At any moment you may have **either** any number of shared references (`&T`) **or** exactly one mutable reference (`&mut T`), never both. This is “aliasing XOR mutation.”

The fix. End the shared borrow before mutating (use the value, then mutate), or copy out what you need.

```
fn main() {
    let mut v = vec![1, 2, 3];
    let first = v[0]; // copy the i32 out; no live borrow
    v.push(4);
    println!("{}", first);
}
```

Watch out. Only one `&mut` may exist at a time, and no `&mut` while any `&` is alive. This is what makes data races impossible. (Chapter 8 — Borrowing and References.)

“Fighting the borrow checker”: common patterns

When the borrow checker rejects code that you *know* is logically fine, reach for one of these patterns, roughly in order of preference:

1. **Shorten the borrow.** Compute and finish reading before you mutate. Thanks to non-lexical lifetimes, a borrow ends at its last use, not at the closing brace.
2. **Copy or clone to break the tie.** Pull a small `Copy` value out, or `.clone()` an owned value to start. Optimize later if it matters.
3. **Split borrows.** Borrow two different fields of a struct mutably at once (allowed), or use `split_at_mut` to get two mutable halves of a slice.

4. **Use indices instead of references.** Store `usize` indices into a `Vec` rather than references between elements. Common for graphs and trees.
5. **Restructure.** Move the borrowing into a smaller function, or change who owns what.
6. **Reach for shared ownership.** `Rc<RefCell<T>>` (single thread) or `Arc<Mutex<T>>` (across threads) when you genuinely need many owners or shared mutable state.

```
fn main() {
    // Split borrows: two halves of a slice, both mutable, no conflict.
    let mut a = [1, 2, 3, 4];
    let (left, right) = a.split_at_mut(2);
    left[0] += 10;
    right[0] += 20;
    println!("{a:?}"); // [11, 2, 23, 4]
}
```

Rule of thumb. When stuck, clone first to make it compile, then remove the clone once you see the data flow. Fighting the checker for an hour is worse than one `.clone()`. (Chapters 8, 9, and 17 — Smart Pointers.)

Drop order is reverse of declaration

The trap. You assume destructors run in declaration order, as in C++ would be the question; in C there are no destructors at all.

Why it surprises you. C has no automatic cleanup, so you may not expect any order. Rust drops (cleans up) values automatically. Within a scope, values are dropped in **reverse** order of declaration — last declared, first dropped — like unwinding a stack.

```
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("drop {}", self.0);
    }
}

fn main() {
    let _a = Noisy("a");
    let _b = Noisy("b");
    // prints: drop b, then drop a
}
```

C vs Rust. Rust's `Drop` is RAI: cleanup is automatic and deterministic, like calling `free` at the right moment for you. (Chapter 7.)

Strings and characters

`char` is 4 bytes, not 1

The trap. You treat `char` like a byte.

Why it surprises you. In C, `char` is one byte and doubles as “small integer” and “ASCII letter.” In Rust, `char` is a **Unicode scalar value**: it is 4 bytes and holds any single character, like `'é'` or `' '`. A single byte is `u8`.

```
fn main() {
    let c: char = ' ';
    println!("{}", std::mem::size_of::<char>()); // 4
    let b: u8 = b'A'; // a byte literal is u8, value 65
    println!("{c} {b}");
}
```

Strings are UTF-8; you cannot index by integer

The trap. You try `s[0]` to get the first character, like a C `char` array.

```
// COMPILER ERROR: `String` cannot be indexed by `{integer}`
fn main() {
    let s = String::from("héllö");
    let first = s[0]; // error[E0277]: no such indexing
    println!("{first}");
}
```

Why it surprises you. A Rust `String` (and `&str`) is UTF-8 bytes. One character may take several bytes, so `s[0]` has no clear meaning and is forbidden. There is also **no NUL terminator** — the length is stored, not found by scanning.

The fix. Iterate by character or by byte, or take a checked byte range.

```
fn main() {
    let s = String::from("héllö");
    let first: char = s.chars().next().unwrap(); // 'h'
    let nth: Option<char> = s.chars().nth(1); // 'é'
    let bytes: &[u8] = s.as_bytes(); // raw UTF-8 bytes
    println!("{first} {nth:?} {}", bytes.len());
}
```

`.len()` is bytes, not characters

The trap. You expect `.len()` to count characters.

```
fn main() {
    let s = "héllö";
    println!("{}", s.len()); // 6, not 5 ('é' is two bytes in UTF-8)
    println!("{}", s.chars().count()); // 5 characters
}
```

Why it surprises you. C’s `strlen` counts bytes up to the NUL, and for ASCII that equals the character count. In UTF-8 they differ. `.len()` returns the number of bytes; use `.chars().count()` for characters. (Chapter 10 — Slices and Strings.)

&str vs String (like `const char *` vs an owned buffer)

The trap. You are not sure which string type to use in a function signature.

Why it surprises you. C has just `char *`. Rust splits the idea in two:

Rust	C analogy	Owns the data?
<code>String</code>	a malloc'd, growable buffer	yes
<code>&str</code>	a <code>const char *</code> view (ptr + len)	no, it borrows

The fix. Take `&str` in parameters (it accepts both a `String` and a literal), return `String` when you build new text.

```
fn shout(s: &str) -> String {
    s.to_uppercase()
}

fn main() {
    let owned = String::from("hi");
    println!("{}", shout(&owned)); // &String coerces to &str
    println!("{}", shout("there")); // a literal is already &str
}
```

The same idea applies to data: take `&[T]` (a slice), return `Vec<T>`. A slice is “pointer + length,” exactly like the (ptr, len) pair you pass around in C.

Collections and indexing

Slices and `Vec` are bounds-checked

The trap. You index past the end.

```
fn main() {
    let v = vec![1, 2, 3];
    let x = v[10]; // panic: index out of bounds: len is 3 but index is 10
    println!("{}", x);
}
```

Why it surprises you. In C, `v[10]` reads whatever memory is there — silent corruption or a crash. Rust checks the bound and panics with a clear message instead of reading out of bounds.

The fix. Use `.get()`, which returns an `Option`, when the index might be invalid.

```
fn main() {
    let v = vec![1, 2, 3];
    match v.get(10) {
        Some(x) => println!("got {x}"),
        None => println!("out of range"),
    }
}
```

Deep dive. The bounds check has a tiny runtime cost, but the optimizer removes it in many loops, and iterators avoid it entirely. (Chapter 16.)

Null, errors, and return values

There is no null — use `Option`

The trap. You look for a null pointer to mean “nothing.”

Why it surprises you. In C, any pointer can be NULL, and forgetting to check is a top cause of crashes. Rust has no null. “Maybe a value” is the type `Option<T>`, which is `Some(value)` or `None`, and the compiler forces you to handle `None`.

```
fn first_word(s: &str) -> Option<&str> {
    s.split_whitespace().next()
}

fn main() {
    match first_word("hello world") {
        Some(w) => println!("first: {w}"),
        None => println!("empty"),
    }
}
```

C vs Rust. `Option<&T>` replaces a nullable pointer, and because of “niche optimization” it takes the same space as the pointer: `None` reuses the all-zeros bit pattern. You get the safety for free. (Chapter 12 — Enums and Pattern Matching.)

Result is `#[must_use]`; ignoring it warns

The trap. You call a function that returns `Result` and ignore the result.

```
use std::fs::File;

fn main() {
    File::create("out.txt"); // warning: unused `Result` that must be used
}
```

Why it surprises you. In C, ignoring a return code is silent; nobody stops you from skipping an error check. Rust marks `Result` as `#[must_use]`, so the compiler warns when you drop it.

The fix. Handle it: `match`, `?`, `.unwrap().expect()` (panics on error), or `.ok()` if you truly do not care.

```
use std::fs::File;

fn main() -> std::io::Result<()> {
    let _f = File::create("out.txt"); // `?` returns the error to the caller
    Ok(())
}
```

Watch out. `.unwrap()` and `.expect()` panic on `Err` or `None`. They are fine in examples and tests; in real code prefer `?` and proper handling. (Chapter 13 — Error

Handling.)

Result/panic are not C++ exceptions

The trap. You expect errors to be thrown and caught.

Why it surprises you. Rust has no exceptions. A recoverable error is a normal return value (`Result`), passed up with `?`. A `panic!` is for bugs that should never happen; it unwinds and usually ends the program. There is no `try/catch`. (Chapter 13.)

Expressions and control flow

`if`, `loop`, and `match` are expressions

The trap. You write a C-style assignment from an `if` and get confused, or you add a stray semicolon that throws away the value.

```
fn main() {  
    // `if` is an expression: it produces a value.  
    let n = 7;  
    let label = if n % 2 == 0 { "even" } else { "odd" };  
    println!("{label}");  
  
    // `loop` can return a value with `break value`.  
    let mut i = 0;  
    let doubled = loop {  
        i += 1;  
        if i == 5 {  
            break i * 2;  
        }  
    };  
    println!("{doubled}");  
}
```

Why it surprises you. In C, `if` and `switch` are statements; you assign inside them. In Rust they are expressions that evaluate to a value, which replaces C's ternary `?:` and many temporary variables.

A trailing semicolon swallows the return value

The trap. You add a semicolon to the last line of a function.

```
// COMPILE ERROR: mismatched types (expected i32, found ())  
fn square(x: i32) -> i32 {  
    x * x; // the `;` turns this into a statement returning ()  
}
```

Why it surprises you. In C, `return` is explicit, so a semicolon is harmless. In Rust, the last expression with no semicolon is the return value. Adding `;` turns the expression into a statement whose value is `()` (the empty “unit” type, like C's `void`).

The fix. Drop the semicolon (or write `return x * x;`).

```
fn square(x: i32) -> i32 {
    x * x // no semicolon: this is the result
}

fn main() {
    println!("{}", square(9));
}
```

match is exhaustive and does not fall through

The trap. You expect C switch behavior: cases fall through, and missing cases are allowed.

```
// COMPILER ERROR: non-exhaustive patterns: `Blue` not covered
enum Color { Red, Green, Blue }

fn name(c: Color) -> &'static str {
    match c {
        Color::Red => "red",
        Color::Green => "green",
        // forgot Blue -> error[E0004]
    }
}
```

Why it surprises you. C's switch falls through to the next case unless you write **break**, and you can omit cases. Rust **match** arms never fall through, and the compiler forces you to cover every case (or add a **_** catch-all).

The fix. Cover all cases, or use **_** for the rest.

```
enum Color { Red, Green, Blue }

fn name(c: Color) -> &'static str {
    match c {
        Color::Red => "red",
        Color::Green => "green",
        Color::Blue => "blue",
    }
}

fn main() {
    println!("{}", name(Color::Blue));
}
```

Rule of thumb. Exhaustiveness is a feature: add a variant to an enum and the compiler lists every **match** you must update. (Chapter 12.)

Macros and functions

println! and friends are macros (the !)

The trap. You write **println(...)** without the **!**.

Why it surprises you. They look like function calls but the `!` marks a **macro** — code that runs at compile time. The macro checks your format string against the arguments, which a normal function cannot do. Common ones: `println!`, `format!`, `vec!`, `panic!`, `assert!`, `write!`. (Chapter 26 — Macros.)

```
fn main() {
    let x = 42;
    println!("x = {x}"); // captures `x` from scope; checked at compile time
}
```

No function overloading, no default arguments

The trap. You define two functions with the same name and different parameters.

Why it surprises you. C does not have overloading either, but you may expect it from C++ or other languages. Rust has no overloading and no default argument values.

The fix. Use different names, an `Option` parameter, a builder, or a trait (generics) to accept several types.

```
fn connect(host: &str, port: u16) { /* ... */ }
fn connect_default(host: &str) {
    connect(host, 8080); // wrapper supplies the "default"
}

fn main() {
    connect_default("localhost");
}
```

(Chapter 6 — Functions and Closures; builders in Chapter 27 — Idioms and Style.)

No inheritance — use traits and composition

The trap. You look for class inheritance to share behavior.

Why it surprises you. Rust has no classes and no inheritance. Shared behavior comes from **traits** (like interfaces, or a C struct of function pointers), and data reuse comes from **composition** (put one struct inside another). Traits can provide default method bodies. (Chapter 15 — Traits.)

Structs and layout

Field order is not guaranteed — use `#[repr(C)]` for FFI

The trap. You assume struct fields are laid out in memory in declaration order, then pass the struct to C.

Why it surprises you. In C, struct fields are in declaration order (with padding). Rust may **reorder** fields to save space. That is fine for pure Rust, but it breaks any code that expects the C layout — including FFI and casting to bytes.

The fix. Add `#[repr(C)]` to force C layout.

```
#[repr(C)]
struct Point {
```

```

    x: i32,
    y: i32,
}

```

Watch out. Only use `#[repr(C)]` when layout matters (FFI, memory-mapped structures). Let Rust optimize layout otherwise. (Chapter 25 — Unsafe and FFI.)

Deref coercion can surprise you

The trap. You call a method and it seems to come from a different type.

Why it surprises you. Rust automatically converts `&String` to `&str`, `&Vec<T>` to `&[T]`, and `&Box<T>` to `&T` when needed. This “deref coercion” is convenient but can be confusing: a method you call on a `String` may actually be a `str` method. It also means you usually do not need to write `&*` or `.as_str()`. (Chapter 17 — Smart Pointers.)

Smart pointers and shared state

Rc is not thread-safe — use Arc across threads

The trap. You share an `Rc` between threads.

```

// COMPILER ERROR: `Rc<i32>` cannot be sent between threads safely
use std::rc::Rc;
use std::thread;

fn main() {
    let data = Rc::new(5);
    let d = Rc::clone(&data);
    thread::spawn(move || {
        println!("{d}"); // error[E0277]: Rc is not Send
    });
}

```

Why it surprises you. `Rc` (reference counting) is fast because its count is *not* atomic. That makes it unsafe to share across threads. The compiler catches this through the `Send` trait.

The fix. Use `Arc` (atomic reference count) for sharing across threads.

```

use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(5);
    let d = Arc::clone(&data);
    let h = thread::spawn(move || println!("{d}"));
    h.join().unwrap();
}

```

(Chapter 17; threads in Chapter 19 — Threads and Concurrency.)

Rc/Arc cycles leak — use Weak

The trap. You build a cycle of Rcs (for example, a parent that points to a child that points back to the parent).

Why it surprises you. Reference counting cannot free a cycle: each node keeps the other alive, so the count never reaches zero. The memory leaks. This is one of the few ways safe Rust can leak.

The fix. Make one direction a Weak reference, which does not keep the value alive.

```
use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct Node {
    parent: RefCell<Weak<Node>>, // weak: does not own the parent
    children: RefCell<Vec<Rc<Node>>>, // strong: owns the children
}
```

(Chapter 17.)

RefCell moves borrow checking to runtime — and panics

The trap. You borrow a RefCell mutably twice at once.

```
use std::cell::RefCell;

fn main() {
    let c = RefCell::new(5);
    let a = c.borrow_mut();
    let b = c.borrow_mut(); // panic: already mutably borrowed: BorrowMutError
    println!("{a} {b}");
}
```

Why it surprises you. RefCell lets you mutate through a shared reference (“interior mutability”). It still enforces the borrow rules, but at **runtime**, not compile time. A violation is a panic, not a compile error.

The fix. Keep borrows short, and do not hold two `borrow_mut()` at once. Drop the first borrow before taking the second. (Chapter 17.)

Concurrency

Data races are caught at compile time

The trap. You move data into a thread, or share it without synchronization, and expect it to “just work” like C.

Why it surprises you. In C, sharing data between threads compiles fine; data races are your problem to avoid, and they cause the worst bugs. In Rust the compiler refuses to compile a data race, using two marker traits: **Send** (safe to move to another thread) and **Sync** (safe to share by reference between threads).

The fix. Use a move closure to give the thread ownership, and wrap shared mutable state in `Arc<Mutex<T>>`.


```

use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..4 {
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut n = c.lock().unwrap(); // lock; auto-unlocks when `n` drops
            *n += 1;
        }));
    }
    for h in handles {
        h.join().unwrap();
    }
    println!("{}", *counter.lock().unwrap()); // 4
}

```

C vs Rust. A Mutex in Rust *wraps* the data it protects, so you cannot read the data without locking. In C the mutex and the data are separate, and nothing stops you from forgetting the lock. (Chapters 19 and 20.)

Key takeaways

- **Immutable by default.** Add `mut` to change a variable.
- **Moves, not copies.** Assigning or passing a non-Copy value moves it; use-after-move is a compile error. Borrow with `&/&mut`, or `.clone()`.
- **Aliasing XOR mutation.** Many `&` or one `&mut`, never both. This is the borrow checker's core rule.
- **No dangling references.** Returning `&local` is a compile error — the opposite of the classic C bug.
- **No null, no implicit conversions.** Use `Option`; convert numbers with `as` or `From`.
- **Strings are UTF-8.** `char` is 4 bytes, `s[i]` does not compile, `.len()` is bytes, there is no NUL terminator.
- **Overflow panics in debug.** Use `wrapping_`/`checked_`/`saturating_`; index with `usize`.
- **Exhaustive match, no fall-through.** Macros end in `!`. **Result is `#[must_use]`.** The last expression (no semicolon) is the return value.
- **Rc is single-thread; use Arc across threads.** Cycles leak — use `Weak`. `RefCell` panics on a borrow violation at runtime.
- **Data races are compile errors.** Use move closures and `Arc<Mutex<T>>`.
- **Slices/Vec are bounds-checked; use `.get()` for an `Option`.** Drop order is reverse of declaration. Prefer `&str`/`&[T]` in parameters, return `String`/`Vec`.

Interview questions

Q: A C programmer writes `let s2 = s1;` for two `Strings` and then uses `s1`. Why does it not compile? **A:** Assigning a non-Copy value moves ownership. After the move, `s1` is invalid,

so using it is a compile error (E0382). This guarantees a single owner and so the buffer is freed exactly once. Borrow (`&s1`) or `.clone()` to keep `s1` usable.

Q: Why does returning a reference to a local variable fail in Rust but compile in C?

A: The local is dropped when the function returns, so the reference would dangle. Rust’s borrow checker proves the reference cannot outlive the value and rejects it (E0515). C compiles it and gives you undefined behavior at run time. Return the owned value instead.

Q: What is the difference between `s.len()` and `s.chars().count()` on a Rust string?

A: `.len()` is the number of UTF-8 *bytes*; `.chars().count()` is the number of characters (Unicode scalar values). They are equal for ASCII but differ when any character takes more than one byte, such as `'é'`.

Q: When does integer overflow panic, and how do you opt into wrapping? A: In debug builds, overflow panics (“attempt to add with overflow”). In release builds it wraps silently. To choose behavior explicitly use `wrapping_add`, `checked_add` (returns `Option`), `saturating_add`, or `overflowing_add`.

Q: Why can you not share an `Rc<T>` between threads, and what do you use instead?

A: `Rc`’s reference count is not atomic, so concurrent updates would race; `Rc` is not `Send`, and the compiler rejects sending it to another thread. Use `Arc<T>`, which uses an atomic count and is safe to share. For shared mutation, wrap the data: `Arc<Mutex<T>>`.

Q: How does Rust prevent data races at compile time? A: Through the `Send` and `Sync` marker traits. `Send` means a type is safe to move to another thread; `Sync` means `&T` is safe to share between threads. The compiler only allows thread APIs on types that satisfy these, so code with a data race does not compile. Shared mutable state must go through a `Mutex/RwLock` (often inside an `Arc`).

Chapter 29 — Interview Questions and Answers

This chapter is a bank of real Rust interview questions with short, correct answers, grouped by topic, plus code puzzles and design questions at the end.

How to use this. Read the question, try to answer out loud before you read the answer, and write the code snippets by hand. The questions double as a review of the whole book; each group maps to chapters you have already read. The “Predict / explain” puzzles are the kind interviewers love, because they reveal whether you really understand ownership, not just the syntax.

Basics

Q: What is the difference between `let` and `let mut`? A: `let` binds an immutable value; you cannot reassign it. `let mut` binds a mutable value you can change. Rust is immutable by default — the opposite of C, where everything is mutable unless you write `const`.

Q: What is shadowing, and how does it differ from mutation? A: Shadowing means declaring a new variable with the same name, which hides the old one. It can change the type, and the old value still existed independently. Mutation changes the same variable in place and keeps its type.

```
fn main() {
    let x = "42"; // &str
    let x: i32 = x.parse().unwrap(); // new variable, new type
    println!("{x}");
}
```

Q: Why is immutability the default? A: It makes code easier to reason about, helps the optimizer, and supports the borrow rules (you can share an immutable value freely between many readers). You opt into mutation with `mut`, which documents intent.

Q: List Rust’s scalar types and their C analogues. A: Integers `i8..i128/u8..u128` plus `isize/usize` (like `size_t`); floats `f32/f64`; `bool`; and `char` (a 4-byte Unicode scalar, not C’s 1-byte `char`). Sizes are fixed by the language, unlike C where `int` width is platform-dependent.

Q: How big is a `char`, and why? A: Four bytes. A Rust `char` is a Unicode scalar value, so it can hold any single character (`'é'`, `' '`), not just ASCII. A single byte is `u8`. Strings store UTF-8 bytes, so a `char` and the bytes used to encode it in a string are not the same size.

Q: What does `as` do, and when should you avoid it? A: `as` is an explicit primitive cast (for example `x as u8`). It can truncate or change sign silently. Prefer `From/into` for lossless conversions; use `as` only when you accept the possible data loss, or `try_into` to get a checked `Result`.

Q: Why is `if/match/loop` being an “expression” useful? A: They evaluate to a value, so you can write `let y = if c { a } else { b }`; with no temporary mutable variable, and `break` value out of a loop. The last expression in a block (without a semicolon) is its value. A stray semicolon turns it into a statement of type `()` and discards the value.

Ownership and borrowing

Q: What is ownership? A: Every value has exactly one owner (a variable). When the owner goes out of scope, the value is dropped (freed) automatically. This replaces manual `malloc/free` and guarantees each value is freed exactly once.

Q: What is the difference between a move and a copy? A: For types that are not `Copy` (like `String`, `Vec`), assignment or passing *moves* ownership and invalidates the source. For `Copy` types (small, fixed-size: integers, `bool`, `char`, references, arrays of `Copy`), the value is duplicated bit-for-bit and the source stays valid.

Q: State the borrow rules and explain why they exist. A: At any time you may have either any number of shared references (`&T`) or exactly one mutable reference (`&mut T`), never both, and no reference may outlive the value. “Aliasing XOR mutation” prevents data races and iterator-invalidation bugs, and it lets the compiler reason about and optimize the code.

Q: Why can there be only one `&mut` at a time? A: If two mutable references existed, two pieces of code could write the same memory at once (a data race) or invalidate each other’s assumptions. Exclusivity makes mutation safe and is what makes `&mut T` analogous to a `restrict` pointer in C — but enforced.

Q: When do you need `.clone()`? A: When you need an independent, owned copy of a non-`Copy` value (for example to keep using the original after giving one away). Cloning is explicit because it may be expensive (a deep copy / allocation). It is a reasonable first move to satisfy the borrow checker, then optimize away if it matters.

Q: What is Drop, and how does it relate to RAI and C? A: Drop is the trait whose drop method runs automatically when a value goes out of scope, like a C++ destructor. It implements RAI: resources (memory, files, locks) are released deterministically. In C you would call free/fclose/unlock by hand; in Rust the compiler inserts the cleanup, in reverse order of declaration.

Q: Can you call drop yourself? A: Not the Drop::drop method directly. You call std::mem::drop(value), which takes ownership and lets the value go out of scope early. This is how you release a lock or free memory before the end of the scope.

Lifetimes

Q: What is a lifetime? A: A lifetime is the span of code during which a reference is valid. It is a compile-time concept with no runtime cost. Annotations like 'a describe relationships between the lifetimes of inputs and outputs so the compiler can prove no reference dangles.

Q: What is lifetime elision? A: A set of rules that let you omit lifetime annotations in common cases. For example, a method that takes &self and returns a reference gets the lifetime of self automatically. Most function signatures need no explicit lifetimes thanks to elision.

Q: What does 'static mean? A: The reference is valid for the entire program. String literals are &'static str because they live in the binary. As a bound (T: 'static) it means the type contains no shorter-lived references. It does *not* mean “leaked forever” — an owned String also satisfies T: 'static.

Q: When does a struct need a lifetime parameter? A: When it stores a reference. The lifetime ties the struct’s validity to the borrowed data, so the struct cannot outlive what it points to.

```
struct Parser<'a> {  
    input: &'a str, // the struct must not outlive `input`  
}
```

Q: Why do lifetimes exist if they have no runtime cost? A: They give the borrow checker the information it needs to reject dangling references at compile time. They are documentation that the compiler also verifies.

Enums and pattern matching

Q: How does Option<T> replace null? A: Option<T> is an enum with Some(T) or None. “No value” is encoded in the type, and the compiler forces you to handle None, so you cannot dereference a null by accident. There is no null pointer in safe Rust.

Q: What is Result<T, E>? A: An enum with Ok(T) for success and Err(E) for failure. It is how fallible functions report errors — Rust has no exceptions. It is #[must_use], so ignoring it warns.

Q: What does it mean that match is exhaustive? A: You must cover every possible case, or the code does not compile. If you add a variant to an enum, the compiler points you at every match you must update. Arms do not fall through, unlike C’s switch.

Q: What is niche optimization? A: The compiler reuses an unused bit pattern (“niche”) to encode an enum’s tag for free. The classic example: Option<&T> is the same size as &T because a

reference is never null, so the all-zeros pattern represents `None`. You get null-safety with no extra space.

Q: How do you handle just one variant without a full match? A: Use `if let` (or `let ... else`) to match one pattern and ignore the rest.

```
fn main() {
    let maybe = Some(7);
    if let Some(n) = maybe {
        println!("got {n}");
    }
}
```

Error handling

Q: When do you return a `Result` versus calling `panic!`? A: Return `Result` for expected, recoverable errors the caller should handle (file not found, bad input). Use `panic!` for bugs and broken invariants that should never happen (a failed assertion, an index that must be valid). A library should almost always return `Result`, not `panic`.

Q: What does the `?` operator do? A: It unwraps an `Ok/Some` value, or returns the `Err/None` early from the current function. It is concise error propagation, replacing C's repetitive "check the return code and return it" pattern.

```
use std::num::ParseIntError;

fn double(s: &str) -> Result<i32, ParseIntError> {
    let n: i32 = s.parse()?; // returns the error on failure
    Ok(n * 2)
}
```

Q: How does `?` convert error types? A: It calls `From::from` on the error, so any error type that implements `From<SourceError>` is converted to the function's declared error type. This lets one function return one error type while calling code that fails in different ways.

Q: What are `thiserror` and `anyhow` for? A: `thiserror` is a derive macro to build custom error enums (typed errors for libraries). `anyhow` provides a single boxed error type (`anyhow::Error`) plus context, convenient for applications where you do not need to match on the error.

Q: What does `#[must_use]` mean on `Result`? A: The compiler warns if you ignore the returned value. It nudges you to handle the error instead of silently dropping it, unlike C where ignoring a return code is invisible.

Q: What does `.unwrap()` do, and when is it acceptable? A: It returns the inner value or panics on `Err/None`. It is fine in examples, prototypes, tests, and cases where the value is provably present. In production paths prefer `?` or explicit handling, and use `.expect("why")` when you do `unwrap` so the panic message explains the assumption.

Traits and generics

Q: What is the difference between static and dynamic dispatch? A: Static dispatch uses generics: the compiler generates a specialized copy of the function per concrete type (monomor-

phization), so calls are direct and inlinable — zero cost. Dynamic dispatch uses a trait object (`dyn Trait`): the concrete type is chosen at run time through a vtable, like a C struct of function pointers. It adds an indirection but allows mixing types at run time.

Q: What is a trait object, and how is it laid out? A: `dyn Trait` is a value of some type that implements `Trait`, accessed behind a pointer. The pointer is “fat”: it carries two words — a data pointer and a pointer to a vtable (the method table). It is the Rust version of a hand-rolled C interface made of function pointers.

```
&dyn Draw = [ data ptr | vtable ptr ]
              |
              v
            [ drop | size | align | draw fn | ... ]
```

Q: What is monomorphization? A: For each concrete type used with a generic, the compiler stamps out a dedicated copy of the code. This gives C++-template-like performance (no runtime dispatch) at the cost of larger binaries and longer compile times.

Q: What is the orphan rule? A: You may implement a trait for a type only if you own the trait or the type. This prevents two crates from giving conflicting implementations for the same pair. To work around it for a foreign trait on a foreign type, wrap the type in a newtype you own.

Q: What is object safety? A: A trait is “object safe” (can be made into `dyn Trait`) only if its methods can be dispatched through a vtable — for example, no generic methods and no methods that return `Self` by value. If a trait is not object safe, you cannot use it as a trait object; use generics instead.

Q: Name common standard traits and what they give you. A: `Debug` (the `{:?}` format), `Clone/Copy` (duplication), `PartialEq/Eq` (equality), `PartialOrd/Ord` (ordering), `Default` (a default value), `Hash` (use as a `HashMap` key), `From/Into` (conversions), `Iterator`, `Display` (user-facing `{}`), and `Drop` (cleanup). Many can be `#[derive]`d.

Q: What is the difference between `impl Trait` and `dyn Trait` in a return type? A: `-> impl Trait` returns one concrete (but unnamed) type chosen at compile time; it is static dispatch and zero cost, but every path must return the *same* type. `-> Box<dyn Trait>` returns a heap-allocated trait object via dynamic dispatch and can return different concrete types from different branches.

Smart pointers

Q: When do you use `Box<T>`? A: To put a value on the heap: for large values you do not want on the stack, for recursive types (a linked list or tree node that contains itself), and to store a trait object (`Box<dyn Trait>`). It is a single owner, like a `unique_ptr`.

Q: What is the difference between `Rc` and `Arc`? A: Both give shared ownership through reference counting; the value is freed when the last owner drops. `Rc` uses a non-atomic counter (fast, single-thread only). `Arc` uses an atomic counter (safe to share across threads, slightly slower). The compiler stops you from sending `Rc` across threads.

Q: What is interior mutability, and what is `RefCell`? A: Interior mutability lets you mutate data through a shared reference (`&T`). `RefCell<T>` provides it by enforcing the borrow rules at *run*

time instead of compile time: `borrow()` and `borrow_mut()` track borrows, and a violation panics. It is for single-threaded code; the thread-safe equivalents are `Mutex/RwLock`.

Q: How do reference-counting cycles cause leaks, and how do you fix them? A: If two `Rc`s point at each other, neither count reaches zero, so neither is freed — a leak. Break the cycle with `Weak<T>`, a non-owning reference you upgrade to `Rc` only when needed. Typically children own via `Rc` and point back to parents via `Weak`.

Q: What is `Rc<RefCell<T>>` for? A: Multiple owners (`Rc`) of a value that any of them can mutate (`RefCell`). It is the common single-threaded pattern for shared mutable state, such as nodes in a graph. The thread-safe version is `Arc<Mutex<T>>`.

Concurrency

Q: What are `Send` and `Sync`? A: Marker traits the compiler uses for thread safety. `Send` means a value can be moved to another thread. `Sync` means `&T` can be shared between threads (i.e. `T` is `Send + Sync`-friendly to reference). Most types are both automatically; types like `Rc` and `RefCell` are not, which is how unsafe sharing is rejected.

Q: What is “fearless concurrency”? A: The idea that the same ownership and `Send/Sync` rules that ensure single-threaded safety also rule out data races at compile time. You can write threaded code aggressively because the compiler refuses to build a race.

Q: How do you share mutable state between threads? A: Wrap it in `Arc<Mutex<T>>` (or `Arc<RwLock<T>>`). `Arc` gives shared ownership across threads; `Mutex` guarantees one writer at a time. The data lives *inside* the mutex, so you cannot access it without locking.

Q: What are channels? A: Message-passing pipes (`std::sync::mpsc`): one or more senders push values, a receiver pulls them. They let threads communicate by transferring ownership of data rather than sharing memory — “do not communicate by sharing memory; share memory by communicating.”

Q: How exactly does Rust prevent a data race? A: A data race needs two threads, at least one writing, the same memory, with no synchronization. Rust breaks this: `&mut T` is exclusive (no other access while it exists), and shared types that allow mutation must be `Sync` (a `Mutex` makes access exclusive). Types that are not thread-safe are not `Send/Sync`, so they cannot cross threads. The check is at compile time.

Q: What is the difference between `async` and threads? A: Threads are OS-scheduled and good for CPU-bound or blocking work; each has its own stack. `Async` tasks are lightweight, cooperatively scheduled by a runtime (like Tokio), and excel at many concurrent I/O operations on few threads. `Async` avoids the per-thread memory cost when you have thousands of waiting connections.

Q: What does “futures are lazy” mean? A: An `async` function returns a `Future` that does nothing until it is `.awaited` or handed to an executor. Creating a future does not start the work, unlike, say, spawning a thread. No runtime polls it, nothing happens.

Memory and runtime

Q: What is the difference between the stack and the heap in Rust? A: The same as in C. The stack holds fixed-size values with automatic, LIFO lifetime (function locals). The heap holds

dynamically sized or long-lived data, reached through owners like `Box`, `Vec`, and `String`. Rust frees heap data automatically when its owner drops.

Q: Rust has no garbage collector — how is memory freed? A: Through ownership and `Drop`. When an owner goes out of scope, its `drop` runs and frees any owned resources, deterministically and at a known point. There are no GC pauses; cleanup cost is predictable, exactly as with manual `free` in C but automatic and correct.

Q: Can safe Rust leak memory? A: Yes. Leaking memory is safe (it does not cause undefined behavior), so it is allowed. You can leak with reference-count cycles (`Rc`), or on purpose with `Box::leak` or `std::mem::forget`. Leaks are a logic bug, not a safety bug.

Q: What is a “zero-cost abstraction”? A: A high-level feature that compiles to code as efficient as the hand-written low-level version. Iterators, generics, `Option`, and `async/await` are examples: you pay no runtime penalty for the convenience. “What you do not use, you do not pay for; what you do use, you could not hand-code better.”

Q: Where do `Vec<T>` and `String` keep their data? A: The header (pointer, length, capacity) lives on the stack or inside its owner; the elements live on the heap. This is exactly a `malloc`’d buffer plus a length and capacity, managed for you.

Unsafe and FFI

Q: What does `unsafe` actually allow? A: Five extra powers: dereference a raw pointer, call an `unsafe` function (incl. FFI), access or modify a mutable static, implement an `unsafe` trait, and access union fields. That is all. It does not turn off the type system.

Q: Does `unsafe` disable the borrow checker? A: No. The borrow checker and type checker still run inside `unsafe` blocks. `unsafe` only unlocks the five operations above. You are promising the compiler you upheld the invariants it cannot check (for example, that a raw pointer is valid).

Q: How do you call a C function from Rust? A: Declare it in an `extern "C"` block with its signature and link the library. Calls are `unsafe` because Rust cannot verify the C side.

```
unsafe extern "C" {
    fn abs(input: i32) -> i32; // from the C standard library
}

fn main() {
    let x = unsafe { abs(-5) };
    println!("{x}");
}
```

Q: Why is `#[repr(C)]` needed for FFI? A: Rust may reorder struct fields for efficiency, so the default layout is not guaranteed. `#[repr(C)]` forces C-compatible layout (declaration order with C padding rules) so the struct matches what the C code expects.

Q: What is the goal of an `unsafe` block in good Rust code? A: To isolate the few operations the compiler cannot verify, wrap them in a safe API that upholds the invariants, and keep the `unsafe` region small and well-documented. The rest of the program stays safe and relies on that wrapper.

Cargo and tooling

Q: What is the difference between a crate and a package? A: A crate is a single compilation unit — one library or one binary. A package is a bundle described by one `Cargo.toml`; it can contain at most one library crate and any number of binary crates.

Q: What is `Cargo.lock`, and should you commit it? A: It records the exact dependency versions resolved for a build, so builds are reproducible. Commit it for applications (binaries). For libraries it is conventionally not committed, so downstream users resolve their own versions.

Q: What is `clippy`? A: The official linter (`cargo clippy`). It catches common mistakes and suggests more idiomatic code beyond what the compiler warns about — for example, “use `if let` instead of `match`,” or flagging a needless `.clone()`.

Q: What are Rust editions? A: Named language epochs (2015, 2018, 2021, 2024) that let Rust make otherwise-breaking syntax changes without splitting the ecosystem. Each crate picks its edition in `Cargo.toml`; crates of different editions interoperate. This book targets edition 2024.

Q: How do you add a dependency? A: `cargo add <crate>` edits `Cargo.toml` for you (or you add it by hand under `[dependencies]`). `cargo build` then fetches and compiles it. Versions follow semantic versioning.

Q: What does `cargo check` do that `cargo build` does not? A: `cargo check` type-checks and borrow-checks without producing a binary, so it is much faster. It is what you run repeatedly while editing; `cargo build` when you actually need the executable.

Predict / explain (small puzzles)

For each puzzle, decide what happens and why before reading the answer.

1. Use after move.

```
// COMPILE ERROR: borrow of moved value: `s`
fn main() {
    let s = String::from("hi");
    let t = s; // move
    println!("{s} {t}");
}
```

Answer: Does not compile (E0382). `String` is not `Copy`, so `let t = s` moves ownership; `s` is invalid afterward. Fix: borrow (`let t = &s;`) or clone (`let t = s.clone();`).

2. Borrow conflict.

```
// COMPILE ERROR: cannot borrow `v` as mutable because it is also borrowed as immutable
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0]; // shared borrow, still alive below
    v.push(4); // needs a mutable borrow -> error[E0502]
    println!("{first}");
}
```

Answer: Does not compile. `push` may reallocate the buffer and invalidate `first`, so the borrow checker rejects mutating `v` while `first` lives. Fix: copy the value out (`let first = v[0];`) before

pushing, or finish using `first` first.

3. Option handling.

```
fn main() {
    let v = vec![10, 20, 30];
    match v.get(5) {
        Some(x) => println!("got {x}"),
        None => println!("nothing"),
    }
}
```

Answer: Compiles and prints `nothing`. `.get()` returns `Option<T>`, and index 5 is out of range, so it is `None`. Compare with `v[5]`, which would panic. This is how you index safely.

4. Iterator laziness.

```
fn main() {
    let nums = vec![1, 2, 3];
    nums.iter().map(|n| println!("{n}")); // no output!
    println!("done");
}
```

Answer: Prints only `done`. `map` is lazy: it builds an iterator but does nothing until consumed. Nothing drives it, so the closure never runs (the compiler also warns about an unused iterator). Fix: use `for n in &nums { println!("{n}"); }`, or add `.for_each(...)`, or `.collect()`.

5. Integer overflow in debug.

```
fn main() {
    let x: u8 = std::hint::black_box(200);
    let y: u8 = std::hint::black_box(100);
    println!("{}", x + y); // ?
}
```

Answer: In a debug build this panics (“attempt to add with overflow”) because $300 > 255$. In a release build it wraps to 44. Do not rely on either: use `x.wrapping_add(y)` to wrap on purpose or `x.checked_add(y)` to get an `Option`. (The `black_box` calls stop the compiler from constant-folding the values; if you write `200u8 + 100u8` as plain literals, the compiler rejects it at compile time.)

6. Rc cycle.

```
use std::rc::Rc;
use std::cell::RefCell;

struct Node {
    next: RefCell<Option<Rc<Node>>>,
}

fn main() {
    let a = Rc::new(Node { next: RefCell::new(None) });
    let b = Rc::new(Node { next: RefCell::new(None) });
```

```

    *a.next.borrow_mut() = Some(Rc::clone(&b));
    *b.next.borrow_mut() = Some(Rc::clone(&a)); // cycle!
    // a and b keep each other alive; memory leaks at end of main.
}

```

Answer: Compiles and runs, but leaks. `a` and `b` reference each other, so neither strong count reaches zero and neither `Node` is dropped. This is a safe leak (no undefined behavior). Fix: make one link a `Weak<Node>` to break ownership of the cycle.

Design questions

These open-ended questions check whether you reach for idiomatic Rust shapes.

Q: Model a traffic-light state machine. A: Use an enum for the states and a method that returns the next state. Enums make illegal states unrepresentable, and exhaustive `match` guarantees every transition is handled.

```

#[derive(Debug, Clone, Copy, PartialEq)]
enum Light {
    Red,
    Green,
    Yellow,
}

impl Light {
    fn next(self) -> Light {
        match self {
            Light::Red => Light::Green,
            Light::Green => Light::Yellow,
            Light::Yellow => Light::Red,
        }
    }
}

fn main() {
    let mut s = Light::Red;
    for _ in 0..4 {
        println!("{s:?}");
        s = s.next();
    }
}

```

Q: Build a thread-safe counter shared by many threads. A: Wrap the count in `Arc<Mutex<T>>`. `Arc` gives each thread shared ownership; `Mutex` makes increments mutually exclusive. The lock auto-releases when the guard drops.

```

use std::sync::{Arc, Mutex};
use std::thread;

fn main() {

```

```

let counter = Arc::new(Mutex::new(0u64));
let mut handles = vec![];
for _ in 0..8 {
    let c = Arc::clone(&counter);
    handles.push(thread::spawn(move || {
        for _ in 0..1000 {
            *c.lock().unwrap() += 1;
        }
    }));
}
for h in handles {
    h.join().unwrap();
}
println!("{}", *counter.lock().unwrap()); // 8000
}

```

For a counter specifically, you could also use `AtomicU64` with no lock at all, which is faster for a single number.

Q: Sketch the design of an LRU (least-recently-used) cache. A: You need $O(1)$ lookup and $O(1)$ “move to most-recently-used.” The classic design combines a hash map (key to node) with a doubly linked list ordered by recency:

- On `get`, find the node via the map and move it to the front of the list.
- On `put`, insert at the front; if over capacity, evict the node at the back and remove it from the map.

In Rust, a doubly linked list with shared mutable nodes fights the borrow checker. Three practical options:

1. Use `std::collections::HashMap` plus a `VecDeque` or an index-based list, storing `usize` indices into a `Vec<Node>` instead of references — the most common safe approach.
2. Use `Rc<RefCell<Node>>` for the nodes and `Weak` for back-pointers to avoid cycles.
3. Use the `lru` crate, which has already solved this carefully (sometimes with a small, well-audited `unsafe` core).

The interview point is to recognize that intrusive doubly linked lists are the one data structure that genuinely fights Rust’s aliasing rules, and to name the index-based or `Rc<RefCell>` workarounds rather than reaching for `unsafe` first.

Chapter 30 — Capstone Projects

What you’ll learn. How the pieces of this book fit together in three complete, runnable programs: a command-line tool, a multithreaded counter, and a small generic data structure with tests. Each project is built up section by section, with the full code, how to run it, and the chapters it draws on.

You have learned the parts of Rust one at a time. This chapter puts them to work. We build three small but complete programs. Each one is the kind of thing a C programmer writes often: a Unix-style command-line filter, a parallel data-processing task, and a reusable container.

Every program here uses **only the standard library**, so it builds offline with no dependencies. You can find the exact same code in this book's `examples/` folder, which is one Cargo package named `rustbook-examples`. The code in this chapter and the code in `examples/` are identical and were compiled and tested together.

How to follow along. Create a package once with `cargo new rustbook-play` (Chapter 2 — Installing Rust and the Cargo Toolchain), then paste each project into the right file as the text explains. Or open the book's `examples/` package and run the binaries directly.

The three projects map onto the package like this:

```
examples/
Cargo.toml           package "rustbook-examples", edition 2024
src/
  lib.rs             Project 3: generic Stack and RPN evaluator + tests
  bin/
    linecount.rs     Project 1: a wc-like counter
    parallel_count.rs Project 2: a concurrent word-frequency counter
```

A package can hold one library crate (`src/lib.rs`) and many binary crates (each file in `src/bin/`). You run a binary with `cargo run --bin <name>` (Chapter 3 — Program Structure: Crates, Modules, and Visibility). The `Cargo.toml` is tiny because we use no external crates:

```
[package]
name = "rustbook-examples"
version = "0.1.0"
edition = "2024"

[dependencies]
```

Project 1 — `linecount`: a `wc`-like command-line tool

Goal. Build a small clone of the Unix `wc` tool. It counts lines, words, and bytes. It reads the files named on the command line, or standard input when no files are given. It supports the flags `-l` (lines), `-w` (words), and `-c` (bytes). With no flag it prints all three, just like `wc`.

This is the classic Unix filter shape every C programmer knows: read input, process it line by line, write a summary, return a status code. We will write it the Rust way — with `Result` and `?` for errors, `BufRead` for buffered input, and explicit `stdout/stderr`.

C vs Rust. In C you would loop with `getline` or `fgets`, track counts in `long` variables, check every return value by hand, and print to `stdout` while sending errors to `stderr`. Rust keeps that exact structure but the error checking rides along on the `?` operator, and the buffered reader is a generic type instead of a `FILE *`.

The data: two small structs

We hold the three counts in one struct, and the chosen flags in another. Both `derive` the traits that make them easy to copy and print (Chapter 15 — Traits). `Counts` derives `Default` so

`Counts::default()` gives an all-zero value, the same idea as zero-initializing a C struct.

```
use std::env;
use std::fs::File;
use std::io::{self, BufRead, BufReader, Write};
use std::process::ExitCode;

/// The three counts we collect for one input.
#[derive(Debug, Default, Clone, Copy)]
struct Counts {
    lines: u64,
    words: u64,
    bytes: u64,
}

impl Counts {
    /// Add another set of counts into this one (used for the total line).
    fn add(&mut self, other: &Counts) {
        self.lines += other.lines;
        self.words += other.words;
        self.bytes += other.bytes;
    }
}

/// Which columns the user asked for.
#[derive(Debug, Clone, Copy)]
struct Flags {
    lines: bool,
    words: bool,
    bytes: bool,
}

impl Flags {
    /// When the user gives no flag at all, show every column.
    fn or_default(self) -> Flags {
        if !self.lines && !self.words && !self.bytes {
            Flags {
                lines: true,
                words: true,
                bytes: true,
            }
        } else {
            self
        }
    }
}
```

`Counts` and `Flags` are `Copy` (Chapter 7 — Ownership and Moves), so passing them around does not move them; they behave like plain C structs of integers and booleans. The `add` method takes

`&self as &mut` and another `&Counts` by shared reference (Chapter 8 — Borrowing and References) — it changes the totals in place without taking ownership of the other value.

The core: counting over any buffered reader

Here is the heart of the program. It is **generic** over the input type: it works for a file, for standard input, or for anything else that implements `BufRead` (Chapter 14 — Generics). That is how one function serves both the file case and the stdin case with no duplication.

```
/// Read everything from `reader` and count lines, words, and bytes.
fn count<R: BufRead>(mut reader: R) -> io::Result<Counts> {
    let mut counts = Counts::default();
    let mut line = Vec::new();

    // read_until keeps the trailing '\n', so byte counts match `wc`.
    loop {
        line.clear();
        let n = reader.read_until(b'\n', &mut line)?;
        if n == 0 {
            break; // end of input
        }
        counts.bytes += n as u64;
        if line.ends_with(b"\n") {
            counts.lines += 1;
        }
        // Words are runs of non-whitespace bytes, split on ASCII whitespace.
        let text = String::from_utf8_lossy(&line);
        counts.words += text.split_whitespace().count() as u64;
    }
    Ok(counts)
}
```

A few points a C programmer will appreciate:

- `fn count<R: BufRead>(...)` is the generic. `R` is any type that implements the `BufRead` trait — a trait is like a contract, the rough Rust equivalent of a struct of function pointers in C (Chapter 15 — Traits). At compile time Rust generates a specialized copy of `count` for each concrete `R` it sees. There is no dynamic dispatch and no runtime cost.
- `reader.read_until(b'\n', &mut line)?` reads bytes up to and including the next newline into `line`. It returns `io::Result<usize>` — the number of bytes read, or an error. The `?` says: if this is an error, stop and return it from `count`; otherwise unwrap the count (Chapter 13 — Error Handling). This replaces C’s “check the return value of every read” boilerplate.
- We read into a reusable `Vec<u8>` of raw bytes, not a `String`, so a file that is not valid UTF-8 cannot make us fail. `String::from_utf8_lossy` then gives a text view for word splitting, replacing any bad bytes (Chapter 10 — Slices and Strings).
- `split_whitespace()` yields each run of non-space characters; `.count()` reports how many. This is the iterator pipeline doing the work a hand-written `for` loop would do in C (Chapter 16 — Collections and Iterators).

Watch out. Counting bytes is not the same as counting characters. We add the

number of **bytes** read, which matches `wc -c`. A UTF-8 file can have more bytes than characters.

Printing one row

`print_row` writes a single line of output, including only the columns the flags asked for. It takes `out: &mut impl Write`, so it can write to a locked `stdout`, a file, or — in a test — a `Vec<u8>` buffer. `impl Write` is shorthand for “some type that implements the `Write` trait” (Chapter 15 — Traits).

```
/// Print one result row, only the columns the flags asked for.
fn print_row(out: &mut impl Write, c: &Counts, flags: Flags, label: &str) -> io::Result<()> {
    if flags.lines {
        write!(out, "{:>8}", c.lines)?;
    }
    if flags.words {
        write!(out, "{:>8}", c.words)?;
    }
    if flags.bytes {
        write!(out, "{:>8}", c.bytes)?;
    }
    if label.is_empty() {
        writeln!(out)
    } else {
        writeln!(out, " {label}")
    }
}
```

`{:>8}` is a format specifier: right-align in a field eight wide, like `printf("%8lu")`. `write!/writeln!` are the buffered cousins of `print!/println!` that send output to a chosen writer instead of always to `stdout` (Chapter 26 — Macros). Each returns `io::Result<()>`, so a broken pipe surfaces as an error instead of being ignored.

Parsing the command line

There is no `getopt` in the standard library, so we parse the few flags by hand. This function turns the raw argument list into a `Flags` value and a list of file paths. It returns a `Result`: an unknown flag is an error, not a panic.

```
/// Parse arguments into flags and the list of file paths.
fn parse_args(args: Vec<String>) -> Result<(Flags, Vec<String>), String> {
    let mut flags = Flags {
        lines: false,
        words: false,
        bytes: false,
    };
    let mut paths = Vec::new();

    for arg in args {
        if arg == "--" {
```



```

        continue;
    }
    if arg.starts_with('-') && arg.len() > 1 {
        for ch in arg.chars().skip(1) {
            match ch {
                'l' => flags.lines = true,
                'w' => flags.words = true,
                'c' => flags.bytes = true,
                other => return Err(format!("unknown flag: -{other}")),
            }
        }
    } else {
        paths.push(arg);
    }
}
Ok((flags, paths))
}

```

Note how one argument like `-lw` is handled: we iterate over its characters after the `-`, so combined short flags work just like in real Unix tools. The `match` on a character is exhaustive in spirit — the `other` arm catches every unknown flag and turns it into an error string (Chapter 12 — Enums and Pattern Matching).

Rule of thumb. For anything beyond a handful of flags, reach for the `clap` crate. It gives you `--help`, value parsing, subcommands, and error messages for free. We hand-roll the parser here only to stay standard-library-only and to show the mechanics. Adding `clap` would be `cargo add clap` (Chapter 22 — Cargo, Crates, and Workspaces).

Driving it all: run and main

Rust convention is to keep `main` tiny and put the real work in a `run` function that returns a `Result`. `main` then maps that result onto a process exit code. This keeps the `?` operator usable throughout `run`.

```

fn run() -> Result<(), String> {
    let raw: Vec<String> = env::args().skip(1).collect();
    let (flags, paths) = parse_args(raw)?;
    let flags = flags.or_default();

    let stdout = io::stdout();
    let mut out = stdout.lock();

    if paths.is_empty() {
        // No files: read standard input.
        let stdin = io::stdin();
        let counts = count(stdin.lock()).map_err(|e| format!("stdin: {e}"))?;
        print_row(&mut out, &counts, flags, "").map_err(|e| e.to_string())?;
        return Ok(());
    }
}

```

```

    }

    let mut total = Counts::default();
    let mut had_error = false;
    let many = paths.len() > 1;

    for path in &paths {
        match open_and_count(path) {
            Ok(counts) => {
                total.add(&counts);
                print_row(&mut out, &counts, flags, path).map_err(|e| e.to_string())?;
            }
            Err(e) => {
                // Report the error but keep going with the other files.
                eprintln!("linecount: {path}: {e}");
                had_error = true;
            }
        }
    }

    if many {
        print_row(&mut out, &total, flags, "total").map_err(|e| e.to_string())?;
    }

    if had_error {
        Err("one or more files could not be read".to_string())
    } else {
        Ok(())
    }
}

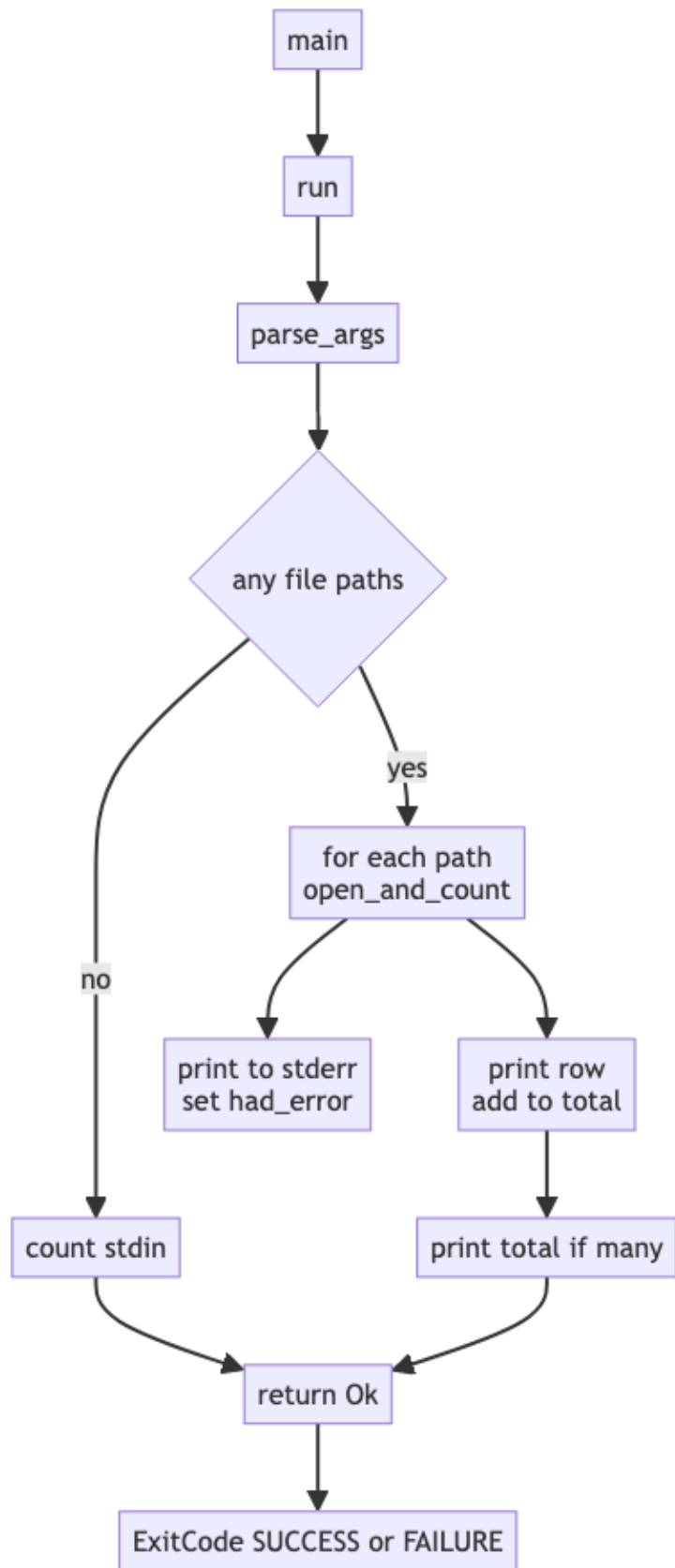
/// Open a file and count it. A helper so `?` can do the error plumbing.
fn open_and_count(path: &str) -> io::Result<Counts> {
    let file = File::open(path)?;
    // BufReader avoids one syscall per byte, like a stdio FILE* buffer in C.
    count(BufReader::new(file))
}

fn main() -> ExitCode {
    match run() {
        Ok(()) => ExitCode::SUCCESS,
        Err(msg) => {
            eprintln!("linecount: {msg}");
            ExitCode::FAILURE
        }
    }
}

```

What to notice:

- `env::args().skip(1).collect()` gathers the command-line arguments into a `Vec<String>`, skipping the program name in slot 0 — the same `argv[0]` you skip in C (Chapter 16 — Collections and Iterators).
- `io::stdout().lock()` takes the lock on stdout once and holds it for the whole run. In C each `printf` re-acquires an internal lock; here we lock once, which is both faster and explicit.
- `BufReader::new(file)` wraps the file in a buffer, exactly like a stdio `FILE *`. Without it, every byte would be its own system call.
- Errors on one file do **not** abort the whole program. We print the problem to stderr with `eprintln!`, set a flag, and keep going — then exit non-zero at the end. This is how `wc` behaves too.
- `main` returns `ExitCode`. `ExitCode::SUCCESS` is status 0; `ExitCode::FAILURE` is non-zero. This is Rust's replacement for `return 0; / return 1;` (Chapter 1 — Why Rust for a C Programmer).



How to run it

Put the code in `src/bin/linecount.rs` and run it with Cargo:

```
# count this file
cargo run --bin linecount -- Cargo.toml

# only lines and words, from a pipe
printf 'hello world\nfoo bar baz\n' | cargo run --bin linecount -- -l -w

# several files, with a total row
cargo run --bin linecount -- src/lib.rs src/bin/linecount.rs
```

The `--` separates Cargo’s own arguments from your program’s arguments. Example output for the pipe above:

```
2      5
```

Chapters and concepts this exercises

- Chapter 3 — Program Structure (binary crates under `src/bin/`).
- Chapter 7 — Ownership and Moves, and Chapter 8 — Borrowing and References (`Copy` structs, `&mut self`, shared references).
- Chapter 10 — Slices and Strings (`&str`, `from_utf8_lossy`, `split_whitespace`).
- Chapter 13 — Error Handling (`Result`, `?`, `map_err`, returning from `main`).
- Chapter 14 — Generics and Chapter 15 — Traits (`BufRead`, `impl Write`).
- Chapter 16 — Collections and Iterators (`env::args`, `.collect()`, `.count()`).

Extend it

- Add a `-m` flag that counts characters (use `text.chars().count()`), and notice how it differs from `-c` on a UTF-8 file.
- Add a “longest line length” column, like `wc -L`.
- Swap the hand-rolled parser for `clap` and add `--help`.
- Read files in parallel using the threads from Project 2.

Project 2 — `parallel_count`: a multithreaded word-frequency counter

Goal. Count how often each word appears in a body of text, using several worker threads. We split the text into chunks, count each chunk on its own thread, and merge the partial results. We use a **bounded** number of workers, and we **join** every thread, so the program always terminates. The result is deterministic: the same input always gives the same totals.

This is the “fearless concurrency” promise in action (Chapter 19 — Threads and Concurrency). The compiler proves at build time that no two threads touch the same data without synchronization. A data race is not a runtime risk to debug; it is a compile error you never even reach.

C vs Rust. In C you would use `pthread_create`, pass each thread a pointer to its slice, guard the shared map with a `pthread_mutex_t`, and hope you locked in all the right places. Forget one lock and you get a race that shows up once a month in production.

In Rust the type system tracks which data crosses thread boundaries (**Send/Sync**), and a missing lock is a type error.

We will use two standard-library tools together:

- `std::thread::scope` — scoped threads that are guaranteed to finish before the scope ends, so they may safely **borrow** data on the parent’s stack. No **Arc** and no **'static** requirement (Chapter 19 — Threads and Concurrency).
- `std::sync::mpsc` — a multi-producer, single-consumer **channel**. Each worker sends its partial result; the main thread receives and merges them (Chapter 20 — Channels and Shared State).

Using a channel means there is **no shared mutable map** at all. Each thread owns its own map and sends it home. That sidesteps locking entirely — a very common and clean Rust pattern.

The input and the per-chunk counter

```
use std::collections::HashMap;
use std::sync::mpsc;
use std::thread;

/// Sample input. In a real tool this would come from a file or stdin.
const TEXT: &str = "\
the quick brown fox
the lazy dog
the fox jumps over the lazy dog
brown fox brown dog
the the the quick brown fox";

/// How many worker threads to run. Bounded on purpose.
const WORKERS: usize = 4;

/// Count words in one slice of lines into a fresh map.
fn count_chunk(lines: &[&str]) -> HashMap<String, u64> {
    let mut local = HashMap::new();
    for line in lines {
        for word in line.split_whitespace() {
            *local.entry(word.to_string()).or_insert(0) += 1;
        }
    }
    local
}
```

`count_chunk` takes a slice of string slices (`&[&str]`) and returns a fresh `HashMap<String, u64>` (Chapter 16 — Collections and Iterators). The `entry(...).or_insert(0)` pattern is the idiomatic “increment a counter, starting at zero if absent” — it returns a mutable reference to the value, which we dereference with `*` to add one. In C you would do a lookup, branch on “not found,” then insert; the `entry` API folds that into one step.

The `const TEXT` is a backslash-continued string literal: the `\` right after the opening quote removes

the first newline, so the text starts at the quick.

Splitting the work

We split the lines into at most `WORKERS` chunks of roughly equal size. Each chunk is a sub-slice that borrows from the original `lines` vector — no copying of the text itself.

```
/// Split the lines into at most `n` roughly equal chunks.
fn split_into_chunks<'a>(lines: &'a [&'a str], n: usize) -> Vec<&'a [&'a str]> {
    if lines.is_empty() {
        return Vec::new();
    }
    let n = n.min(lines.len()).max(1);
    let chunk_size = lines.len().div_ceil(n);
    lines.chunks(chunk_size).collect()
}
```

The lifetime `'a` says: the chunk slices we return borrow from the same data as the `lines` slice we were given, so they may not outlive it (Chapter 9 — Lifetimes). The `slice::chunks(chunk_size)` method is a standard helper that hands back non-overlapping windows over the slice. `div_ceil` is integer division that rounds up, so we never leave a leftover line uncounted.

The parallel core

This is the function that does the threading. Read the comments closely — every line earns its place.

```
/// Run the parallel count and return the merged table.
fn parallel_word_count(text: &str) -> HashMap<String, u64> {
    let lines: Vec<&str> = text.lines().collect();
    let chunks = split_into_chunks(&lines, WORKERS);

    let mut totals: HashMap<String, u64> = HashMap::new();

    // A channel: workers send partial maps, the main thread receives them.
    let (tx, rx) = mpsc::channel::<HashMap<String, u64>>();

    // `scope` guarantees all spawned threads finish before it returns, so the
    // threads may safely borrow `chunks` (no 'static requirement, no Arc).
    thread::scope(|s| {
        for chunk in &chunks {
            let tx = tx.clone();
            let chunk = *chunk; // copy the &[&str] slice handle, not the data
            s.spawn(move || {
                let local = count_chunk(chunk);
                // If the receiver is gone we just drop the result; ignore error.
                let _ = tx.send(local);
            });
        }
    })
    // Drop the original sender so the receiver loop can end once all
```

```

    // worker clones are dropped at the end of their threads.
    drop(tx);

    // Merge results as they arrive. The loop ends when every sender is gone.
    for partial in rx {
        for (word, n) in partial {
            *totals.entry(word).or_insert(0) += n;
        }
    }
});

totals
}

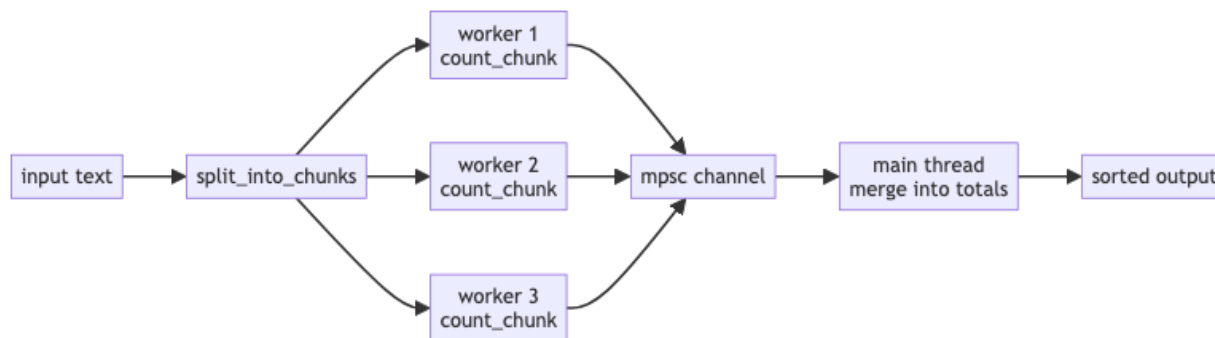
```

Step by step:

- `mpsc::channel()` creates a sender `tx` and a receiver `rx`. The sender can be cloned so many threads can send; the receiver is single (Chapter 20 — Channels and Shared State).
- `thread::scope(|s| { ... })` opens a thread scope. Any thread started with `s.spawn(...)` is guaranteed to be joined before `scope` returns. That promise is what lets the closures `borrow chunk`, which points into the `lines` vector that lives on this stack frame. Plain `thread::spawn` would require the closure to be `'static` and force us to copy or `Arc`-wrap the data.
- `let tx = tx.clone();` makes a per-worker sender. `let chunk = *chunk;` copies the small slice handle (a pointer and a length), not the words it points to.
- `move ||` moves the cloned `tx` and the copied `chunk` into the closure (Chapter 6 — Functions and Closures). Each worker counts its chunk and sends the result. `let _ = tx.send(local);` ignores the send error that would happen only if the receiver were already gone.
- `drop(tx)` drops the **original** sender. The receiver's `for partial in rx` loop ends only when **all** senders are gone. The worker clones drop when their threads finish, but the original would linger to the end of the scope and block the loop forever — so we drop it explicitly before draining.
- The `for partial in rx` loop receives each worker's map and folds it into `totals`. Because addition is commutative, the order of arrival does not change the final counts. That is why the result is deterministic.

Watch out. Forgetting `drop(tx)` is the classic mpsc deadlock: the receiver loop waits for a sender that never disappears, and the program hangs. The rule of thumb is “every sender must be dropped for the receive loop to end.”

There is **no Mutex here** because there is no shared mutable state. Each thread owns its map. This message-passing style — “do not communicate by sharing memory; share memory by communicating” — is often simpler and safer than locks. The book also shows the `Arc<Mutex<T>>` shared-state style in Chapter 20; both are valid.



Printing and main

The `main` function runs the count and prints the table. A `HashMap` has no defined order, so for stable, human-friendly output we sort the rows by count (highest first) and then by word.

```
fn main() {
    let totals = parallel_word_count(TEXT);

    // Sort for stable, human-friendly output: by count desc, then by word.
    let mut rows: Vec<(&String, &u64)> = totals.iter().collect();
    rows.sort_by(|a, b| b.1.cmp(a.1).then(a.0.cmp(b.0)));

    println!("word frequencies ({} unique words):", rows.len());
    for (word, count) in rows {
        println!("{count:>3} {word}");
    }
}
```

The closure `|a, b| b.1.cmp(a.1).then(a.0.cmp(b.0))` compares two `(word, count)` pairs. We compare `b`'s count to `a`'s count (note the reversed order, which gives descending counts), and `.then(...)` adds a tie-breaker by word so equal counts print in alphabetical order. This deterministic sort is what makes the output — and our tests — reliable.

Tests prove it is deterministic and race-free

A binary crate can hold its own tests in a `#[cfg(test)]` module, just like a library (Chapter 23 — Testing). These tests check the totals, the empty-input case, and that two runs give identical results.

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn counts_are_correct_and_deterministic() {
        let totals = parallel_word_count(TEXT);
        assert_eq!(totals["the"], 7);
        assert_eq!(totals["fox"], 4);
        assert_eq!(totals["brown"], 4);
    }
}
```

```

        assert_eq!(totals["dog"], 3);
        assert_eq!(totals["quick"], 2);
        assert_eq!(totals["jumps"], 1);
    }

    #[test]
    fn empty_input_yields_empty_table() {
        let totals = parallel_word_count("");
        assert!(totals.is_empty());
    }

    #[test]
    fn merging_order_does_not_change_totals() {
        // Running twice must give identical results (no data race, no nondeterminism).
        let a = parallel_word_count(TEXT);
        let b = parallel_word_count(TEXT);
        assert_eq!(a, b);
    }
}

```

How to run it

```

cargo run --bin parallel_count
cargo test --bin parallel_count

```

Expected output:

word frequencies (8 unique words):

```

7 the
4 brown
4 fox
3 dog
2 lazy
2 quick
1 jumps
1 over

```

Chapters and concepts this exercises

- Chapter 6 — Functions and Closures (move closures, sort comparator).
- Chapter 9 — Lifetimes (the 'a on `split_into_chunks`).
- Chapter 16 — Collections and Iterators (`HashMap`, `entry`, `chunks`, `collect`).
- Chapter 19 — Threads and Concurrency (`thread::scope`, `Send/Sync`, `joining`).
- Chapter 20 — Channels and Shared State (`mpsc::channel`, `sender/receiver`).
- Chapter 23 — Testing (`#[cfg(test)]`, `#[test]`).

Extend it

- Read the text from a file named on the command line, reusing Project 1's `BufReader` approach.

- Replace the channel with an `Arc<Mutex<HashMap<String, u64>>>` shared map and compare the two styles (Chapter 20 — Channels and Shared State).
- Make a configurable `WORKERS` count and measure speed on a large input with `cargo build --release`.
- Lowercase words and strip punctuation so `Fox` and `fox.` count as `fox`.

Project 3 — `Stack<T>` and an RPN evaluator: a generic library with tests

Goal. Build a reusable, generic stack — a `Stack<T>` that works for any element type — and use it to write a small reverse-polish-notation (RPN) calculator. This project lives in the **library crate** (`src/lib.rs`) and ships with both `#[test]` unit tests and a **documentation test** (an example in a `///` comment that Cargo compiles and runs).

RPN, also called postfix, writes the operator after its operands: `2 3 +` means `2 + 3`. It is the natural job for a stack: push numbers, and when you see an operator, pop two numbers, combine them, and push the result.

C vs Rust. A generic container in C means either macros that paste code for each type, or a `void *stack` plus element size and a lot of casting that the compiler cannot check. Rust generics are real, type-checked, and **monomorphized**: the compiler stamps out a specialized `Stack<i64>`, `Stack<String>`, and so on, each as fast as hand-written code, with full type safety (Chapter 14 — Generics).

The crate documentation and a doc test

A `///!` comment at the top of `lib.rs` documents the whole crate. The fenced code block inside it is a **doc test**: `cargo test` compiles and runs it, so the example in the docs can never drift out of date (Chapter 23 — Testing).

```
///! Project 3: a small generic data structure with tests.
///!
///! This crate gives a generic `Stack<T>` and, on top of it, a tiny
///! reverse-polish-notation (RPN) calculator. It exercises generics, traits,
///! `Option`, and `Result`, and it ships with both `#[test]` unit tests and a
///! documentation test (the example on `Stack` below is compiled and run).
///!
///! ```
///! use rustbook_examples::Stack;
///!
///! let mut s: Stack<i32> = Stack::new();
///! s.push(10);
///! s.push(20);
///! assert_eq!(s.len(), 2);
///! assert_eq!(s.peek(), Some(&20));
///! assert_eq!(s.pop(), Some(20));
///! assert_eq!(s.pop(), Some(10));
///! assert_eq!(s.pop(), None);
///! assert!(s.is_empty());
```

```
//! ...
```

```
use std::fmt;
```

The generic Stack<T>

The stack is a thin, safe wrapper around a `Vec<T>`. A `Vec<T>` is already a growable heap array that knows its length and capacity — like a `malloc`'d array plus a length you maintain by hand, but automatic (Chapter 16 — Collections and Iterators). We add the stack vocabulary on top: `push`, `pop`, `peek`.

```
/// A growable last-in, first-out stack of `T`, backed by a `Vec<T>`.
///
/// This is generic over any element type, the same way a C version would use
/// `void *` plus a size — but here the type is checked at compile time and the
/// code is monomorphized, so there is no runtime cost.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Stack<T> {
    items: Vec<T>,
}

impl<T> Stack<T> {
    /// Create an empty stack.
    pub fn new() -> Self {
        Stack { items: Vec::new() }
    }

    /// Create an empty stack with room for `cap` items before reallocating.
    pub fn with_capacity(cap: usize) -> Self {
        Stack {
            items: Vec::with_capacity(cap),
        }
    }

    /// Push a value onto the top.
    pub fn push(&mut self, value: T) {
        self.items.push(value);
    }

    /// Remove and return the top value, or `None` if the stack is empty.
    pub fn pop(&mut self) -> Option<T> {
        self.items.pop()
    }

    /// Look at the top value without removing it.
    pub fn peek(&self) -> Option<&T> {
        self.items.last()
    }
}
```

```

/// Number of items on the stack.
pub fn len(&self) -> usize {
    self.items.len()
}

/// True when the stack has no items.
pub fn is_empty(&self) -> bool {
    self.items.is_empty()
}
}

```

Key ideas for a C programmer:

- `Stack<T>` declares one type parameter `T`. `impl<T> Stack<T>` then provides methods for every `T`. The `items` field is **private** (no `pub`), so callers cannot poke at the inner vector and break the stack's invariants (Chapter 3 — Program Structure). They must go through the methods.
- `pop` returns `Option<T>`: `Some(value)` if there was one, or `None` if the stack was empty. There is no null and no sentinel value to forget to check (Chapter 12 — Enums and Pattern Matching). The caller must handle the empty case; the compiler enforces it.
- `peek` returns `Option<&T>` — a **borrow** of the top item, not a copy. The value stays on the stack (Chapter 8 — Borrowing and References).
- `#[derive(Debug, Clone, PartialEq, Eq)]` gives the stack printing, cloning, and equality for free, as long as `T` supports them. This is how the unit tests can compare two stacks with `assert_eq!` (Chapter 15 — Traits).

Two standard traits: `Default` and `FromIterator`

Implementing well-known traits makes a type feel native. `Default` lets `Stack::default()` produce an empty stack and fits APIs that ask for a default. `FromIterator` lets any iterator build a stack with `.collect()` — the same way you can `.collect()` into a `Vec` or a `HashMap` (Chapter 15 — Traits, Chapter 16 — Collections and Iterators).

```

// A common Rust idiom: implement `Default` so `Stack::default()` works and so
// the type fits APIs that need a default value.
impl<T> Default for Stack<T> {
    fn default() -> Self {
        Stack::new()
    }
}

// Let any iterator of `T` build a stack with `.collect()`.
impl<T> FromIterator<T> for Stack<T> {
    fn from_iter<I: IntoIterator<Item = T>>(iter: I) -> Self {
        Stack {
            items: iter.into_iter().collect(),
        }
    }
}

```

Rule of thumb. When you write a collection type, implement `Default`, `Debug`, and (where it makes sense) `FromIterator` and `IntoIterator`. It makes your type behave like the standard ones, so other code “just works” with it.

A typed error and the `Error` trait

The evaluator can fail in a few specific ways. We model them with an `enum`, the Rust replacement for an error-code `#define` list — except each variant can carry data, and a `match` on it is exhaustive (Chapter 12 — Enums and Pattern Matching). We implement `Display` so the error prints a clean message, and the empty `Error` impl marks it as a real error type usable with `Box<dyn Error>` and the `?` operator (Chapter 13 — Error Handling).

```
/// The reason an RPN expression could not be evaluated.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum RpnError {
    /// A token was neither a number nor a known operator.
    BadToken(String),
    /// An operator ran but the stack did not have two operands.
    NotEnoughOperands,
    /// Division (or remainder) by zero.
    DivideByZero,
    /// The expression finished with more or fewer than one value left.
    LeftoverOperands(usize),
}

// Implementing `Display` makes `RpnError` print nicely and (with the blanket
// impl in std) usable as a boxed error. This is the trait-driven error style.
impl fmt::Display for RpnError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            RpnError::BadToken(t) => write!(f, "bad token: {t}"),
            RpnError::NotEnoughOperands => write!(f, "not enough operands"),
            RpnError::DivideByZero => write!(f, "divide by zero"),
            RpnError::LeftoverOperands(n) => {
                write!(f, "expression left {n} values on the stack")
            }
        }
    }
}

impl std::error::Error for RpnError {}
```

The evaluator

Now the calculator itself. It uses our `Stack<i64>`, returns a `Result`, and leans on `?` and `ok_or` to turn missing operands into errors instead of panics.

```
/// Evaluate a reverse-polish-notation expression of `i64` integers.
///
```

```

/// Supported operators: `+`, `-`, `*`, `/`, `%`. Tokens are separated by
/// whitespace. Returns the single result or an [RpnError].
///
/// ```
/// use rustbook_examples::eval_rpn;
///
/// assert_eq!(eval_rpn("2 3 +"), Ok(5));
/// assert_eq!(eval_rpn("4 2 - 3 *"), Ok(6)); // (4 - 2) * 3
/// assert!(eval_rpn("1 0 /").is_err());
/// ```
pub fn eval_rpn(expr: &str) -> Result<i64, RpnError> {
    let mut stack: Stack<i64> = Stack::new();

    for token in expr.split_whitespace() {
        if let Ok(n) = token.parse::<i64>() {
            stack.push(n);
            continue;
        }

        // An operator: pop the two operands (note the order: b is on top).
        let b = stack.pop().ok_or(RpnError::NotEnoughOperands)?;
        let a = stack.pop().ok_or(RpnError::NotEnoughOperands)?;

        let result = match token {
            "+" => a + b,
            "-" => a - b,
            "*" => a * b,
            "/" => {
                if b == 0 {
                    return Err(RpnError::DivideByZero);
                }
                a / b
            }
            "%" => {
                if b == 0 {
                    return Err(RpnError::DivideByZero);
                }
                a % b
            }
            other => return Err(RpnError::BadToken(other.to_string())),
        };
        stack.push(result);
    }

    match stack.len() {
        1 => Ok(stack.pop().unwrap()),
        n => Err(RpnError::LeftoverOperands(n)),
    }
}

```

```
}
```

How it works:

- `expr.split_whitespace()` yields each token. `token.parse::i64()` tries to read it as an integer; if `let Ok(n) = ...` pushes it when it parses (Chapter 12 — Enums and Pattern Matching). Otherwise the token is an operator.
- `stack.pop().ok_or(RpnError::NotEnoughOperands)?` is the key line. `pop` gives an `Optioni64`; `ok_or(err)` turns `None` into `Err(err)` and `Some(v)` into `Ok(v)`; the `?` then unwraps `Ok` or returns the error from `eval_rpn` (Chapter 13 — Error Handling). This is the bridge from “missing value” to “real error.”
- Order matters: the value popped first (`b`) was pushed last, so it is the right-hand operand. `4 2 -` must compute `4 - 2`, not `2 - 4`.
- Division and remainder check for a zero divisor and return `DivideByZero`. In C, integer divide-by-zero is undefined behavior; here it is a value you handle.
- At the end, a valid expression leaves exactly one value. Anything else is a `LeftoverOperands` error. `stack.pop().unwrap()` is safe in the `1` arm because we just checked the length is one.

Unit tests

The `#[cfg(test)] mod tests` block holds the unit tests. They cover the stack for several element types (proving the generic really is generic), the `collect` path, the happy RPN cases, every error case, and the `Display` text (Chapter 23 — Testing).

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn push_pop_peek() {
        let mut s = Stack::new();
        assert!(s.is_empty());
        s.push("a");
        s.push("b");
        assert_eq!(s.peek(), Some(&"b"));
        assert_eq!(s.len(), 2);
        assert_eq!(s.pop(), Some("b"));
        assert_eq!(s.pop(), Some("a"));
        assert_eq!(s.pop(), None);
    }

    #[test]
    fn works_for_different_types() {
        // The same generic code handles i32 and String with no extra work.
        let mut ints: Stack<i32> = Stack::new();
        ints.push(1);
        ints.push(2);
        assert_eq!(ints.pop(), Some(2));
    }
}
```



```

    let mut words: Stack<String> = Stack::new();
    words.push("hello".to_string());
    assert_eq!(words.peek().map(|s| s.as_str()), Some("hello"));
}

#[test]
fn collect_into_stack() {
    let s: Stack<i32> = (1..=3).collect();
    assert_eq!(s.len(), 3);
    assert_eq!(s.peek(), Some(&3));
}

#[test]
fn rpn_basic() {
    assert_eq!(eval_rpn("2 3 +"), Ok(5));
    assert_eq!(eval_rpn("10 2 /"), Ok(5));
    assert_eq!(eval_rpn("7 3 %"), Ok(1));
    assert_eq!(eval_rpn("2 3 4 * +"), Ok(14));
}

#[test]
fn rpn_errors() {
    assert_eq!(eval_rpn("1 +"), Err(RpnError::NotEnoughOperands));
    assert_eq!(eval_rpn("1 2"), Err(RpnError::LeftoverOperands(2)));
    assert_eq!(eval_rpn("1 0 /"), Err(RpnError::DivideByZero));
    assert_eq!(
        eval_rpn("1 2 foo"),
        Err(RpnError::BadToken("foo".to_string()))
    );
}

#[test]
fn rpn_error_displays() {
    let msg = eval_rpn("1 0 /").unwrap_err().to_string();
    assert_eq!(msg, "divide by zero");
}
}

```

use `super::*`; pulls every item from the parent module (the crate root) into the test module, so the tests can name `Stack`, `eval_rpn`, and `RpnError` directly. This glob import is the one place glob imports are encouraged (Chapter 3 — Program Structure).

How to run it

```

cargo test --lib           # runs the unit tests
cargo test --doc           # runs the documentation tests
cargo test                 # runs everything: unit, doc, and the binaries' tests
cargo doc --open           # builds the HTML docs; the examples appear there

```

`cargo test` prints a section for the unit tests and a separate “Doc-tests” section. Seeing both pass is the proof that your examples are correct, not just your code.

Chapters and concepts this exercises

- Chapter 3 — Program Structure (a library crate, private fields, `use super::*`).
- Chapter 8 — Borrowing and References (`peek` returns `Option<&T>`).
- Chapter 12 — Enums and Pattern Matching (`Option`, the `RpnError` enum, `match`).
- Chapter 13 — Error Handling (`Result`, `?`, `ok_or`, the `Error` trait).
- Chapter 14 — Generics (`Stack<T>`, monomorphization).
- Chapter 15 — Traits (`derive`, `Display`, `Default`, `FromIterator`).
- Chapter 23 — Testing (unit tests and documentation tests).

Extend it

- Add floating-point support by making the evaluator generic over a numeric trait, or by switching to `f64`.
- Add a `IntoIterator` impl so you can write `for x in stack`.
- Turn `Stack<T>` into a bounded stack with a maximum size that returns an error on overflow.
- Build a tiny REPL: read a line from stdin (Project 1’s technique), evaluate it, print the result or the error, and loop.

Key takeaways

- A single Cargo package can hold one library crate (`src/lib.rs`) and many binary crates (`src/bin/*.rs`); run them with `cargo run --bin <name>`.
- **Project 1** shows the Unix-filter shape in Rust: read with `BufRead`, count generically over any reader, propagate errors with `Result` and `?`, write to locked stdout, report problems on stderr, and return an `ExitCode`.
- **Project 2** shows fearless concurrency: `thread::scope` lets worker threads borrow stack data safely, and an `mpsc` channel merges per-thread results with no shared mutable state — so there is no lock and no possible data race.
- **Project 3** shows a clean generic API: `Stack<T>` wraps a `Vec<T>`, returns `Option/Result` instead of nulls and sentinels, implements standard traits, and is covered by both unit tests and a documentation test.
- Across all three, the same Rust habits repeat: keep `main` thin, make illegal states hard to reach, prefer borrowing to copying, and let the compiler prove safety before the program runs.

Watch out (gotchas for C programmers)

- `cargo run --bin name -- args`: the `--` separates Cargo’s arguments from your program’s. Forgetting it makes Cargo eat your flags.
- **Byte count is not character count.** `wc -c` and our `-c` count bytes; a UTF-8 file has more bytes than characters.
- **`mpsc` deadlock.** A receive loop ends only when every sender is dropped. Keep a stray `tx` alive and the program hangs. Drop the original sender before draining.
- **`thread::scope` vs `thread::spawn`.** Scoped threads can borrow local data and are auto-joined; plain `spawn` needs `'static` data and a manual `join`.

- **Struct fields are private by default**, even in a `pub struct`. `Stack`'s inner `Vec` is hidden on purpose so callers cannot break its invariants.
- **`pop` returns `Option`, not a value.** There is no sentinel like `-1` or `NULL`; you must handle the empty case, and the compiler makes sure you do.

Interview questions

Q: Why split a CLI program into a `run` function and a thin `main`? A: `run` returns a `Result`, so you can use the `?` operator throughout for clean error propagation. `main` then maps that `Result` onto a process exit code (`ExitCode::SUCCESS/FAILURE`) and prints any error to `stderr`. This separation also makes the logic testable without spawning a process.

Q: In the multithreaded counter, why is there no `Mutex`? A: Because there is no shared mutable state. Each worker thread owns its own `HashMap` and sends it back through an `mpsc` channel; the main thread is the only one that writes the merged totals. Message passing replaces locking, which removes both the locking overhead and the chance of a deadlock or a forgotten lock.

Q: What does `thread::scope` give you that `thread::spawn` does not? A: Scoped threads are guaranteed to finish before the scope returns, so the compiler lets their closures borrow data from the parent stack frame without a `'static` lifetime and without `Arc`. With plain `thread::spawn`, the closure must own or `Arc`-wrap everything it touches, and you must join the handles yourself.

Q: How does a generic `Stack<T>` in Rust differ from a `void * stack` in C? A: A `void *` `stack` erases the element type, so the compiler cannot check what you push or pop, and you store an element size and cast by hand. A Rust `Stack<T>` is type-checked and monomorphized: the compiler generates a specialized version for each concrete type, so it is type-safe and as fast as hand-written code, with no casts and no separate size field.

Q: What is a documentation test and why is it useful? A: It is a code example inside a `///` or `///!` doc comment that `cargo test` compiles and runs. It keeps documentation honest — if the example stops compiling or its assertions fail, the test fails — so the examples in your docs can never silently drift out of sync with the code.

Try it

1. Run all three projects from the book's `examples/` package: `cargo run --bin linecount`, `-- Cargo.toml`, `cargo run --bin parallel_count`, and `cargo test`.
2. Break Project 2 on purpose by deleting the `drop(tx)` line and run it. Watch it hang, then restore the line and confirm it finishes.
3. Add one new operator (for example `**` for power) to the RPN evaluator and a matching unit test, then run `cargo test --lib`.

Appendix A — C to Rust Cheat Sheet

What you'll learn. A dense, scannable lookup table from C to Rust: types, declarations, memory, strings, pointers, control flow, errors, concurrency, the standard library headers, common idioms, and a `cargo` command reference.

This is a desk reference, not a tutorial. Each row maps something you do in C to the idiomatic Rust way to do it. Chapters are named so you can read the full story.

Types

C	Rust	Note
<code>char</code>	<code>i8 / u8</code>	C <code>char</code> signedness is platform-defined; Rust is explicit.
<code>signed char / unsigned char</code>	<code>i8 / u8</code>	A raw byte is <code>u8</code> .
<code>short / unsigned short</code>	<code>i16 / u16</code>	Fixed width in Rust.
<code>int / unsigned int</code>	<code>i32 / u32</code>	<code>int</code> is usually 32-bit; Rust pins it.
<code>long</code>	<code>i32 or i64</code>	C <code>long</code> varies by platform; pick the exact Rust type.
<code>long long / unsigned long long</code>	<code>i64 / u64</code>	Fixed width.
<code>__int128</code>	<code>i128 / u128</code>	Built in, no extension needed.
<code>size_t</code>	<code>usize</code>	Index/length type; pointer-sized.
<code>ptrdiff_t, intptr_t</code>	<code>isize</code>	Pointer-sized signed integer.
<code>float / double</code>	<code>f32 / f64</code>	IEEE-754, same as C.
<code>_Bool / bool</code>	<code>bool</code>	<code>true / false</code> ; size 1 byte.
<code>void</code> (no value)	<code>()</code> (unit)	The empty tuple; size 0.
<code>void</code> (return: never)	<code>!</code> (never)	For functions that never return.
<code>enum { ... }</code>	<code>enum</code>	Rust enums are tagged unions, far more powerful (Ch. 12).
<code>struct { ... }</code>	<code>struct</code>	Same idea; see declarations below (Ch. 11).
<code>union { ... }</code>	<code>enum</code> (safe) or <code>union</code> (unsafe)	Prefer <code>enum</code> ; raw <code>union</code> needs <code>unsafe</code> .
<code>char *</code> (text)	<code>&str / String</code>	UTF-8; not null-terminated (Ch. 10).
<code>T *</code>	<code>&T / &mut T / Box<T> / *const T / *mut T</code>	Pick by ownership and mutability.
<code>T[N]</code>	<code>[T; N]</code>	Fixed-size array, length known at compile time.
<code>T * + length</code>	<code>&[T] / Vec<T></code>	Slice (borrowed) or owned growable array.

Declarations

Thing	C	Rust
Variable	<code>int x = 5;</code>	<code>let x: i32 = 5;</code> (type optional)
Mutable variable	<code>int x = 5;</code>	<code>let mut x: i32 = 5;</code>

Thing	C	Rust
Constant	<code>#define N 10 / const int N = 10;</code>	<code>const N: i32 = 10;</code>
Global mutable	<code>static int c = 0;</code>	<code>static mut C: i32 = 0;</code> (needs <code>unsafe</code> ; prefer atomics)
Immutable global	<code>static const int K = 1;</code>	<code>static K: i32 = 1;</code>
Array	<code>int a[3] = {1,2,3};</code>	<code>let a: [i32; 3] = [1, 2, 3];</code>
Repeated array	<code>int a[100] = {0};</code>	<code>let a = [0; 100];</code>
Pointer	<code>int *p = &x;</code>	<code>let p: &i32 = &x;</code> (or <code>&mut</code> , <code>*const</code> , <code>*mut</code>)
Struct	<code>struct P { int x, y; };</code>	<code>struct P { x: i32, y: i32 }</code> <code>struct P(i32, i32);</code>
Tuple struct	(none)	<code>enum C { Red, Green }</code>
Enum	<code>enum C { Red, Green };</code>	<code>type Id = u32;</code> (alias) or <code>struct Id(u32);</code> (newtype)
Typedef	<code>typedef uint32_t Id;</code>	<code>fn add(a: i32, b: i32) -> i32</code>
Function	<code>int add(int a, int b)</code>	<code>fn(i32) -> i32</code> or <code>Box<dyn Fn(i32) -> i32></code>
Function pointer	<code>int (*f)(int)</code>	<code>extern "C" { fn f() -> i32; }</code> (Ch. 25)
Extern function	<code>extern int f(void);</code>	

C vs Rust. Variables are immutable by default in Rust. Where C has mutable everything, Rust makes you write `mut`. Types come *after* the name, not before.

Memory management

C	Rust	Note
<code>malloc(sizeof(T))</code>	<code>Box::new(value)</code>	One heap value, freed when the <code>Box</code> drops (Ch. 17).
<code>free(p)</code>	(automatic)	<code>Drop</code> runs at end of scope; no manual <code>free</code> (Ch. 7).
<code>calloc(n, sizeof(T))</code>	<code>vec![T::default(); n] / vec![0; n]</code>	Zero-initialized buffer.
<code>malloc(n * sizeof(T))</code> (array)	<code>Vec::with_capacity(n)</code> then <code>push</code> , or <code>vec![x; n]</code>	Growable array (Ch. 16).
<code>realloc(p, n)</code> (grow)	<code>vec.resize(n, value) / vec.push(x)</code>	<code>Vec</code> reallocates for you.
<code>realloc</code> (shrink)	<code>vec.truncate(n) / vec.shrink_to_fit()</code>	
<code>sizeof(T)</code>	<code>std::mem::size_of::<T>()</code>	Compile-time size in bytes.
<code>sizeof(expr)</code>	<code>std::mem::size_of_val(&expr)</code>	Size of a value.
<code>_Alignof(T)</code>	<code>std::mem::align_of::<T>()</code>	Alignment in bytes.

C	Rust	Note
<code>memcpy(dst, src, n)</code>	<code>dst.copy_from_slice(src) / slice::copy_from_slice</code>	Safe, checked.
<code>memset(p, 0, n)</code>	<code>slice.fill(0)</code>	
<code>memmove</code>	<code>slice.copy_within(range, dest)</code>	Overlap-safe.
Move semantics	(manual)	<code>let b = a;</code> <i>moves</i> ownership; <code>a</code> is invalid after (Ch. 7).
Reference count	(manual)	<code>Rc<T></code> (single thread) / <code>Arc<T></code> (threads) (Ch. 17, 20).

Mental model. A `Box<T>` is `malloc` + a guaranteed `free` at scope end. You never call `free`; the compiler inserts it exactly once.

Strings

C	Rust	Note
<code>char * literal</code>	<code>&str ("hi")</code>	UTF-8, length known, not null-terminated.
<code>char buf[N]</code> (owned text)	<code>String</code>	Heap-allocated, growable (Ch. 10).
<code>strlen(s)</code>	<code>s.len()</code>	Byte length; <code>s.chars().count()</code> for characters.
<code>strcmp(a, b) == 0</code>	<code>a == b</code>	<code>==</code> compares contents.
<code>strcmp</code> ordering	<code>a.cmp(b)</code>	Returns <code>Ordering::Less, Equal, Greater</code> .
<code>strcpy(d, s) / strdup(s)</code>	<code>s.to_owned() / s.to_string() / String::from(s)</code>	Owned copy.
copy a <code>String</code>	<code>s.clone()</code>	Deep copy.
<code>strcat(d, s)</code>	<code>d.push_str(s)</code>	In place.
<code>a + b</code> (concat)	<code>a + &b</code> (<code>a</code> is <code>String</code> , <code>b</code> is <code>&str</code>)	Or <code>format!("{a}{b}")</code> .
<code>strchr / strstr</code>	<code>s.find(c) / s.find("sub")</code>	Returns <code>Option<usize></code> .
<code>strtok</code>	<code>s.split(sep)</code>	Returns an iterator.
<code>printf("%d\n", x)</code>	<code>println!("{x}")</code>	Type-checked format string (Ch. 5).
<code>fprintf(stderr, ...)</code>	<code>eprintln!(...)</code>	
<code>sprintf(buf, ...)</code>	<code>let s = format!(...);</code>	Returns a <code>String</code> ; no buffer overflow.
<code>snprintf</code>	<code>format!(...)</code>	Always bounded; allocates as needed.
<code>puts(s)</code>	<code>println!("{s}")</code>	
<code>atoi / strtol</code>	<code>s.parse::<i32>()? </code>	Returns <code>Result</code> , no silent failure.

C	Rust	Note
C string for FFI	<code>std::ffi::CString</code> / <code>CStr</code>	Null-terminated bridge to C (Ch. 25).

Watch out. A Rust `&str`/`String` is **not** null-terminated and is UTF-8. Indexing by byte (`s[0]`) is not allowed; use `s.as_bytes()[0]` or `s.chars()`.

Pointers and references

C	Rust	Note
<code>const T *p</code> (read)	<code>&T</code>	Shared reference; many allowed at once.
<code>T *p</code> (write)	<code>&mut T</code>	Exclusive reference; only one at a time (Ch. 8).
Owning pointer	<code>Box<T></code>	Single owner on the heap.
<code>T *</code> (raw, unchecked)	<code>*const T</code> / <code>*mut T</code>	Only dereferenced in <code>unsafe</code> (Ch. 25).
NULL	<code>Option::None</code>	A nullable <code>&T</code> becomes <code>Option<&T></code> (Ch. 12).
<code>p == NULL</code> check	<code>match opt { Some(x) => .., None => .. }</code>	Compiler forces the check.
<code>p->field</code>	<code>p.field</code>	Rust auto-dereferences; no <code>-></code> .
<code>(*p).field</code>	<code>(*p).field</code> or <code>p.field</code>	<code>.</code> works through references.
<code>&x</code>	<code>&x</code>	Take a reference.
<code>*p</code>	<code>*p</code>	Dereference.
<code>p + 1</code> (pointer math)	<code>slice[i]</code> / iterators / <code>ptr.add(1)</code> (<code>unsafe</code>)	No arithmetic on safe refs.

C vs Rust. A `&T`/`&mut T` can never be null and never dangle — the borrow checker proves the referent outlives the reference (Ch. 9). For “maybe absent,” use `Option<&T>`, which is the same size as a pointer thanks to niche optimization.

Control flow

C	Rust
<code>if (c) { } else { }</code>	<code>if c { } else { }</code> (no parentheses; braces required)
ternary <code>c ? a : b</code>	<code>if c { a } else { b }</code> (an expression)
<code>switch (x) { case ...: }</code>	<code>match x { pattern => expr, _ => expr }</code> (Ch. 12)
<code>while (c) { }</code>	<code>while c { }</code>
<code>do { } while (c);</code>	<code>loop { ...; if !c { break; } }</code>
<code>for (i=0; i<n; i++)</code>	<code>for i in 0..n { }</code>
<code>for over array</code>	<code>for x in &arr { }</code> / <code>for x in arr.iter() { }</code>

C	Rust
infinite loop	<code>loop { }</code>
<code>break / continue</code>	<code>break / continue</code> (and labeled <code>break 'outer'</code>)
<code>goto</code>	(none)
fallthrough case	<code>match x { 1 \ 2 => ... }</code>

Watch out. `match` is **exhaustive**: you must cover every case or add `_`. There is no implicit fallthrough between arms.

Error handling

C	Rust	Note
return code int	<code>Result<T, E></code>	Errors are values you cannot ignore (Ch. 13).
<code>errno</code>	the <code>E</code> in <code>Result<T, E></code>	Carry rich error data, not a global int.
check-and-return on error NULL for “no value”	<code>let x = f()?;</code> <code>Option<T></code>	<code>?</code> returns the error early. <code>Some/None</code> (Ch. 12).
<code>assert(cond)</code>	<code>assert!(cond) /</code> <code>debug_assert!(cond)</code>	
<code>abort()</code>	<code>std::process::abort()</code>	
<code>exit(code)</code>	<code>std::process::exit(code)</code>	
fatal error	<code>panic!("msg")</code>	Unwinds the stack, then aborts (Ch. 13).
<code>setjmp/longjmp</code> (exceptions)	<code>Result + ?</code> ; or <code>panic!/catch_unwind</code>	Rust has no exceptions.
<code>goto cleanup; (RAII)</code>	<code>Drop</code> runs automatically	No manual cleanup label needed.

```
use std::num::ParseIntError;

fn double(s: &str) -> Result<i32, ParseIntError> {
    let n = s.parse::<i32>()?; // `?` returns Err early, like checking a return code
    Ok(n * 2)
}

fn main() {
    println!("{:?}", double("21")); // Ok(42)
    println!("{:?}", double("x")); // Err(ParseIntError { .. })
}
```

Concurrency

C (POSIX)	Rust	Note
<code>pthread_create</code>	<code>std::thread::spawn({ .. })</code>	Returns a <code>JoinHandle</code> (Ch. 19).
<code>pthread_join</code>	<code>handle.join().unwrap()</code>	
<code>pthread_mutex_t</code> + lock/unlock	<code>std::sync::Mutex<T> + .lock()</code>	Lock guards the data; auto-unlocks on drop.
<code>pthread_rwlock_t</code>	<code>std::sync::RwLock<T></code>	
<code>pthread_cond_t</code>	<code>std::sync::Condvar</code>	
shared owned data	<code>Arc<T></code>	Atomic reference count (Ch. 20).
shared mutable data	<code>Arc<Mutex<T>></code>	The standard pattern.
<code>_Atomic int</code>	<code>std::sync::atomic::AtomicI32</code> With <code>Ordering</code> .	
<code>atomic_fetch_add</code>	<code>a.fetch_add(1, Ordering::SeqCst)</code>	
thread-local <code>__thread</code>	<code>thread_local!</code> macro	
message passing	<code>std::sync::mpsc::channel()</code>	<code>tx.send / rx.recv</code> (Ch. 20).

C vs Rust. The compiler checks thread safety with the `Send` and `Sync` marker traits. Sharing non-synchronized mutable data across threads simply does not compile — data races are caught before the program runs.

libc headers → Rust

C header	Rust equivalent	Note
<code><stdio.h></code>	<code>std::io, print!/println!, std::fs</code>	I/O and files.
<code><stdlib.h></code>	<code>std</code> (alloc via <code>Box/Vec</code>), <code>std::process, std::env</code>	<code>exit, getenv, rand</code> (use the <code>rand</code> crate).
<code><string.h></code>	<code>&str/String/[T]</code> methods	<code>len, find, split, copy_from_slice</code> .
<code><stdint.h></code>	built-in <code>i8..i128, u8..u128, usize/isize</code>	No header needed.
<code><stdbool.h></code>	<code>bool</code>	Built in.
<code><stddef.h></code>	<code>usize, (), core::ptr</code>	
<code><math.h></code>	<code>f64/f32</code> methods, <code>std::f64::consts</code>	<code>x.sqrt(), x.sin(), PI</code> .
<code><float.h> / <limits.h></code>	<code>f64::MAX, i32::MAX, u8::MIN, etc.</code>	Associated constants.
<code><time.h></code>	<code>std::time::{Instant, Duration, SystemTime}</code>	Monotonic and wall clock.
<code><assert.h></code>	<code>assert!, debug_assert!</code>	
<code><errno.h></code>	<code>std::io::Error / Result</code>	No global <code>errno</code> .
<code><pthread.h></code>	<code>std::thread, std::sync</code>	Threads, mutexes, channels.
<code><ctype.h></code>	<code>char</code> methods	<code>c.is_alphabetic(), c.to_ascii_uppercase()</code> .
<code><stdarg.h></code> (varargs)	macros / slices / generics	Use <code>&[T]</code> or a macro instead.

```

use std::time::Instant;

fn main() {
    let start = Instant::now();
    let r = (1.0_f64).sin() + std::f64::consts::PI.sqrt(); // math.h methods
    println!("{r:.4} in {:?}", start.elapsed());           // time.h
}

```

Idioms (the Rust way)

Idea	C habit	Rust idiom	Chapter
Cleanup	<code>goto cleanup;</code> <code>free(...)</code>	RAII: <code>Drop</code> frees on scope exit	7, 17
Ownership	“who frees this?” comments	One owner, enforced by compiler	7
Distinct types	<code>typedef uint32_t UserId;</code>	Newtype: <code>struct UserId(u32);</code>	27
Conversions	implicit casts	From/Into: <code>let s: String = x.into();</code>	15, 27
Errors	return codes / <code>errno</code>	<code>Result<T, E> + ?</code>	13
“No value”	<code>NULL</code> / sentinel	<code>Option<T></code>	12
Polymorphism	structs of function pointers	traits + <code>dyn</code>	15
Generic code	macros / <code>void *</code>	generics (monomorphized)	14
Invalid states	runtime checks	“make illegal states unrepresentable” with enums	27
Builders	many init functions	builder pattern returning <code>Self</code>	27

cargo command cheat sheet

Command	What it does
<code>cargo new my_app</code>	Create a new binary project (<code>--lib</code> for a library).
<code>cargo init</code>	Turn the current directory into a Cargo project.
<code>cargo build</code>	Compile (debug). <code>--release</code> for an optimized build.
<code>cargo run</code>	Build and run the binary. Args after <code>--</code> : <code>cargo run -- foo</code> .
<code>cargo check</code>	Type-check without producing a binary (fast).
<code>cargo test</code>	Run unit, integration, and doc tests (Ch. 23).
<code>cargo bench</code>	Run benchmarks.
<code>cargo add serde</code>	Add a dependency to <code>Cargo.toml</code> (<code>--features ...</code>).

Command	What it does
<code>cargo remove serde</code>	Remove a dependency.
<code>cargo update</code>	Update dependencies within semver, rewriting <code>Cargo.lock</code> .
<code>cargo fmt</code>	Format code with <code>rustfmt</code> .
<code>cargo clippy</code>	Lint with Clippy; <code>--fix</code> to auto-apply (Ch. 24).
<code>cargo doc --open</code>	Build and open HTML docs.
<code>cargo clean</code>	Delete the <code>target/</code> build directory.
<code>cargo tree</code>	Show the dependency graph.
<code>cargo install ripgrep</code>	Install a binary crate globally.

Common one-liners

```
// Read all of stdin into a String
let mut input = String::new();
std::io::Read::read_to_string(&mut std::io::stdin(), &mut input).unwrap();

// Parse and sum whitespace-separated integers
let total: i64 = "1 2 3".split_whitespace().map(|s| s.parse::<i64>().unwrap()).sum();

// Command-line arguments (skip(1) drops the program name)
let args: Vec<String> = std::env::args().skip(1).collect();

// Read a file to a String (returns Result)
let text = std::fs::read_to_string("data.txt")?;

// Sort and remove duplicates
let mut v = vec![3, 1, 2, 1]; v.sort(); v.dedup();

// Filter, transform, collect
let evens: Vec<i32> = (0..10).filter(|n| n % 2 == 0).map(|n| n * n).collect();

rustc hello.rs && ./hello      # compile and run a single file (no Cargo)
cargo run --release            # optimized build + run
RUST_BACKTRACE=1 cargo run     # show a backtrace on panic
```

Appendix B — Glossary

What you'll learn. Every important Rust term used in this book, defined in one to three plain sentences, with a C comparison where it helps.

Terms are listed alphabetically. Bold words inside a definition are themselves defined elsewhere in this glossary.

adaptor (iterator adaptor) — A method that takes an iterator and returns a new, lazy iterator, such as `map`, `filter`, or `take`. Nothing runs until a consumer (like `collect` or `sum`) pulls values through the chain.

Arc — “Atomically Reference Counted” pointer (**Arc**<T>): shared ownership of a heap value that is safe to send between threads. Like **Rc** but with thread-safe (atomic) counting; use it instead of **Rc** across threads.

associated type — A type slot declared inside a trait, written **type Item**;, that each implementer fills in. For example, **Iterator** has **type Item**, so each iterator names the type it yields without an extra generic parameter.

blanket impl — A trait implementation written for *all* types that satisfy some bound, such as **impl**<T: **Display**> **ToString** for T. One **impl** block thereby covers many types at once.

borrow — To temporarily access a value through a **reference** without taking ownership. A borrow ends when the reference goes out of scope; the owner keeps the value. Like passing a pointer in C, but the compiler proves it stays valid.

borrow checker — The part of the Rust compiler that enforces the borrowing rules: you may have many shared **&T** borrows or exactly one **&mut T** borrow, never both, and no borrow may outlive its referent. It prevents dangling pointers and data races at compile time.

Box — **Box**<T> is the simplest smart pointer: a single owned value on the heap, freed automatically when the **Box** drops. It is Rust’s **malloc** + guaranteed **free** for one value.

Cargo — Rust’s build tool and package manager. It compiles, tests, formats, lints, fetches dependencies, and builds docs — replacing Makefiles plus a package manager.

Cargo.lock — A generated file that records the exact versions of every dependency used in a build, so builds are reproducible. Commit it for binaries; libraries usually do not.

Cargo.toml — The manifest file that describes a **package**: its name, version, edition, and dependencies. The analog of a Makefile plus dependency list.

Cell — **Cell**<T> provides **interior mutability** for **Copy** types by moving values in and out, with no references handed out. It allows mutation through a shared **&** without runtime borrow tracking.

Clone — The trait for explicit, possibly expensive deep copies, via **.clone()**. Unlike **Copy**, cloning is never implicit; you ask for it.

closure — An anonymous function that can capture variables from its surrounding scope, written **|args| body**. Like a C function pointer plus a captured environment, but type-safe and without manual context structs.

Copy — A marker trait for small types (like integers) that are duplicated bit-for-bit on assignment instead of **moved**. If a type is **Copy**, **let b = a**; leaves **a** still usable.

crate — The unit of compilation in Rust: either a binary (has **main**) or a library. Roughly one tree of modules compiled together; a **package** contains one or more crates.

deref coercion — The compiler’s automatic conversion from **&T** to **&U** when T implements **Deref**<Target = U>, e.g. **&String** to **&str**, or **&Box**<T> to **&T**. It lets methods and arguments work across these pointer-like types without manual conversion.

Drop — The trait whose **drop** method runs automatically when a value goes out of scope, releasing resources. It is Rust’s destructor and the basis of **RAII** — the compiler-inserted equivalent of **free/close**.

dyn — The keyword marking a **trait object**, e.g. `&dyn Trait` or `Box<dyn Trait>`. It selects dynamic dispatch through a **vtable** instead of static dispatch.

edition — A named, opt-in language revision (2015, 2018, 2021, 2024) set in `Cargo.toml`. Editions can change syntax and idioms without breaking older crates, which keep their own edition. This book targets edition 2024.

elision (lifetime elision) — A set of compiler rules that infer common **lifetime** annotations so you do not have to write them. For example, a method taking `&self` usually returns a reference tied to `self` automatically.

enum — A type that is exactly one of several named **variants**, each able to carry data. It is a tagged union (a C **union** plus a tag) but type-safe, with the tag checked by `match`.

exhaustiveness — The rule that a `match` must handle every possible value, or include a `_` wildcard. The compiler rejects an incomplete match, so adding an enum variant forces you to update all matches.

FFI — Foreign Function Interface: calling C from Rust (or Rust from C) across the language boundary. It requires `extern "C"`, `unsafe`, and often `repr(C)` types.

Fn / FnMut / FnOnce — The three traits a **closure** can implement, by how it uses captures: `Fn` reads them (`&`), `FnMut` mutates them (`&mut`), and `FnOnce` consumes them (by value, callable once). Function arguments accept whichever they need.

generic — A type or function parameterized over types, written with `<T>`. Rust generics are real and type-checked, unlike C macros or `void *`, and are compiled by **monomorphization**.

interior mutability — The ability to mutate data through a shared `&` reference, provided by types like `Cell`, `RefCell`, `Mutex`, and atomics. It moves the aliasing-vs-mutation check from compile time to runtime (or to atomic hardware).

iterator — A value that produces a sequence on demand via its `next` method (the `Iterator` trait). Iterators are lazy and compile to tight loops — a **zero-cost abstraction**.

lifetime — A named region of code, written `'a`, over which a **reference** is valid. The compiler uses lifetimes to prove no reference outlives its data; they are erased at compile time and have no runtime cost.

'static — The longest **lifetime**: data that lives for the entire program, such as string literals (`&'static str`) and `const/static` items. As a bound (`T: 'static`), it means the type holds no shorter-lived references.

macro — Code that generates code at compile time, invoked with a `!` such as `println!` or `vec!`. Far more powerful and hygienic than C's text-substitution `#define`.

macro_rules — The built-in system for *declarative* macros, which match on syntax patterns and expand to code. The everyday way to write your own macros.

match — Rust's pattern-matching control structure, like a `switch` on steroids: it matches structure and binds values, and it must be **exhaustive**. It is also an expression that yields a value.

module — A namespace inside a crate, declared with `mod`, that groups items and controls visibility. The replacement for C's header-and-file organization, without separate `.h` files.

monomorphization — The compiler’s strategy of generating a separate specialized copy of a **generic** function or type for each concrete type used. This makes generics zero-cost (no runtime dispatch), at the price of larger code.

move — Transfer of ownership from one binding to another, e.g. `let b = a;` for a non-**Copy** type, after which `a` is invalid. It guarantees a single owner, so each value is freed exactly once.

&mut — An exclusive, mutable **reference** (`&mut T`). While it exists, no other reference to the same value may exist, which is how Rust prevents data races and aliasing bugs.

Mutex — `Mutex<T>` wraps data and grants access only while locked, via `.lock()`. The lock guard unlocks automatically on drop, so you cannot forget to unlock; combine with **Arc** to share across threads.

newtype — A single-field tuple struct that wraps another type to give it a distinct identity, e.g. `struct Meters(f64);`. It prevents mixing up values, adds methods, and sidesteps the **orphan rule**.

niche optimization — A layout trick where the compiler stores an enum’s tag inside an unused bit pattern of its fields, so `Option<&T>` is the same size as `&T` (using null for `None`). It makes many `Option` and enum types free of extra space.

Option — The enum `Option<T>` with variants `Some(T)` and `None`, representing a value that may be absent. It replaces null pointers and sentinel values; the compiler forces you to handle `None`.

orphan rule — The coherence rule that you may implement a trait for a type only if you define the trait or the type in your crate. It prevents conflicting implementations across crates; the **newtype** pattern works around it.

ownership — The core rule that every value has exactly one owner, and the value is dropped (freed) when the owner goes out of scope. It replaces manual `malloc/free` and makes use-after-free and double-free impossible.

package — A bundle managed by **Cargo**, defined by one `Cargo.toml`, containing one or more **crates** (often one library and/or one binary).

panic — An unrecoverable error that stops the current thread, by default **unwinding** the stack and running destructors. Triggered by `panic!`, failed `assert!`, out-of-bounds indexing, or `unwrap` on `None/Err`.

pattern — The structure on the left of a `match` arm or in a `let`, which both tests a value’s shape and binds parts of it (e.g. `Some(x)`, `Point { x, .. }`). Destructuring and matching in one step.

procedural macro — A macro written as Rust code that transforms a token stream into new tokens, used for custom `#[derive]`, attributes, and function-like macros. More flexible than **macro_rules** but lives in its own crate.

pub — The visibility keyword that exports an item from its **module** (e.g. `pub fn`, `pub struct`). Without it, items are private to their module; the analog of choosing what goes in a C header.

RAII — “Resource Acquisition Is Initialization”: tying a resource’s lifetime to a value’s scope, so cleanup happens automatically via **Drop**. The same idiom C++ uses; in Rust it is built into ownership.

raw pointer — `*const T` or `*mut T`: an unchecked C-style pointer with no borrow or lifetime guarantees. Dereferencing one requires **unsafe**; used for **FFI** and low-level code.

Rc — “Reference Counted” pointer (`Rc<T>`): shared ownership of a heap value within a single thread, freed when the last **Rc** drops. Not thread-safe — use **Arc** across threads.

RefCell — `RefCell<T>` provides **interior mutability** by checking the borrow rules at *runtime* instead of compile time, handing out `Ref/RefMut` guards. Violating the rules panics rather than failing to compile.

reference — A non-owning pointer to a value, `&T` (shared) or `&mut T` (exclusive), guaranteed by the compiler to be non-null and never dangling. Like a C pointer, but checked.

repr(C) — An attribute (`#[repr(C)]`) that lays out a struct or enum using C’s field order and alignment rules. Required for types shared across **FFI**.

Result — The enum `Result<T, E>` with variants `Ok(T)` and `Err(E)`, representing success or failure. It replaces return codes and `errno`; the **? operator** propagates the `Err` case.

rustfmt — The official code formatter, run via `cargo fmt`, that enforces a standard style. It removes formatting debates and keeps diffs clean.

rustup — The official toolchain installer and manager. It installs **rustc**, **Cargo**, and components, and switches between stable, beta, nightly, and cross-compilation targets.

Send — A marker trait meaning a value can be safely *moved* to another thread. Most types are **Send**; **Rc** and raw pointers are not. The compiler uses it to prevent unsafe thread transfers.

shadowing — Declaring a new variable with the same name as an earlier one using `let`, which hides the old binding (and may change its type). Different from mutation: it creates a fresh variable.

slice — A view into a contiguous sequence, `&[T]` (or `&str` for text), carrying a pointer and a length. It is a C pointer-plus-length pair, but bounds-checked and lifetime-tracked.

String — A heap-allocated, growable, owned UTF-8 string. The owned counterpart to `&str`; like a managed `char *` buffer that always knows its length and capacity.

`&str` — A borrowed string slice: a view into UTF-8 text as a pointer and length. Comparable to a `const char *` plus length, but not null-terminated and guaranteed valid UTF-8.

supertrait — A trait that another trait requires, written `trait B: A`, so every type implementing B must also implement A. It lets B’s methods rely on A’s.

Sync — A marker trait meaning a value is safe to *share* between threads by reference (`&T` is **Send**). Types guarded by **Mutex** or atomics are **Sync**; **RefCell** and **Cell** are not.

trait — A named set of methods a type can implement, defining shared behavior. Like a C++ interface or a C struct of function pointers, but checked at compile time; the basis of generics’ bounds.

trait object — A value accessed through `dyn Trait` (behind a pointer like `&dyn Trait` or `Box<dyn Trait>`), enabling runtime polymorphism via a **vtable**. The Rust equivalent of a pointer to a struct of function pointers.

turbofish — The `::<Type>` syntax used to specify a generic type argument explicitly, as in

`"42".parse::<i32>()` or `vec.iter().collect::<Vec<_>>()`. Use it when inference cannot decide the type.

unsafe — A keyword marking code where you, not the compiler, uphold the safety rules: dereferencing **raw pointers**, calling **extern/unsafe** functions, or accessing **static mut**. It is “ordinary C,” scoped and searchable.

unwinding — The default response to a **panic**: walking back up the stack, running each value’s **Drop**, until the thread stops. It can be caught with **catch_unwind** or replaced by **abort** via a build setting.

usize — The pointer-sized unsigned integer used for indexing and lengths (Rust’s `size_t`). `isize` is its signed counterpart (`ptrdiff_t`).

variant — One of the named alternatives of an **enum**, possibly carrying data (e.g. `Some(T)` and `None` are the variants of `Option`). A **match** selects on which variant a value is.

Vec — `Vec<T>`, a heap-allocated, growable array storing a pointer, length, and capacity. The Rust equivalent of a `malloc`’d array tracked with its length, freed automatically.

vtable — A table of function pointers (plus size and alignment) that a **trait object** uses to find the right method at runtime. Equivalent to a hand-built C struct of function pointers used for dynamic dispatch.

where clause — A clause that lists trait bounds for generics in a readable block after the signature, e.g. `where T: Clone + Debug`. Useful when bounds are long or involve associated types.

zero-cost abstraction — The principle that high-level features (iterators, generics, traits) compile to machine code as efficient as hand-written low-level code. You do not pay at runtime for the convenience.

? operator — The postfix `?` that, applied to a **Result** or **Option**, returns the value on success or returns early with the error/`None` on failure. It replaces repetitive check-and-return code around C return codes.

References and Further Reading

What you’ll learn. Where to go next: the official docs, the best books, places to practice, the everyday tools, the community, and a short list aimed specifically at C programmers.

Every link below is to an official or well-known resource. Where a resource has no stable official URL, it is named without a link.

Official documentation

- **The Rust Programming Language** (“the book”) — the canonical introduction. <https://doc.rust-lang.org/book/>
- **Rust by Example** — runnable examples for each feature. <https://doc.rust-lang.org/rust-by-example/>
- **The Standard Library (std) docs** — API reference for everything in `std`. <https://doc.rust-lang.org/std/>

- **The Rust Reference** — the detailed language reference (syntax and semantics). <https://doc.rust-lang.org/reference/>
- **The Rustonomicon** — the dark arts of `unsafe` Rust. <https://doc.rust-lang.org/nomicon/>
- **The Rust API Guidelines** — how to design idiomatic, predictable APIs. <https://rust-lang.github.io/api-guidelines/>
- **The Cargo Book** — the full guide to Cargo and `Cargo.toml`. <https://doc.rust-lang.org/cargo/>
- **Asynchronous Programming in Rust** (“the async book”) — `async/.await`, futures, and executors. <https://rust-lang.github.io/async-book/>
- **The Edition Guide** — what changed in each edition, including 2024. <https://doc.rust-lang.org/edition-guide/>

Books

- **Programming Rust** — Jim Blandy, Jason Orendorff & Leonora Tindall (O’Reilly). Thorough and systems-focused; excellent for C and C++ programmers.
- **Rust for Rustaceans** — Jon Gjengset (No Starch Press). Intermediate; deepens your understanding once you know the basics.
- **Rust Atomics and Locks** — Mara Bos (O’Reilly). The clearest treatment of low-level concurrency, atomics, and memory ordering. Read online: <https://marabos.nl/atomics/>
- **Zero To Production In Rust** — Luca Palmieri. A hands-on guide to building real web services (APIs, databases, testing, deployment). <https://www.zero2prod.com/>

Practice

- **Rustlings** — small fix-the-code exercises that track the official book. <https://github.com/rust-lang/rustlings>
- **Exercism — Rust track** — graded exercises with mentor feedback. <https://exercism.org/tracks/rust>
- **Advent of Code** — yearly programming puzzles; a popular, fun way to practice Rust on real problems. <https://adventofcode.com/>

Tools

- **Clippy** — the official linter; catches mistakes and unidiomatic code. <https://doc.rust-lang.org/clippy/>
- **rustfmt** — the official code formatter. <https://github.com/rust-lang/rustfmt>
- **rust-analyzer** — the language server for editor support (completion, go-to, inline errors). <https://rust-analyzer.github.io/>
- **Miri** — an interpreter that detects undefined behavior in `unsafe` code. <https://github.com/rust-lang/miri>
- **Tokio** — the most widely used async runtime. <https://docs.rs/tokio/> and <https://tokio.rs/>
- **Serde** — the standard serialization/deserialization framework. <https://serde.rs/>
- **clap** — the standard command-line argument parser. <https://docs.rs/clap/>

Community

- **The Rust Users Forum** — ask questions, get help. <https://users.rust-lang.org/>
- **r/rust** — the Rust subreddit, news and discussion. <https://www.reddit.com/r/rust/>

- **This Week in Rust** — a weekly newsletter of news, blog posts, and crates. <https://this-week-in-rust.org/>

Especially for C programmers

- **The Rustonomicon — FFI chapter** — calling C from Rust and back across the boundary. <https://doc.rust-lang.org/nomicon/ffi.html>
- **std::ffi docs** — CString, CStr, OsString, and C-compatible types. <https://doc.rust-lang.org/std/ffi/>
- **The bindgen User Guide** — auto-generate Rust bindings from C headers. <https://rust-lang.github.io/rust-bindgen/>
- **The cbindgen User Guide** — generate C headers from a Rust library so C code can call it. <https://github.com/mozilla/cbindgen/blob/master/docs.md>